chapter - 1 (FD's and normalization)

Sunday, June 9, 2024 4:38 PM

(3-4 marks)

Functional dependencies and normalization

topic - relational database management system

Database that is in a form of table (relation)

Data

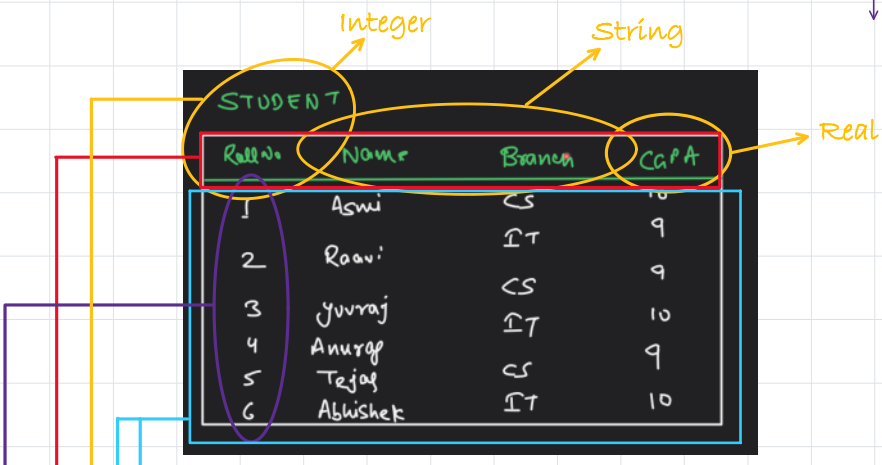
information

Database (collection of logically related data)

DBMS

ROW
Tuple/
Record

COLUMN
Attribute/
Field



1. Arity (degree) : Number of attributes (4).

2. Cardinality : Number of tuples (6).

3. Relational schema (intension) (fixed): Table abstraction/heading.
   - table name
   - attribute name
   - domain

4. Relational Instance (Relation state/extension) (can change): Set of records (tell us detail about tuples)

5. Domain: Set of allowed value for each attribute.
   (data type)

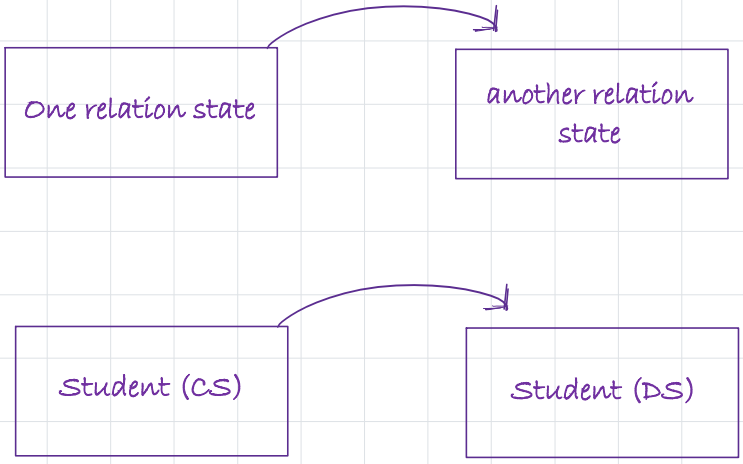
Attribute values are required to be atomic, that is indivisible.
The special value - Null (member of every domain)

Relation Schema and Instance

- **A relation schema R**, denoted by $R(A_1, A_2, , A_n)$, is made up of a relation name R, a list of attributes, $A_1, A_2, , A_n$ & Each attribute A_i domain D. D is called the domain of A_i and is denoted by $dom(A_i)$.
- The **degree (or arity)** of a relation is the number of attributes n of its relation schema.
- STUDENT(Name, Ssn, Home_phone, Address, Office_phone, Age, Gpa)
- A relation of degree seven, which stores information about university students, would contain seven attributes describing each student as follows:

Schema Using the data type of each attribute, the definition is sometimes written as:

- **STUDENT(Name: string, Ssn: string, Home_phone: string, Address: string, Office_phone: string, Age: integer, Gpa: real)**



SCHEMA - fixed. #STATE - changed.

EN.	name	Dept.	salary
1	A	Support	50k
2	B	MR.	60k
3	C	developer	70k
4	D	technology	80k

Relational Instance (State/Extension) TUPLES.

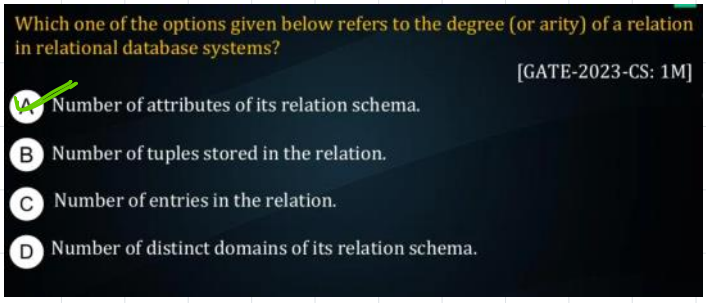
EN.	name	Dept.	salary
1	P	designer	40k
2	Q	HR.	70k
3	R	SDE	90k
4	S	ML eng.	30k

Relational Instance (State/Extension) TUPLES.

SCHEMA - fixed. #STATE - varies

EN.	name	Dept.	salary
1	A	Support	50k
2	B	MR.	60k
3	C	developer	70k
4	D	technology	80k

Relation is a subset of domain.
Domain : 1000 Employee
chose only 4.



chapter - functional dependency.

1. Functional dependency: $X \rightarrow Y$ (whenever X value repeat, corresponding Y value must be same)

X(Determinate) functionally determine Y(Determiner/Dependent). whenever my roll number repeats, my result will be displayed.

Let R be the Relational schema, X and Y be the attribute set of relation R and t_1, t_2 Such that $X \rightarrow Y$ exist.

If $t_1.x = t_2.x$ then $t_1.y = t_2.y$ must be same.

X	Y
1	4
2	5
3	6
4	7
5	8
2	5
4	8

X	Y
1	7
2	7
3	7
4	8
5	8
6	8

X	Y
x_1	y_1
x_2	y_5
x_3	y_7
x_1	y_1
x_2	y_5
x_3	y_4

X	Y
x_1	y_5
x_2	y_6
x_3	y_7
x_1	y_5
x_2	y_6

X	Y
1	8
2	8
3	8
4	9
5	9
6	5

X	Y
1	5
2	5
3	6
1	5
2	5
3	6

here x value repeat but Y value is not same hence invalid.

Types of FD.

1. Trivial FD - $X \rightarrow Y$ is a trivial FD if and only if $X \geq Y$ (x is a subset of Y, Y is a subset of X)
(khudka khudi se)

R.H.S. must be equal or part of L.H.S.

Ex.
$AB \rightarrow A$
$AB \rightarrow B$
$AB \rightarrow AB$

2. Non-Trivial FD - $X \rightarrow Y$ is a Non - trivial FD if and only if $X \cap Y = \Phi$

Nothing is common and $X \rightarrow Y$ must satisfy FD definition.

Ex.
$A \rightarrow C$
$A \rightarrow B$

note :
Trivial FD Always valid.

TRIVIAL	VS	NON-TRIVIAL
$A \rightarrow A$		$A \rightarrow B$
$AB \rightarrow A$		$AB \rightarrow C$

Problems-

Cases :

(A)

A → B

9 → 9

9 → 4

A → C

9 → 9

9 → 1

A → BC

9 → 99

9 → 49

(B)

B → A

9 → 9

9 → 4

B → C

9 → 9

9 → 4

B → AC

9 → 99

9 → 44

(C)

C → A

1 → 4

1 → 9

C → B

9 → 9

9 → 4

C → AB

9 → 99

9 → 94

(D)

AB → C

49 → 4

49 → 1

BC → A

AC → B

99 → 9

99 → 4

(A)

X → Y

X → Z

X → YZ

(B)

Y → X

Y → Z

Y → XZ

(C)

Z → X

Z → Y

Z → XY

(D)

XY → Z

YZ → X

ZX → Y

note :
Rule out the FD's based on table

(A)

X → Y

(B)

Y → X

(C)

Z → X

(D)

XY → Z

(A) $X \rightarrow Y$

(B) $Y \rightarrow X$

(C) $Z \rightarrow X$

(D) $XY \rightarrow Z$

$X \rightarrow Z$

$Y \rightarrow Z$

$Z \rightarrow Y$

$YZ \rightarrow X$

$X \rightarrow YZ$

$Y \rightarrow XZ$

$Z \rightarrow XY$

$ZX \rightarrow Y$

(A) $A \rightarrow B$

(B) $B \rightarrow C$

(C) $C \rightarrow A$

(D) $AB \rightarrow C$

$A \rightarrow C$

$B \rightarrow A$

$C \rightarrow B$

$BC \rightarrow A$

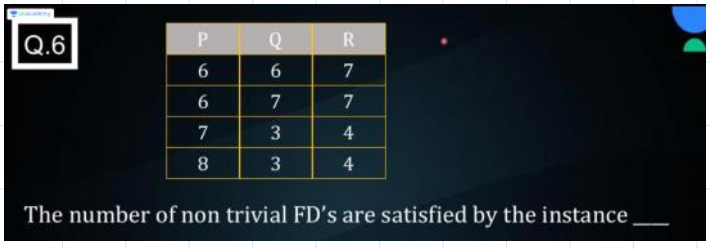
$A \rightarrow BC$

$B \rightarrow AC$

$C \rightarrow AB$

$CA \rightarrow B$

None of these



(A) $P \rightarrow Q$

(B) $Q \rightarrow R$

(C) $R \rightarrow P$

(D) $PQ \rightarrow R$

$P \rightarrow R$

$Q \rightarrow P$

$R \rightarrow Q$

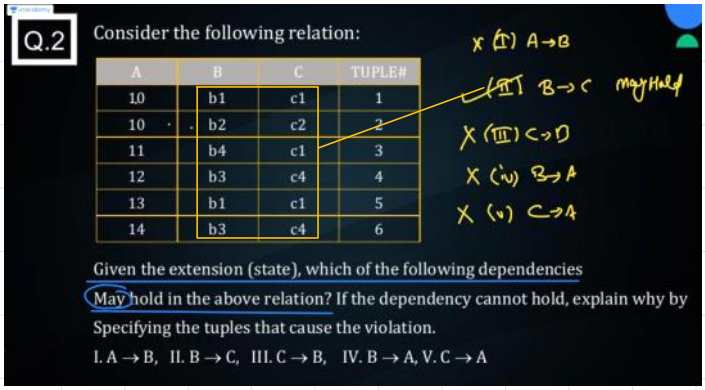
$PR \rightarrow Q$

$P \rightarrow QR$

$Q \rightarrow PR$

$R \rightarrow PQ$

$QR \rightarrow P$



By looking at some instance, we cannot conclude - B→C (~~always satisfy~~) (may hold)

On the behalf of 6 tuples we cannot conclude true(always satisfy) for whole table.

Conclusion :

1. Trivial FD's are always valid.
2. Rule out the FD based on Instances.
3. Trivial FD always hold.

Instance - CGPA - 9 and more
Table - University of 1 lakh students.

Instance ke behalf par conclusion nahi de sakte because for true in future it may become false but we can guarantee for failure because once failed means it is never going to hold in future also.

A→B
C→B
B→A
C→A

Q.9 Given the following relation instance. [2000: 2 Marks]

X	Y	Z
1	4	2
1	5	3
1	6	3
3	2	2

Which of the following functional dependencies are satisfied by the instance?

A. XY → Z and Z → Y
B. YZ → X and Y → Z
C. YZ → X and X → Z
D. XZ → Y and Y → X

Q.10 From the following instance of a relation scheme R (A, B, C), we can conclude that: [2002: 2 Marks]

A	B	C
1	1	1
1	1	0
2	3	2
2	3	2

A. A functionally determines B and B functionally determines C
B. A functionally determines B and B does not functionally determines C
C. B does not functionally determines C
D. A does not functionally determines B and B does not functionally determines C

note :
Because, on the behalf of instance we cannot conclude always satisfy. But failure is guaranteed.

[MSQ]

Consider a relation scheme R(A, B, C, D, E) and following table is an instance of relation R.

A	B	C	D	E
a	1	5	2	3
b	2	6	3	4
c	2	6	4	4
d	3	7	6	5
e	3	7	7	5

F is set of functional dependencies hold on R. Which of the following is definitely true (or we conclude)

A. B does not functionally determine D.
B. B functionally determine C and D functionally determine E.
C. C functionally determine E.
D. C does not functionally determine D.

Or trivial FD's

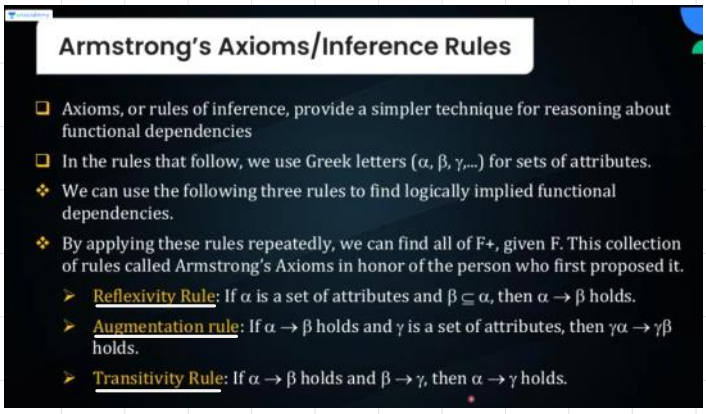
Q.11 Consider the relation X(P, Q, R, S, T, U) with the following set of functional dependencies [2015: 1 Marks]

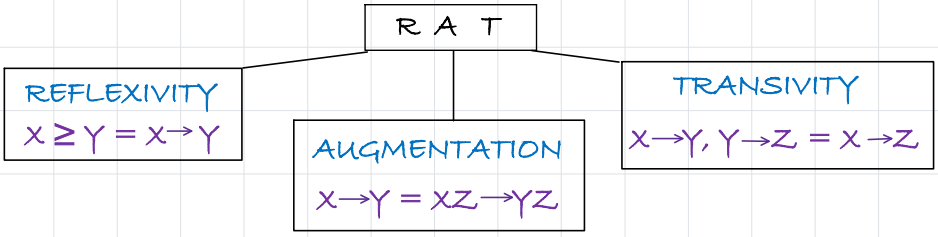
F = {
{P, R} → {S, T}
{P, S, U} → {Q, R}
}

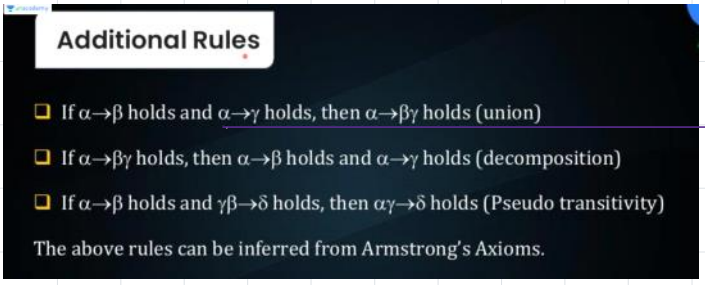
Which of the following is the trivial functional dependency in F⁺ is closure of F?

A. {P, R} → {S, T}
B. {P, R} → {R, T}
C. {P, S} → {S}
D. {P, S, U} → {Q}

Properties of FD-







A→BC
A→B
A→C

wrong

AB→C
A→C
B→C

topic - Attribute closure $[X]^+$

let R be the relation schema, X be the attribute set of relation R

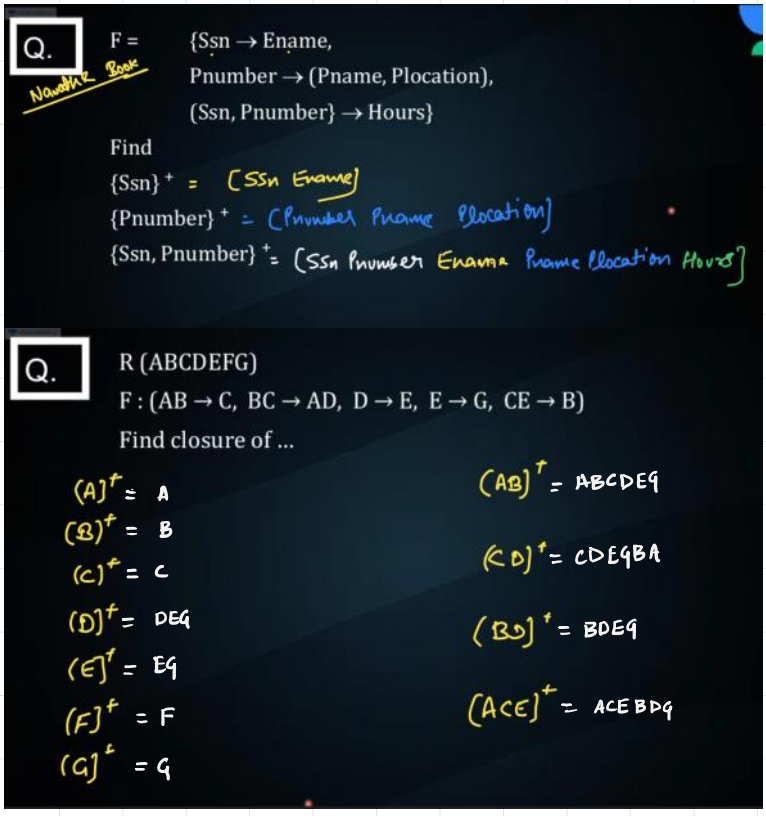
set of all possible attributes which logically functionally determined by attribute X (kya kya nikal sakte from the attribute X or from its combinations)

R(ABCDEF) [A →B, B →C, C →D, D →E, E →F]

$[A]^+$ = ABCDEF
$[B]^+$ = BCDEF
$[C]^+$ = CDEF
$[D]^+$ = DEF
$[E]^+$ = EF
$[F]^+$ = F
$[AF]^+$ = AFBCDE
$[DE]^+$ = DEF

$[A]^+ = A,B$ — what more we can determine from either A or B or C or from the combination of AB....

A,B,C — what more we can determine from either A or B or C or from the combination of AB, BC, AC, ABC....



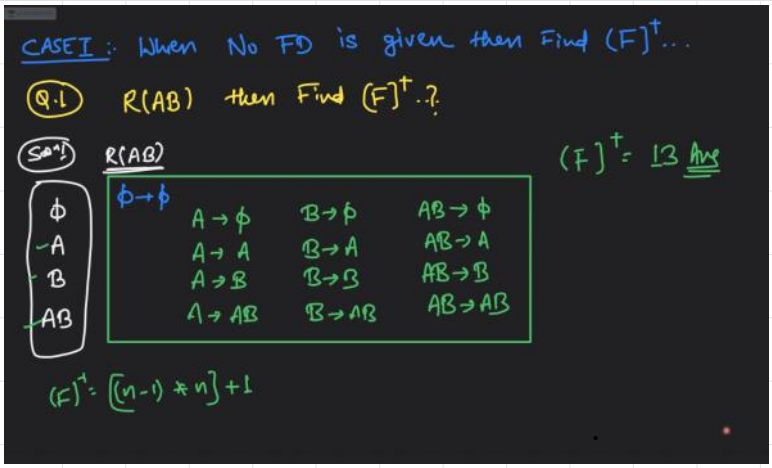
topic - closure of FD set $[F]^+$

Set of all possible FD's which can be determined by the given FD set.

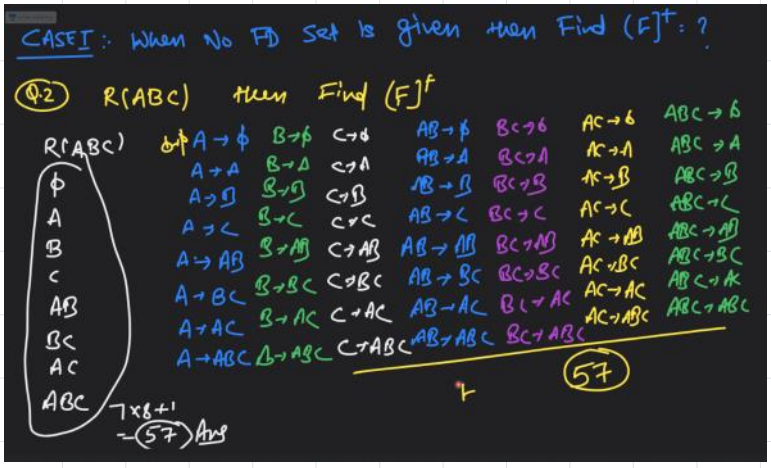
Closure of FD set $[F]^+$

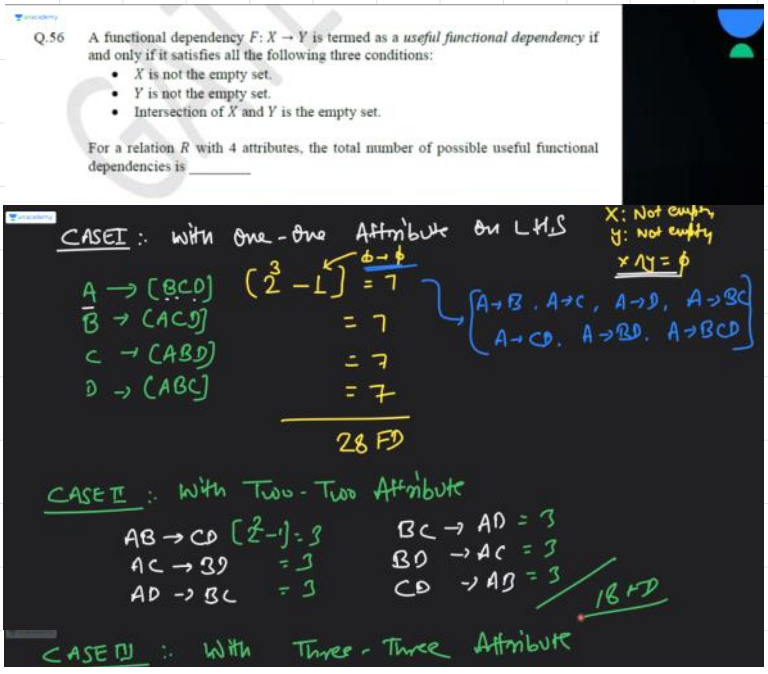
- when FD set is not given then find $[F]^+$
- when FD set is given then find $[F]^+$

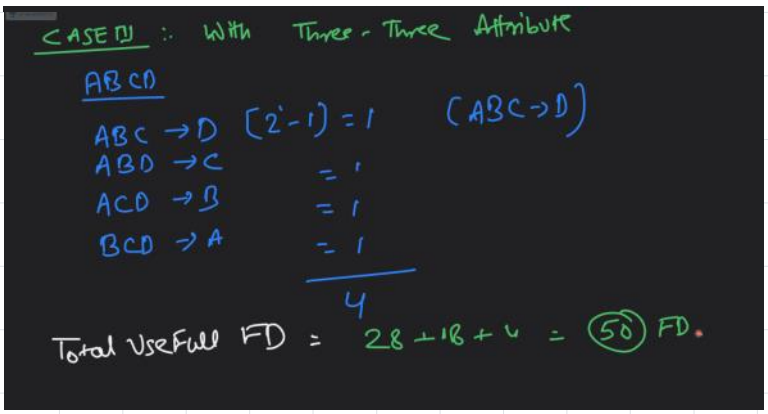
case 1: when FD set is not given then find $[F]^+$



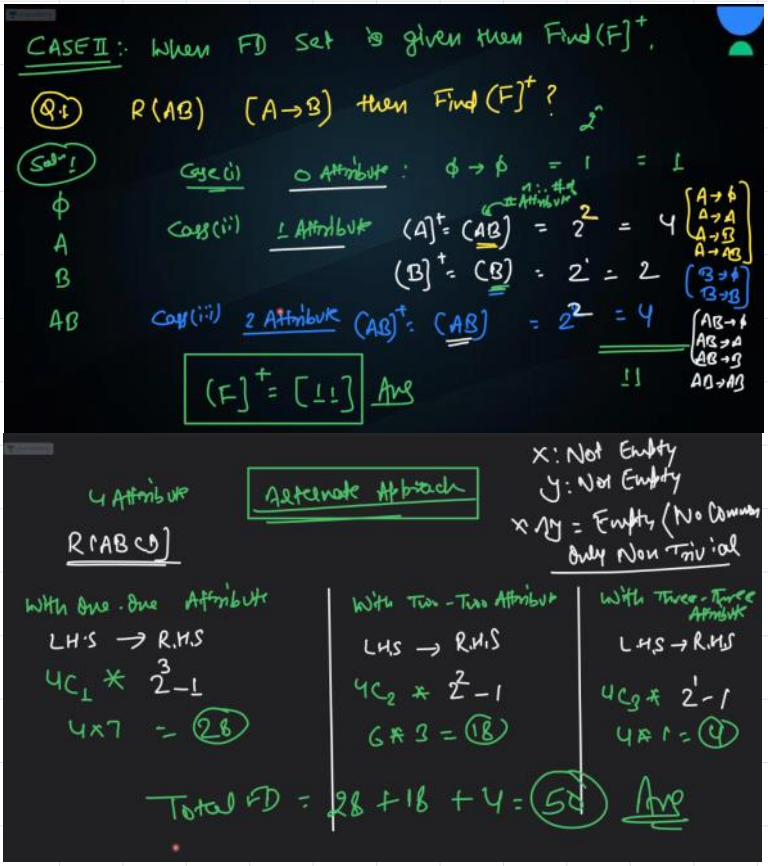
Formula to find :
$[F]^+ = [(n-1)n] + 1$



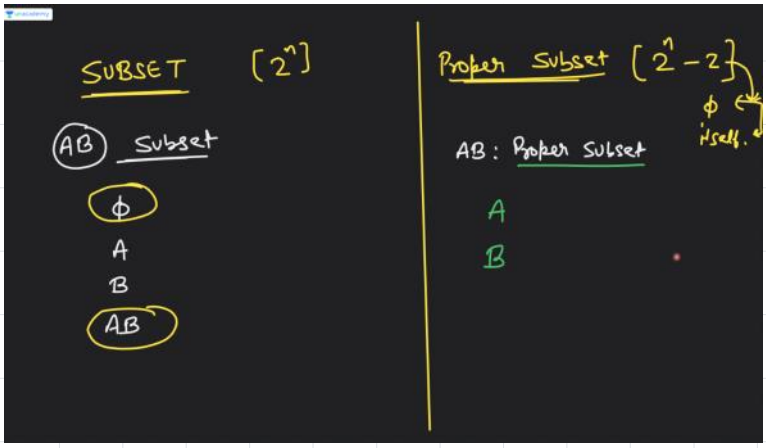




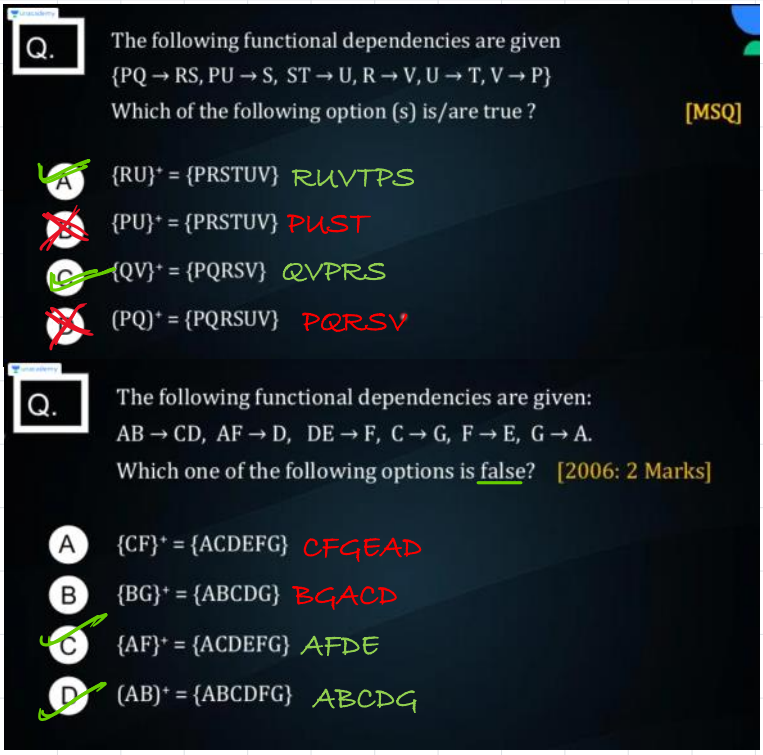
case 2: when FD set is given then find $[F]^+$



Difference between Subset and Proper subset



Homework



Keys Concept

1. Super key : Let R be the relational schema
Let X be the attribute of Relation R

if [X] - Determine all attribute of Relational Schema R, then X is a super key
(jis attribute se mai rest of all attributes ko determine kar sakta hoon)

R(ABCDE) [A → B, B → C, C → D, D → E]

[A]+ = ABCDE ——— superkey
[B]+ = BCDE

$[A]^+ = ABCDE$ — superkey

$[B]^+ = BCDE$

$[C]^+ = CDE$

$[D]^+ = DE$

$[E]^+ = E$

any superset of 'A' will be a superkey

note :
Any superset of superkey is also superkey

superkey — superkey

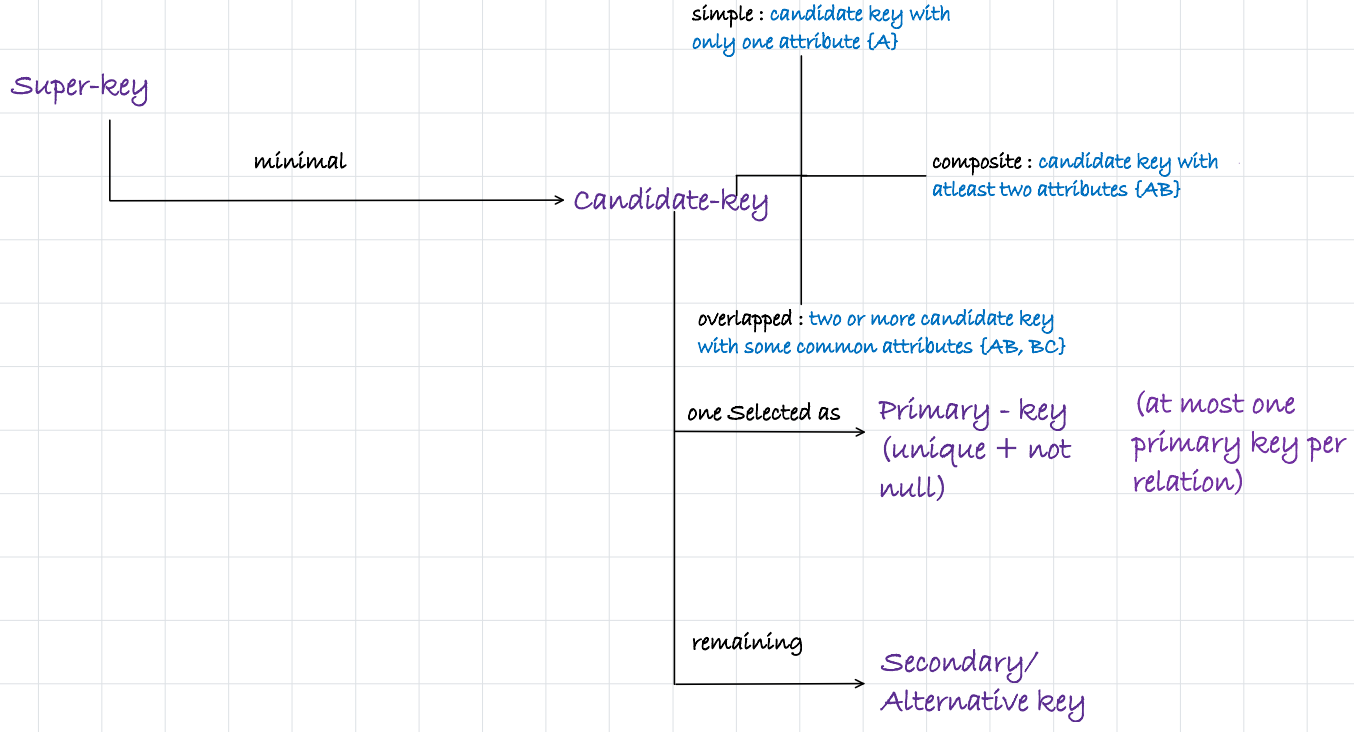
A + ____ = A____

2. Candidate key : Let R be the relational schema

Minimum se minimum se minimum attribute or set of attribute which cannot be further Decomposed and is mandatory and required to identify each tuple in relation

(OR)

If any proper subset of super-key is also super-key then that proper subset is called candidate key.



example : bank account

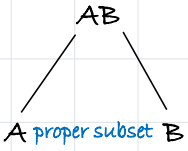
Super-key
(P.K + A.K)

minimal → Candidate-key

Candidate-key → one Selected as → Primary - key
(account number)

Candidate-key → remaining → Secondary/
Alternative key
(aadhar card, PAN card, registered mobile number and dob.)

R(ABCDE) [AB → C, C → D, B → EA]
$[AB]^+$ = ABCDE | AB is superkey

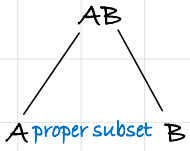


$[A]^+$ = A
$[B]^+$ = BEACD (ABCDE)

B is candidate key (b can determine all)

B is minimal

R(ABCDE) [AB → C, C → D, B → E]
$[AB]^+$ = ABCDE | AB is superkey



$[A]^+$ = A
$[B]^+$ = BE

AB is candidate key (alone A or B cannot determine all)

AB is minimal

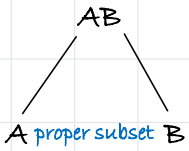
note :
(i) Every candidate key is super-key
(ii) Every super-key is not a candidate key because candidate key is a minimal of super-key

3. Prime/key attribute : Set of all attribute which is present/ belongs to any/some candidate key.

4. Non-Prime/Non-key attribute : Set of all attribute which is not present/ does not belong to any/some candidate key.

R(ABCDE) [AB → C, C → D, B → EA]

$[AB]^+$ = ABCDE | AB is superkey



$[A]^+$ = A

$[B]^+$ = BEACD (ABCDE)

B is candidate key superkey also
(b can determine all)

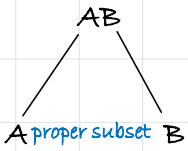
B is minimal

Prime/key attribute
B

Non-Prime/Non-key attribute
ACDE

R(ABCDE) [AB → C, C → D, B → E]

$[AB]^+$ = ABCDE | AB is superkey



$[A]^+$ = A

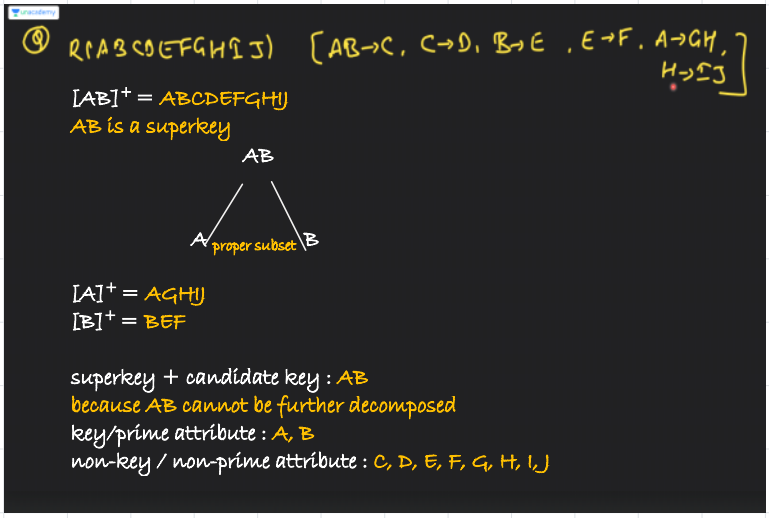
$[B]^+$ = BE

AB is candidate key (alone A or B cannot determine all)

AB is minimal

Prime/key attribute
AB

Non-Prime/Non-key attribute
CDE



5. Finding Multiple Candidate key :

Procedure - First find any one candidate key, then that attribute is called key/prime attribute.

(OR)

If any (X) attribute → Prime attribute
Then multiple candidate keys are possible.

If any (X) attribute ⟶ Prime attribute
Then multiple candidate keys are possible.

simple bhi ho sakta aur composite bhi

Assume B is a candidate key -

Prime/key attribute - B

If that B is coming from D

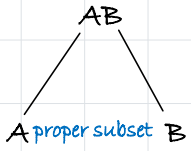
EG.
1. D→B
D→BC (D is also candidate key)

2. AD→B
Check

AD → [A]+
AD → [D]+

If we are deriving a 2 attribute candidate key then firstly break it and check if individual closure are giving us possible attributes or not.

R(ABCDE) [AB →C, C →D, D →E, B →A, C →B]
[AB]+ = ABCDE | AB is superkey



[A]+ = A
[B]+ = BACDE

B is candidate key

Prime attribute : [B, C]

If any X attribute ⟶ Prime attribute
koi attribute prime attribute (b) ko determine kar paa raha hai?

C→B

[C]+ = CBADE
C is candidate key

6. Composite key : A key with 2 or more attribute.

Problems -

Q.1

R(ABCDEF) {A → B, B → C, D → CEF}

Find candidate keys for the relation R ?

$[A]^+ = ABC$
$[B]^+ = BC$
$[C]^+ = C$
$[D]^+ = DCEF$
$[AD]^+ = ABCDEF$

AD is superkey
$[A]^+ = A$
$[D]^+ = DCEF$

AD cannot be further decomposed hence AD is also candidate key

Prime : A and D

no attribute can determine prime hence there is only one candidate key : AD

Q.3

R(ABCD) {A → B, B → C, C → D, D → A}

Find candidate keys for the relation R ?

$[A]^+ = ABCD$
A is superkey + candidate key

$[B]^+ = BCDA$
B is superkey + candidate key

$[C]^+ = CDAB$
C is superkey + candidate key

$[D]^+ = DABC$
B is superkey + candidate key

Prime : A, B, C and D
(i) A
D→A | D is already a candidate key
(ii) B
A→B | A is already a candidate key
(iii) C
B→C | B is already a candidate key
(iv) D
C→D | C is already a candidate key

A, B, C and D are the candidate keys.

Q.4

R(ABCDEF) {A → BCDE, BC → AD, D → EF}

Find candidate keys for the relation R ?

$[A]^+ = ABCDEF$
A is superkey + candidate key

Prime : A
(i) A
BC→AD
we have to break down BC in order to check if it is possible to determine the relation R from B or C independently
$[B]^+ = B$
$[C]^+ = C$
we are not able to determine the relation R from B or C independently hence the candidate keys are A and BC
Prime : A, B, C
cannot determine A, B or C from any attribute.

Q.5

R(ABCD) F: {AB → C, B→ D, C → B, D → B}

Find all candidate key of R?

$[AB]^+ = ABCD$
$[A]^+ = A$
$[B]^+ = BD$
AB is superkey + candidate key

Prime : A, B
(i) A
no attribute can determine A
(ii) B
AD→B (D replaced B in AB)
$[AD]^+ = ADBC$
$[A]^+ = A$
$[D]^+ = DB$
AC→B (C replaced B in AB)
$[AC]^+ = ACBD$
$[A]^+ = A$
$[C]^+ = CBD$
Prime : A, B, C, D

we are not able to determine any prime attribute anymore hence candidate keys are AB, AD, AC

Q.6

Consider the following relational schema R(ABCDEF) with functional dependency {AB → C, C → D, D → E, E → F, F → B}
The number of candidate keys for relation R ?

Q.6

Consider the following relational schema R(ABCDEF) with functional dependency {AB → C, C → D, D → E, E → F, F → B} The number of candidate keys for relation R ?

$[AB]^+ = ABCDEF$
$[A]^+ = A$
$[B]^+ = B$
AB is superkey + candidate key

Prime : A, B
(i) A
no attribute can determine A
(ii) B
AF→B (F replaced B in AB)
$[AF]^+ = AFBCDE$
$[A]^+ = A$
$[F]^+ = FB$
Prime : A, B, F
(iii) F
AE→F (E replaced F in AF)
$[AE]^+ = AEFBCD$
$[A]^+ = A$
$[E]^+ = EFB$

Prime : A, B, F, E
(iv) E
AD→E (D replaced E in AE)
$[AD]^+ = ADEFBC$
$[A]^+ = A$
$[D]^+ = DEFB$
Prime : A, B, F, E, D
(v) D
AC→D (C replaced D in AD)
$[AC]^+ = ACDEFB$
$[A]^+ = A$
$[C]^+ = BCDEFB$
Prime : A, B, F, E, D, C

we are not able to determine any prime attribute anymore hence candidate keys are AB, AF, AE, AD, AC

Q.7

R(ABCDE) : {AB→C, BC →D}
Find Candidate keys for the Relation R?

$[AB]^+ = ABCD$
cannot determine E hence we have to add E from the relation R
$[ABE]^+ = ABCDE$

hence, ABE is the candidate key.
Prime : A, B, E

note :
whenever any attribute not present in FD then ADD that in the candidate key

Q.8

R(ABCDEFG) : {A→B, B → C, C →D, D → E, F → G}
Find Candidate keys for the Relation R?

$[AF]^+ = ABCDEFG$
$[A]^+ = ABCDE$
cannot determine F and G
$[F]^+ = FG$

hence, AF is the candidate key.
Prime : A, F

Q.9

R(ABCDEFGH) : {AB→C, AC→B, AD→E, B→D, BC→A, E→G}
Find Candidate keys for the Relation R?

$[AB]^+ = ABCDEG$
$[A]^+ = A$
$[B]^+ = B$
FH is missing hence we have to add FH
$[ABFH]^+ = ABCDEFGH$
ABFH is a superkey + candidate key

Prime : A, B, F, H
(i) A
BC→A (A replaced by C in AB)
$[BC]^+ = BCADEG$
$[B]^+ = B$
$[C]^+ = C$
BCFH is a candidate key

Prime : A, B, F, H, C
(ii) B
AC→B (B replaced by A in BC)
$[AC]^+ = ACBDEGFH$
$[A]^+ = A$
$[C]^+ = C$
ACFH is a candidate key

hence, ABFH , BCFH and ACFH is the candidate key.
Prime : A, B, F, H, C

Q.10

R(ABDLPT), {B→PT, T → L, A → D}
Find candidate keys for the relation R?

$[AB]^+ = ABPTLD$
$[A]^+ = AD$
$[B]^+ = BPTL$

hence, AB is the candidate key.
Prime : A, B

Q.11 R(ABCDEFGHIJ) = {AB→C, A → DE, B → F, F → GH, D → IJ]

Find candidate keys for the relation R?

$[AB]^+$ = ABCDEFGHIJ
$[A]^+$ = ADEIJ
$[B]^+$ = BFGH

hence, AB is the candidate key.
Prime : A, B

Q.12 R(ABCDEFG)

[A→B, B → A, C → D, D → E, E → FG]

Find candidate keys for the relation R?

$[AC]^+$ = ABCDEFG
$[A]^+$ = AB
$[C]^+$ = CDEFG

Prime : A, C
(i) A
B→A (B replaced A in AC)
$[BC]^+$ = BACDEFG
$[B]^+$ = BA
$[C]^+$ = CDEFG
BC is candidate key

hence, AC and BC is the candidate key.
Prime : A, B, C

Q.13 R(ABCDEFG) {AB→CDEF, C→ADE, D→EBF, F→DA, BE→AF}

Find candidate keys for the relation R?

$[AB]^+$ = ABCDEF
$[A]^+$ = A
$[B]^+$ = B
G is missing hence we have to add G
$[ABG]^+$ = ABCDEFG
ABG is a superkey + candidate key

Prime : A, B, G
(i) A
BE→AF (A replaced by E in AB)
$[BE]^+$ = BEAFDC
$[B]^+$ = B
$[E]^+$ = E
BEG is a candidate key
Prime : A, B, G, E
F→DA (A replaced by F in AB)
$[F]^+$ = FADEBC
BFG is a candidate key

Prime : A, B, G, E, F
C→ADE (B replaced by A in BC)
$[C]^+$ = CADEBF
CBG is a candidate key
Prime : A, B, G, E, F, C
(ii) E
D→EBF
$[D]^+$ = DEBFAC
DEG is a candidate key
BDG is a candidate key
ADG is a candidate key

hence, ABG, BEG, BFG, CBG, DEG, BDG is the candidate key.
Prime : A, B, G, E, F, C, D

Q.14 R(ABCDEFGH) {A→BC, B → DEF, DE → AGH}

Find candidate keys for the relation R?

$[A]^+$ = ABCDEFGH
A is a superkey + candidate key

Prime : A
(i) A
DE→AGH (A replaced by DE in A)
$[DE]^+$ = DEAGHBC
$[D]^+$ = D
$[E]^+$ = E
DE is a candidate key
Prime : A, D, E
(ii) D
B→DEF (D replaced by B in DE)
$[BE]^+$ = BEDFAGH
BE is a candidate key

hence, A, DE, BE is the candidate key.
Prime : A, D, E, B

Q.15 R(ABCDE) {A → BC, CD → E, B → D, E → A}

Find candidate keys for the relation R ?

$[A]^+$ = ABCDE

primary key : A, E, C, D, B

Find candidate keys for the relation R ?

$[A]^+ = ABCDE$
$[E]^+ = EABCD$
$[CD]^+ = CDEAB$
$[CB]^+ = CBDEA$

primary key : A, E, C, D, B
candidate key : A, E, CD, CB

four candidate keys :

A
E
CD
CB

1 select as primary key
- A

rest select as alternate
(OR) secondary key
- E
- CD
- CB

Q.16 R(ABCDEFGH)

{AB → CD, D → EG, F → H, C → EF, H → A, G → B, A → B}

Find candidate keys for the relation R ?

- $[A]^+ = ABCDEFGH$
- $[H]^+ = HABCDEFG$
- $[F]^+ = FHABCDEG$
- $[C]^+ = CEFHABDG$

P : A, H, F, C

7 A, H, F, C

GATE Questions

Q.1 Consider the relation scheme R(A,B,C) with the following functional dependencies

Q.1 Consider the relation scheme R(A,B,C) with the following functional dependencies

$AB \rightarrow C$

$C \rightarrow A$

Determine the minimal keys of relations R. [1995: 2 Marks]

① AB.
∘ C→A

② CB
CBA

P: A,B,C

Q.2 Let R = (A,B,C,D,E,F) be a relation scheme with the following dependencies C→F, E→A, EC→D, A→B. Which of the following is a key for R? [1999: 1 Mark]

(A) CD - CDF
(B) EC ECDABF
(C) AE - AEB
(D) AC - ACFB

Q.3 The relation scheme student Performance (name course No, rollNo, grade) has the following functional dependencies:

name, courseNo → grade
RollNo, courseNo → grade
name → rollNo
rollNo → name
Find candidate keys? [1999: 1 Mark]

② (NC) (RC)
N → NR
RC → RCGN
(NC)+ ⇒ NCGR

Q.4 Consider a relation scheme R = (A, B, C, D, E, H) on which of the following functional dependencies hold:

{A → B, BC → D, E → C, D → A}

What are the candidate keys of R? [2005: 2 Marks]

(A) AE, BE
(B) AE, BE, DE
(C) AEH, BEH, BCH
(D) AEH, BEH, DEH

[AE]+ = ABECD
[BCH]+ = [BCHD
[DEH]+ = DAECHB

Q.5 Let R(A,B,C,D,E,F,P,G) be a relational schema in which of the following functional dependencies are known to hold:

AB→CD, DE→P, C→E, P→C and B→G.

Find candidate key of Relations R?

P: A,B,F

Find candidate key of Relations R?

P : A, B, F

$[AB]^+$: ABCDEFG

✓ $[ABF]^+$ = ABCDEFG

Q.6 Consider a relation R with five attributes V, W, X, Y, and Z. The following functional dependencies hold : VY → W, WX → Z, and ZY → V. Which of the following is a candidate key for R?

[2006: 2 Marks]

A. VXZ — $[VXZ]^+$ = [VXZ]

B. VXY — $[VXY]^+$ = [VXYWZ]

C. VWXY

D. VWXYZ

] S.key (not C.K) (C.K - minimal of S.K)

Q.7 Relation R has eight attributes ABCDEFGH. Fields of R contain only atomic values.

F = {CH → G, A → BC, B → CFH, E → A, F → EG} is a set of functional dependencies (FDs) so that F is exactly the set of FDs that hold for R.

How many candidate keys does the relation R have? [2013: 2 Marks]

AD, ED, FD, BD.

A. 3

B. 4

C. 5

D. 6

Q.8 Consider the relation scheme R = (E, F, G, H, I, J, K, L, M, N) and the set of functional dependencies{{EF} → {G}, {F} → {I,J}, {E,H} → {K,L},{K} → {M},{L} → {N}no R. What is the key of R?

[2014: 1 Mark]

A. {E,F}

B. {E,F,H}

C. {E,F,H,K,L}

D. {E}

- $[EF]^+$ = EFGIJ
- $[EFH]^+$ = EFHGIJKLMN

Q.9 A prime attribute of a relation scheme R is an attribute that appears [2014: 1 Mark]

A. In all candidate keys of R.

B. In some candidate key of R.

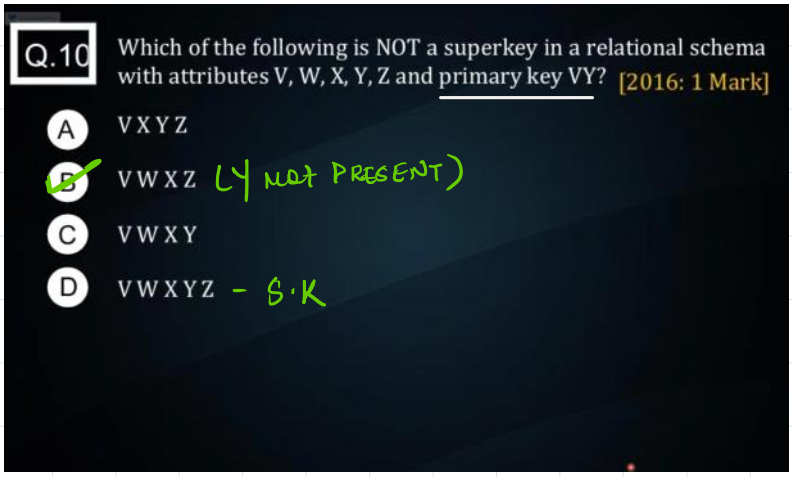
C. In a foreign key of R.

D. Only in the primary key of R.

note :
4 CK - A,E,H,CD
Prime - A,C,D,E,H
A is not appearing in every candidate key!

note :
Default we call key as a candidate key.

Which of the following is NOT a superkey in a relational schema



topic - membership - set

Let F be the given FD set
F : [----------]
X → y is a member of FD set or not?

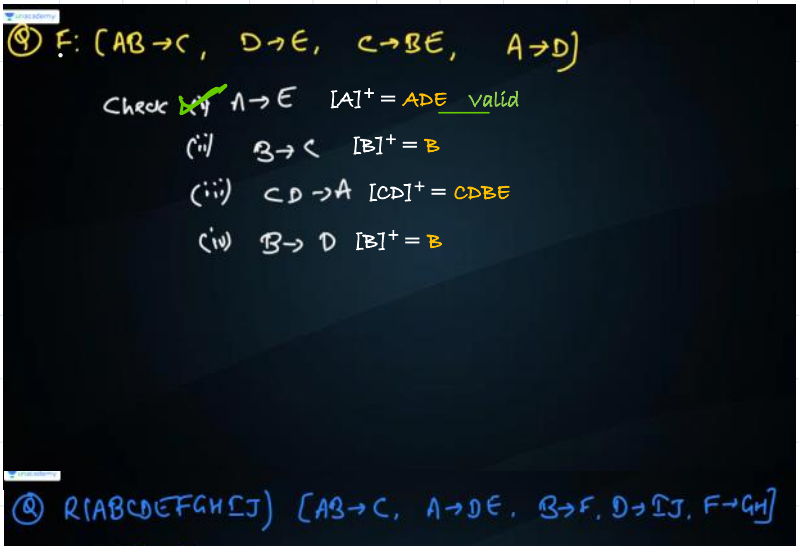
X → y is a member of FD set F if and only if X → y logically implied in F
X → y logically implied means from the closure of X determines Y

$[X]^+ = [-----Y]$
Member, Inferred, Valid FD or Logically Implied.
Example :

F : [A → B, B → C]

Check A → C

$[A]^+$ = ABC (logically implied)



Q R(ABCDEFGHIJ) {AB→C, A→DE, B→F, D→IJ, F→GH}

CHECK ?

(i) A → J $[A]^+$ = ADEIJ

(ii) B → H $[B]^+$ = BFGH

(iii) C → G $[C]^+$ = C

(iv) B → I $[B]^+$ = BFGH

(v) BD → G $[BD]^+$ = BFDIJGH

(vi) AF → H $[AF]^+$ = AFGH

Q.

In a schema with attributes A, B, C, D and E following set of functional dependencies are given

A → B

A → C

CD → E

B → D

E → A

Which of the following functional dependencies is NOT implied by the above set

[MCQ: GATE - 2M]

A. CD → AC $[CD]^+$ = CDEAC

B. BD → CD $[BD]^+$ = BD

C. BC → CD $[BC]^+$ = BCDEA

D. AC → BC $[AC]^+$ = ABCDE

Q.

Suppose the following functional dependencies hold on a relation U with attributes P, Q, R, S and T:

P → QR

RS → T

Which of the following functional dependencies can be inferred/implied from the above functional dependencies?

[MSQ: 2021 - 2M]

A. PS → T $[PS]^+$ = PQRST

B. R → T $[R]^+$ = R

C. P → R $[P]^+$ = PQR

D. PS → Q $[PS]^+$ = PSQRT

GATE 2024 IISc Bengaluru

Computer Science and Information Technology Set 1 (CS1)

Q.44 The symbol → indicates functional dependency in the context of a relational database. Which of the following options is/are TRUE?

(A) $(X,Y) \rightarrow (Z,W)$ implies $X \rightarrow (Z,W)$ $[XY]^+$ = ZW and $[X]^+$ = X, not getting ZW from X

(B) $(X,Y) \rightarrow (Z,W)$ implies $(X,Y) \rightarrow Z$ $[XY]^+$ = ZW and $[XY]^+$ = XYZW, we are getting Z from XY

(C) $((X,Y) \rightarrow Z$ and $W \rightarrow Y)$ implies $(X,W) \rightarrow Z$ $[XY]^+$ = Z and $[W]^+$ = Y so $[XW]^+$ = XWYZ, we are getting Z from X,W

(D) $(X \rightarrow Y$ and $Y \rightarrow Z)$ implies $X \rightarrow Z$ $[X]^+$ = Y and $[Y]^+$ = Z, so $[X]^+$ = XYZ, we are getting Z from X

GATE 2024 IISc Bengaluru

Data Science and Artificial Intelligence (DA)

Q.46 Given the relational schema $R = (U,V,W,X,Y,Z)$ and the set of functional

we are getting Z from X

Q.46	Given the relational schema $R = (U, V, W, X, Y, Z)$ and the set of functional dependencies: $\{U \rightarrow V, U \rightarrow W, WX \rightarrow Y, WX \rightarrow Z, V \rightarrow X\}$ Which of the following functional dependencies can be derived from the above set?
(A) ✓	$VW \rightarrow YZ$ $[VW]^+ = VXWYZ$
(B) ✓	$WX \rightarrow YZ$ $[WX]^+ = WXYZ$
(C)	$VW \rightarrow U$ $[VW]^+ = VWXYZ$
(D) ✓	$VW \rightarrow Y$ $[VW]^+ = VWXYZ$

topic - equality between FD's set

Let FD set F & G are given
F : [----------] and G : [----------]
F and G equals if and only if $[F]^+ = [G]^+$

F Equivalent to G when -
F cover G : TRUE
G cover F : TRUE

(i) F cover G : (F) covers all the FD's of (G) FD set
{OR}
All (G) FD's should be logically implied in (F) FD set

(ii) G cover F : (G) cover all the FD's of (F) FD set
{OR}
All (F) FD's should be logically implied in (G) FD set.

(i) F cover G : TRUE FALSE TRUE FALSE

(ii) G cover F : FALSE TRUE TRUE FALSE
F≥G G≥F F≡G incomparable

Q. F : [AB→CD, B →C, C →D]
G : [AB→C, AB →D, C →D]

F cover G

(i) F cover G :
write down all the

G cover F

(ii) G cover F :
Write down all the FD's of (F)

(i) F cover G :
write down all the FD's of (G) and then take the closure of (G) in (F).

If (F) is covering all the FD's of (G) then it is true.

F cover G

$AB \rightarrow C$	$[AB]^+ = ABCD$
$AB \rightarrow D$	$[AB]^+ = ABCD$
$C \rightarrow D$	$[C]^+ = CD$

G cover F

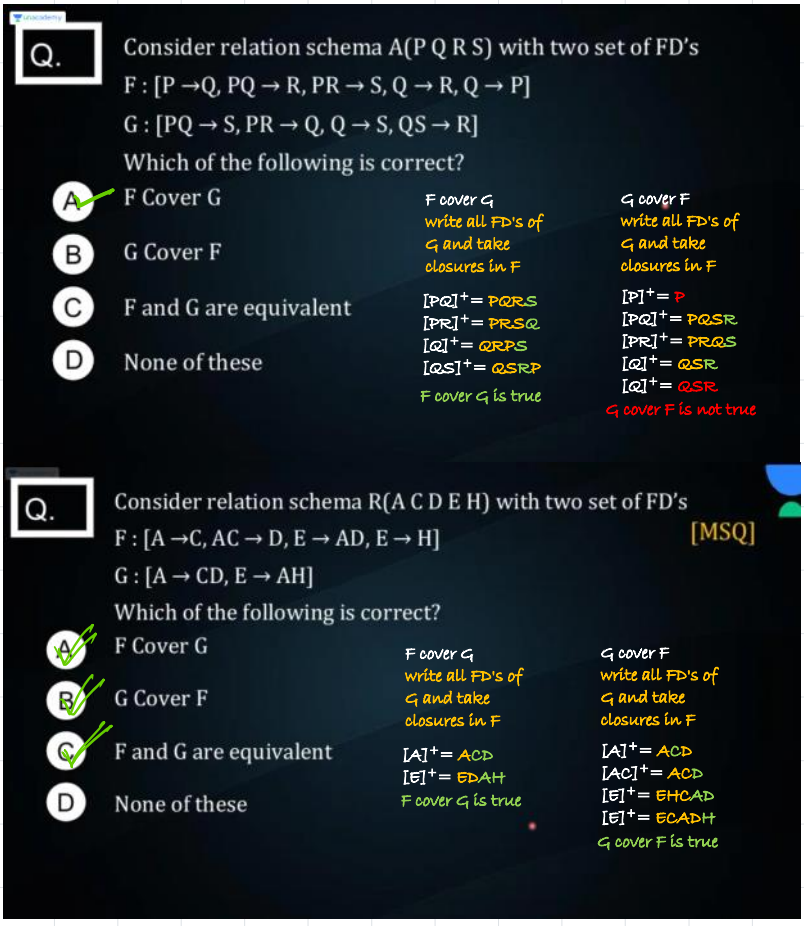
$AB \rightarrow CD$	$[AB]^+ = ABCD$
$B \rightarrow C$	$[B]^+ = B$
$C \rightarrow D$	$[C]^+ = CD$

(ii) G cover F :
Write down all the FD's of (F) and then take the closure of (F) in (G).

If (G) is covering all the FD's of (F) then it is true.

F cover G is true but G cover F is not true

$F \geq G$



topic : minimal/Canonical cover or irreducible set.

Sets of functional dependencies may have redundant dependencies that can be inferred from the others.
For example :
$A \rightarrow C$ is redundant in : $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$

Redundant FD : Extra FD. If we delete that FD from the FD set F then deletion of that FD does not affect the power of FD set

$\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$

Assume,

A→C is redundant in :{A →B, B →C, A →C}

G :

{A→B, B→C}

$[A]^+ = [ABC]$

So,

{A→C} is a Redundant FD.

How to check any particular FD is an extra FD?

Method 1 :

F : [----------] given original FD

After deletion of any assumed Redundant FD we get new FD set G

G : [----------] after deletion of Redundant FD

Check-

If and only if F is equivalent to G

F cover G : TRUE

G cover F : TRUE

Then we can confirm X →Y is Redundant FD.

How to find Redundant FD's from FD set F?

Method 2 :

Process : Assume you want to check X →Y is Redundant FD in FD set F, then Hide X Y FD and then take the closure of X in all other remaining FD, if we are getting $[x]^+$ =[----y] then X Y is Redundant FD.

1. A→B hide this and check in FD (2) and (3)
2. B→C hide this and check in FD (1) and (3)
3. A →C hide this and check in FD (1) and (2)

$[A]^+ = [AC]$ $[B]^+ = [B]$ $[A]^+ = [ABC]$

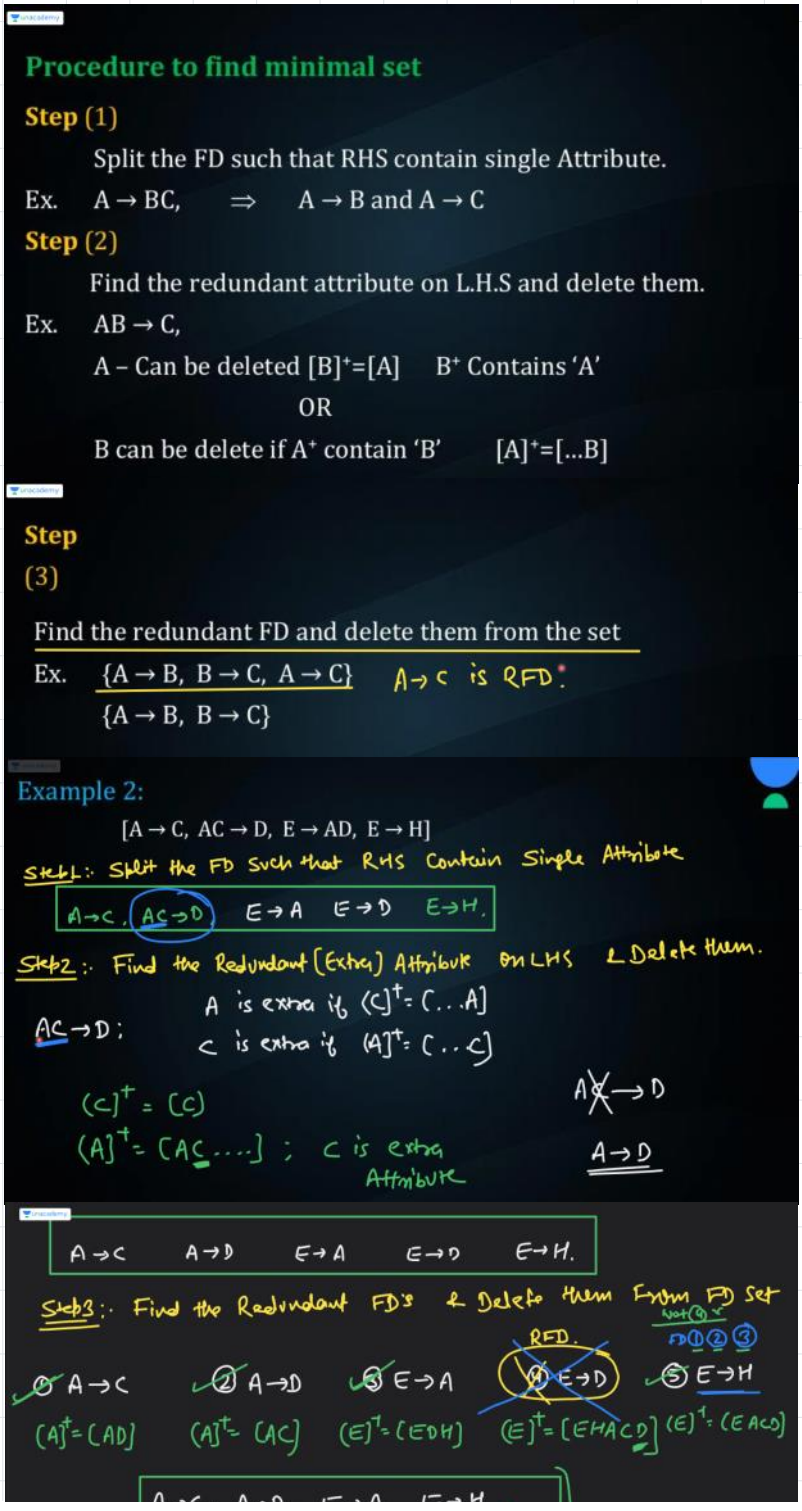
Hence, Redundant FD is A →C.

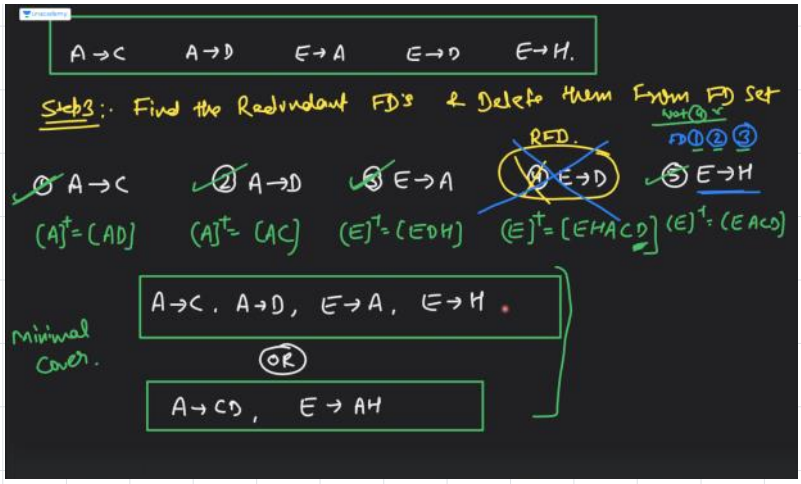
Procedure to find Minimal cover.

Step 1 : Split the FD such that RHS contain single attribute

EX: X →YZ = X →Y, X →Z.

Step 2 : Find redundant attribute on LHS and delete them.
EX: AB → C
A is extra if $[B]^+$ contains A.
Means A is extra when B contains A, basically A is coming from B.
B is extra if $[A]^+$ contains B.
Means B is extra when A contains B, basically B is coming from A.
Step 3 : Find redundant FD and delete them from FD set.
EX : A → B, B → C, A → C: A → C is redundant.
Minimal Cover : [A → B, B → C]





How to check given minimal cover is correct or not?

F : [----------] given original FD

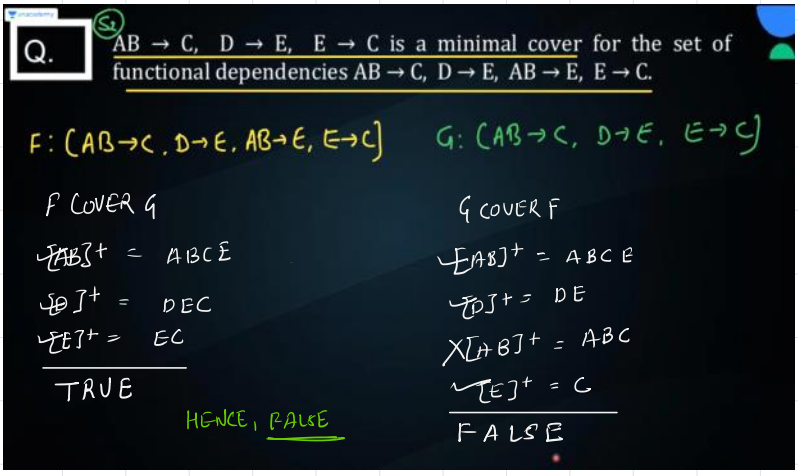
G : [----------] Minimal cover

If and only if F is equivalent to G

F cover G : TRUE

G cover F : TRUE

Then given minimal cover is correct.



note :
Minimal cover may or may not be unique
Sometimes we can have multiple minimal cover for a particular question.

Example 1:

$[AB \to CD,\ A \to E,\ E \to C]$

S① $AB \to CD$, $A \to E$, $E \to C$ | S② $AB \to C$

$AB \to C$

$AB \to D$

RFD

$[A]^+ = AEC$ not Extra.

$[B]^+ = B$

S ③ $AB \to C$ $AB \to D$ $A \to E$ $E \to C$

$[AB]^+ = ABDEC$ $[A]^+ = A$

$[AB]^+ = ABEC$ $[E]^+ = E$

Example 3:

$[B \to A,\ D \to A,\ AB \to D]$

① $B \to A$, $D \to A$, $AB \to D$ ($B \to D$) — $[A]^+ = A$, $[B]^+ = BAD$

② R.F.D $B \to A$, $D \to A$, $B \to D$

③ $[B]^+ = BDA$

$[D]^+ = D$

$[B]^+ = B$

Example 4:

$[A \to BC,\ CD \to E,\ E \to C,\ D \to AEH,\ ABH \to BD,\ DH \to BC]$

① $A \to B$, $A \to C$, $E \to C$, $D \to A$, $D \to E$, $D \to H$, $DH \to B$, $DH \to C$, $ABH \to B$, $ABH \to D$, $CD \to E$, $D \to E$

$DH \to B$ $DH \to C$ $ABH \to B$ $ABH \to D$

② $DH \to B$ — $[D]^+ = DAEH$

$DH \to C$ — $[D]^+ = DAEH$

$ABH \to B$ — $[A]^+ = ABC$

$ABH \to D$ — $[A]^+ = ABC$

$CD \to E$ — $[C]^+ = C$, $[D]^+ = DEACB$

③ $A \to B$, $A \to C$, $E \to C$, $D \to A$, $D \to E$, $AB \to H$, $D \to B$, $D \to C$, $AH \to B$, $AH \to D$, $D \to E$ (R.F.D)

$[A]^+$ AC — $[A]^+$ AB — $[E]^+$ E — $[D]^+$ DEHBC — $[D]^+$ DHBCE — $[D]^+$ DABCE — $[D]^+$ DAEHCB — $[D]^+$ DAHBC — $[AH]^+$ AHB — $[AH]^+$ AHBC — $[D]^+$ DHACB

RFD - $D \to E$, $D \to B$, $D \to C$, $AH \to B$. | $A \to BC$, $AH \to D$, $D \to AEH$, $E \to C$

note :
Don't check/consider previously deleted FD.

Q. Consider the following FD Set:
$\{P \to QR, Q \to PR, R \to PQ\}$ which of the following is/are the minimal cover for the above FD set?

(A) $P \to Q, Q \to R, R \to P$

(B) $P \to R, Q \to R, R \to PQ$

(C) $Q \to P, P \to R, R \to Q$

(D) $P \to QR, Q \to P, R \to P$

RFD RFD

$P \to Q$, $P \to R$, $Q \to P$, $Q \to R$, $R \to P$, $R \to Q$

$[P]^+$ PRQ — $[P]^+$ P — $[Q]^+$ QRP — $[Q]^+$ Q — $[RJ^+$ RQ — $[R]^+$ RP

$P \to R$, $Q \to R$, $R \to PQ$

$P \to R$, $Q \to R$, $R \to P$, $R \to Q$

$P \to R, Q \to R, R \to P, R \to Q$

$P \to QR, \quad Q \to PR, \quad R \to PQ.$

① $P \to Q$ ② $P \to R$ ③ $Q \to P$ ④ $Q \to R$ ⑤ $R \to P$ ⑥ $R \to Q$

↳ option (b)

② $P \to Q$ ① $P \to R$ ③ $Q \to P$ ④ $Q \to R$ ⑤ $R \to P$ ⑥ $R \to Q$

↳ option (a)

③ $P \to Q$ ④ $P \to R$ ⑤ $Q \to P$ ⑥ $Q \to R$ ① $R \to P$ ② $R \to Q$

↳ option (c)

⑤ $P \to Q$ ⑥ $P \to R$ ② $Q \to P$ ① $Q \to R$ ④ $R \to P$ ③ $R \to Q$

↳ option (d)

Q. $[XYZ \to W, \quad X \to Y, \quad Y \to Z]$ DIRECT.

$X \to W, \quad X \to Y, \quad Y \to Z$

$[X]^+ = XWYZ$

minimal - $X \to W, X \to Y, Y \to Z$

Q. Given the following two statements:

S1: Every table with two single-valued attributes is in 1NF, 2NF, 3NF and BCNF.

S2: $AB \to C, D \to E, E \to C$ is a minimal cover for the set of functional dependencies $AB \to C, D \to E, AB \to E, E \to C$.

Which one of the following is CORRECT? [MCQ: 2014: 2M

R.F.D $AB \to C, D \to E, AB \to E, E \to C$

① $AB \to C$, $D \to E$, $AB \to E$, $E \to C$

$[AB]^+ = ABEC$, $[D]^+ = D$, $[AB]^+ = AB$, $[E]^+ = E$

A S1 is TRUE and S2 is FALSE.

B Both S1 and S2 are TRUE.

C S1 is FALSE and S2 is TRUE.

D Both S1 and S2 are FALSE.

Q. The following functional dependencies hold true for the relational schema R{V, W, X, Y, Z}:

$V \to W$
$VW \to X$
$Y \to VX$
$Y \to Z$

Which of the following is irreducible equivalent for this set of functional dependencies? [MCQ:2017]

A $V \to W$, $V \to X$, $Y \to V$

B $V \to W$, $W \to X$, $Y \to V$

C $V \to W$, $V \to X$, $Y \to V$

D $V \to W$, $W \to X$, $Y \to V$

Q. The following functional dependencies hold true for the relational schema R{V, W, X, Y, Z}:

$V \to W$
$VW \to X$
$Y \to VX$
$Y \to Z$

Which of the following is irreducible equivalent for this set of functional dependencies? [MCQ:2017]

A) $V \to W$, $V \to X$, $Y \to V$, $Y \to Z$

B) $V \to W$, $W \to X$, $Y \to V$, $Y \to Z$

C) $V \to W$, $V \to X$, $Y \to V$, $Y \to X$, $Y \to Z$

D) $V \to W$, $W \to X$, $Y \to V$, $Y \to X$, $Y \to Z$

$V \to W, VW \to X, Y \to VX, Y \to Z$

(i) $V \to W$, $VW \to X$, $Y \to V$, $Y \to X$, $Y \to Z$

$[V]^+ = VW$, $[W]^+ = W$, $V \to X$

(3) $V \to W$, $V \to X$, $Y \to V$, $Y \to X$, $Y \to Z$ (R.F.D)

$[V]^+ = VX$, $[V]^+ = VW$, $[Y]^+ = YXZ$, $[Y]^+ = YVXWZ$, $[Y]^+ = YX$

Final set - $V \to W$, $V \to X, Y \to X$, $Y \to Z$

$F: [P \to QR, Q \to PR, R \to PQ]$

F cover G

G		F
$P \to Q$	$[P]^+ = PQR$	$P \to QR$ $[P]^+ = PQR$
$Q \to R$	$[Q]^+ = QPR$	$Q \to PR$ $[Q]^+ = QRP$
$R \to P$	$[R]^+ = PQ$	$R \to PQ$ $[R]^+ = RPQ$

G cover F

(1) $G: [P \to Q, Q \to R, R \to P]$

G		F
$P \to R$	$[P]^+ = PQR$	$P \to QR$ $[P]^+ = PRQ$
$Q \to R$	$[Q]^+ = QPR$	$Q \to PR$ $[Q]^+ = QRP$
$R \to PQ$	$[R]^+ = PQR$	$R \to PQ$ $[R]^+ = RPQ$

(2) $G: [P \to R, Q \to R, R \to PQ]$

G		F
$P \to R$	$[P]^+ = PQR$	$P \to QR$ $[P]^+ = PRQ$
$Q \to R$	$[Q]^+ = QPR$	$Q \to PR$ $[Q]^+ = QPR$
$R \to PQ$	$[R]^+ = RPQ$	$R \to PQ$ $[R]^+ = RQP$

(3) $G: [Q \to P, P \to R, R \to Q]$

G		F
$Q \to P$	$[Q]^+ = QPR$	$P \to QR$ $[P]^+ = PRQ$
$P \to R$	$[P]^+ = PQR$	$Q \to PR$ $[Q]^+ = QPR$
$R \to Q$	$[R]^+ = RPQ$	$R \to PQ$ $[R]^+ = RQP$

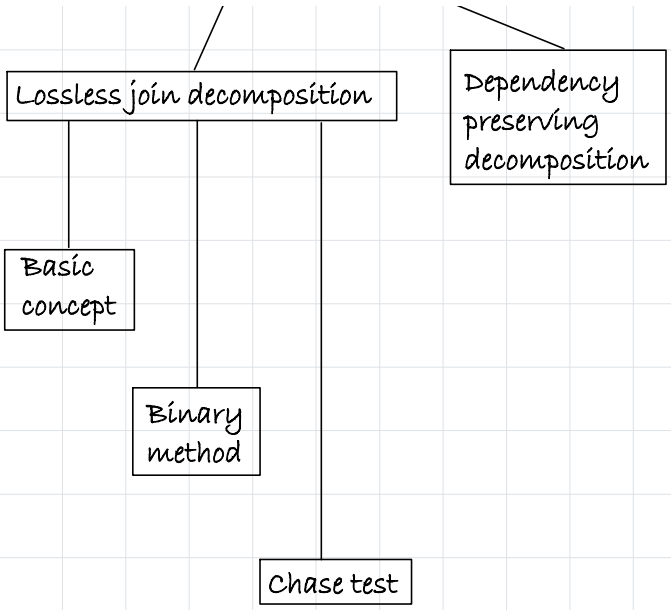
(4) $G: [P \to QR, Q \to P, R \to P]$

G		F
$P \to QR$	$[P]^+ = PQR$	$P \to QR$ $[P]^+ = PQR$
$Q \to P$	$[Q]^+ = QPR$	$Q \to PR$ $[Q]^+ = QPR$
$R \to P$	$[R]^+ = RPQ$	$R \to PQ$ $[R]^+ = RPQ$

note :
- If MSQ then check F Equivalent G
- minimal cover may or may not be unique

topic : properties of decomposition

- Lossless join decomposition
- Dependency



(i) Basic concept - [Not for GATE]

Let R be the relation schema with instance R, is decomposed into sub relation R_1, R_2, R_3 ... R_n with instance r_1, r_2, r_3 ... r_n respectively.

If $R_1 \bowtie R_2 \bowtie R_3 \ldots \bowtie R_n = R$
Lossless join decomposition.
but
If $R_1 \bowtie R_2 \bowtie R_3 \ldots \bowtie R_n > R$
Lossy join decomposition.

- Natural Join $\bowtie$
(derived)

Step 1 : Cross product [cartesian product] of R & S.

R	S	R×S = N_1 X N_2 Tuple (tuple multiplies)
N_1 Tuple	N_2 Tuple	$C_1 + C_2$ Attribute (column adds)
C_1 Attribute	C_2 Attribute	

Step 2 : Select the tuples which satisfied equality condition on all common attributes of R & S (from R & S).

Step 3 : Projection of distinct attributes.

R(ABC)

A	B	C
1	5	5
2	5	8
3	8	8

Step 1 : Cross product [cartesian product] of R & S.

$R_1(AB)$ $R_2(BC)$

A	B
1	5
2	5
3	8

B	C
5	5
5	8
8	8

R_1A	R_1B	R_2B	R_2C
1	5	5	5
1	5	5	8
1	5	8	8
2	5	5	5
2	5	5	8
2	5	8	8
3	8	5	5
3	8	5	8
3	8	8	8

Step 2 : Select the tuples which satisfied equality condition on all common attributes of R & S (from R & S). ($R_1.B = R_2.B$)

R_1A	R_1B	R_2B	R_2C
1	5	5	5
1	5	5	8
1	5	8	8
2	5	5	5
2	5	5	8
2	5	8	8
3	8	5	5
3	8	5	8
3	8	8	8

R_1A	R_1B	R_2B	R_2C
1	5	5	5
1	5	5	8
2	5	5	5
2	5	5	8
3	8	8	8

Step 3 : Projection of distinct attributes.

$R_1 \bowtie R_2$

A	B	C
1	5	5
1	5	8
2	5	5
2	5	8
3	8	8

Spurious tuples - extra tuples.

$R_1 \bowtie R_2 > R$

Lossy join, because of extra tuples

R(ABC)

A	B	C
1	5	5
2	5	8
3	8	8

Step 1 : Cross product [cartesian product] of R & S.

$R_1(AB)$ $R_2(AC)$

A	B
1	5
2	5
3	8

B	C
1	5
2	8
3	8

R_1A	R_1B	R_2A	R_2C
1	5	1	5
1	5	2	8
1	5	3	8
2	5	1	5
2	5	2	8
2	5	3	8
3	8	1	5
3	8	2	8
3	8	3	8

Step 2 : Select the tuples which satisfied equality condition on all common attributes of R & S (from R & S). ($R_1.A = R_2.A$)

R_1A	R_1B	R_2A	R_2C
1	5	1	5
1	5	2	8
1	5	3	8
2	5	1	5
2	5	2	8
2	5	3	8
3	8	1	5
3	8	2	8
3	8	3	8

R_1A	R_1B	R_2A	R_2C
1	5	1	5
2	5	2	8
3	8	3	8

Step 3 : Projection of distinct attributes.

$R_1 \bowtie R_2$

A	B	C
1	5	5
2	5	8
3	8	8

$R_1 \bowtie R_2 = R$

Lossless join, because of exact tuples

(ii) Binary method -

Let R be the relation schema with FD set F is decomposed into 2 sub relation R_1 and R_2 then $R_1 \bowtie R_2$ Lossless if and only

If $R_1 \cup R_2 \equiv R$
If common attribute of R_1 and R_2 either super-key of R_1

Let R be the relation schema with FD set F is decomposed into 2 sub relation R_1 and R_2 then $R_1 \bowtie R_2$ is Lossy if and only

If $R_1 \cup R_2 \equiv R$
If common attribute R_1 and R_2 is neither super-key of R_1

If $R_1 \cup R_2 \equiv R$
If common attribute of R_1 and R_2 either
super-key of R_1
(OR)
super-key of R_2

If $[R_1 \cap R_2]^+ \rightarrow R_1$

(OR)

$[R_1 \cap R_2]^+ \rightarrow R_2$

If $R_1 \cup R_2 \equiv R$
If common attribute R_1 and R_2 is neither
super-key of R_1
(OR)
super-key of R_2

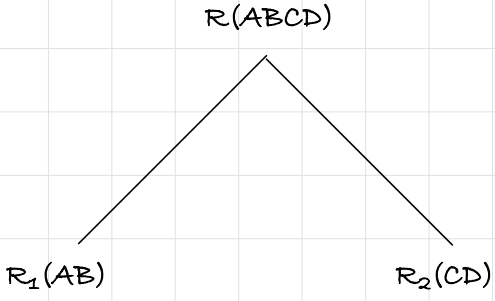
If $[R_1 \cap R_2]^+ \not\rightarrow R_1$

(OR)

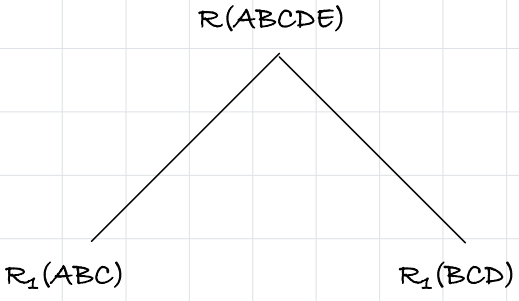
$[R_1 \cap R_2]^+ \not\rightarrow R_2$

Que 1 : R(ABC) [A → C]
$R_1(AB)$ & $R_2(AC)$

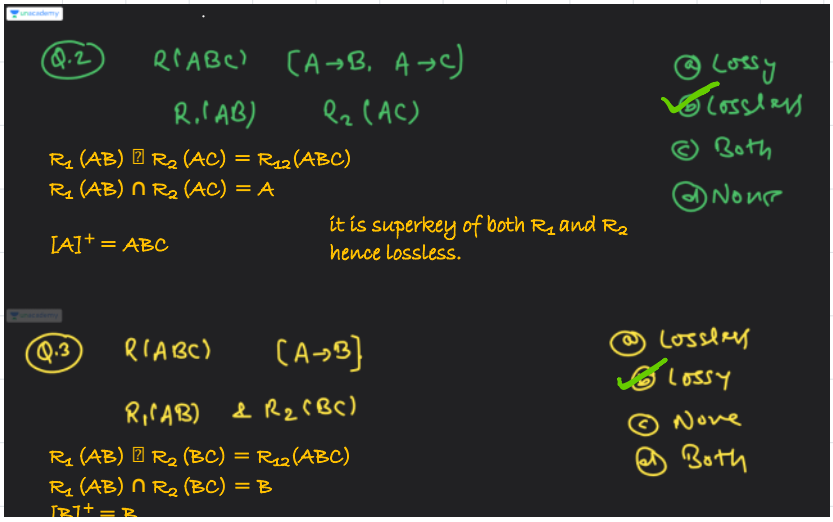
$R_1(AB) \cup R_2(AC) = R(ABC)$
$R_1(AB) \cap R_2(AC) = A$
$[A]^+ = [AC]$
Super-key of R_2
Hence, lossless join.



Lossy because no common Attribute



Lossy because Attribute (E) is missing



$R_1(AB) \bowtie R_2(BC) = R_{12}(ABC)$
$R_1(AB) \cap R_2(BC) = B$
$[B]^+ = B$
it is not a superkey of either R_1 or R_2
hence lossy

(a) Both

Q.4 $R(ABC)$ $[A \to B,\ A \to C]$
$R_1(AB)$ $R_2(BC)$

(a) Lossy
(b) Lossless
(c) Both
(d) None

$R_1(AB) \bowtie R_2(BC) = R_{12}(ABC)$
$R_1(AB) \cap R_2(BC) = B$
$[B]^+ = B$
it is not a superkey of either R_1 or R_2
hence lossy

Q.1 $R(ABCDEFG)\ \{AB \to CD,\ D \to E,\ E \to FG\}$
Decomposed into $R_1(ABCD)$ and $R_2(DEFG)$

(a) Lossless
(b) Lossy

$R_1(ABCD) \bowtie R_2(DEFG) = R_{12}(ABCDEFG)$
$R_1(ABCD) \cap R_2(DEFG) = D$
$[D]^+ = DEFG$
it is a superkey of either R_2
hence lossless

Q.2. $R(ABCDEFG)\ \{AB \to C,\ C \to D,\ D \to EFG\}$
Decomposed into $R_1(ABCE)$ and $R_2(DEFG)$

(a) Lossy
(b) Lossless

$R_1(ABCE) \bowtie R_2(DEFG) = R_{12}(ABCDEFG)$
$R_1(ABCE) \cap R_2(DEFG) = E$
$[E]^+ = E$
it is not a superkey of either R_1 or R_2
hence lossy

Q3 $R(ABCDEFG)\ \{AB \to C,\ AC \to B,\ AD \to E,\ B \to D,\ BC \to A,\ E \to G\}$
Decomposed into $R_1(ABC)$ $R_2(ACDE)$ and $R_3(ADG)$

$R_1(ABC) \bowtie R_2(ACDE) = R_{12}(ABCDE)$
$R_1(ABC) \cap R_2(ACDE) = AC$
$[AC]^+ = ACBD$
it is a superkey of R_1

$R_{12}(ABCDE) \bowtie R_3(ADG) = R_{123}(ABCDEG)$
$R_{12}(ABCDE) \cap R_2(ADG) = AD$
$[AD]^+ = ADEG$
it is a superkey of R_2
hence lossless

note : R_1 and R_2 and R_3 are calculated in pairs either as
-R_1 and R_2
OR as
-R_1 And R_3
OR as
-R_2 And R_3

Q. $R(ABCDEFG)\ \{AB \to C,\ AC \to B,\ AD \to E,\ B \to D,\ BC \to A,\ E \to CG\}$
1. Decomposed into $R_1(AB)$ $R_2(BC)$ $R_3(ABDE)$ and $R_4(EG)$
2. Decomposed into $R_1(AB)$ $R_2(BC)$ $R_3(ABDE)$ and $R_4(ECG)$

$R_1(AB) \bowtie R_1(BC) = R_{12}(ABC)$
$R_1(AB) \cap R_2(BC) = B$
$[B]^+ = BD$
it is not a superkey of either R_1 or R_2

$R_{12}(ABC) \bowtie R_3(ABDE) = R_{123}(ABCDE)$
$R_{12}(ABC) \cap R_3(ABDE) = AB$
$[AB]^+ = ABCDEG$
it is a superkey of both R_{12} and R_3

$R_{123}(ABCDE) \bowtie R_4(EG) = R_{1234}(ABCDEG)$
$R_{123}(ABCDE) \cap R_4(EG) = E$
$[E]^+ = ECG$
it is a superkey of either R_2

$R_1(AB) \bowtie R_1(BC) = R_{12}(ABC)$
$R_1(AB) \cap R_2(BC) = B$
$[B]^+ = BD$
it is not a superkey of either R_1 or R_2

$R_{12}(ABC) \bowtie R_3(ABDE) = R_{123}(ABCDE)$
$R_{12}(ABC) \cap R_3(ABDE) = AB$
$[AB]^+ = ABCDEG$
it is a superkey of both R_{12} and R_3

$R_{123}(ABCDE) \bowtie R_4(ECG) = R_{1234}(ABCDEG)$
$R_{123}(ABCDE) \cap R_4(ECG) = EC$
$[EC]^+ = ECG$
it is a superkey of either R_2

note : R_1 and R_2 and cannot join because it is not able to produce any super-key.
But don't declare lossy/lossless on the behalf of single result.

$R_{123}(ABCDE) \bowtie R_4(EG) = R_{1234}(ABCDEG)$
$R_{123}(ABCDE) \cap R_4(EG) = E$
$[E]^+ = ECG$
it is a superkey of either R_2

$R_{123}(ABCDE) \bowtie R_4(ECG) = R_{1234}(ABCDEG)$
$R_{123}(ABCDE) \cap R_4(ECG) = EC$
$[EC]^+ = ECG$
it is a superkey of either R_2

both are lossless

note : Combine all for the final result and then declare the answer.

Q. Let R(A, B, C, D) be a relational schema with the following function dependencies:

$A \to B$, $B \to C$, $C \to D$ and $D \to B$.

The decomposition of R into (A, B), (B, C), (B, D) [MCQ: 2M]

$R_1(AB) \bowtie R_1(BC) = R_{12}(ABC)$
$R_1(AB) \cap R_2(BC) = B$
$[B]^+ = BCD$
it is a superkey of R_2

$R_{12}(ABC) \bowtie R_3(BD) = R_{123}(ABCD)$
$R_{12}(ABC) \cap R_3(BD) = B$
$[B]^+ = BCD$
it is a superkey of R_3

hence it is lossless

Q. Consider the relation R (P, Q, S, T, X, Y, Z, W) with the following functional dependencies.

$PQ \to X$; $P \to YX$; $Q \to Y$; $Y \to ZW$

D_1: R = [(P, Q, S, T); (P, T, X); (Q, Y); (Y, Z, W)]
D_2: R = [(P, Q, S); (T, X); (Q, Y); (Y, Z, W)]

D1
① $R_1(PQST) \cap R_2(PTX) = PT$
$[PT]^+ = PTYXZW$ — Superkey of R_2
② $R_{12}(PQSTX) \cap R_3(QY) = Q$
$[Q]^+ = QYZW$ - Superkey of R_3
③ $R_{123}(PQSTXY) \cap R_4(YZW)$
$[Y]^+ = YZW$ — Superkey of R_4
$R_{1234}(PQSTXYZW)$ — LOSSLESS

D2
① $R_1(PQS) \cap R_3(QY) = Q$
$[Q]^+ = QY$ — Superkey of R_3
② $R_{13}(PQSY) \cap R_4(YZW) = Y$
$[Y]^+ = YZW$ - Superkey of R_4
③ $R_{134}(PQSYZW) \cap R_2(TX)$
NO common attribute — LOSSY

3. Chase test -

Step 1 : In the chase test we create a matrix in which row/tuple represent the sub relation and column represent the respective attribute.

Step 2 : Now fill the cell/entries with 'a' (any value) in

corresponding attribute of respective sub relation.

Step 3 : Now fill the table entries with the help of given FD's

For X→Y : If 2 values of X is present and 1 value of Y is present then write same Y value. [a]

If $t_1.x=t_2.x$ then $t_1.y=t_2.y$ must be same.

For X→Y : If 2 values of X is present

X	Y
a	a
a	a

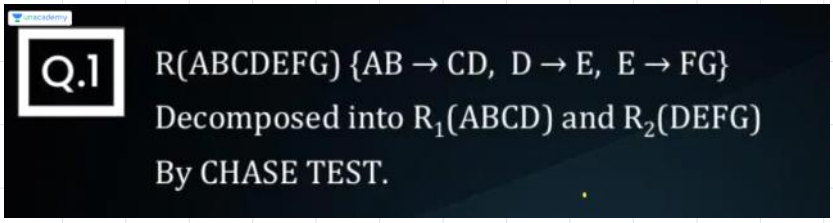
If one Y value 'a' is present then write same Y value 'a'

X	Y
a	b
a	b

If zero Y value is present then write same Y value 'b' instead of 'a'

note :
If we get a tuple (any one tuple) with all 'a' entries then lossless join decomposition

Chase test will stop either we get any one tuple with 'a' entries (lossless)
(OR)
There is no further updation in table



Step 1 : create a matrix in which row/tuple represent the sub relation and column represent the respective attribute.

	A	B	C	D	E	F	G
R_1 (ABCD)							
R_2 (DEFG)							

Step 2 : Now fill the cell/entries with 'a' (any value) in corresponding attribute of respective sub relation.

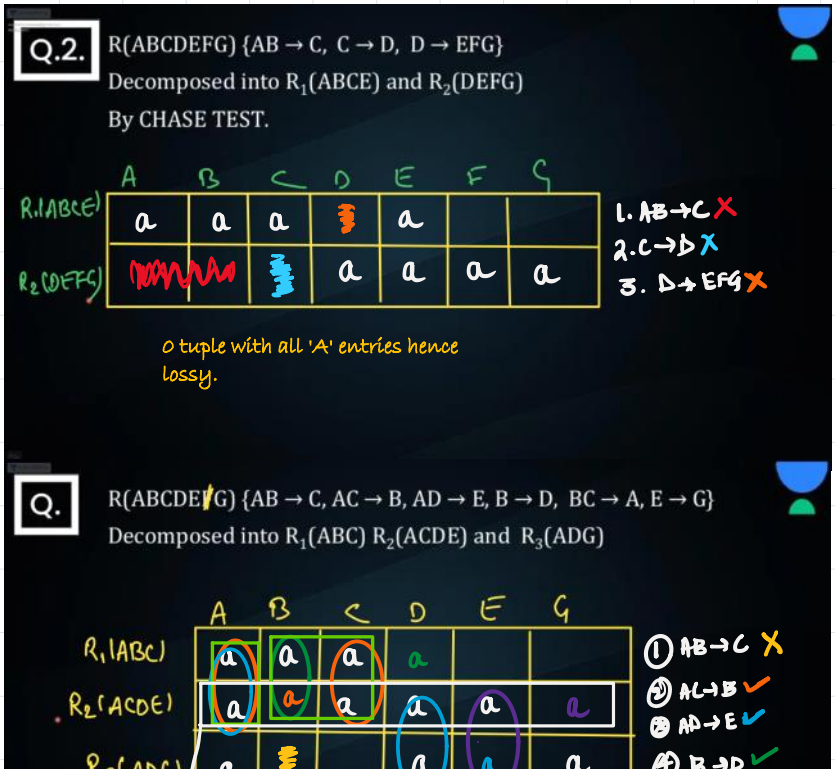
	A	B	C	D	E	F	G
R_1 (ABCD)	a	a	a	a			
R_2 (DEFG)				a	a	a	a

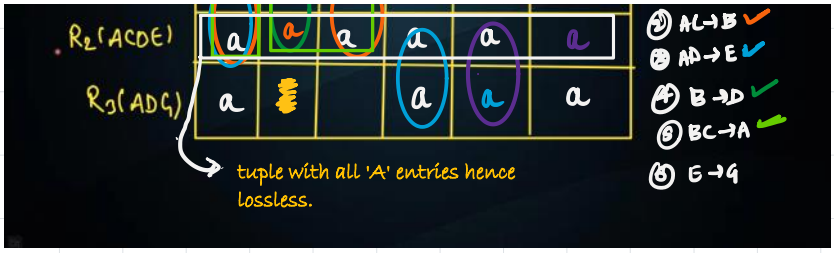
Step 3 : Now fill the table entries with the help of given FD's
For X→Y : If 2 values of X is present and 1 value of Y is present then write same Y value. [a]

	A	B	C	D	E	F	G
R_1 (ABCD)	a	a	a	a	a	a	a
R_2 (DEFG)				a	a	a	a

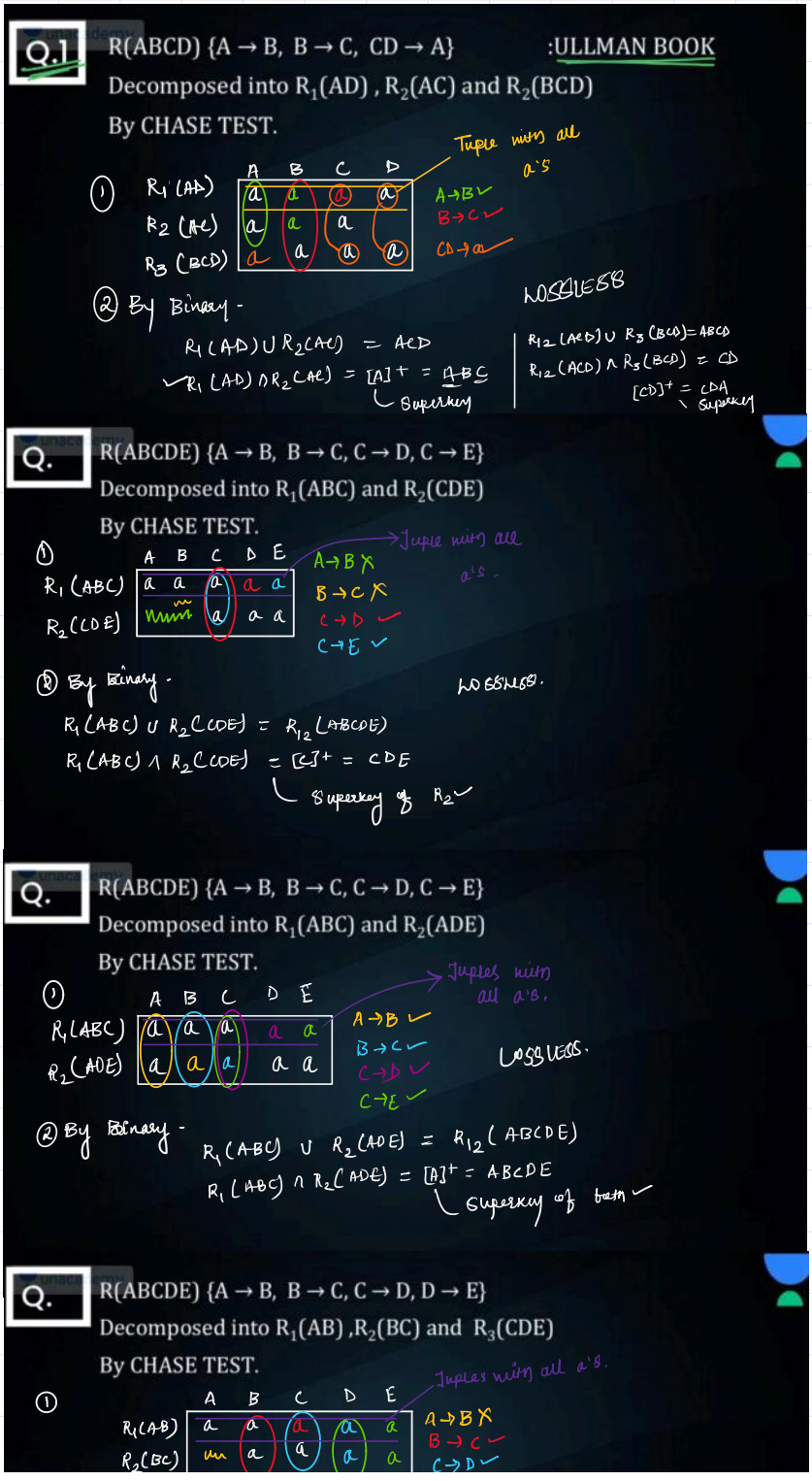
Getting a tuple with all 'a' entries (lossless)

✗ AB →CD (values missing)
✓ D→E
✓ E→FG




R2(ACDE)
R3(ADG)
tuple with all 'A' entries hence lossless.
1) AC→B
2) AD→E
3) B→D
4) BC→A
5) G→G

Practice -


Q.1
R(ABCD) {A → B, B → C, CD → A}
:ULLMAN BOOK
Decomposed into R1(AD), R2(AC) and R2(BCD)
By CHASE TEST.
Tuple with all a's
R1 (AD)
R2 (AC)
R3 (BCD)
A→B
B→C
CD→A
By Binary -
R1(AD) ∪ R2(AC) = ACD
R1(AD) ∩ R2(AC) = [A]+ = ABC
Superkey
LOSSLESS
R12(ACD) ∪ R3(BCD) = ABCD
R12(ACD) ∩ R3(BCD) = CD
[CD]+ = CDA
Superkey
Q.
R(ABCDE) {A → B, B → C, C → D, C → E}
Decomposed into R1(ABC) and R2(CDE)
By CHASE TEST.
Tuple with all a's
A→B
B→C
C→D
C→E
By Binary -
LOSSLESS.
R1(ABC) ∪ R2(CDE) = R12(ABCDE)
R1(ABC) ∩ R2(CDE) = [C]+ = CDE
Superkey of R2
Q.
R(ABCDE) {A → B, B → C, C → D, C → E}
Decomposed into R1(ABC) and R2(ADE)
By CHASE TEST.
Tuples with all a's.
A→B
B→C
C→D
C→E
LOSSLESS.
By Binary -
R1(ABC) ∪ R2(ADE) = R12(ABCDE)
R1(ABC) ∩ R2(ADE) = [A]+ = ABCDE
Superkey of both
Q.
R(ABCDE) {A → B, B → C, C → D, D → E}
Decomposed into R1(AB), R2(BC) and R3(CDE)
By CHASE TEST.
Tuples with all a's.
A→B
B→C
C→D

①

	A	B	C	D	E
$R_1(AB)$	a	a	a	a	a
$R_2(BC)$	w	a	a	a	a
$R_3(CDE)$			a	a	a

$A \to B$ ✗
$B \to C$ ✓
$C \to D$ ✓
$D \to E$ ✓

②

By Binary -

$R_1(AB) \cup R_2(BC) = R_{12}(ABC)$

$R_1(AB) \cap R_2(BC) = [B]^+ = BCDE$

↳ S.K of R_2

LOSSLESS.

$R_{12}(ABC) \cup R_3(CDE) = R_{123}(ABCDE)$

$R_{12}(ABC) \cap R_3(CDE) = [C]^+ = CDE$

S.K of R_3

Q.2 R(ABCDEFG) {AB → CD, D → E, E → FG}

Decomposed into R_1(ABCD) and R_2(DEFG)

By CHASE TEST.

①

	A	B	C	D	E	F	G
$R_1(ABCD)$	a	a	a	a	a	a	a
$R_2(DEFG)$	ww			a	a	a	a

Tuples with all a's.

$AB \to CD$ ✗
$D \to E$ ✓
$E \to FG$ ✓

LOSSLESS

②

By Binary -

$R_1(ABCD) \cup R_2(DEFG) = R_{12}(ABCDEFG)$

$R_1(ABCD) \cap R_2(DEFG) = [D]^+ = DEFG$

↳ Superkey - R_2

Let R(A, B, C, D, E) be a relational schema with the following function dependencies:

$A \to B$, $B \to C$, $C \to D$ and $D \to BE$.

Decomposed into R_1(AB) R_2(BC) R_3(CD) and R_4(DE)

①

	A	B	C	D	E
$R_1(AB)$	a	a	a	a	a
$R_2(BC)$	w	a	a	a	a
$R_3(CD)$			a	a	
$R_4(DE)$				a	a

Tuples with all a's.

$A \to B$ ✗
$B \to C$ ✓
$C \to D$ ✓
$D \to BE$ ✓

LOSSLESS ✓

② ∘ $R_1(AB) \cup R_2(BC) = R_{12}(ABC)$

✓ $R_1(AB) \cap R_2(BC) = [B]^+ = BC$

∘ $R_{12}(ABC) \cup R_3(CD) = R_{123}(ABCD)$

✓ $R_{12}(ABC) \cap R_3(CD) = [C]^+ = CDE$

∘ $R_{123}(ABCD) \cup R_4(DE) = R_{1234}(ABCDE)$

✓ $R_{123}(ABCD) \cap R_4(DE) = [D]^+ = DBE$

Q. Consider a schema R(A, B, C, D) and functional dependencies $A \to B$ and $C \to D$. Then the decomposition of R into R_1(AB) and R_2(CD) is [MCQ: 2M]

- (A) Dependency preserving and lossless join
- (B) Lossless join but not dependency preserving
- (C) Dependency preserving but not lossless join
- (D) Not dependency preserving and not lossless join

①

	A	B	C	D
$R_1(AB)$	a	a	ww	
$R_2(CD)$	ww		a	a

$A \to B$ ✗
$C \to D$ ✗

② $R_1(AB) \cup R_2(CD)$ Nothing common.

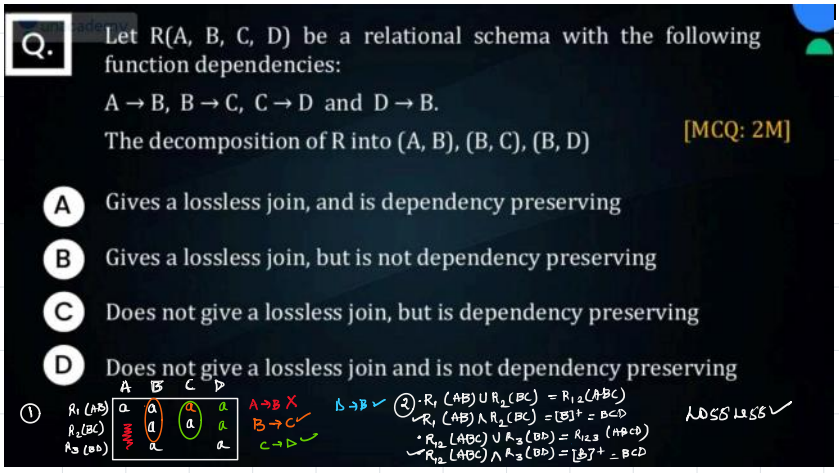
LOSSY ✓

Q. Let R(A, B, C, D) be a relational schema with the following function dependencies:

$A \to B$, $B \to C$, $C \to D$ and $D \to B$.

The decomposition of R into (A, B), (B, C), (B, D) [MCQ: 2M]

- (A) Gives a lossless join, and is dependency preserving
- (B) Gives a lossless join, but is not dependency preserving



topic : Dependency preserving decomposition

Let R be the relational schema with FD set F is decomposed into sub relations R_1 , R_2 , R_3 ... and R_n with FD set F_1, F_2, F_3....F_n respectively.

So, if union karne par EXACT FD set aa jata hai then we call it dependency preserving decomposition.

$F_1 \cup F_2 \cup F_3 .... \cup F_n \equiv F$

(OR)

IF union karne par less FD set aata hai then dependency not preserving.

$F_1 \cup F_2 \cup F_3 .... \cup F_n \subset F$

Process to find dependency preserving -

Step 1 : first take the closure of each attribute.

Step 2 : Write non trivial FD's in the respective relations

if $F_1 \cup F_2 \cup F_3 .... \cup F_n \equiv F$
Hence, dependency preserving.

Let R(A, B, C, D, E) be a relational schema with the following function dependencies:
A → B, B → C, C → D and D → BE.
Decomposed into R_1(AB) R_2(BC) R_3(CD) and R_4(DE)

Step 1 : first take the closure of each attribute.

$[A]^+ = [ABCDE]$
$[B]^+ = [BCDE]$

$[C]^+ = [CDEB]$
$[D]^+ = [DBEC]$
$[E]^+ = [E]$

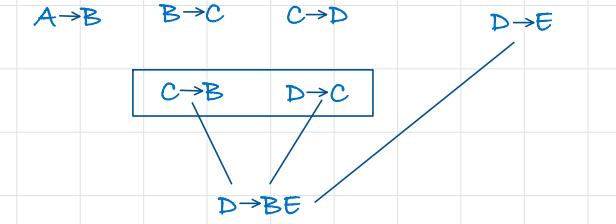
Step 2 : Write non trivial FD's in the respective relations

$[A]^+ = [ABCDE]$ Deriving B from A
$[B]^+ = [BCDE]$ Deriving C from B
$[C]^+ = [CDEB]$ Deriving B from C and
Deriving D from C
$[D]^+ = [DBEC]$ Deriving E and C from D
$[E]^+ = [E]$ Cannot derive anything

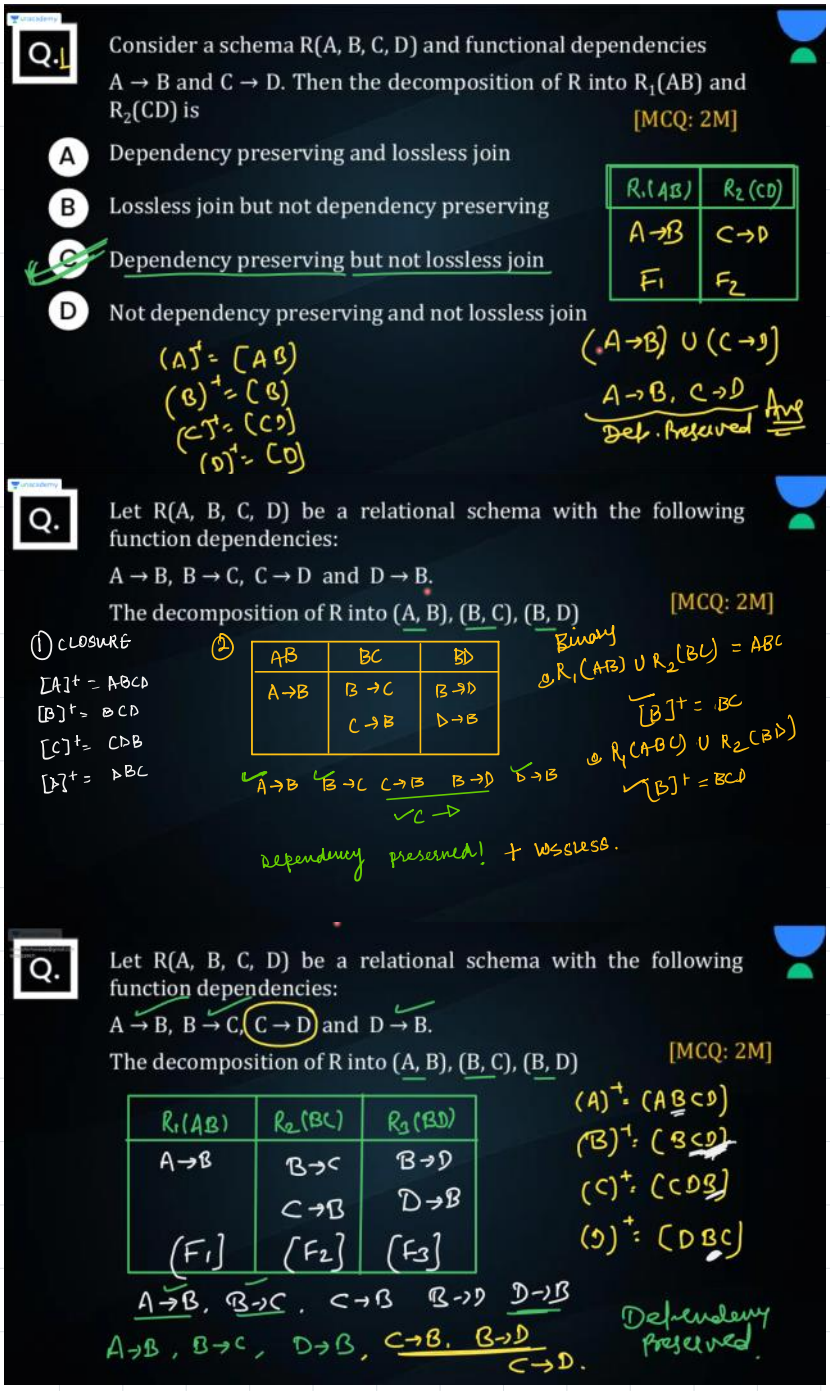
R_1 (AB)	R_2 (BC)	R_3 (CD)	R_4 (DE)
A→B	B→C C→B	C→D D→C	D→E

Step 3 : if $F_1 \cup F_2 \cup F_3 \ldots \cup F_n \equiv F$
(Dependency preserving)

$A \rightarrow B,\ B \rightarrow C,\ C \rightarrow D$ and $D \rightarrow BE.$

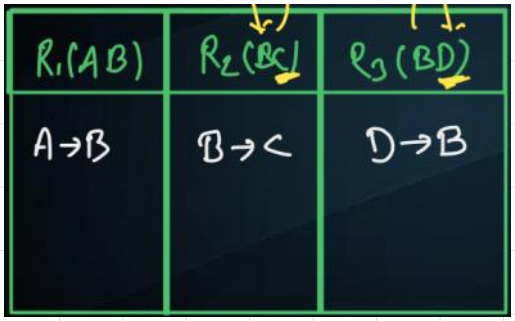


Hence, dependency preserving

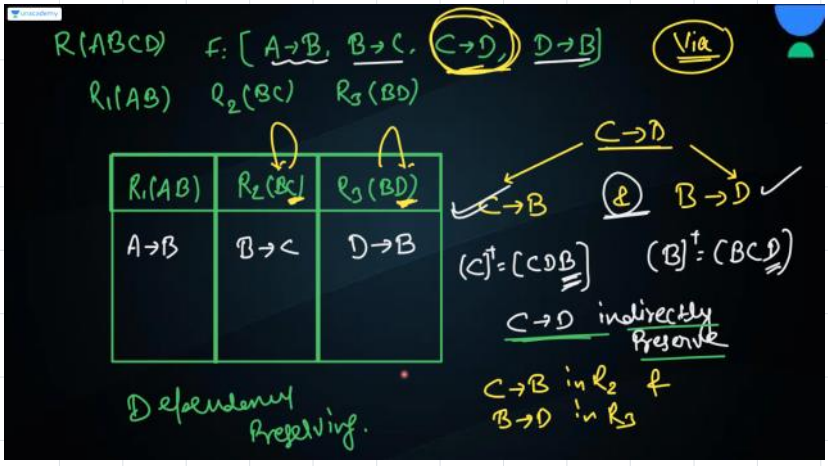


Direct method - Checking VIA

Step 1 : Make the same table and then write down all the non-trivial FD's in respective column.



Step 2 : Check the remaining FD's by taking closure. If are able to derive the attribute indirectly from closure then we can say it is indirectly preserved.



There is no direct preservation in R_2 and R_3 for C → D because C and D both are in different relations.

But indirectly we are able to preserve

In the closure of C we got B that means C → B

In the closure of B we got D that means B → D

So we can say that C → D is indirectly preserved.

topic : Finding number of super-keys -

First approach

Let R be the relation schema with n attributes $(A_1, A_2, A_3 ..... A_n)$

Then how many super-keys are possible with only

(A) With only candidate key A_1

Total super-keys = 2^{n-1}

(n is number of attributes)

Example : R(ABCD) with candidate key "A" then how many super-keys?

Solution : R(ABCD) candidate key = [A]

Method 1 : Number of Super-keys = 2^{n-1}

Number of Super-keys = 2^{n-1}

$= 2^{4-1}$

$= 2^3$

Total number of super-keys are = 8 Super-keys

Method 2 :

R(ABCD)

"A" is fixed.

$2^3 = 8$ Super-keys

Method 3 : Digital

R(ABCD)

A	B	C	D	
	0	0	0 →	A
	0	0	1 →	AD
	0	1	0 →	AC
	0	1	1 →	ACD
	1	0	0 →	AB
	1	0	1 →	ABD
	1	1	0 →	ABC
	1	1	1 →	ABCD

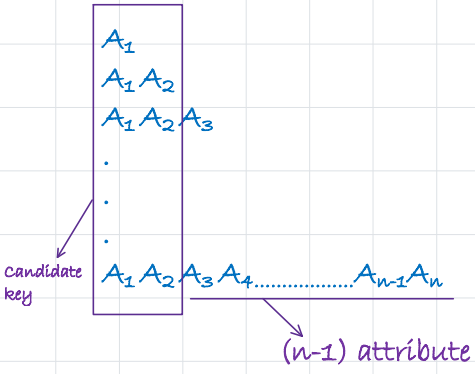
Method 4 : Combinations

R(ABCD)

1. Single attribute : A
2. Two attribute : AB, AC, AD
3. Three attribute : ABC, ABD, ACD
4. Four attribute : ABCD

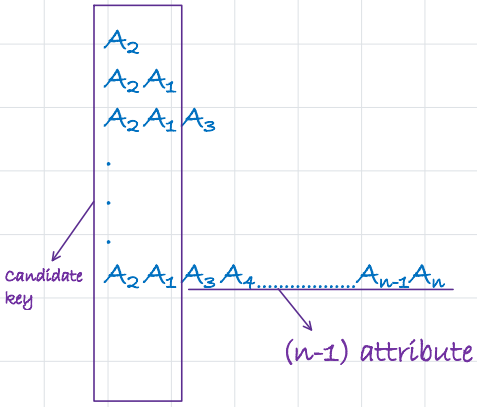
Total number of super-keys= 8

(B) With Only candidate key A_1, A_2



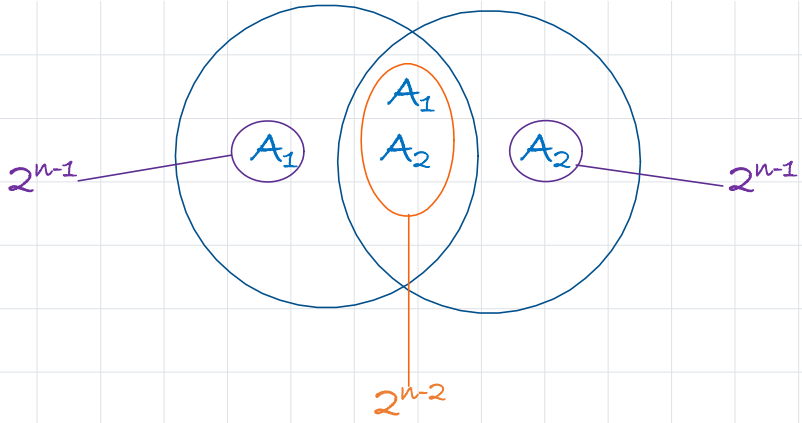
Total super-keys = 2^{n-1}

(n is number of attributes)



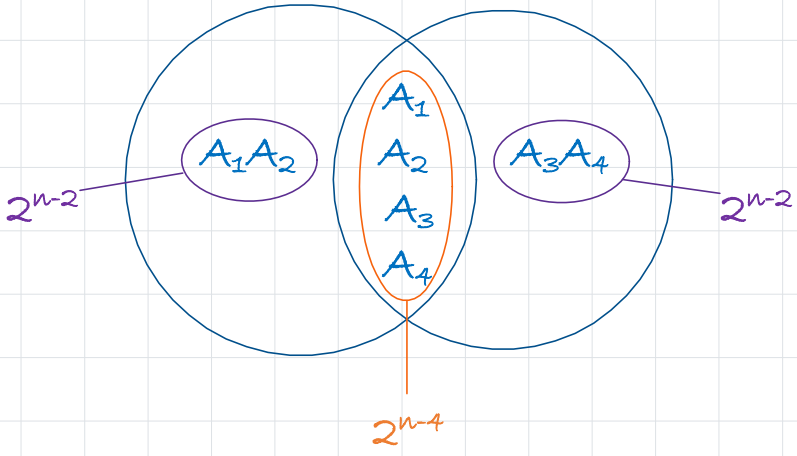
Total super-keys = 2^{n-1}

(n is number of attributes)



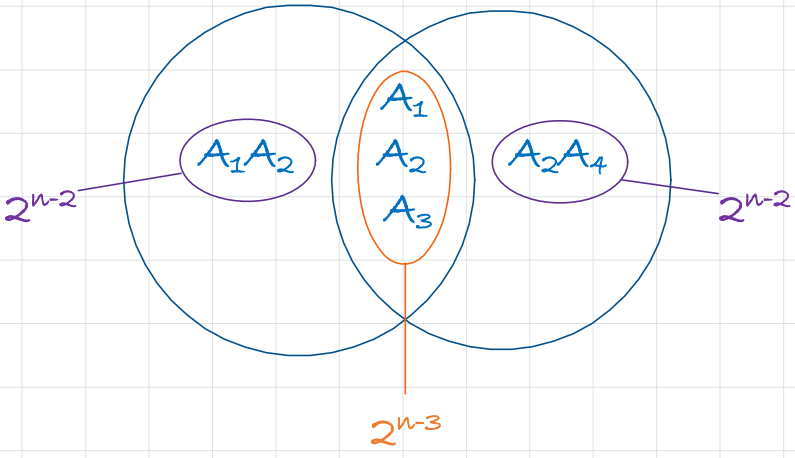
Total super-keys = $2^{n-1}+2n^{-1}-2^{n-2}$

(C) With only candidate key A_1A_2, A_3A_4



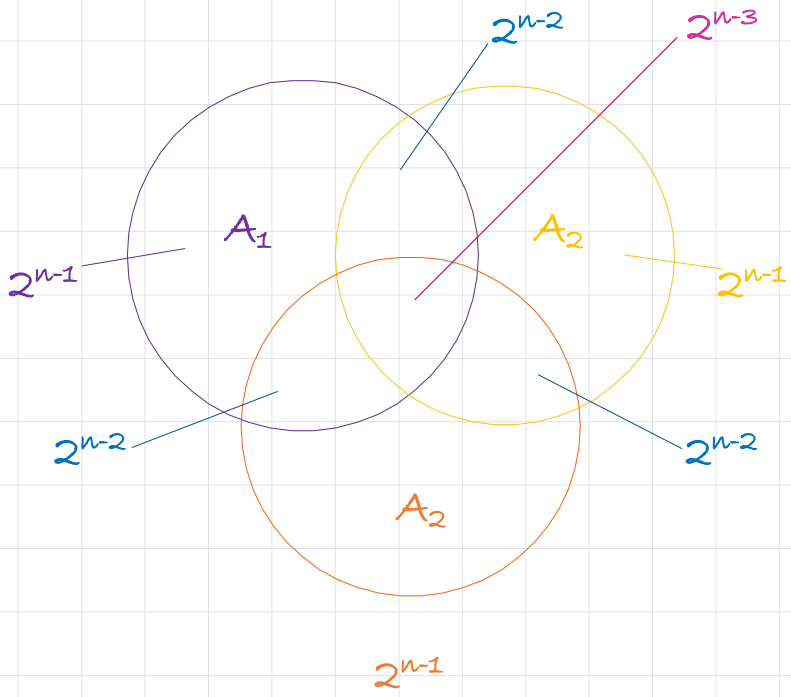
Total super-keys = $2^{n-2}+2n^{-2}-2^{n-4}$

(D) With only candidate key A_1A_2, A_2A_3



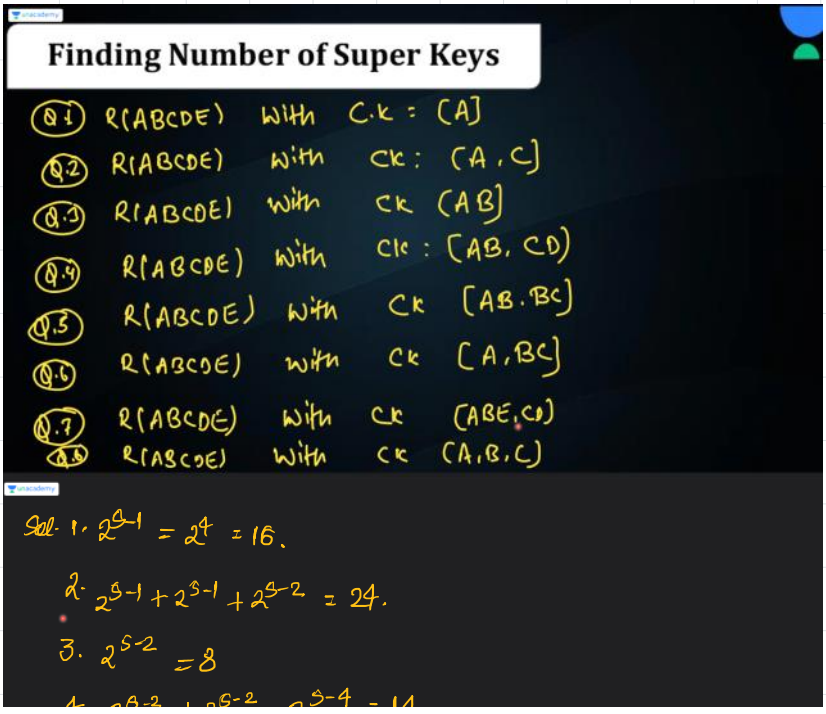
Total super-keys = $2^{n-2}+2^{n-2}-2^{n-3}$

(E) With only candidate key A_1, A_2, A_3



Total super-keys = $2^{n-1}+2^{n-1}+2^{n-1}-2^{n-2}-2^{n-2}-2^{n-2}+2^{n-3}$

Practice -



3. $2^{5-2} = 8$

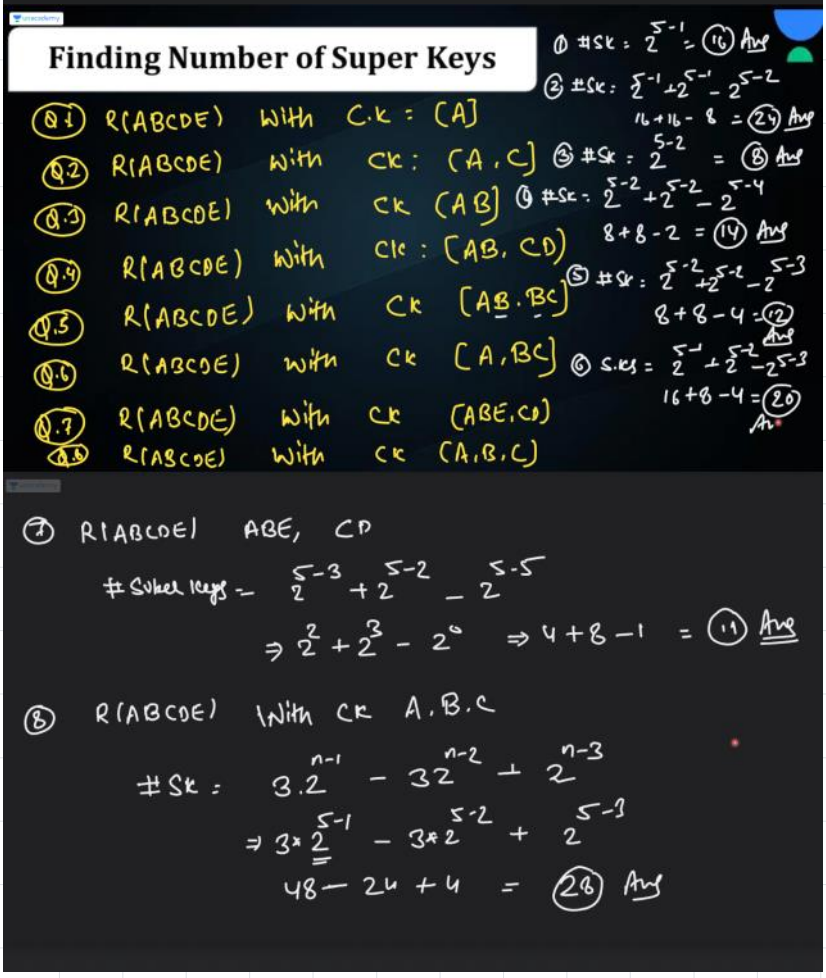
4. $2^{8-2} + 2^{5-2} - 2^{5-4} = 14$

9. $2^{5-2} + 2^{5-2} - 2^{5-3} = 12$

6. $2^{5-1} + 2^{5-2} - 2^{5-3} = 20$

7. $2^{8-3} + 2^{8-2} - 2^{0} = 11$

8. $2^{5-1} + 2^{5-1} + 2^{8-1} - 2^{5-2} - 2^{5-2} - 2^{5-2} + 2^{5-3} = 28$



Second approach to find number of super-keys -

Example : R(ABCDE) with candidate key "A, BC" then how many super-keys?

First approach -

Total super-keys = $2^{5-1}+2^{5-2}-2^{5-3}$

$= 2^4+2^3-2^2$

$= 16 + 8 - 4$

$= 20$

Second approach - (not reliable)

1. Combinations due to Candidate key "A" -

ABCDE = $2^4 = 16$

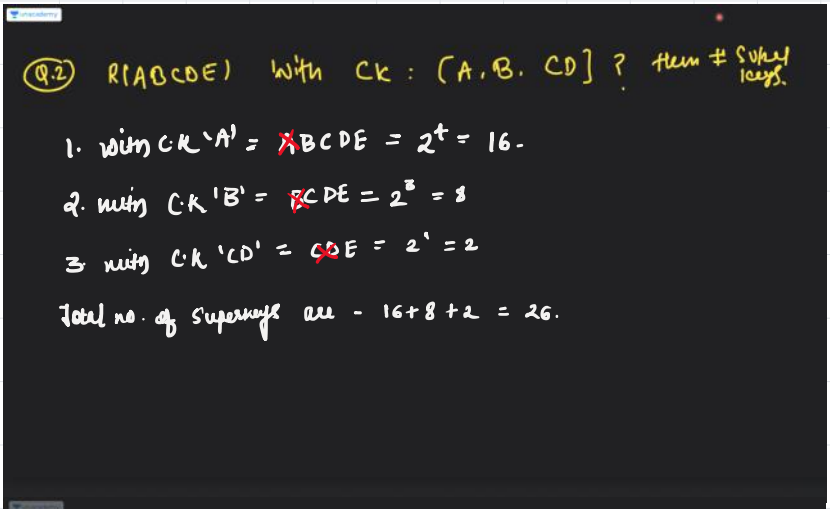
2. Combinations due to Candidate key "BC" -

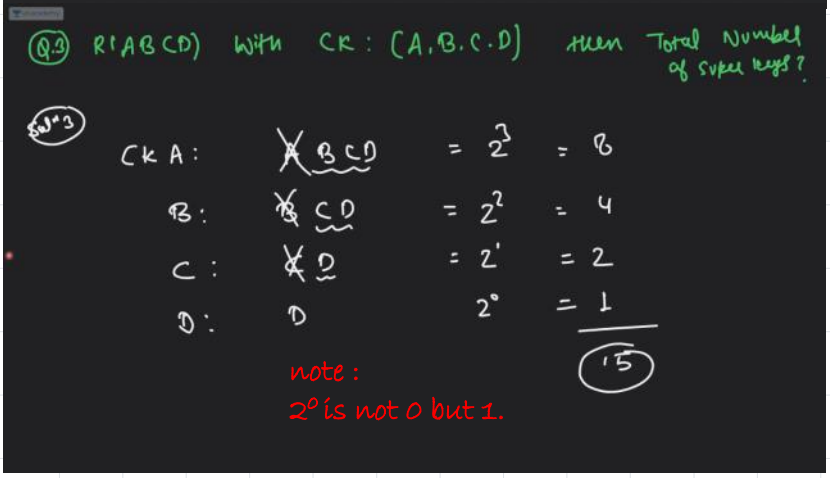
BC DE = 2^2 = 4

[BC, BCD, BCE, BCDE]

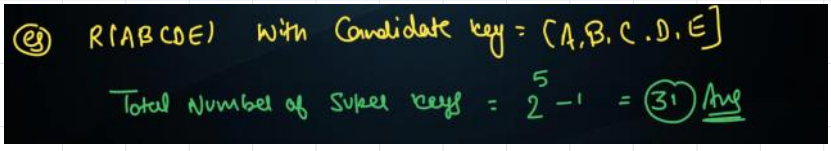
Total Number of super-keys are 16 + 4 = 20.

Practice -

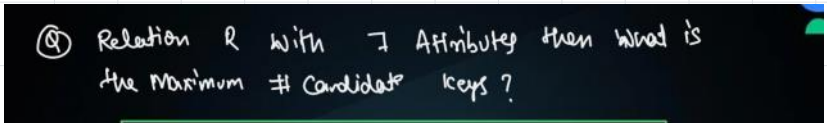


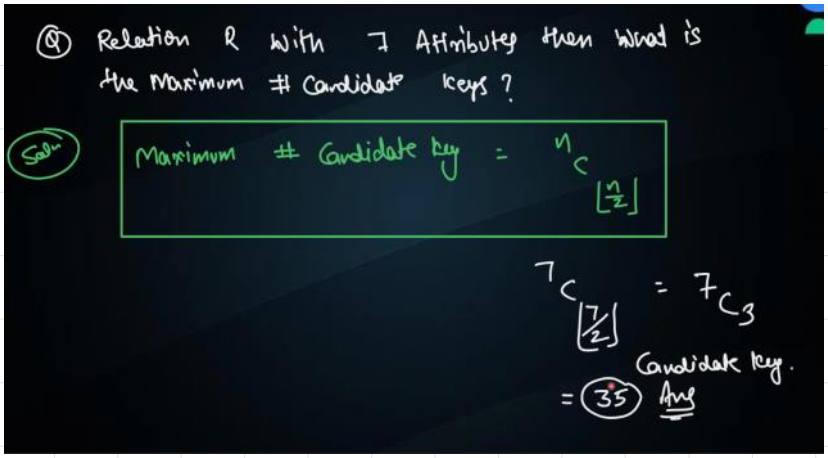


note :when in a relation each single attribute is a candidate key then Total number of super-keys are=$2^n - 1$



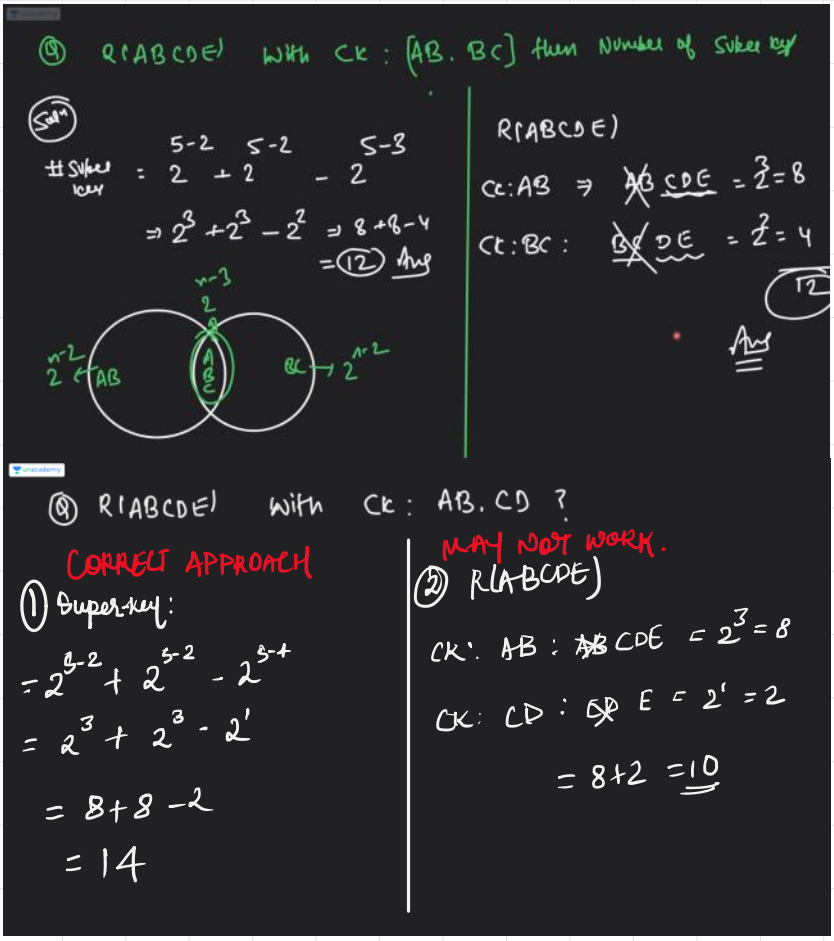
note : when in a relation each single attribute is a candidate key then maximum Number of candidate keys are = $^nC_{n/2}$





Example : R(ABCDEF) then what is the maximum number of candidate key?

1. If each single attribute is candidate key $= {}^{6}C_1 = 6$
2. IF every 2-2 attribute is candidate key $= {}^{6}C_2 = 15$
3. IF every 3-3 attribute is candidate key $= {}^{6}C_3 = 20$
4. IF every 4 attribute is candidate key $= {}^{6}C_4 = 15$
5. IF every 5 attribute is candidate key $= {}^{6}C_5 = 6$
6. IF every 6 attribute is candidate key $= {}^{6}C_6 = 1$



Third approach to find number of super-keys -

Third approach - (most reliable)

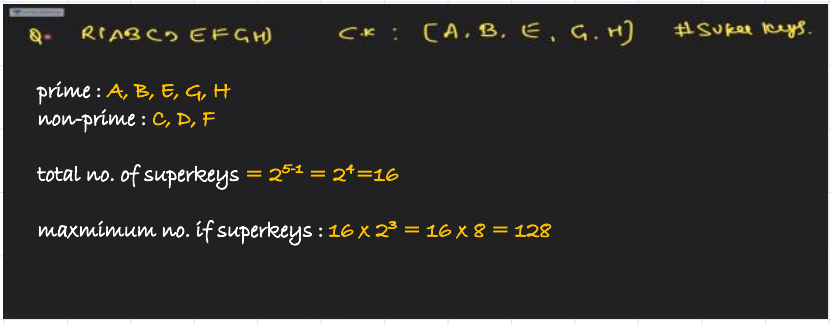
Total Number of super- keys = [Number of super-keys Among the Prime attribute Of Relation R] X $2^{\text{no. Of non-prime attributes}}$

When in a relation each single attribute is a candidate key then

Total number of super-keys are=$2^n - 1$

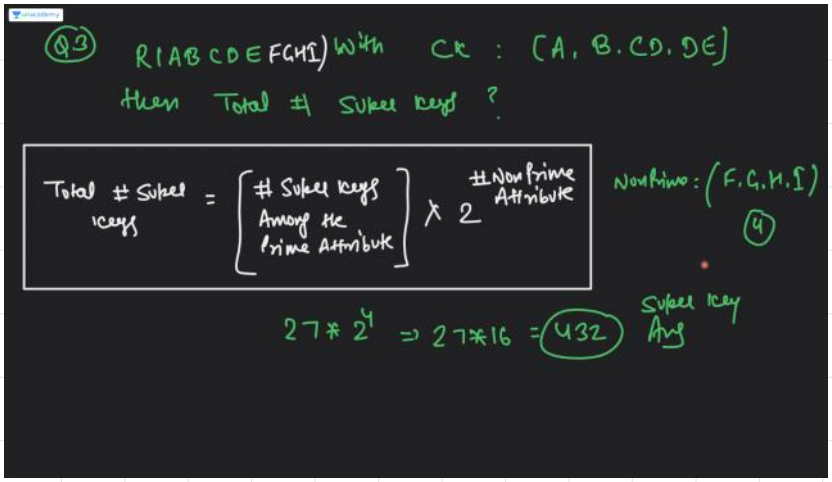
Example : R(ABCDEFGH) with candidate key [A,B,C,D] then total number of super-keys are?
Solution : Prime attributes : [A,B,C,D]
and Non-Prime attributes : [E,F,G,H]
When in a relation each single attribute is a candidate key
Total number of super-keys are=$2^n - 1 = 2^4 - 1 = 15$

Total Number of super- keys = $15 \times [2]^4$
$= 15 \times 16$
$= 240.$

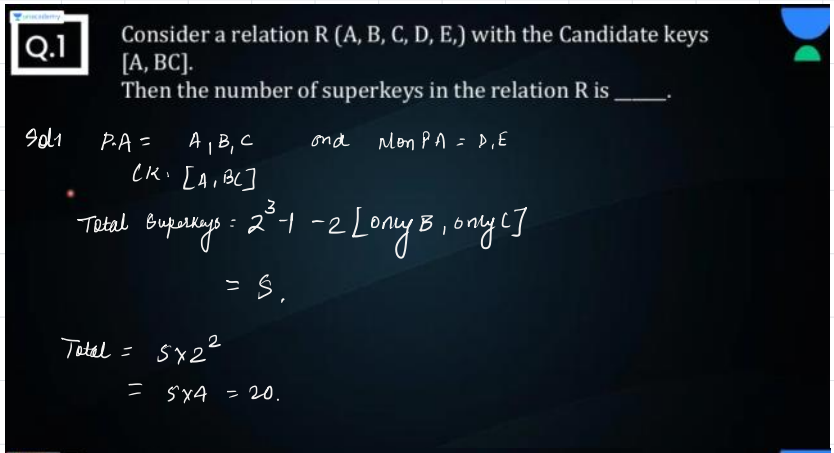


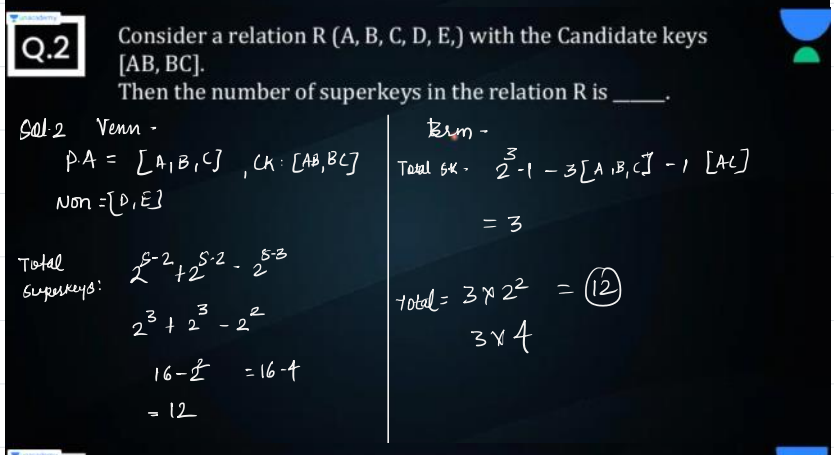
When multi-candidate keys are given.

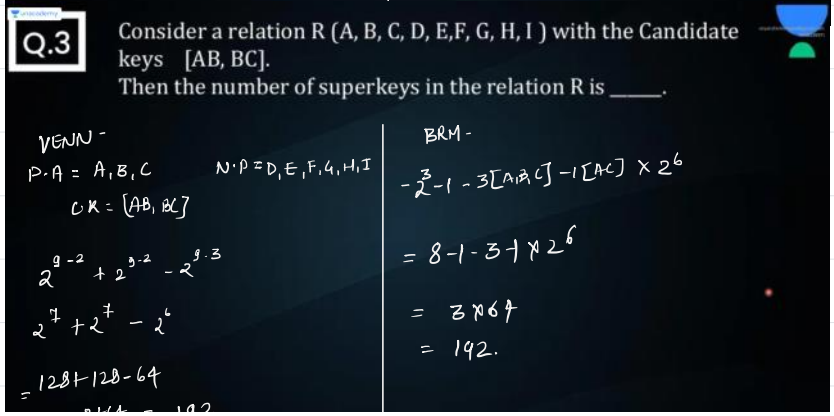
Example : R(ABCDE) with candidate key [A,B,CD,DE] then total number of super-keys are?
Solution : Prime attributes : [A,B,C,D,E]
and Non-Prime attributes : [0]
When in a relation each single attribute is a candidate key
Total number of super-keys are=$2^5 - 1 - 3$ [only C,D,E is not key] - 1 [only CE is not key]
$= 27$


Q.3 R(A B C D E F G H I) with CK : [A, B, CD, DE]
then Total # Super keys ?
Total # Super keys = [# Super keys Among the Prime Attribute] x 2 # Non Prime Attribute
Non Prime : (F, G, H, I)
4
27 * 2^4 => 27*16 = 432
Super key Ans

Practice -


Q.1
Consider a relation R (A, B, C, D, E,) with the Candidate keys [A, BC].
Then the number of superkeys in the relation R is _____.
Sol 1 P.A = A, B, C and Non PA = D, E
CK: [A, BC]
Total Superkeys : 2^3 - 1 - 2 [only B, only C]
= 5.
Total = 5 x 2^2
= 5 x 4 = 20.


Q.2
Consider a relation R (A, B, C, D, E,) with the Candidate keys [AB, BC].
Then the number of superkeys in the relation R is _____.
Sol 2 Venn -
P.A = [A, B, C] , CK: [AB, BC]
Non = [D, E]
Total Superkeys: 2^(5-2) + 2^(5-2) - 2^(5-3)
2^3 + 2^3 - 2^2
16 - 4 = 16 - 4
= 12
Brm -
Total SK. 2^3 - 1 - 3[A, B, C] - 1 [AC]
= 3
Total = 3 x 2^2 = 12
3 x 4


Q.3
Consider a relation R (A, B, C, D, E,F, G, H, I) with the Candidate keys [AB, BC].
Then the number of superkeys in the relation R is _____.
VENN -
P.A = A, B, C N.P = D, E, F, G, H, I
CK = [AB, BC]
2^(9-2) + 2^(9-2) - 2^(9-3)
2^7 + 2^7 - 2^6
= 128 + 128 - 64
Brm -
- 2^3 - 1 - 3[A, B, C] - 1 [AC] x 2^6
= 8 - 1 - 3 + 1 x 2^6
= 3 x 64
= 192.

$2^7 + 2^7 - 2$

$= 128 + 128 - 64$

$= 128 + 64 = 192.$

$= 192.$

Q.4 Consider a relation R (A, B, C, D, E,) with the Candidate keys [AB, CD].
Then the number of superkeys in the relation R is _____.

Sol:- VENN - C.K : [AB, CD]
P.A = [A, B, C, D] Non P.A = [E]

$2^{5-2} + 2^{5-2} - 2^{5-4}$

$2^3 + 2^3 - 2^1$

$8 + 8 - 2$

$16 - 2 = 14$ ✓

BRM

$2^4 - 1 - 4 - 2 - 2 \times 2^1$

$16 - 1 - 4 - 2 - 2 \times 2$

$16 - 9 \times 2$

7×2

$= 14$

Q. Consider a relation R (A, B, C, D, E) with the following three functional dependencies.
$AB \rightarrow C$; $BC \rightarrow D$; $C \rightarrow E$; [AB] C.K
The number of superkeys in the relation R is _____. [NAT:2022]

Sol. VENN

P : A, B Non P : C, D, E
C.K = [AB]

$= 2^{5-2} + 2^{5-2} - 2^{5-2}$

$= 2^3$

$= 8$

BRM -

$2^2 - 1 - 2 \times 2^3$

$4 - 1 - 2 \times 3$

1×8

$= 8.$

Q.1 Consider a relation R (A, B, C, D, E, F, G) with the Candidate keys [A, B, C, DE].
Then the number of superkeys in the relation R is _____.

Sol:1 VENN

$= 2^{7-3} + 2^{7-3} - 2^{7-}$

$= 4 + 4 -$

BRM

$2^5 - 1 - 2 \times 2^2$

$31 - 2 \times 4$

29×4

$= 116$

Q.2 Consider a relation R (A, B, C, D, E, F, G, H , I ,J) with the Candidate keys [A, B, C, D, EF, FG].
Then the number of superkeys in the relation R is _____.

Sol 2

$= 2^7 - 1 - 3[EFG] - 1[EG]$

$= 127 - 4$

$= 123 \times 2^3$

$= 123 \times 8$

$= 984.$

$2^{10-6} + 2^{10-6} - 2^{10-7}$

$2^4 + 2^4 - 2^3$

$32 - 8$

24

Consider a relation R (A, B, C, D, E, F, G) with the Candidate

Q.3 Consider a relation R (A, B, C, D, E, F, G) with the Candidate keys [A, B, C, D, E, F, G].
Then the number of superkeys in the relation R is _____.

Sol.

$= 2^7 - 1 \times 2^0$

$= 128 - 1 \times 1$

$= 127.$

Q.4 Consider a relation R (A, B, C, D, E, F, G, H) with the Candidate keys [A, BC, D, CE].
Then the number of superkeys in the relation R is _____.

Sol.4

$2^5 - 1 - 3 - 1 \times 2^3$ (BCE, BC)

$32 - 5 \times 8$

27×8

216

Q.5 Consider a relation R (A1, A2,A3,A4,A5,A6,A7,A8) with the Candidate keys [A1A2A3].
Then the number of superkeys in the relation R is _____.

Sol.

$= 2^3 - 1 - 3[A1, A2, A3] - 3[A1A2, A2A3, A1A3] \times 2^5$

$= 8 - 1 \times 32$

$= 1 \times 32$

$= 32.$

Q Relation Schema R with n attribute $(A_1 A_2 A_3 .. A_n)$ and 'K' Simple Candidate keys in R where $K < n$ then How many Super keys are possible in Relation R?

Soln K Simple Ck :⇒ Prime Attribute = (K) Non Prime = (n − K)

Super Keys Among Prime Attribute $= 2^K - 1$

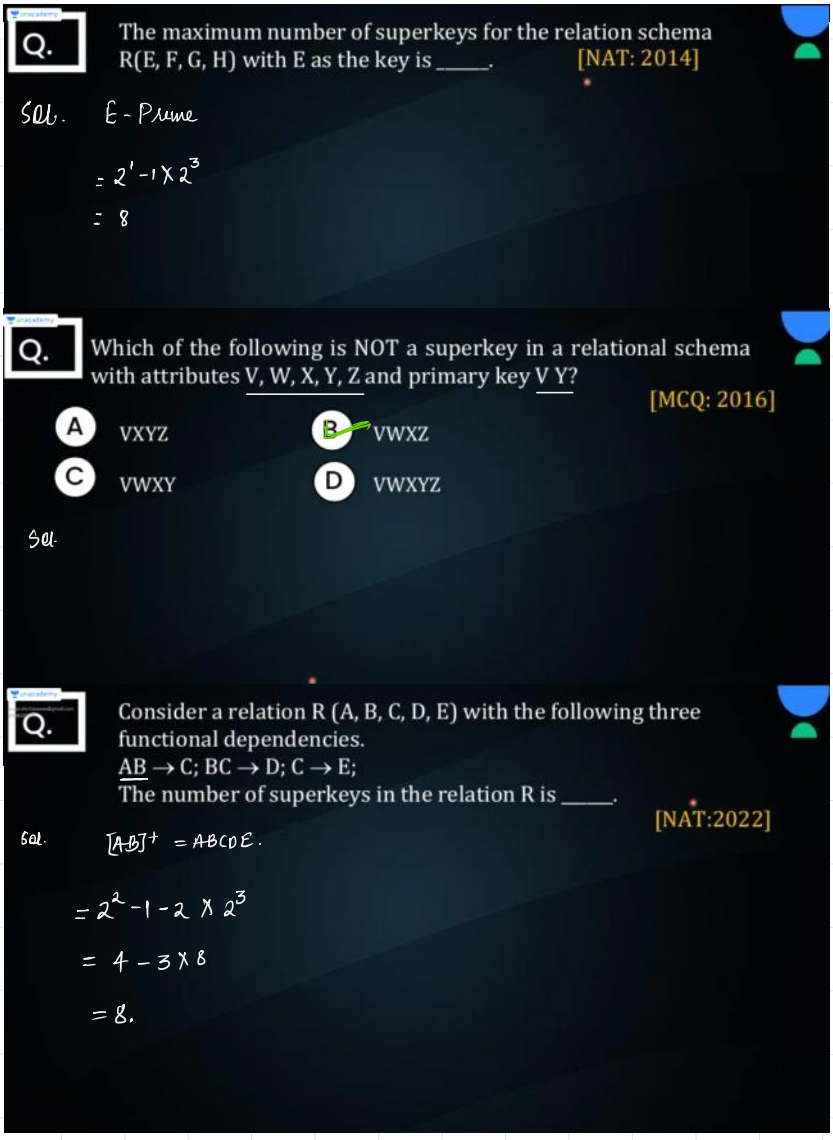
Total # Super keys $= (2^K - 1) * 2^{n-K}$ Ans

Q. The maximum number of superkeys for the relation schema R(E, F, G, H) with E as the key is _____. [NAT: 2014]

Sol. E - Prime

$= 2^1 - 1 \times 2^3$

$= 8$



topic : normal forms

Normal form is a set of rules which are used to reduce/eliminate the redundancy.

Redundancy is the unnecessary repetition of data.

The process of removal is called normalization.

There are various normal forms-
1. 1NF [first normal form]
2. 2NF [second normal form]
3. 3NF [third normal form]
4. BCNF [Boyce Codd normal form]

1NF [first normal form]: A Relation schema R is in 1NF is R does not contain any multivalued attribute

(OR)

Not contain composite attribute

(OR)

A relation schema R is in 1NF if all attributes of R are atomic.

Atomic means not divisible.

Not in 1NF

Student

Roll no.	Name	Course
1	Aryan	C/Java

Multivalued Attribute

In 1NF

Student

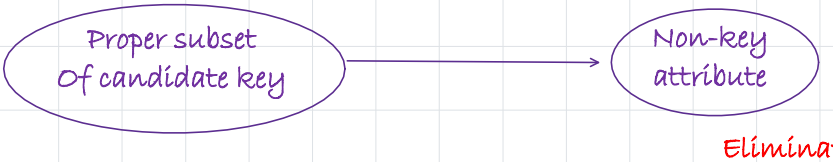
Roll no.	Name	Course
1	Aryan	C
2	Aryan	Java

note :
1. Default RDBMS is in 1NF
2. 1NF ensured by candidate key
3. RDBMS does not accept multivalued attribute
4. In 1NF redundancy level is too high.

Redundancy level : 1NF > 2NF > 3NF > BCNF(0%)

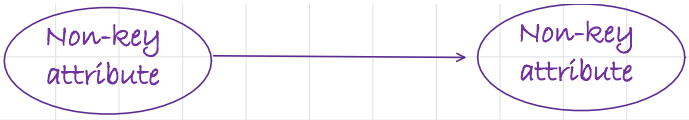
Possible Non trivial FD which cause redundancy

CASE 1:



Eliminated by 2NF

CASE 2 :



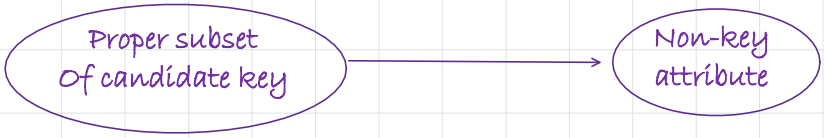
Eliminated by 3NF

CASE 3 :

Eliminated by BCNF

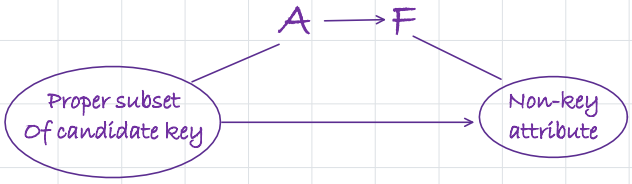
Cases with examples :

CASE 1:



R(ABCDEF) [AB → C, C → D, D → E, A → F]

Candidate key = [AB]
Non-Prime/Non key attribute = [C,D,E,F]



CASE 2 :

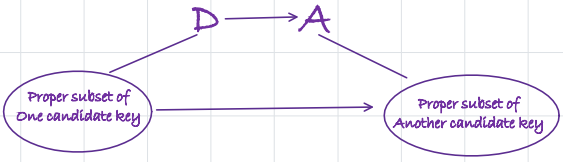
R(ABCDEF) [AB → C, C → D]

Candidate key = [AB]
Non-Prime/Non key attribute = [C,D]

CASE 3 :

R(ABCD) [AB →CD, D →A]

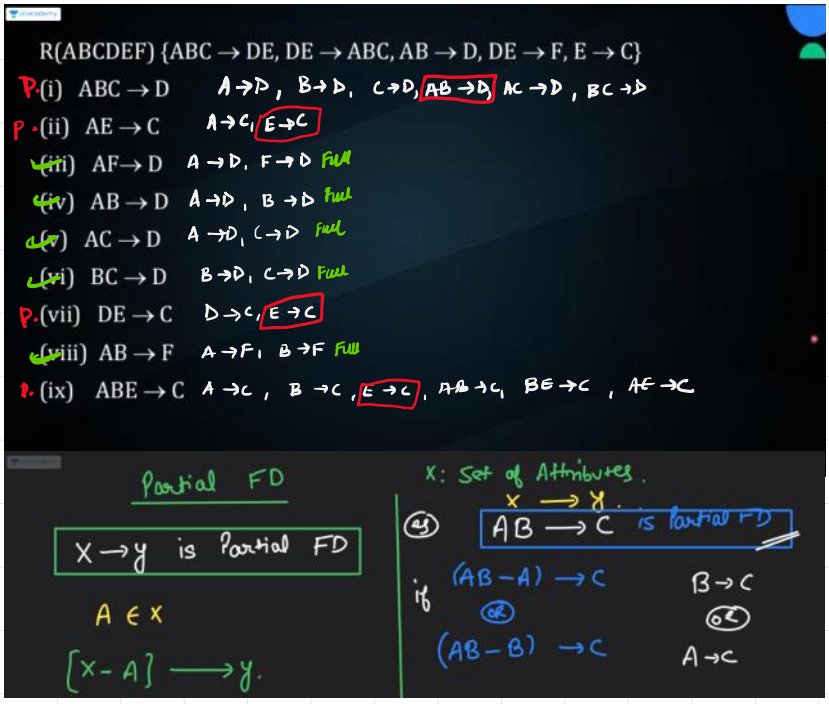
Candidate key = [AB, DB]



note :
1. If relation R is in 2NF, R is already in 1NF
2. If R is in 3NF , R is already in 2NF and 1NF
3. If R is in BCNF, R is already in 3NF
4. Default RDBMS is in 1NF.

Partial Dependency: (Take closure)

AB→C is a partial FD if I can determine
A→C
B→C
Then, AB→C is a partial FD.



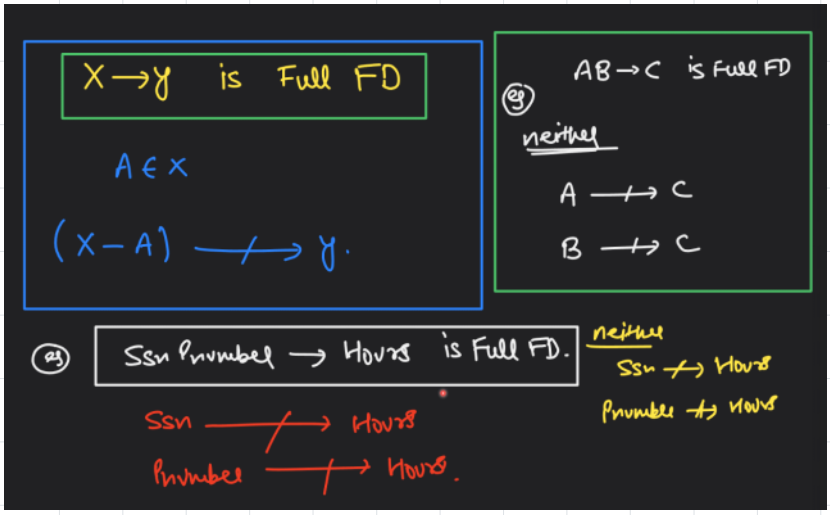
Full Dependency: AB→C is a full FD if I cannot determine.

We cannot remove any attribute from L.H.S.

$A \not\rightarrow C$

$B \not\rightarrow C$

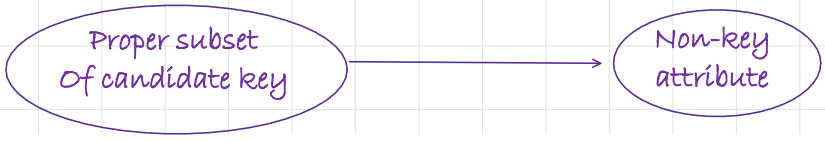
Then, AB→C is a full FD.

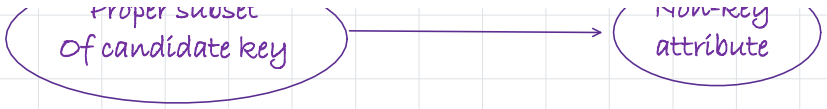


2NF [second normal form]: A relation schema R is in 2Nf only if

(1) R is in 1NF

(2) Check that R should not contain this case.





This cause violation of 2NF

A relation schema R is in 2NF if every non-prime attribute 'A' in 'R' is fully functionally dependent on the primary key of R.
In 2NF we can have partial dependency but that type of partial dependency does not create any problem.

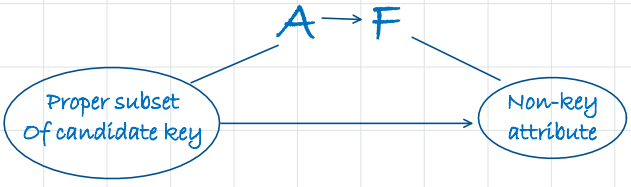
2NF

1. R(ABCDEF) [AB →CD, CD →AB, B →D]
Candidate key = [AB, CD]
Prime attribute : [A,B,C,D]
Non-prime attribute : 0
AB →CD
B→D
AB →D is a partial dependency
But both are already prime attributes.
Hence, R is in 2NF.

Not in 2NF

2. R(ABCDEF) [AB →C, C →D, D →E, A →F]
Candidate key = [AB]
AB→C
A→C, $[A]^+ = AF$
B→C, $[B]^+ = B$
I can determine F from A here.
AB→F is a partial Dependency.
Prime attribute - [A,B]
Non-Prime attribute - [C,D,E,F]

Check 2NF-



Hence, not in 2NF

FD: [Ssn → Pnumber → Hours, Ssn → Ename, Pnumber → Pname Plocation]

Ssn Pnumber Hours Ename Pname Plocation

R[A B C D E F] (or) [AB → C, A → D, B → EF]

[AB]+ = ABCDEF Prime: A, B Non-prime: C, D, E, F

[A]+ = AD [B]+ = BEF

A → D

AB → D is a Partial FD. Hence Not in 2NF.

(A) Proper subset of C.K → (D) Non-key attribute.

(Q) FD: [Ssn → Pnumber → Hours, Ssn → Ename, Pnumber → Pname Plocation]

Ssn Pnumber Hours Ename Pname Plocation

R[A B C D E F] (or) [AB → C, A → D, B → EF]

(Sol^n) Candidate key = [AB]

Non Key | Non Prime Attribute = [C, D, E, F]

2NF CHECKING ?

A → D B → EF

Proper Subset of CK → Non key Attribute Not in 2NF

Here Non Prime | Non Key Attribute D, E, & F is Not Fully Dependent on Key.

AB → D ∵ A → D

AB → EF ∵ B → EF

Not in 2NF.

(Q) R(ABCDE) [AB → D, C → B, E → A]

Sol.

[EC]+ = ECABD.

PRIME: E, C. | Non-Prime: A, B, D.

[E]+ = EA E → A – Non-prime. (Proper subset)

[C]+ = CB C → B

EC → AB (Partial FD)

(Q) R(ABCDE) [AB → CDE, B → C]

Sol.

[AB]+ = ABCDE

Prime: A, B Non-Prime: C, D, E

[A]+ = A

[B]+ = BC

AB → C — Partial FD. Not in 2NF.

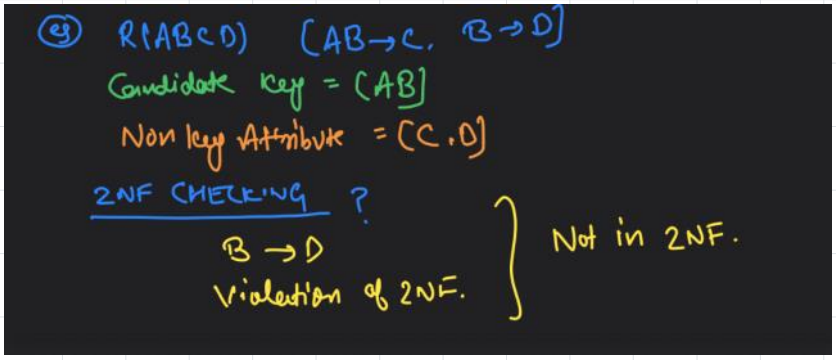
B → C

B – Subset of C.K

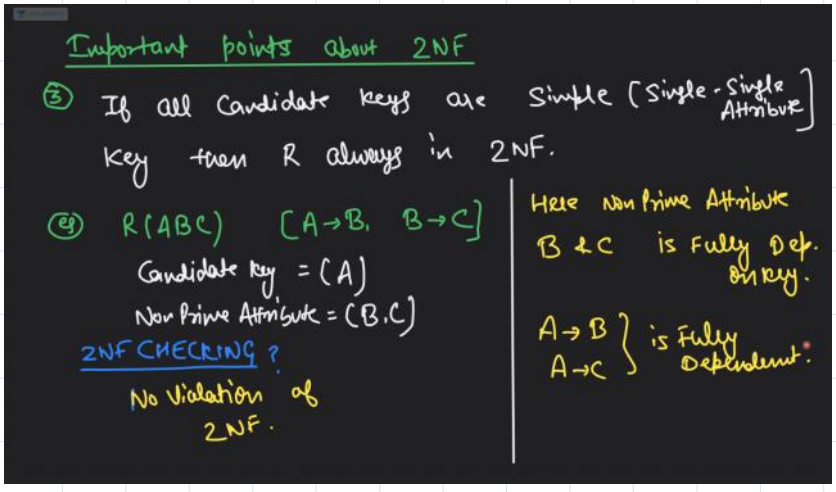
C – Non-Prime

important points :

1. If relation R has only one candidate key then relation R may or may not in 2NF.

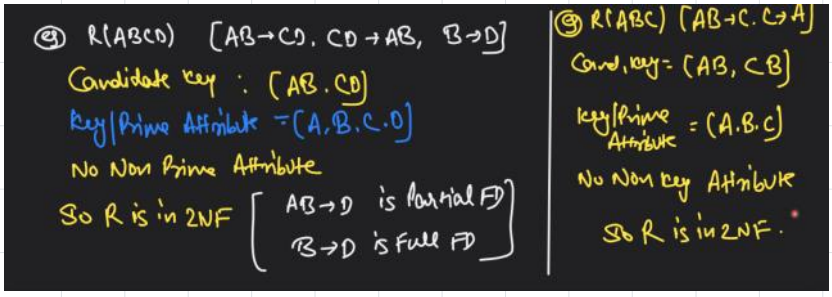


2. If all candidate keys are simple (single attribute)then R always in 2NF.

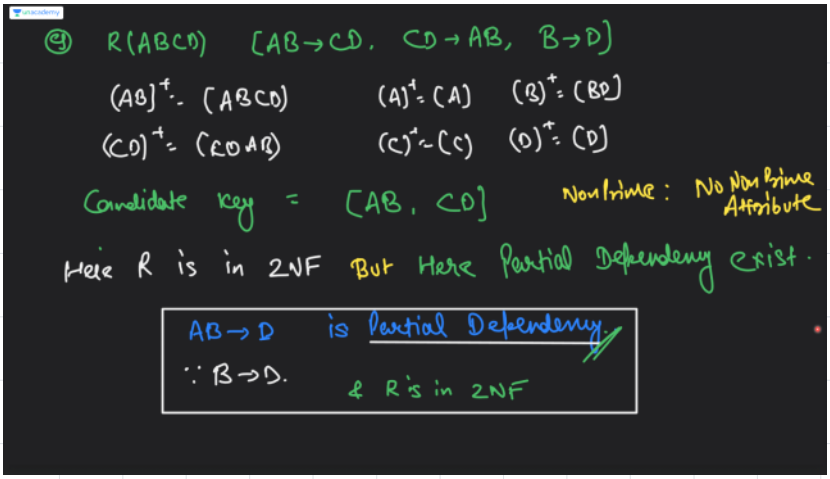


Fully dependent
not on subsets

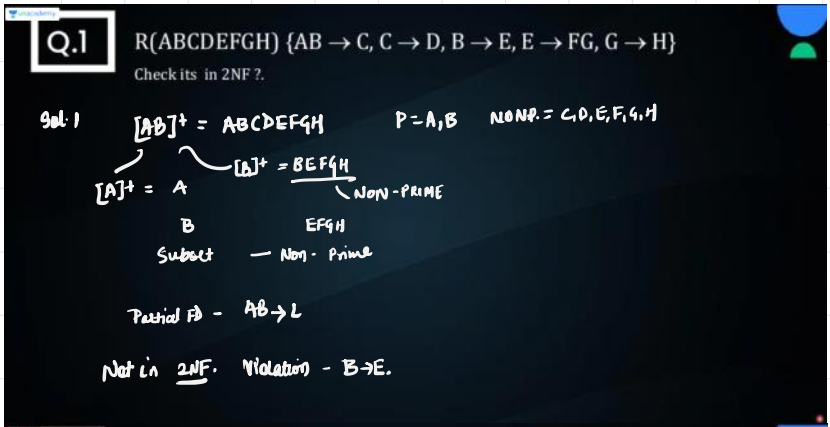
3. IF all attributes of relation R are key/prime attribute then R always in 2NF.

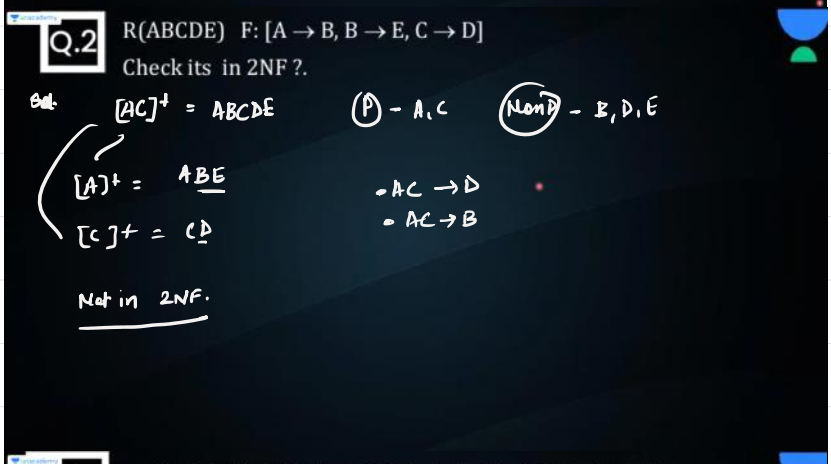


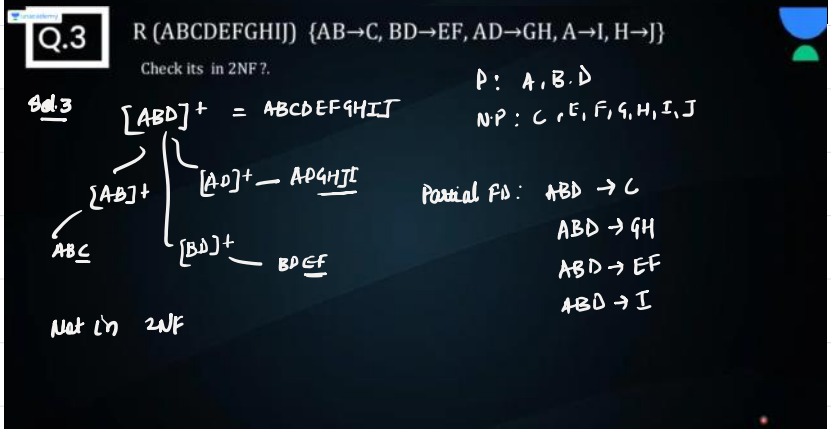
4. In 2NF we can have partial dependency but that type of partial dependency does not create any problem

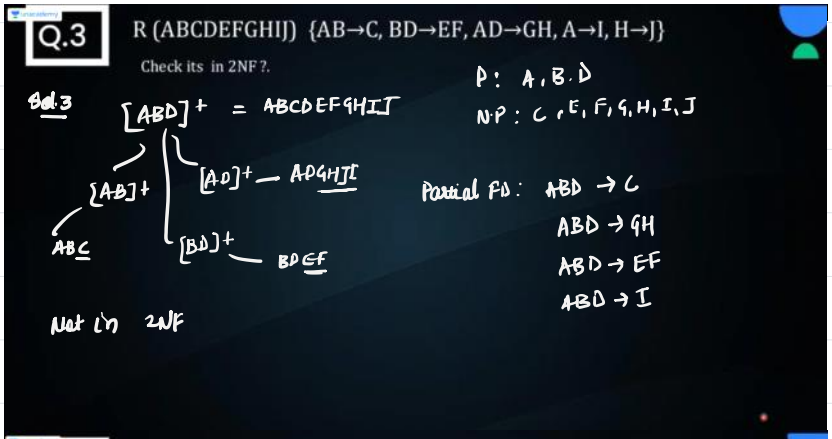

R(ABCD) [AB→CD, CD→AB, B→D]
(AB)+ = (ABCD)
(A)+ = (A)
(B)+ = (BD)
(CD)+ = (CDAB)
(C)+ = (C)
(D)+ = (D)
Candidate key = [AB, CD]
Non prime : No Non Prime Attribute
Here R is in 2NF But Here Partial Dependency exist.
AB→D is Partial Dependency
∵ B→D.
& R's in 2NF

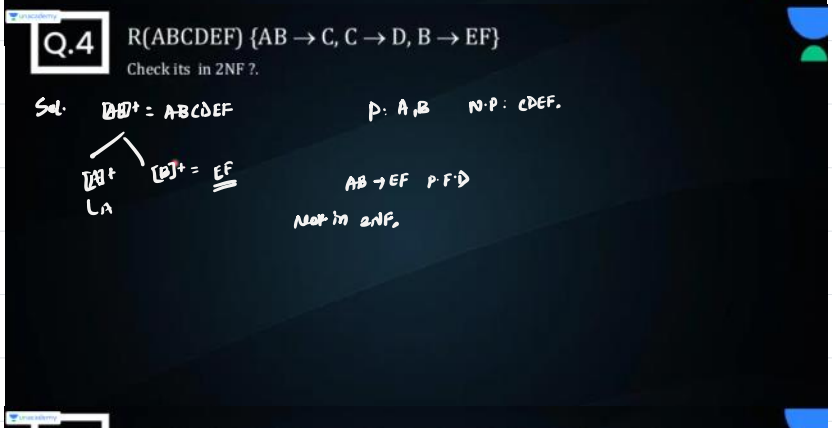
Practice -

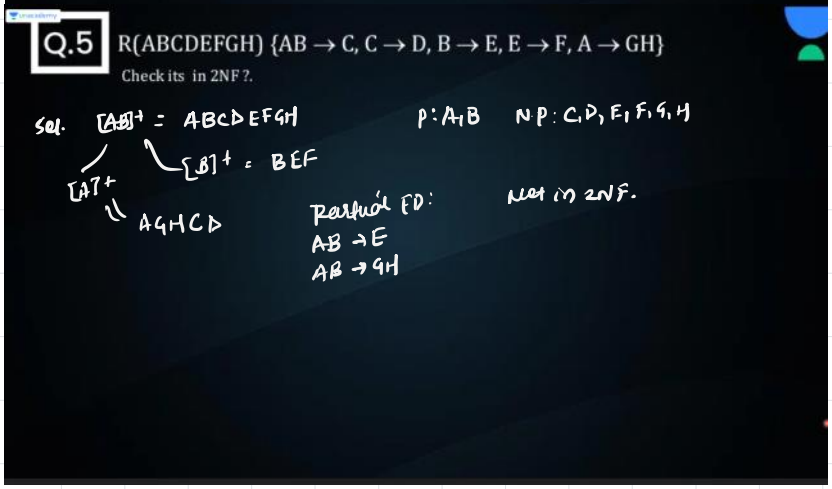

Q.1
R(ABCDEFGH) {AB → C, C → D, B → E, E → FG, G → H}
Check its in 2NF ?.
Sol 1
[AB]+ = ABCDEFGH
P = A, B
NONP. = C, D, E, F, G, H
[A]+ = A
[A]+ = BEFGH
NON-PRIME
B
EFGH
Subset — Non-Prime
Partial FD - AB → C
Not in 2NF. Violation - B→E.


Q.2
R(ABCDE) F: [A → B, B → E, C → D]
Check its in 2NF ?.
Sol.
[AC]+ = ABCDE
(P) - A, C
(Non P) - B, D, E
[A]+ = ABE
[C]+ = CD
• AC → D
• AC → B
Not in 2NF.


Q.3
R (ABCDEFGHIJ) {AB→C, BD→EF, AD→GH, A→I, H→J}
Check its in 2NF ?.
Sol.3
[ABD]+ = ABCDEFGHIJ
P: A, B, D
NP: C, E, F, G, H, I, J
[AB]+
ABC
[AD]+ — ADGHIJ
[BD]+
BDEF
Partial FD: ABD → C
ABD → GH
ABD → EF
ABD → I
Not in 2NF







Transitive FD : Not possible.

A → B and B → C then A → C is a transitive FD. (B → A) or (C → A)

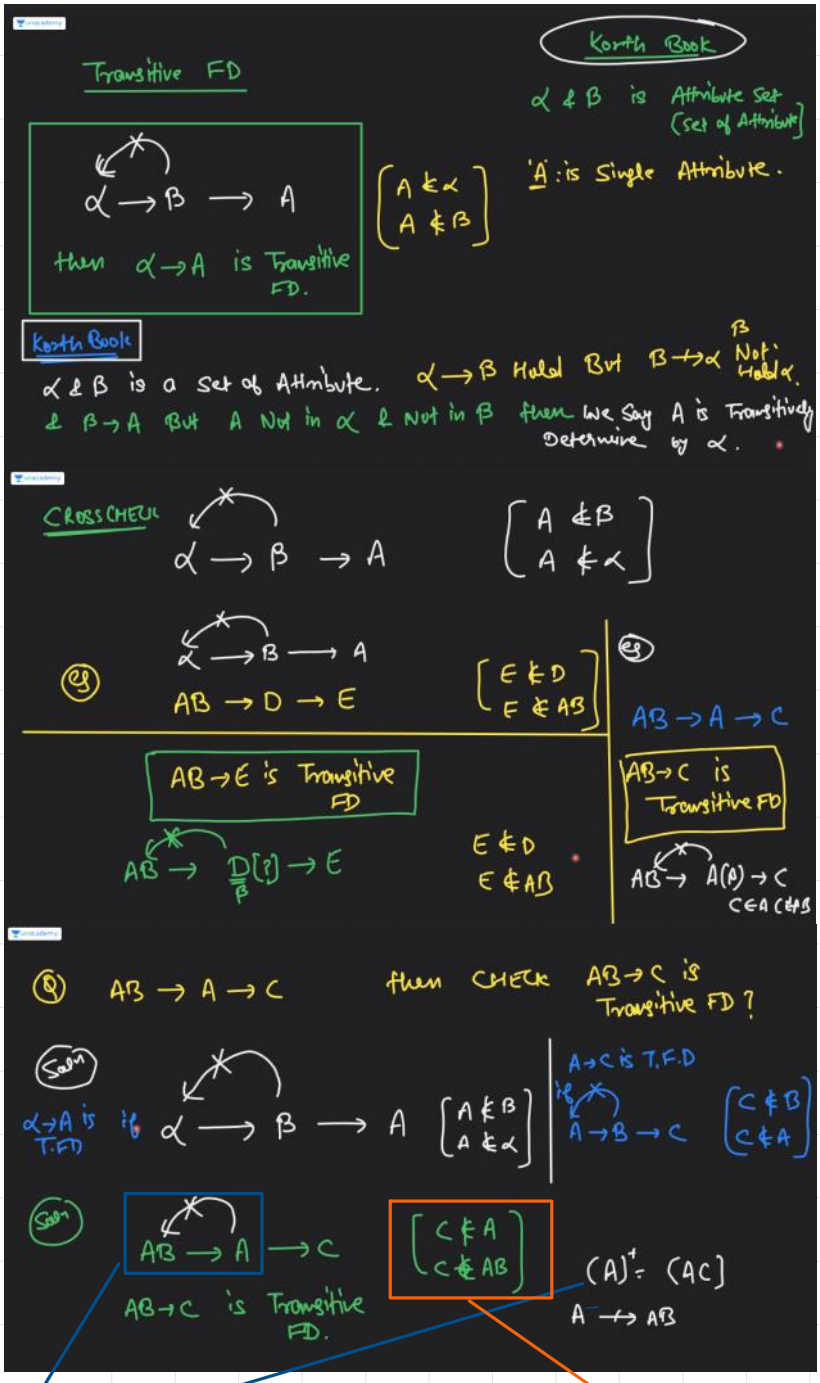
(OR)

C is transitively determined by A

Then X → Z is a transitive FD.

(OR)

Z is transitively determined by X



I am not able to determine AB from A

C does not belong to A and also C does not belong to AB

Hence, i can say AB tends to C is my transitive FD.

Important points -

R(ABC) [A → C]

Check AB→C is a partial FD, transitive FD?

1. Partial FD:

AB→C

Candidate key : [AB]

$[A]^+ = AC$

So, AB→C is a partial FD.

2. Transitive FD.

AB→C

Candidate key : [AB]

A→C

AB→A→C

A cannot determine AB

C is not present in AB

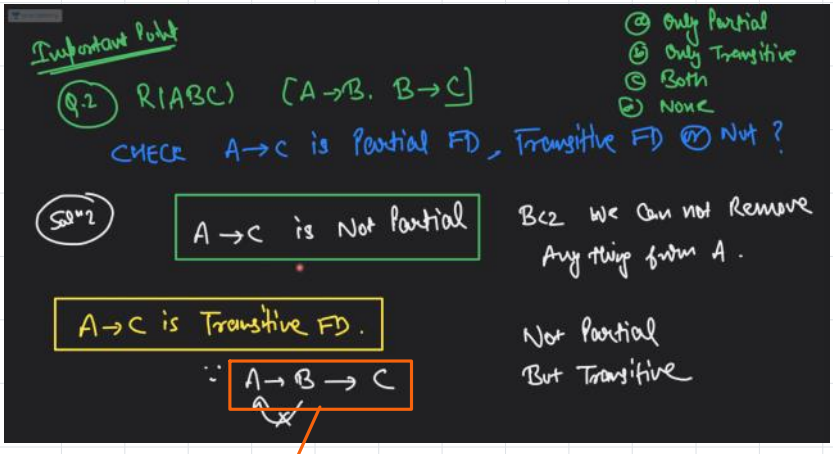
C is not present in A

Hence it is transitive.

In question A →C is given but B →C is not given

We are not getting C in the closure of B

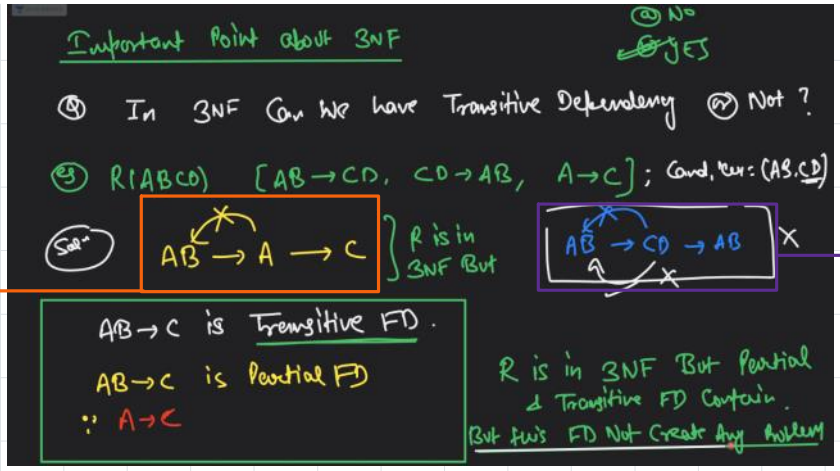
$[B]^+ = [B]$



Cannot determine A from B hence transitive

note :

1. Partial FD may be transitive FD.
2. Transitive FD may or may not be Partial FD.
3. In 3NF we can have partial and transitive FD but this FD does not create any problem.



We cannot determine AB from A.
AB to A because of trivial FD.

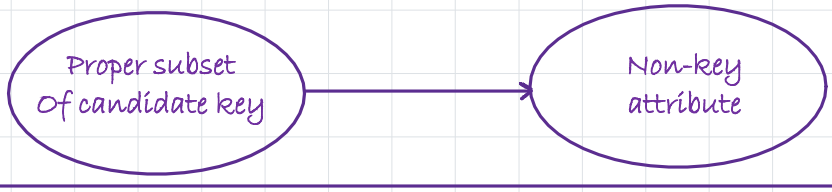
Wrong case because both are candidate keys.

3. In 3NF we can have partial and transitive FD
but this FD does not create any problem because the transitively dependent attribute is a key or prime attribute.
R not in 2NF :
It creates problem when transitively dependent attribute is non-key or non-prime attribute.

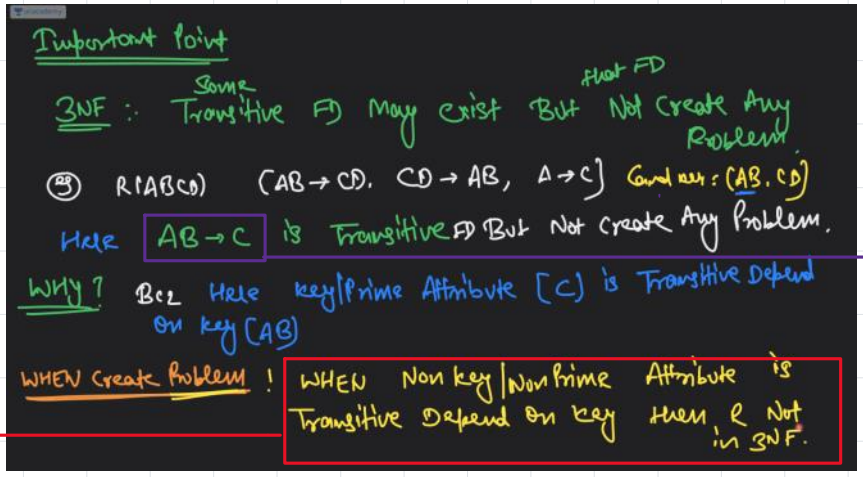
C is transitively dependent upon AB but it is in 3NF/2NF because C is a prime attribute.

2NF : Partial FD exist but that FD does not create any problem.
Due to this case only partial FD create problems, hence it cause violation of 2NF

2NF : Partial FD exist but that FD does not create any problem.
Due to this case only partial FD create problems, hence it cause violation of 2NF



4. in 3NF some transitive FD may exist but that FD does not create any problem.

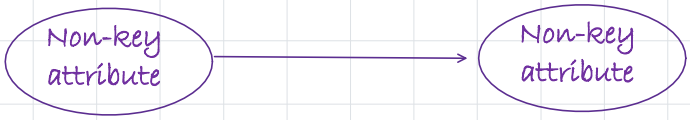


C is transitively dependent upon AB but it is in 3NF because C is a prime attribute.

3NF : Some transitive FD exist but that FD does not create any problem.
Due to this case only partial FD create problems, hence it cause violation of 3NF

3NF [third normal form]: A relation schema R is in 3Nf only if

(1) R is in 2NF
(2) R should not contain this case

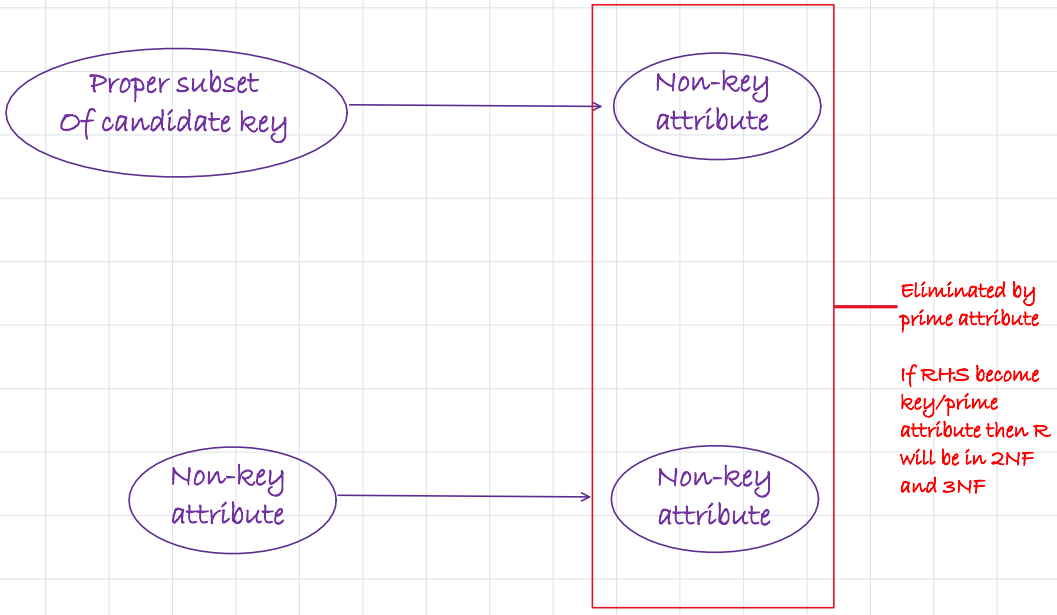


This cause violation of 3NF

A relation schema R is in 3NF if every X → Y non trivial FD satisfy the following condition :
(A) X : super-key
(OR)
(B) Y : key or prime attribute

CASE 1:

First candidate key is a super-key



R is In 3NF if it satisfies 2NF and no non-prime attribute of R is transitively dependent on the primary key.

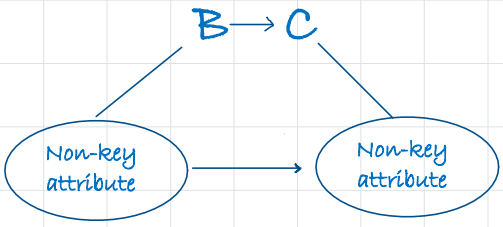
3NF

1. R(ABCD) [AB →CD, D →A]
Candidate key = [AB, DB]
Prime attribute : [A,B,D]
Non-prime attribute : [C]
AB→CD (AB is a super-key)
D→A (A is a prime attribute)
Hence, R is in 3NF.

Not in 3NF

1. R(ABC) [A →B, B →C]

Candidate key = [A]
Prime attribute : [A]
Non-prime attribute : [B, C]



Hence, R is not in 3NF.

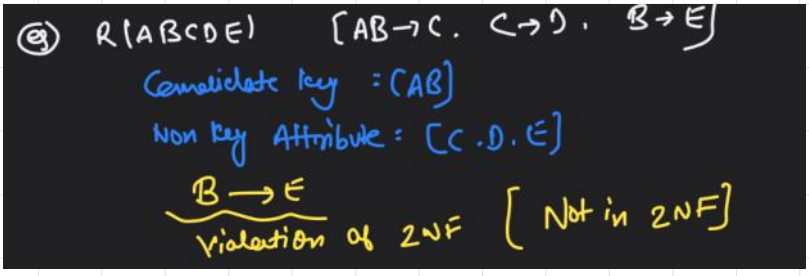
BCNF [Boyce Codd normal form]: A relation schema R is in BCNF only if whenever a non-trivial functional dependency
X⟶A holds in R, then X is a super-key of R.

Every X →Y must satisfy the following condition :
X: super-key

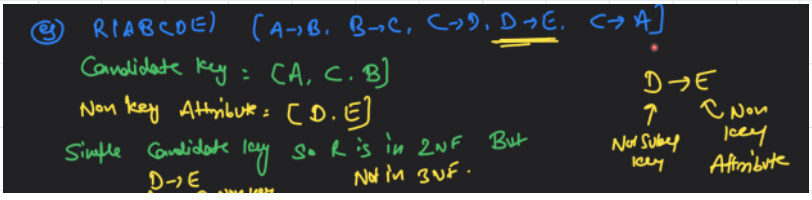
2NF checking	3NF checking	BCNF checking
Proper subset of candidate key → Non-key attribute This cause violation of 2NF	A relation schema R is in 3NF if every X →y non trivial FD satisfy the following condition : (A) X : super-key (OR) (B) Y : key or prime attribute	Every X→Y must satisfy the following condition : X: super-key

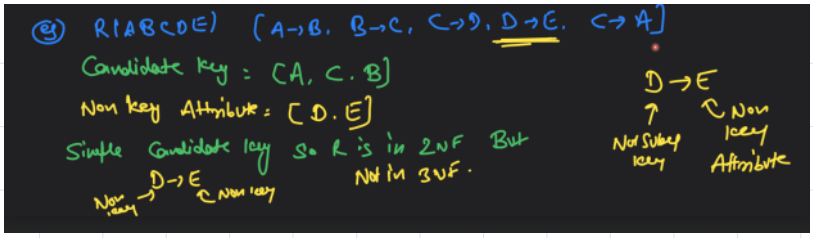
Important points about normal form -

1. If a relation R has only one candidate key then R always in [1NF] but may or may not be in 2NF, 3NF, BCNF

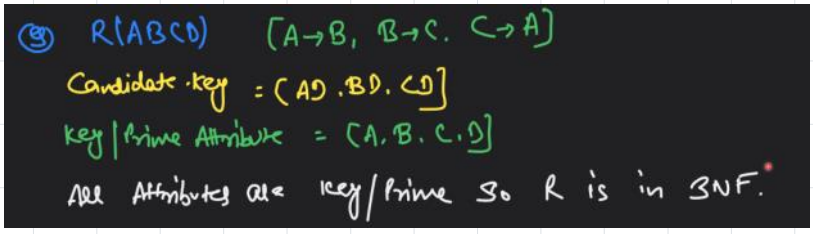


2. If in relation R, all candidate keys are simple candidate keys (no composite keys) then relation R is always in 2NF but may or may not be in 3NF or BCNF

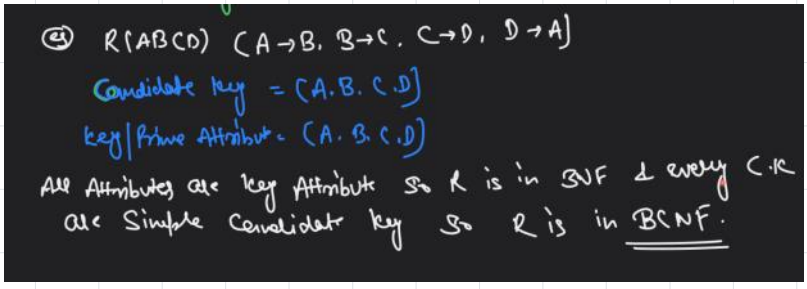




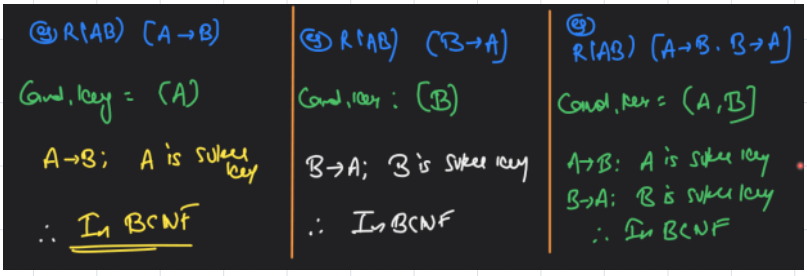
3. If in relation R, all attribute are key or prime attribute then relation R is always in 3NF, 2NF but may or may not be in BCNF.



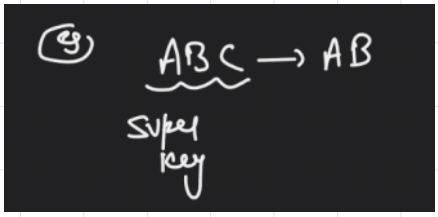
4. If R is in 3NF(all attribute must be key attribute) and all the candidate keys are simple candidate keys then R is always in BCNF.



5. Binary relation (relation with 2 attribute) always is in BCNF.



6. A relation with no non-trivial FD (trivial FD) is in BCNF.



Functional dependency that does not create redundancy

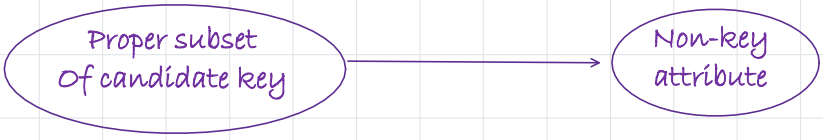
1. Trivial FD does not create redundancy.
2. Non-trivial FD X → Y in which X is super-key does not create redundancy.

Design goal	1NF	2NF	3NF	BCNF
0% Redundancy	✗	✗	✗	✓
Lossless Join	✓	✓	✓	✓
Dependency Preserving	✓	✓	✓	May/ May not

note :
1. BCNF has 0% redundancy but suffers from multivalued functional dependencies.[X →→Y]
2. In BCNF dependency may or may not be possible to decompose.

topic : normal form decompositions.

2NF [second normal form]: A relation schema R is in 2NF if every non-prime attribute 'A' in 'R' is fully functionally dependent on the primary key of R.



This cause violation of 2NF

2NF Decomposition :

Step one : check 2NF and take the closure of the FD that is violating 2NF and make a separate table.

Step two : write the main relation R and cut the attributes that we are determining from the closures of the FD that is violating 2NF.

Step three : write down all the remaining attributes separately.

Step four : Make FD's and check for lossy, lossless and dependency.

Q.1 R(ABCDEFGH) [AB →C, C →D, B →E, E →FG, G →H]
Candidate key : [AB]

Non-key attribute : [C,D,E,F,G,H]

Check 2NF?

$B \rightarrow E$

Not in 2NF.

Partial FD = AB ~~→~~ E

2NF Decomposition

Step one : check 2NF and take the closure of the FD that is violating 2NF and make a separate table(relations).

$[B]^+ = [BEFGH]$ BEFGH

Super-key of relation

Step two : write the main relation R and cut the attributes that we are determining from the closures of the FD that is violating 2NF.

R(ABCDEFGH) BEFGH

R(ABCDEFGH)

Step three : write down all the remaining attributes separately.

New relation

R_1(ABCD)

AB C

C ~~→D~~

No violation.

Super Key of relation

R_2(BEFGH)

B E

E →FG

G →H

No violation.

Step four : Make FD's and check for lossy, lossless and dependency

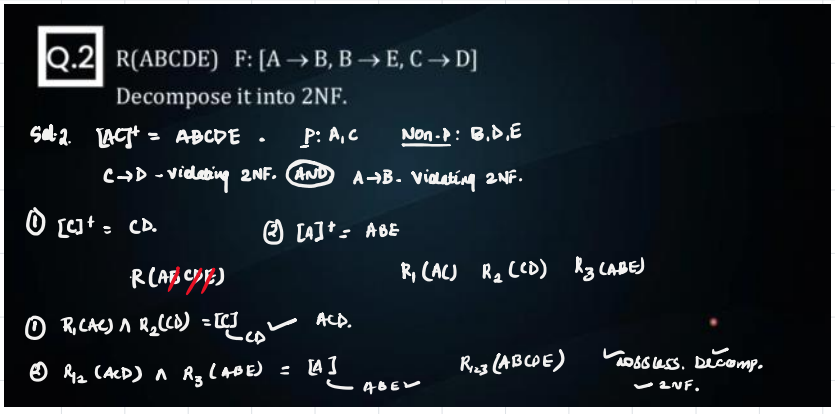
R_1(ABCD) R_2(BEFGH) = R_{12}(ABCDEFGH)

R_2(ABCD) ∪ R_2(BEFGH) = [B]

$[B]^+ = [BEF\cancel{A}GH]$

Super-key of R_2

Hence, lossless.



Q.3 R (ABCDEFGHIJ) {AB→C, BD→EF, AD→GH, A→I, H→J}

Sol:3. C.K. - $[ABD]^+$ = ABCDEFGHIJ. P: A, B, D Non-P: C, E, F, G, H, I, J

(1) AB→C: ABC; ABC, A→I

(2) BD→EF: BDEF; ABDEFGHIJ; ABD

(3) AD→GH: ADGHIJ; ADGHIJ, A→I; AD→GH, H→J, A→I

(4) A→I: AI

ABC, ABD, ADGHIJ, AI, BDEF

- R_1 (ABC) ∪ R_2(ABD) = R_{12}(ABCD)
  R_1 (ABC) ∧ R_2(ABD) = $[AB]^+$ = ABC ✓
- R_{12} (ABCD) ∪ R_3(ADGHIJ) = R_{123} (ABCDGHIJ)
  R_{12} (ABCD) ∧ R_3(ADGHIJ) = $[AD]^+$ = ADGHIJ ✓
- R_{123} (ABCDGHIJ) ∪ R_4(AI) = R_{1234} (ABCDGHIJ)
  R_{123} (ABCDGHIJ) ∧ R_4(AI) = $[A]^+$ = AI ✓

- R_{1234} (ABCDGHIJ) ∪ R_5(BDEF) = R_{12345} (ABCDEFGHIJ)
  R_{1234} (ABCDGHIJ) ∪ R_5(BDEF) = $[BD]^+$ = BDEF

Lossless + 2NF + Decomposition ✓.

Q.4 R(ABCDEF) {AB → C, C → D, B → EF}

Sol:4 $[AB]^+$ = ABCDEF. · P: A, B | · Non-P: C, D, E, F

B → EF, $[B]^+$ = BEF

R (ABCDEF) . R_1 (BEF) ∪ R_2 (ABCD) = R_{12} (ABCDEF)
R_1 (BEF) ∧ R_2 (ABCD) = $[B]^+$ = BEF ✓

☆ - 2NF + Decomposed + Lossless.

Q.5 R(ABCDEFGH) {AB → C, C → D, B → E, E → F, A → GH}

Sol:5 $[AB]^+$ = ABCDEFGH. P: A, B Non-P: C, D, E, F, G, H

(1) B→E: BE; ABCDEFGH; ABCDF

(2) A→GH: AGH

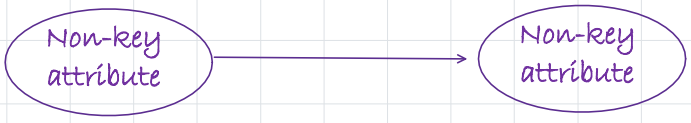
R_1(BE) R_2 (AGH) R_3(ABCDF)

- R_2(AGH) ∪ R_3(ABCDF) = R_{23} (ABCDFGH)
  R_2(AGH) ∧ R_3(ABCDF) = $[A]^+$ = AGH ✓
- R_{23} (ABCDFGH) ∪ R_1(BE) = R_{123} (ABCDEFGH)
  R_{23} (ABCDFGH) ∧ R_1(BE) = $[B]^+$ = BE ✓

Lossless + 2NF + decomposition.

3NF [third normal form]: A relation schema R is in 3NF only if

(1) R is in 2NF
(2) R should not contain this case



This cause violation of 3NF

A relation schema R is in 3NF if every X → non trivial FD satisfy the following condition :
(A) X : super-key
(OR)
(B) Y : key or prime attribute

If non-key attribute is Transitively dependent on prime attribute then also it is not in 3NF.

3NF Decomposition:
Step one : check 2NF and take the closure of the FD that is violating 2NF and make a separate table(relations).

Step two : write the main relation R and cut the attributes that we are determining from the closures of the FD that is violating 2NF.

Step three : write down all the remaining attributes separately and make the relation.

Step four : Check 3NF and write down the FD's(non-key to non-key FD)that are violating 3NF.

Step five : write the main relation R that consists violated FD and cut the attributes that we are determining from the closures of the FD that is violating 3NF.

Step six : Make FD's and check for lossy, lossless and dependency.

Q1 R(ABCDEFGHIJ) [AB → C, BD → EF, AD → GH, A → I, H → J]
Candidate key : [ABD]
Non-key attribute : [C,E,F,G,H]

2NF Decomposition
Step one : check 2NF and take the closure of the FD that is violating 2NF and make a separate table(relations).

1. A → I
$[A]^+ = [AI]$
[ADGHJ] [AI]

2. AD → GH
$[AD]^+ = [AIDGHJ]$
[AIDGHJ] [AI]

3. BD → EF
$[BD]^+ = [BDEF]$

4. AB → C
$[AB]^+ = [AIBC]$
[AIBC] [ABC] [AI]

Super-key of relation

Step two : write the main relation R and cut the attributes that we are determining from the closures of the FD that is violating 2NF.
R(ABCDEFGHIJ) R_1(ADGHJ) R_2(BDEF) R_3(ABC) R_4(AI)

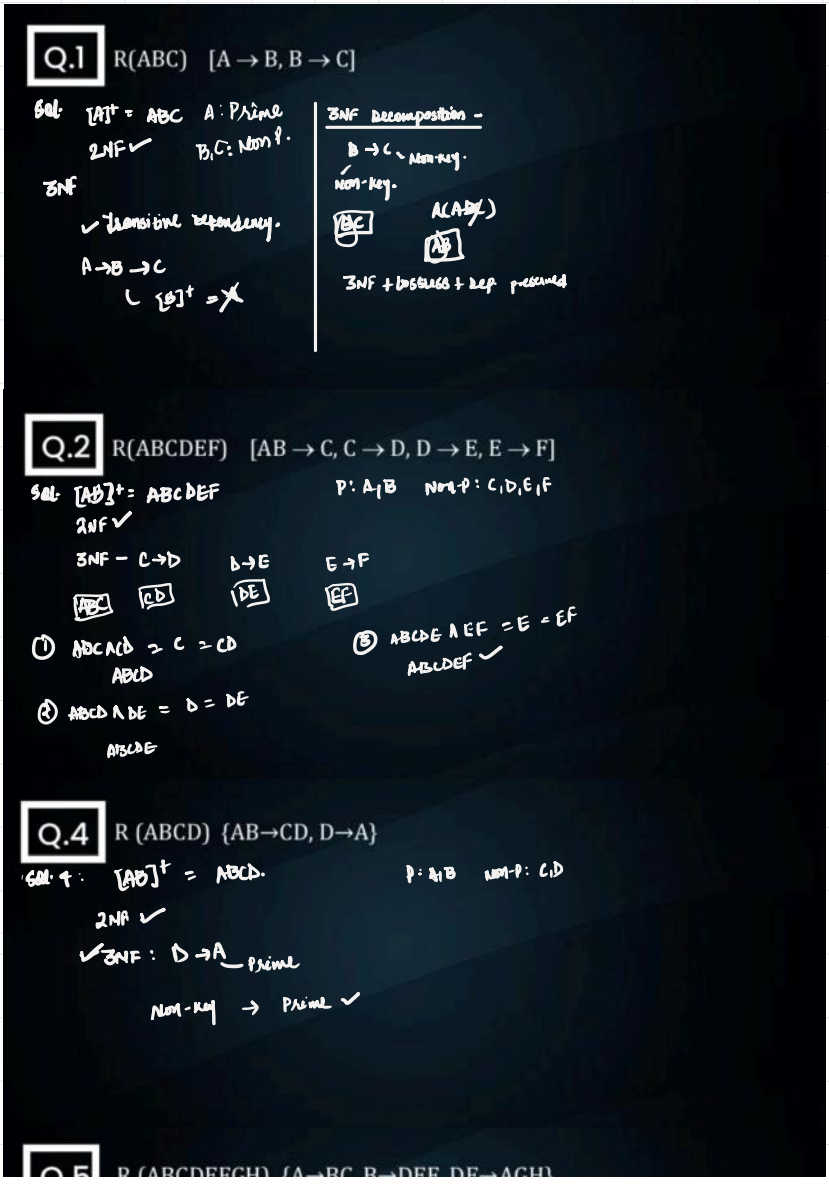
R(ABCDEFGHIJ)
R_5(ABD)

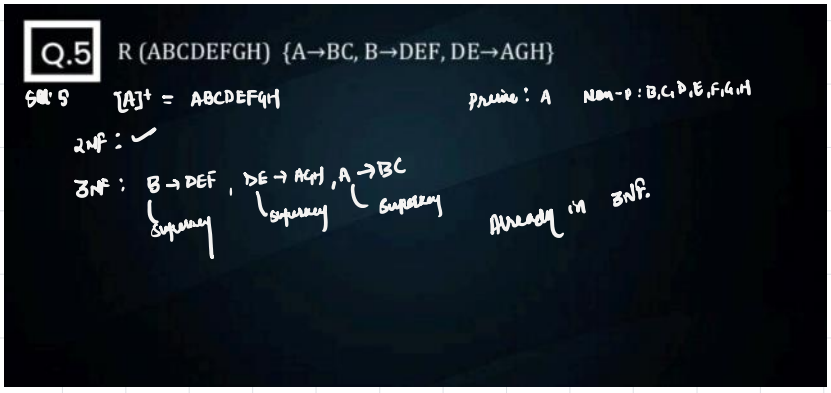
Step three : write down all the remaining attributes separately.
New relation
$R_1(ADGHJ)$ $R_2(BDEF)$ $R_3(ABC)$ $R_4(AI)$ $R_5(ABD)$
Step four : Check 3NF and write down the FD's (non-key to non-key FD) that are violating 3NF.
Check 3NF?
$H \rightarrow J$ (Violating)
$[H]^+ = [HJ]$

Step five : write the main relation R that consists violated FD and cut the attributes that we are determining from the closures of the FD that is violating 3NF.
[HJ] — Super-key of relation

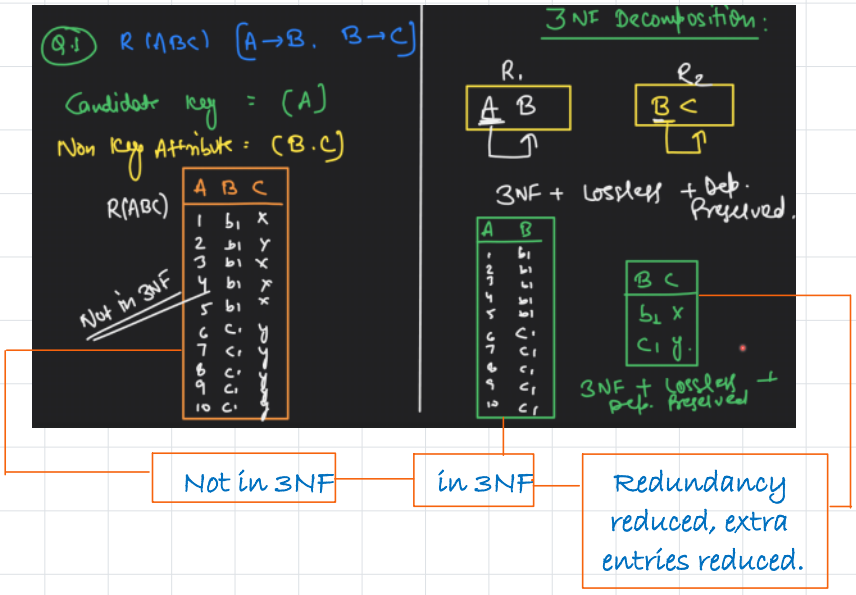
$R_1(ADGHJ)$ $R_2(BDEF)$ $R_3(ABC)$ $R_4(AI)$ $R_5(ABD)$
$R_1(ADGH)$ $R_2(BDEF)$ $R_3(ABC)$ $R_4(AI)$ $R_5(ABD)$ $R_6(HJ)$

Step six : Make FD's and check for lossy, lossless and dependency.
Lossless + 2NF + 3NF + Dependency preserved.





note : In removing redundancy, Tables become more but entries become less.
- Redundancy reduces because of the removal of extra and repeating values



Not in 3NF → in 3NF → Redundancy reduced, extra entries reduced.

BCNF [Boyce Codd normal form]: A relation schema R is in BCNF only if whenever a non-trivial functional dependency
X → A holds in R, then X is a super-key of R.

Every X → Y must satisfy the following condition :
X: super-key

BCNF Decomposition : for every non trivial FD X → Y
X : should be super-key.

BCNF Decomposition:
Step one : check BCNF and write down the FD that is violating BCNF. X must be super-key.

Step two : write the main relation R and write the FD's and then start checking one by one, if you can turn FD into relation or not. Cut down the same attributes of main relation of the FD that you created into relation R and then for next FD repeat the procedure, if you can't turn that FD in relation R then delete it.

Step three : Make FD's and check for lossy, lossless and dependency.

Q.1 R(ABCDE) [A →B, B →C, C →D, D →E]
Candidate key = [A]
Non-key attribute = [B,C,D,E]

Step one : check BCNF and write down the FD that is violating BCNF. X must be super-key.
B → C
C → D
D → E

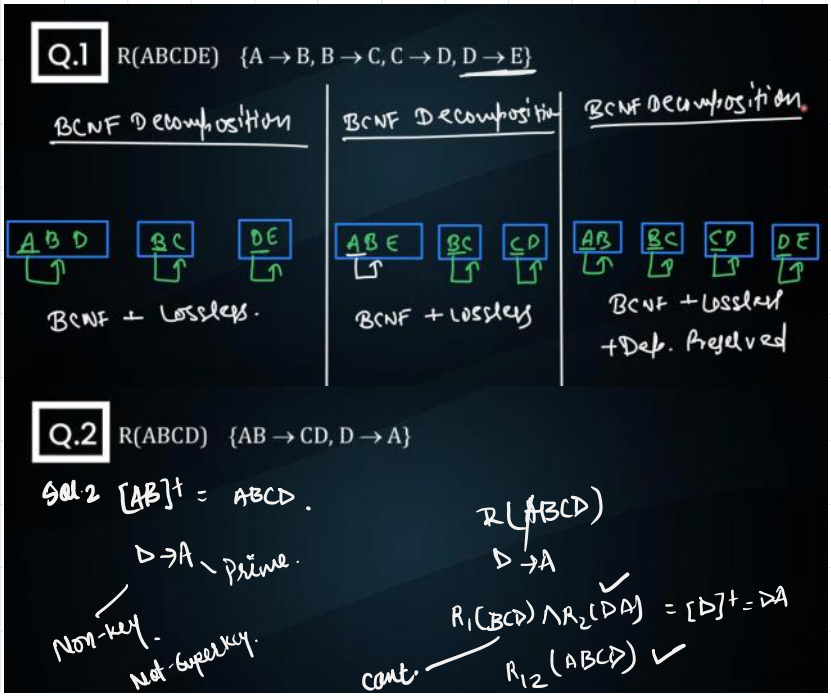
Step two : write the main relation R and write the FD's and then start checking one by one, if you can turn FD into relation or not. Cut down the same attributes of main relation of the FD that you created into relation R and then for next FD repeat the procedure, if you can't turn that FD in relation R then delete it.
R_1(ABCDE)
1. B → C
R_2[BC]

R_1(ABCDE)
2. C → D
C is not available.

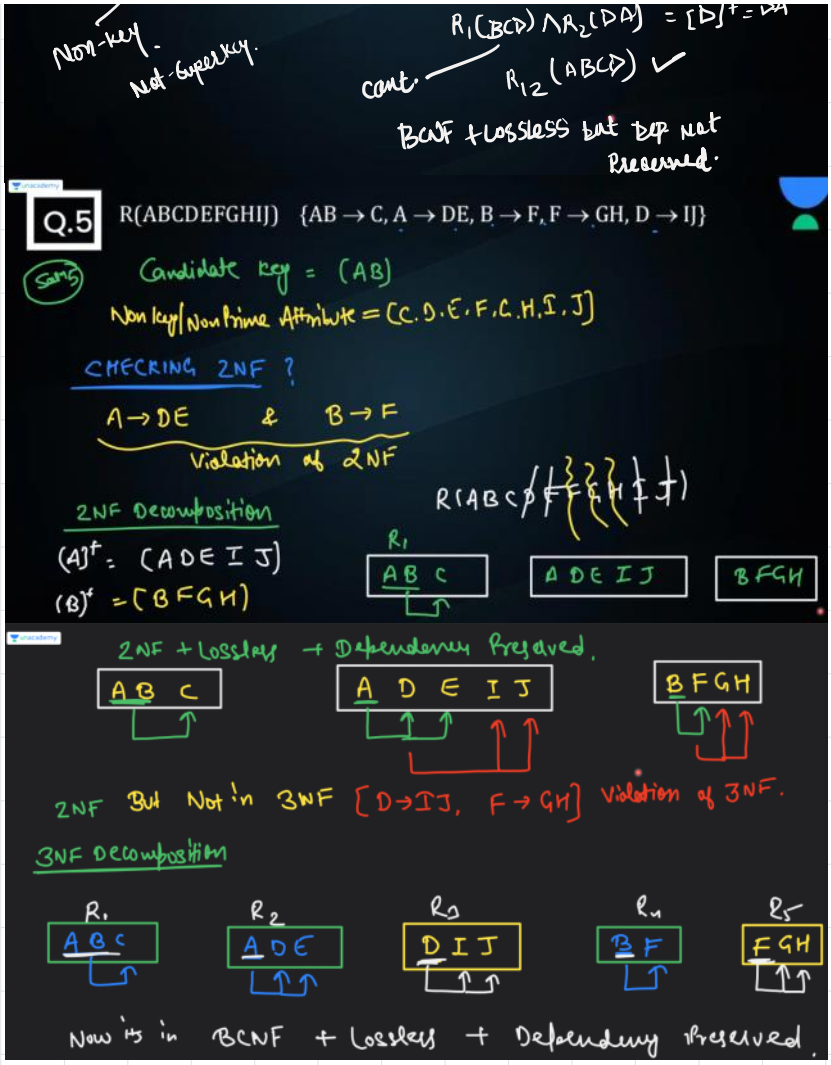
R_1(ABCDE)
3. D → E
R_3[DE]

R_1[ABD] R_2[BC] R_3[DE]

Step three : Make FD's and check for lossy, lossless and dependency.



note :
Dependency not preserved because Candidate key must be



Candidate key must be present in any relation. AB is not present in any relation.

Transaction and concurrency control

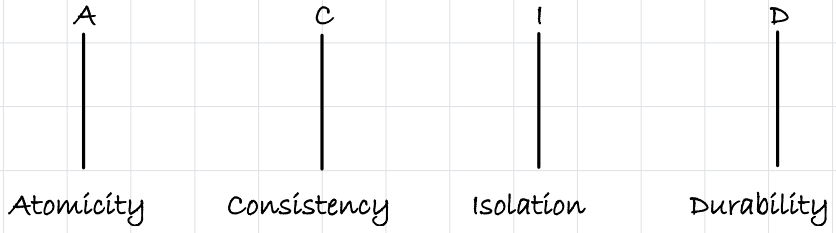
TOPIC : Transaction

Transaction : Transaction is a set of logically related operations (Read, Write) that perform as a unit of work.

```
Read (A)
A = A - 1000
Write (A)
Read (B)
B = B + 1000
Write (B)
```

Transferring rs.1000 from A to B is considered as a one transaction

To maintain the integrity, transaction must satisfy ACID properties -



1. Atomicity: Either execute all the operations of the transaction successfully or none of them.
[Full or none]

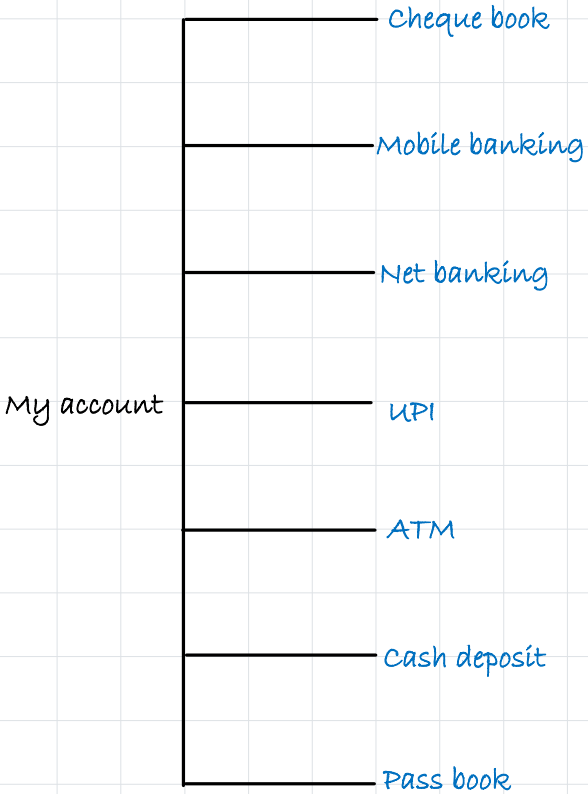
Reasons of transaction failure :
1. Power failure
2. Hardware crash
3. Software crash
4. System or network error

Due to any of these reasons if atomicity is getting failed then recovery management component are there
- If transaction is failed before commit then recovery management component Rollback (undo all modifications) of the transaction with the help of logs.
- Logs : log contain all the activities of transaction.

2. Consistency : Before and after the transaction database must be consistent.

Before		After
A: 4000	500	A: 3500
+B: 2000	A → B	+B: 2500
6000		6000

3. Isolation : Multiple transaction may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.



Multiple transactions occurring at same time but no intermediate results until the process is not completely finished. Intermediate results are hidden from each other. Transactions executing in isolated manner.

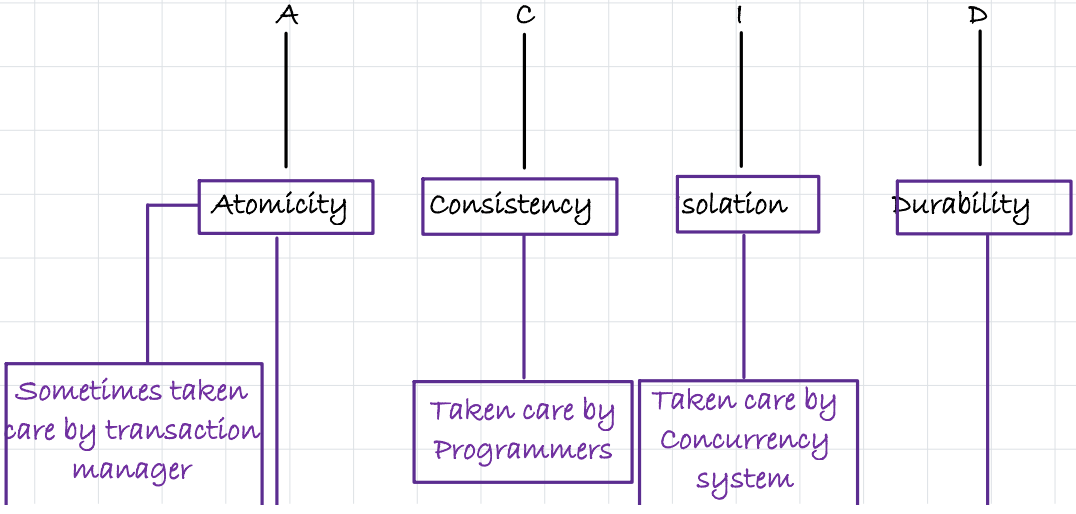
4. Durability : Any change in the database must persist for a long period of time

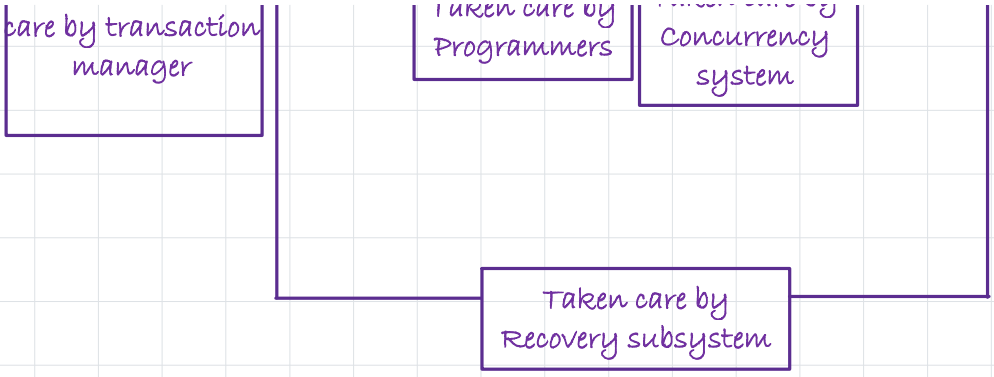
(OR)

Database must be able to recover under any case of failure.

(OR)

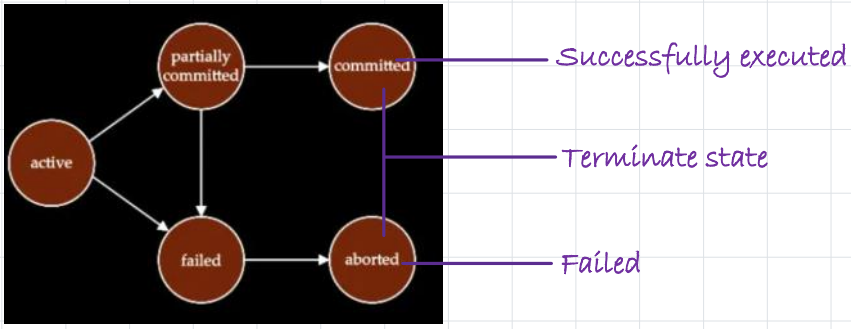
Once transaction is completed successfully, the change it has made to the database must persist, even under any case of failure.



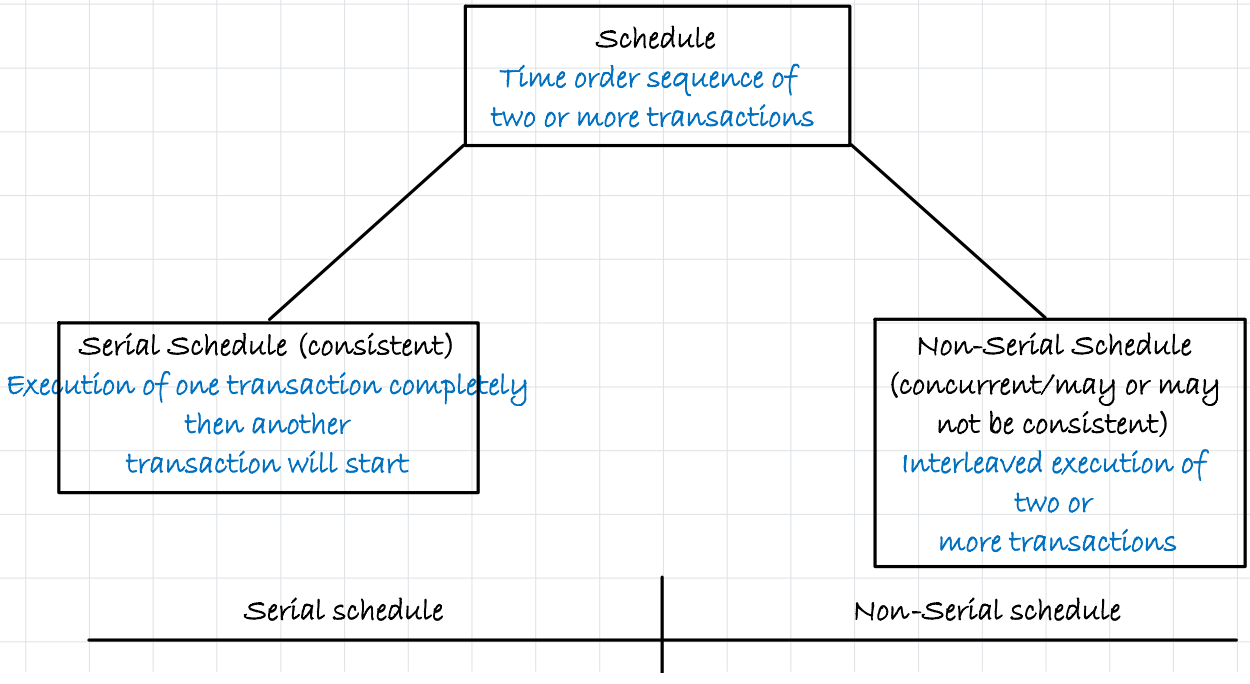


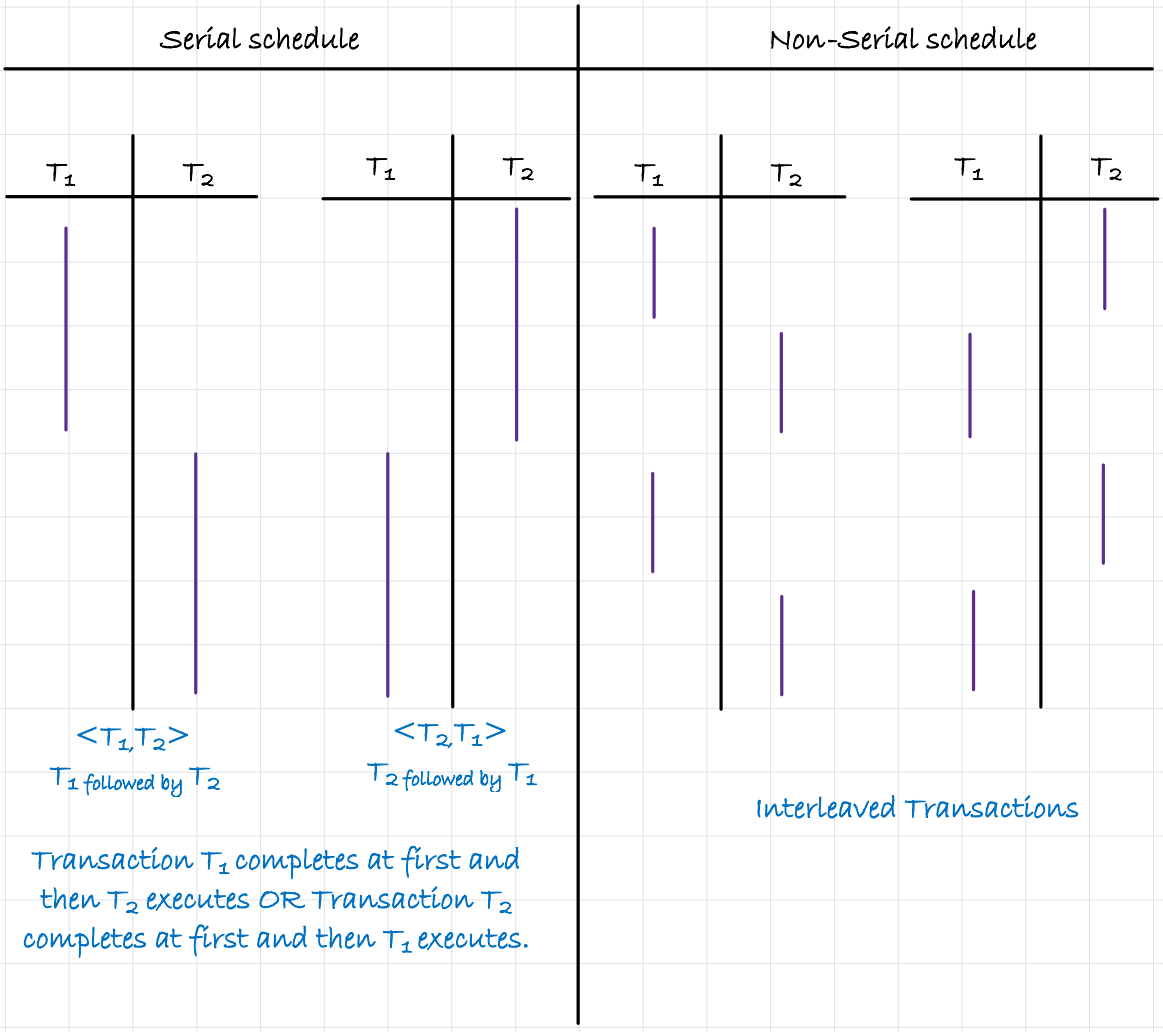
Operations :

1. Read (A): Accessing the data item 'A' from the database to main memory
2. Write (A) : Update the data item 'A'
3. Commit : Commit indicate successfully completing of the transaction
   (OR)
   Indicate transaction execute successfully
4. Abort/Roll : when transaction fails.



TOPIC : Schedule and its types.





1. Non-serial schedule : Total schedule - Serial Schedule
2. Total concurrent schedule : $\frac{(n_1+n_2)}{[(n_1)!][(n_2)!]}$

3. for n transactions : n! serial schedule.

EG. If 2 transactions then '2' Serial schedule

1. <T2,T1>
   T2 followed by T1

2. <T1,T2>
   T1 followed by T2

EG. If 3 transactions then '6' Serial schedule

If there are 3 Transaction
then 3! ⇒ 6 Serial Schedule.

$\langle T_1, T_2, T_3 \rangle$
$\langle T_1, T_3, T_2 \rangle$
$\langle T_2, T_1, T_3 \rangle$
$\langle T_2, T_3, T_1 \rangle$
$\langle T_3, T_1, T_2 \rangle$
$\langle T_3, T_2, T_1 \rangle$

} 6 Serial Schedule

Schedules

- Schedule: a sequences of instructions that specify the chronological order in which instructions of concurrent transactions are executed.
  - A schedule for a set of transactions must consist of all instructions of those transactions
  - Must preserve the order in which the instructions appear in each individual transaction.
- A transaction that successfully completes its execution will have a commit instructions as the last statement
  - By default transaction assumed to execute commit instruction as its last step
- A transaction that fails to successfully complete its execution will have an abort instruction as the last statement.

Successfully : commit

Fail : abort

Types:

(A) Serial schedules:

T_1 transaction

A $\xrightarrow{100rs.}$ B

Schedule 1

T_1	T_2
read (A)	
A: = A – 100	
write (A)	
read (B)	
B: = B + 100	
write (B)	
commit	
	read (A)
	temp := A * 0.1
	A := A – temp
	write (A)
	read (B)
	B := B + temp
	write (B)
	Commit

$S_1 < T_1 T_2 >$

Serial schedule in which T_1 is followed by T_2:

T_2 transaction

A $\xrightarrow{10\%}$ B

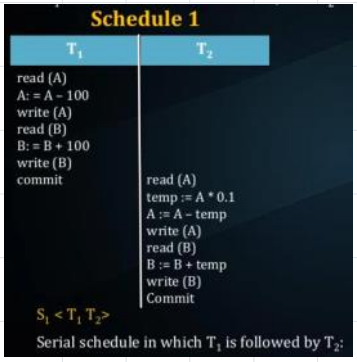
Temporary Variable.

2 Transaction = 2 Serial schedule.

1. $\langle T_2, T_1 \rangle$
   T_2 followed by T_1

2. $\langle T_1, T_2 \rangle$
   T_1 followed by T_2

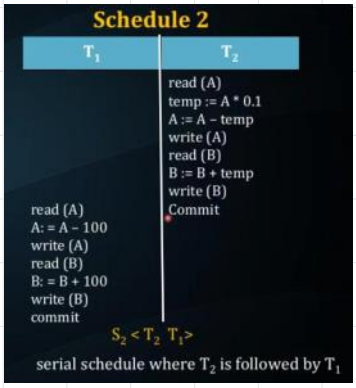
Assume A : 2000
B : 3000
5000



consistent

T_1	T_2
Read (A)	
2000	
A: 2000-100	
Write (A)	
1900	
Read (B)	
3000	
B: 3000+100	
Write (B)	
3100	
commit	
	Read (A)
	1900
	Temp: 1900 x 0.1
	A: 1900 - 190
	Write (A)
	1710
	Read (B)
	3100
	B: 3100+190
	Write (B)
	3290
	commit

5000



consistent

T_1	T_2
	Read (A)
	2000
	Temp: 2000 x 0.1
	A: 2000 - 200
	Write (A)
	1800
	Read (B)
	300
	B: 3000+200
	Write (B)
	3200
	commit
Read (A)	
1800	
A: 1800-100	
Write (A)	
1700	
Read (B)	
3200	
B: 3200+100	
Write (B)	
3300	
commit	

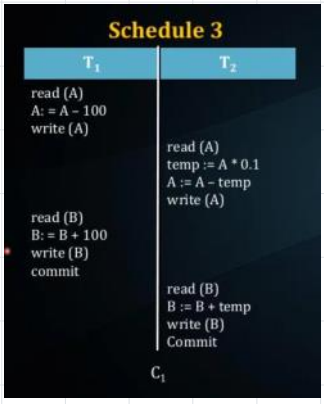
5000

Note:

Serial schedules (n!) are always consistent (correct result)

(B) Non-Serial schedules:
[Concurrent execution]

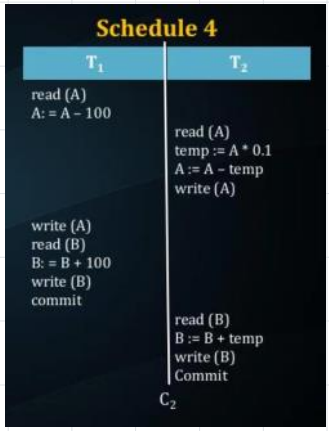
Assume A : 2000
B : 3000
5000



consistent

T_1	T_2
Read (A)	
2000	
A: 2000-100	
Write (A)	
1900	
	Read (A)
	1900
	Temp: 1900 x 0.1
	A: 1900 - 190
	Write (A)
	1710
Read (B)	
3000	
B: 3000+100	
Write (B)	
3100	
commit	
	Read (B)
	3100
	B: 3100+190
	Write (B)
	3290
	commit

5000



Inconsistent

T_1	T_2
Read (A)	
2000	
A: 2000-100	
	Read (A)
	2000
	Temp: 2000 x 0.1
	A: 2000-200
	Write (A)
	1800
Write (A)	
1900	
Read (B)	
3000	
B: 3000+100	
Write (B)	
3100	
commit	
	Read (B)
	3100
	B: 3100+200
	Write (B)
	3300
	commit

No write operation

5200

Note:

Non-Serial schedules (concurrent execution) may or may not be consistent

But we prefer to execute (non-serial schedule) concurrent execution because

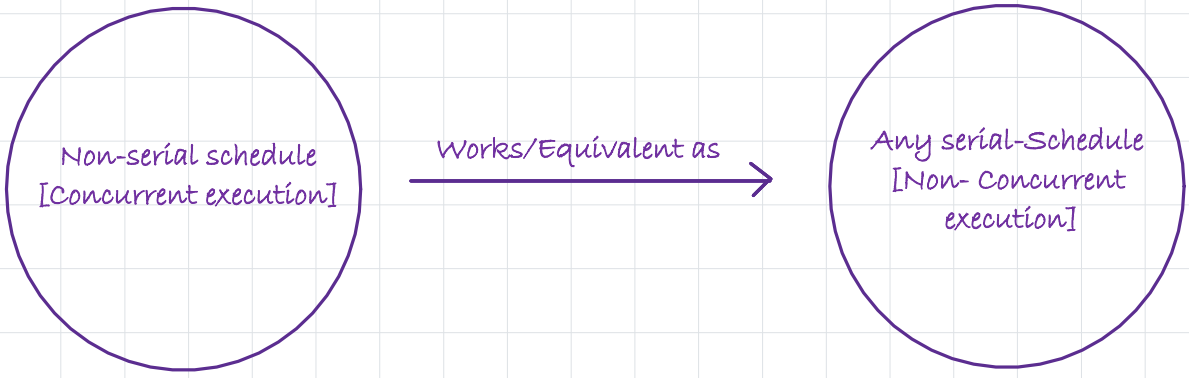
1. To enhance CPU utilization
2. To enhance throughput
3. Fast response
4. Reduce the waiting time
5. Effective utilization of resources

   And many more...

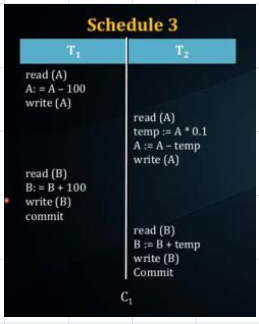
Because in serial schedule

1. Low CPU utilization
2. Low throughput
3. High waiting time
4. Low response
5. Less degree of concurrency

1. Serializable schedule : if a non-serial schedule (concurrent execution) has been executed and that could have same effect on database as a sschedule execute without any concurrent execution (serial schedule) is called serializable schedule.

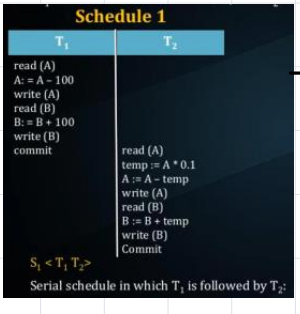


consistent



T_1	T_2
Read (A)	
2000	
A: 2000-100	
Write (A)	
1900	
	Read (A)
	1900
	Temp: 1900 x 0.1
	A: 1900 - 190
	Write (A)
	1710
Read (B)	
3000	
B: 3000+100	
Write (B)	
3100	
commit	
	Read (B)
	3100
	B: 3100+190
	Write (B)
	3290
	commit

Works/Equivalent as



consistent

T_1	T_2
Read (A)	
2000	
A: 2000-100	
Write (A)	
1900	
Read (B)	
3000	
B: 3000+100	
Write (B)	
3100	
commit	
	Read (A)
	1900
	Temp: 1900 x 0.1
	A: 1900 - 190
	Write (A)
	1710
	Read (B)
	3100
	B: 3100+190
	Write (B)
	3290
	commit

B: 3100+190
Write (B)
3290
commit

5000

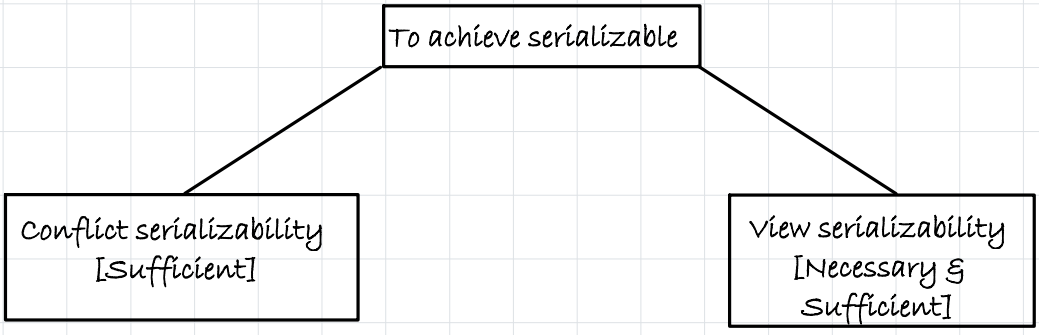
B: 3100+190
Write (B)
3290
commit

5000

Note:

Serializable schedule are always consistent and this is called serializability.

A schedule is a serializable schedule if it is equivalent to serial schedule.



Note:

A schedule is serializable either if it is conflict (or) view (or) both

(A) Conflict serializability - four methods to do

(i) Basic concept [swapping of non-conflict instruction]
(ii) Testing method [Precedence graph method]
(iii) Conflict operations pair
(iv) Conflict equivalent to any serial schedule

(A) Conflict instruction / operation : 2 transactions in which both are operating on same data item (A) hence, swapping not possible.

(RW)
(WR)
(WW)

T_i	T_j	
R(A)	W(A)	$i \neq j$ [Different transactions]
W(A)	R(A)	And same data item (A) and atleast
W(A)	W(A)	one write operation.

(B) Non-conflict instruction / operation : 2 transactions in which both are reading same data item (A) hence, swapping possible

T_i	T_j	
R(A)	R(A)	$i \neq j$ [Different transactions] And same data item (A).

(OR)

operating on different data item.

T_i		T_j
R(A)	-	W(B)
W(A)	-	R(C)
W(A)	-	W(B)

T_i and T_j operating on different data item (A) and (B)

Conflict serializable (only sufficient condition) : let us consider a schedule 'S' in which there are two consecutive instruction of T_i and T_j of transaction T_i and T_j respectively. [i=j]

1. If T_i and T_j refer to different data item : non-conflict instruction. [swapping possible]

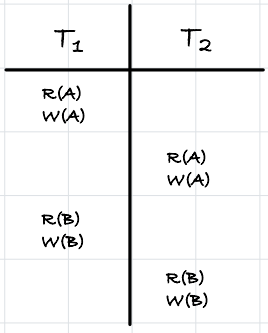
2. If T_i and T_j refer to same data item : Ti : Read(Q) and Tj : Read(Q) hence non-conflict [swapping possible]

3. Conflict operations / Instructions : Ti : Read(Q) and Tj : write(Q) [they conflict]
   [swapping not possible] Ti : Write(Q) and Tj : Read(Q) [they conflict]
   Ti : Write(Q) and Tj : write(Q) [they conflict]

Conflict serializable : If i can make any concurrent schedule (non-serial) into serial schedule (non-concurrent) with the help of series of swapping of non-conflict instructions then it will be conflict equivalent.

S : Given non series schedule S' : Any serial schedule
If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions then we can say S and S' are conflict equivalent.

(i) Basic concept : convert 'S' non serial schedule into any serial schedule S' pf S by swapping of non conflict instructions



C_4

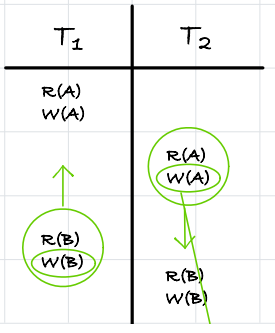
If 2 transactions then '2' Serial schedule

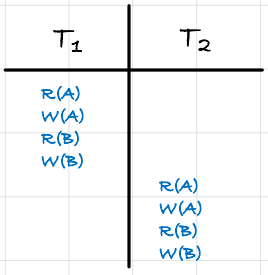
1. $<T_2,T_1>$ T_2 followed by T_1
2. $<T_1,T_2>$ T_1 followed by T_2

Task : try to convert C_4 into either

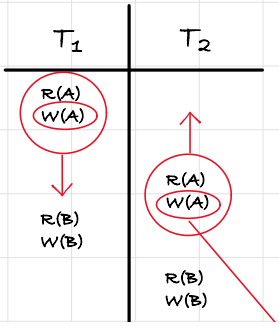
1. $<T_2,T_1>$ T_2 followed by T_1
   (OR)
2. $<T_1,T_2>$ T_1 followed by T_2

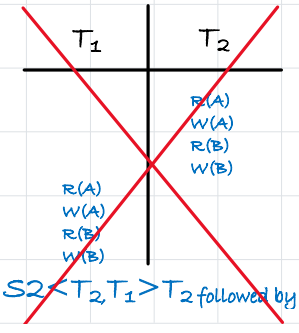
By swapping of non-conflicting instructions



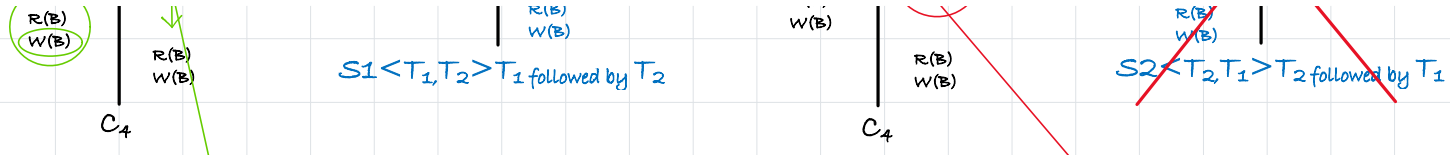


S1 $<T_1,T_2>$ T_1 followed by T_2





S2 $<T_2,T_1>$ T_2 followed by T_1



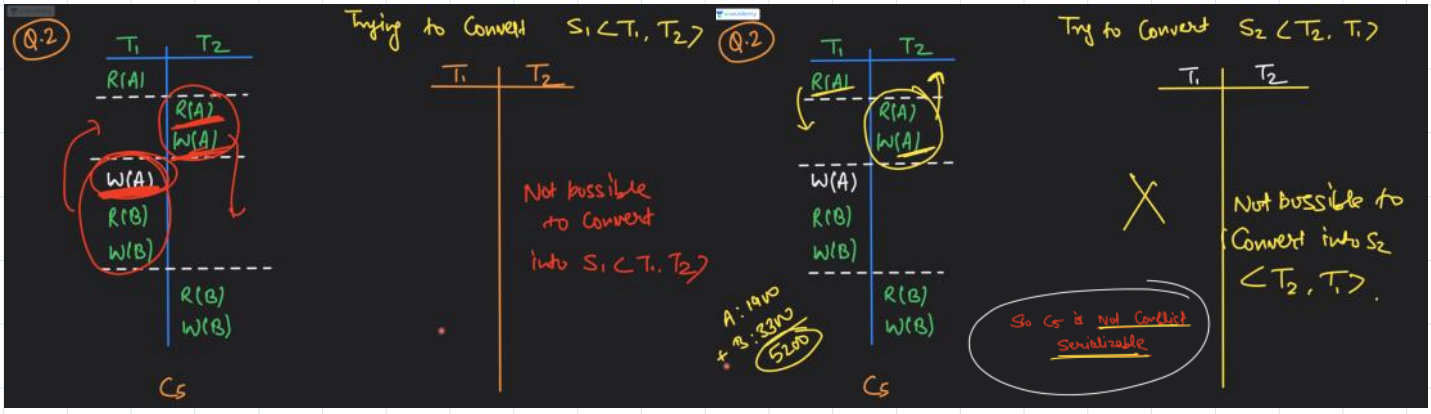
Non-conflict (different data item)

Conflict (writing on same data item)

$C_4 = S1 < T_1, T_2 > T_1$ followed by T_2

$C_4 \neq S2 < T_2, T_1 > T_2$ followed by T_1

C_4 is conflict serializable and it is equivalent to serial schedule $S1 < T_1, T_2 > T_1$ followed by T_2

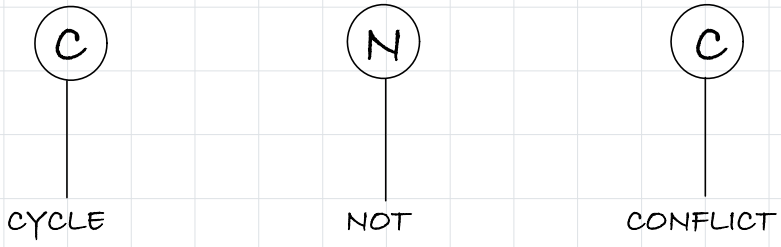


(ii) Testing for conflict serializability :
[Precedence graph method]

Directed graph -

G(V,E) V[vertex]: set of transaction

E[edge]: $T_i \longrightarrow T_j$ edge occur when any conflict instruction encounter. Construct a directed edge $T_i \longrightarrow T_j$ from for conflict operation $T_i \longrightarrow T_j$

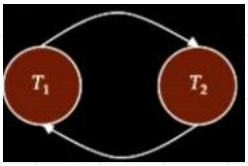


Note:

A schedule is conflict serializable if and only if its precedence graph is acyclic

Note:

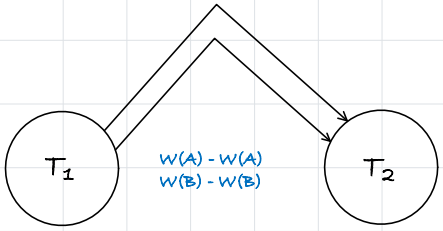
If predecence graph contain any one cycle then schedule is not conflict serializable



Example :

Q. S: $R_1(A)\ W_1(A)\ R_2(A)\ W_2(A)\ R_1(B)\ W_1(B)\ R_2(B)\ W_2(B)$

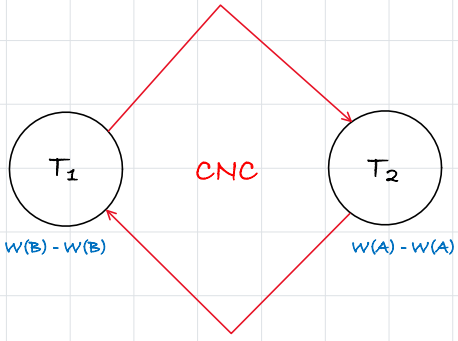
T_1	T_2
R(A)	
W(A)	
	R(A)
	W(A)
R(B)	
W(B)	
	R(B)
	W(B)



$<T_1, T_2> T_1$ followed by T_2

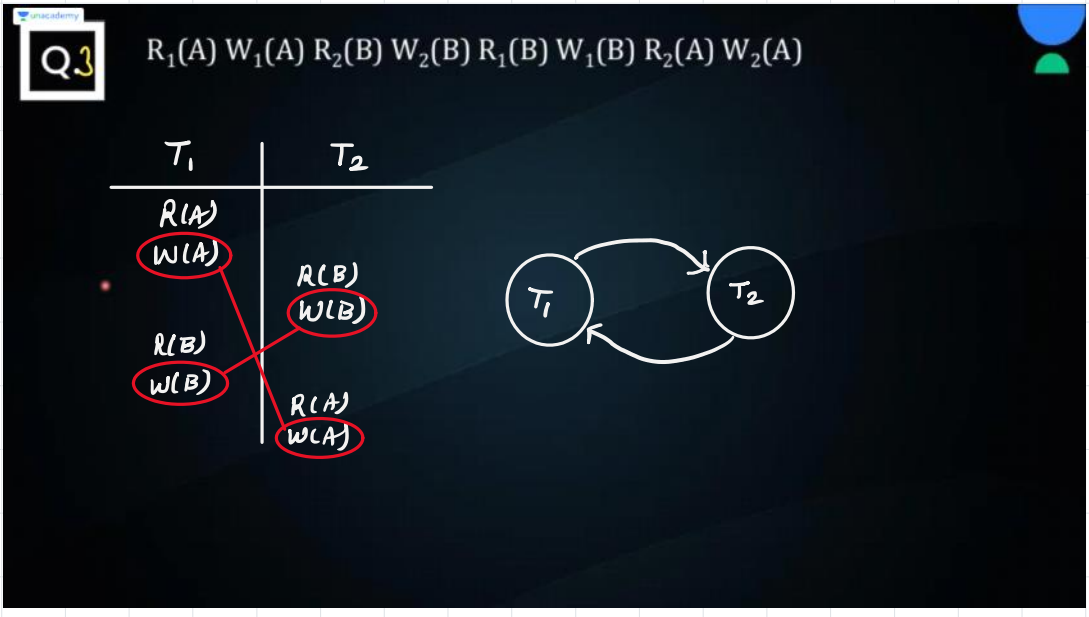
Q. S: $R_1(A)\ R_2(A)\ W_2(A)\ W_1(A)\ R_1(B)\ W_1(B)\ R_2(B)\ W_2(B)$

T_1	T_2
R(A)	
	R(A)
	W(A)
W(A)	
R(B)	
W(B)	
	R(B)
	W(B)



cyle not conflict serializable

Questions :



Q.1

Consider the following schedules involving two transactions. Which one of the following statements is TRUE?

S_1: $r_1(X); r_1(Y); r_2(X); r_2(Y); w_2(Y); w_1(X)$

S_2: $r_1(X); r_2(X); r_2(Y); W_2(Y); r_1(Y); w_1(X)$

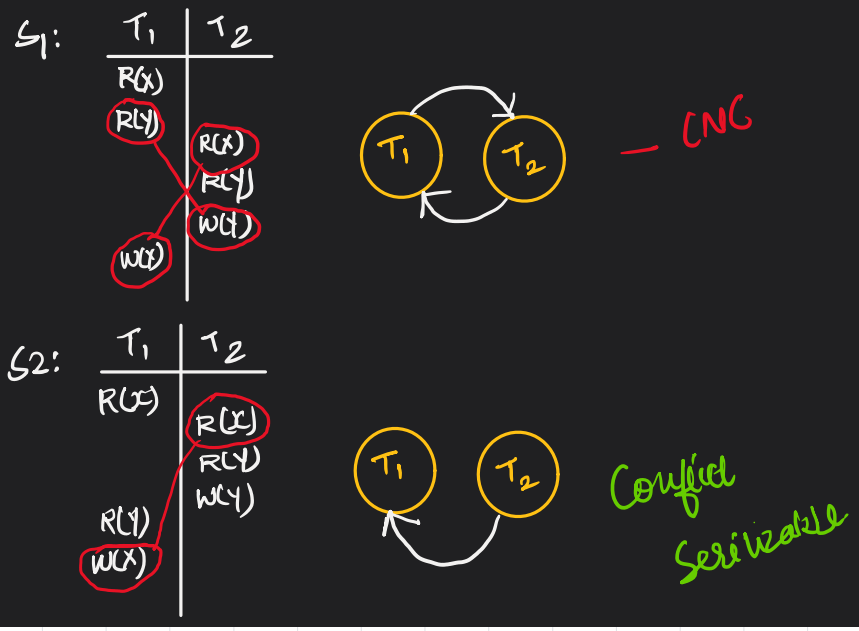
[2007: 2 Marks]

A. Both S_1 and S_2 are conflict serializable

B. S_1 is conflict serializable and S_2 is not conflict serializable

C. S_1 is not conflict serializable and S_2 is conflict serializable ✓

D. Both S_1 and S_2 are not conflict serializable



Q.2

Consider the following four schedules due to three transactions (indicated by the subscript) using read and write on a data item x, denoted by r(x) and w(x) respectively. Which one of them is conflict serializable?

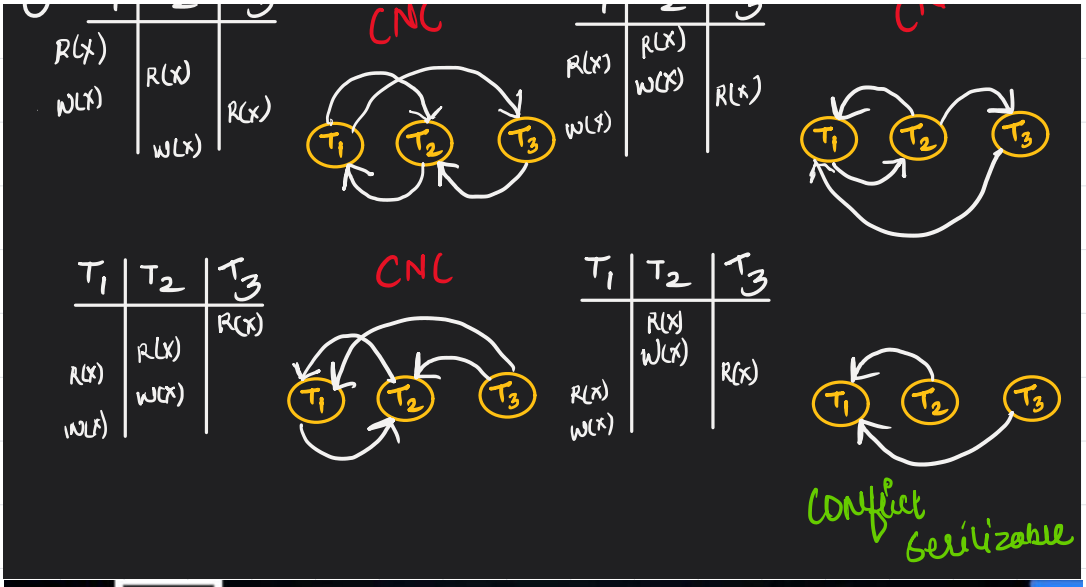
[2014(Set-1): 2 Marks]

A. $r_1(x); r_2(x); w_1(x); r_3(x); w_2(x)$

B. $r_2(x); r_1(x); w_2(x); r_3(x); w_1(x)$

C. $r_3(x); r_2(x); r_1(x); w_2(x); w_1(x)$

D. $r_2(x); w_2(x); r_3(x); r_1(x); w_1(x)$ ✓



MCQ Q.3

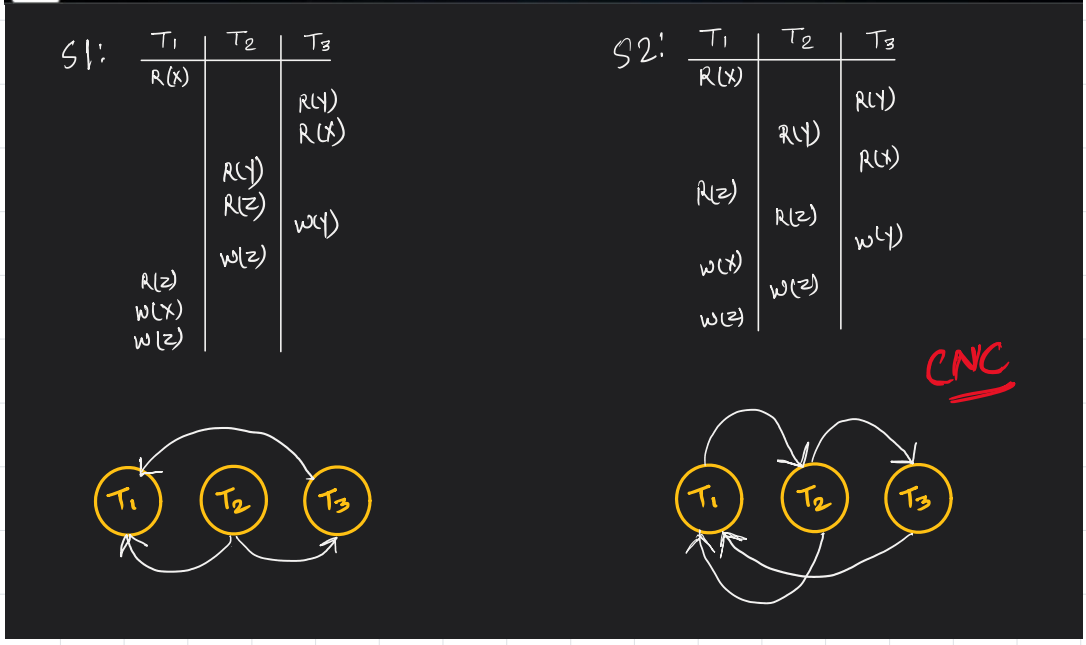
Consider the transactions T1, T2 and T3 and the schedules S1 and S2 given below.

T1: r1(X); r1(Z); w1(X); w1(Z)
T2: r2(Y) ; r2(Z) ; w2(Z)
T3: r3(Y); r3(X); w3(Y)
S1: r1(X); r3(Y); r3(X); r2(Y); r2(Z); w3(Y); w2(Z); r1(Z); w1(X); w1(Z)
S2: r1(X); r3(Y); r2(Y); r3(X); r1(Z); r2(Z); w3(Y); w1(X); w2(Z); w1(Z)

Which one of the following statements about the schedules is TRUE?

[GATE-2014-CS: 2M]

(A) Only S1 is conflict-serializable.
(B) Only S2 is conflict-serializable.
(C) Both S1 and S2 are conflict-serializable.
(D) Neither S1 nor S2 is conflict-serializable.



Q.4 Let $r_i(z)$ and $w_i(z)$ denote read and write operations respectively on a data item by a transaction T_i. Consider the following two schedules.

S_1: $r_1(x)\ r_1(y)\ r_2(x)\ r_2(y)\ w_2(y)\ w_1(x)$

S_2: $r_1(x)\ r_2(x)\ r_2(y)\ w_2(y)\ r_1(y)\ w_1(x)$

Which one of the following options is correct? [MCQ: 2021: 2M]

A. S_1 is conflict serializable, and S_2 is not conflict serializable.

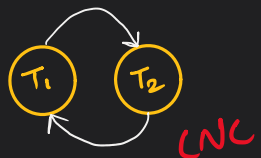
B. S_1 is not conflict serializable, and S_2 is conflict serializable.

C. Both S_1 and S_2 are conflict serializable.

D. Neither S_1 nor S_2 is conflict serializable.

S1:

T_1	T_2
R(X)	
R(Y)	
	R(X)
	R(Y)
	W(Y)
W(X)	



S2:

T_1	T_2
R(X)	
	R(X)
	R(Y)
	W(Y)
R(Y)	
W(X)	

Q.5 Let $R_i(z)$ and $W_i(z)$ denote read and write operations on a data element z by a transaction T_i, respectively. Consider the schedule S with four transactions.

S: $R_4(x), R_2(x), R_3(x), R_1(y), W_1(y), W_2(x), W_3(y), R_4(y)$

Which one of the following serial schedules is conflict equivalent to S? [2022: 2 Marks]

A. $T_1 \rightarrow T_3 \rightarrow T_4 \rightarrow T_2$

B. $T_1 \rightarrow T_4 \rightarrow T_3 \rightarrow T_2$

C. $T_4 \rightarrow T_1 \rightarrow T_3 \rightarrow T_2$

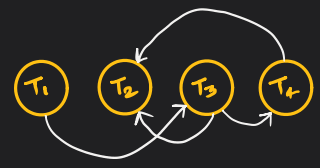
D. $T_3 \rightarrow T_1 \rightarrow T_4 \rightarrow T_2$

T_1	T_2	T_3	T_4
			R(X)

Q.5 S: $R_4(x), R_2(x), R_3(x), R_1(y), W_1(y), W_2(x), W_3(y), R_4(y)$

Conflict operation on

T_1	T_2	T_3	T_4
			R(x)
	R(x)		
		R(x)	
R(y)			
W(y)			
	W(x)		
		W(y)	
			R(y)



$T_1 \rightarrow T_3 \rightarrow T_4 \rightarrow T_2$

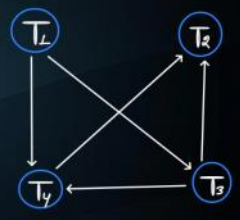
Q.5 S: $R_4(x), R_2(x), R_3(x), R_1(y), W_1(y), W_2(x), W_3(y), R_4(y)$

Conflict operation on

Data Item x
- $R_4(x) - W_2(x)$: $T_4 \rightarrow T_2$
- $R_3(x) - W_2(x)$: $T_3 \rightarrow T_2$

Data Item y
- $R_1(y) - W_3(y)$: $T_1 \rightarrow T_3$
- $W_1(y) - W_3(y)$: $T_1 \rightarrow T_3$
- $W_1(y) - R_4(y)$: $T_1 \rightarrow T_4$
- $W_3(y) - R_4(y)$: $T_3 \rightarrow T_4$

$T_1 \rightarrow T_3 \rightarrow T_2$
$T_1 \rightarrow T_4 \rightarrow T_2$
$T_3 \rightarrow T_4$
$T_1 \rightarrow T_3 \rightarrow T_4 \rightarrow T_2$



Q.6 Consider the following transaction involving two bank accounts x and y.

read(x); x: = x – 50; write (x); read (y); y: = y + 50; write (y)

The constraint that the sum of the accounts x and y should remain constant is that of [2015(Set-2): 1 Marks]

x	y
R(x)	
X := X-50	
W(x)	
	R(y)
	Y := Y+50
	W(y)

A. Atomicity – Full or None.

B. Consistency ✓

C. Isolation – Unaware.

D. Durability – long term.

MCQ Q.7

Which one of the following is NOT a part of the ACID properties of database transactions?

[GATE-2016-CS: 1M]

A. Atomicity

B. Consistency

C. Isolation

D. Deadlock-freedom ✓

MCQ Q.8

Suppose a database schedule S involves transaction T_1....., T_n. Construct the precedence graph of S with vertices representing the transactions and edges representing the conflicts. If S is serializable, which one of the following orderings of the vertices of the precedence graph is guaranteed to yield a serial schedule?

[GATE-2016-CS: 2M]

A Topological order

B Depth-first order

C Breadth-first order

D Ascending order of transaction indices

MCQ Q.9

T1	T3	T3
Read(X)		
	Read (Y)	
		Read (Y)
	Write (Y)	
Write (X)		
		Write (X)
	Read (X)	
	Write (X)	

$T_1 \rightarrow T_3 \rightarrow T_2$

A T 1 →T 3 →T 2

B T 2 →T 1 →T 3

C T2 → T3 → T1

D T3 →T 1 →T 2

MCQ Q.10

Consider two transactions T_1 and T_2, and four schedules S_1, S_2, S_3, S_4 of T_1 and T_2 as given below:

T_1: $R_1[x]$ $W_1[x]$ $W_1[y]$;
T_2: $R_2[x]$ $R_2[y]$ $W_2[y]$;
S_1: $R_1[x]$ $R_2[x]$ $R_2[y]$ $W_1[x]$ $W_1[y]$ $W_2[y]$;
S_2: $R_1[x]$ $R_2[x]$ $R_2[y]$ $W_1[x]$ $W_2[y]$ $W_1[y]$;
S_3: $R_1[x]$ $W_1[x]$ $R_2[x]$ $W_1[y]$ $R_2[y]$ $W_2[y]$;
S_4: $R_2[x]$ $R_2[y]$ $R_1[x]$ $W_1[x]$ $W_1[y]$ $W_2[y]$;

Which of the above schedules are conflict serializable?

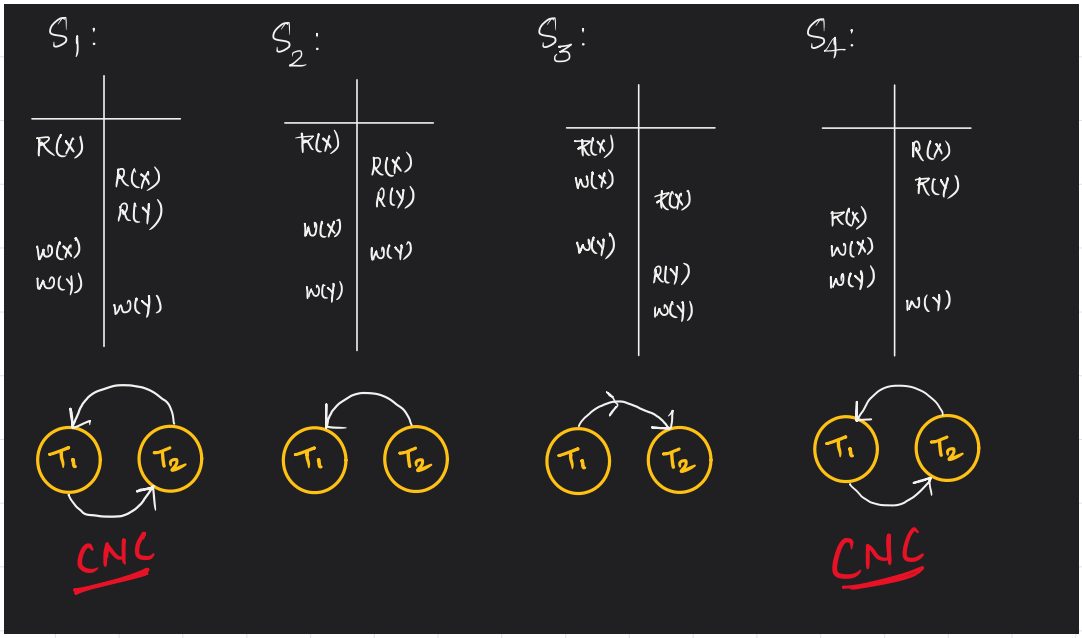
[GATE-2009-CS: 2M]

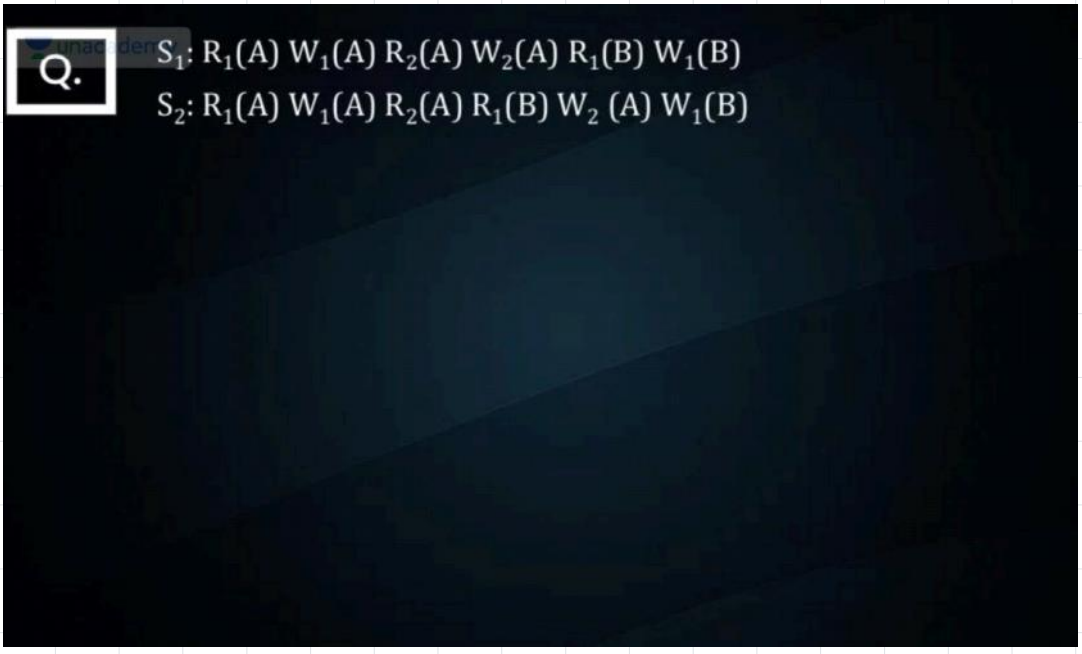
A S_1 and S_2

B S_2 and S_3

C S_3 only

D S_4 only

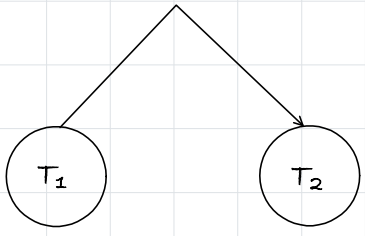




TOPIC : Serializability order (topological sorting)

serializability order means if schedule is conflict serializable (acyclic graph (not cycle in any graph)) then serializability order indicate that schedule is equivalent to which serial schedule

example :



C_1 is equivalent to serial schedule $<T_1, T_2>$ T_1 followed by T_2

$<T_1, T_2>$ Conflict serializable

Serializability order : $<T_1, T_2>$ T_1 followed by T_2

Serializability order tells us the graph is equivalent to which serial schedule.

#Serializability order through topological sorting.
[always acyclic precedence graph][conflict serializable]

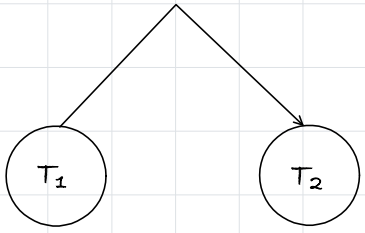
Process :
Step 1 : start from the vertex which having indegree = 0 (no incoming edge)

Step 2 : Delete that vertex and all connecting edges related to it from that vertex.

Step 3 : Repeat this process until all vertex are traverse.

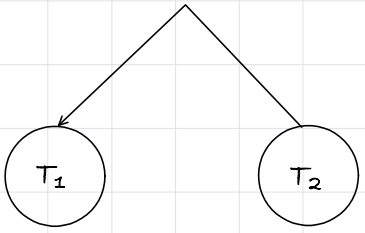
serializability order

(A)



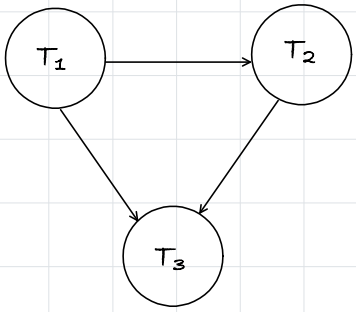
$<T_1, T_2>$ T_1 followed by T_2

(B)



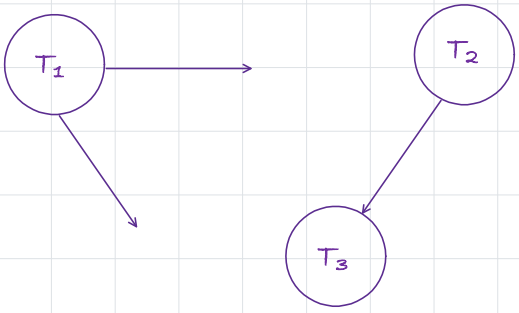
$<T_2, T_1>$ T_2 followed by T_1

(C)



$<T_1, T_2, T_3>$

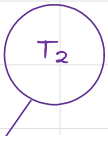
(i)



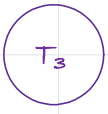
Indegree = 0 (T_1) | Cut down all the edges related to it

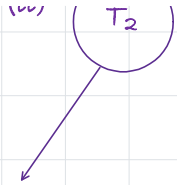
$<T_1,$

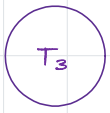
(ii)



(iii)







Indegree = 0 (T_2) | Cut down all the edges related to it

$<T_1, T_2$

Indegree = 0 (T_3) | no edges remaining.

$<T_1, T_2, T_3>$

TOPIC : Conflict equivalent schedule.

Two schedules are said to be conflict equivalent, if all conflicting operations in both schedules must be executed in the same order.

Q S_1: $R_1(X)\ W_1(X)\ R_2(Y)\ W_2(Y)\ R_1(Y)$

S_2: $R_1(X)\ W_1(X)\ R_1(Y)\ R_2(Y)\ W_2(Y)$

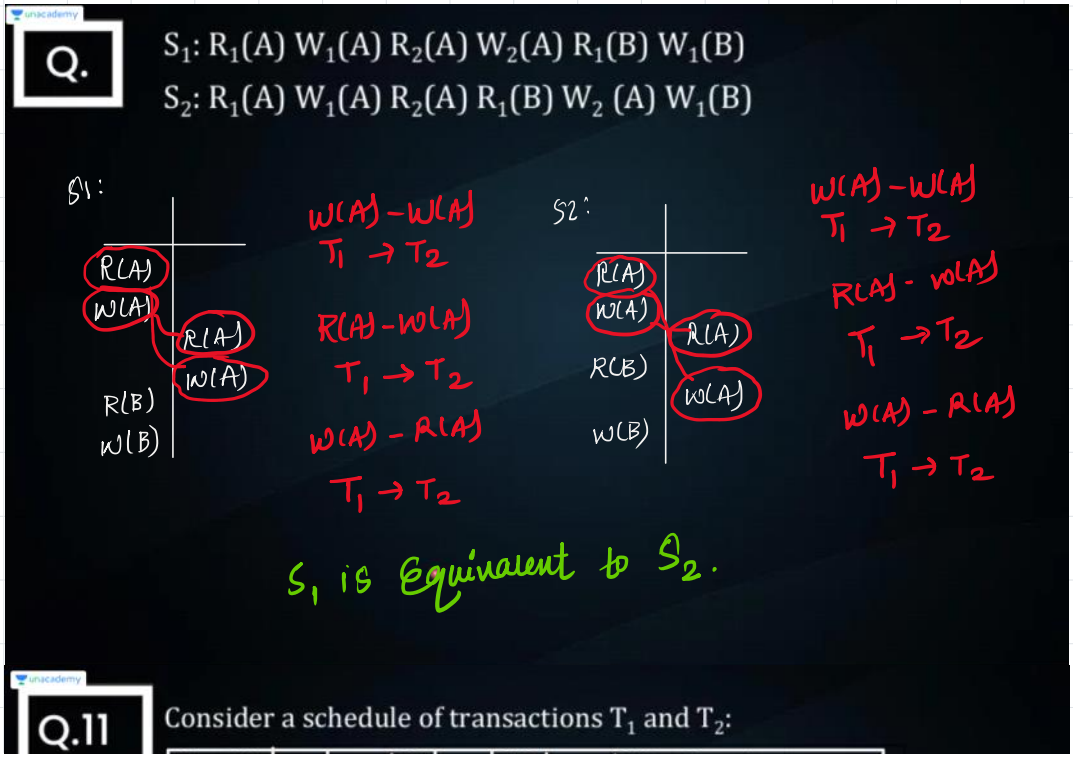
T_1	T_2
R(X)	
W(X)	
	R(Y)
	W(Y)
R(Y)	

W(Y) - R(Y) : T_2 to T_1

T_1	T_2
R(X)	
W(X)	
R(Y)	
	R(Y)
	W(Y)

R(Y) - W(Y) : T_1 to T_2

S_1 is not equivalent to S_2



Q.11

Consider a schedule of transactions T_1 and T_2:

T_1	RA			RC		WB		WD	Commit
T_2		RB	WB		RD		WC		Commit

Here, RX stands for "Read(X)" and WX stands for "Write(X)". Which one of the following schedules is conflict equivalent to the above schedule?

[2020: 2 Marks]

A

T_1					RA	RC	WD	WB	Commit	
T_2	RB	WB	RD	WC						Commit

B

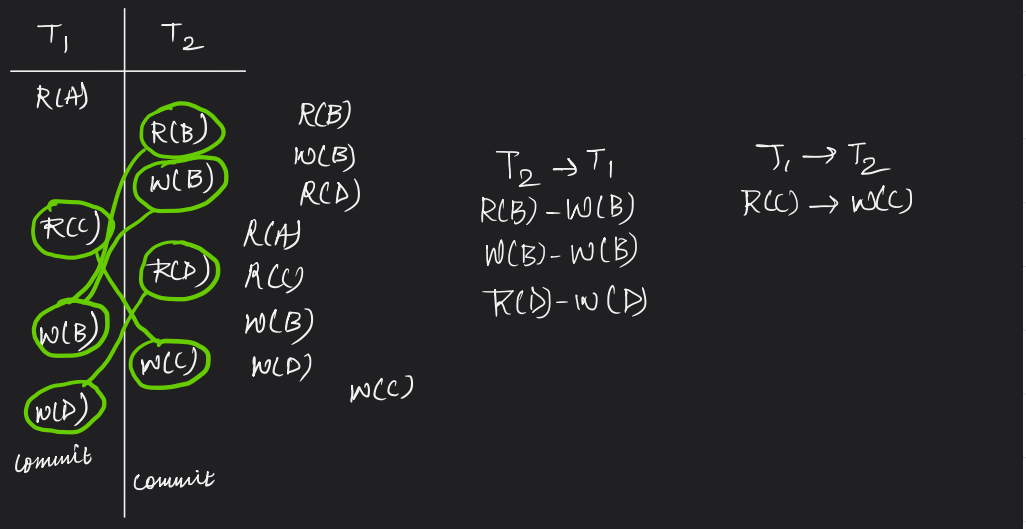
T_1				RA	RC	WD	WB		Commit	
T_2	RB	WB	RD					WC		Commit

C

T_1	RA	RC	WD	WB					Commit	
T_2					RB	WB	RD	WC		Commit

D

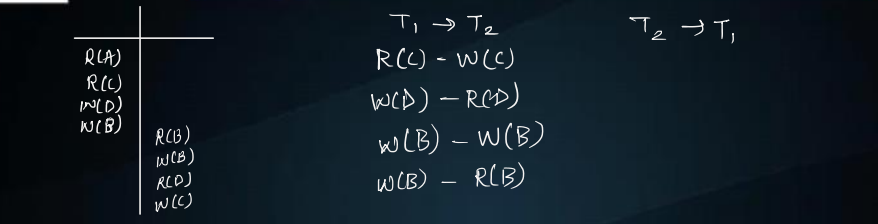
T_1	RA	RC	WD	WB					Commit	
T_2					RB	WB	RD	WC		Commit



Q.11

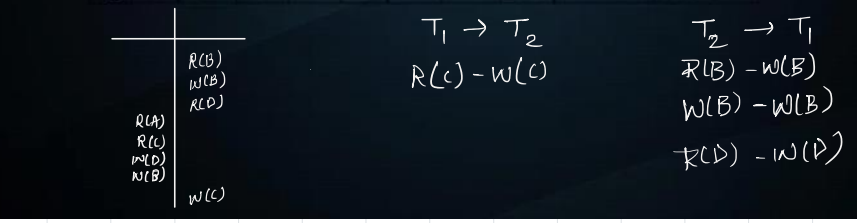
A

T_1					RA	RC	WD	WB	Commit	
T_2	RB	WB	RD	WC						Commit



B

T_1				RA	RC	WD	WB		Commit	
T_2	RB	WB	RD					WC		Commit



Note:

- If schedule is conflict serializable then its already view serializable.
- If schedule is not conflict serializable (CNC) then it may or may not be serializable.

TOPIC : View serializable

if my schedule is conflict serializable then its fine.
but if its not conflict serializable then we check view serializable.

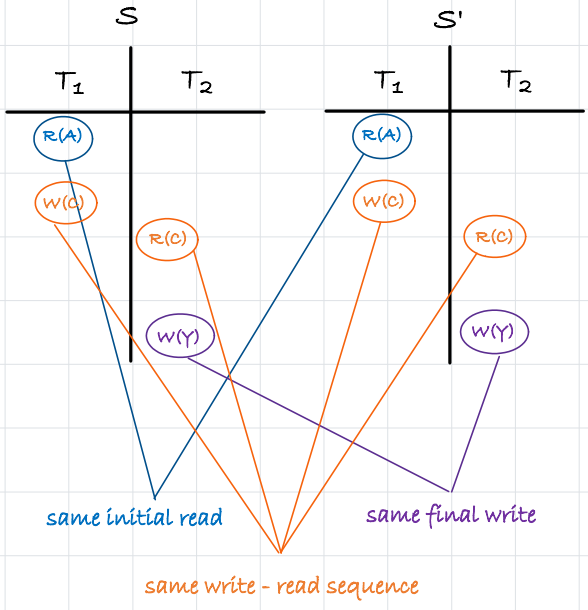
S [non-serial schedule] and S' [any serial schedule of S]

for each data item these 3 conditions must satisfy
(i) Initial read
(ii) Final write
(iii) Updated read [write-read-sequence]

Let S and S' be two schedules with the same set of transactions. S and S' are **view equivalent** if the following three conditions are met, for each data item Q.

1. If in schedule S, transaction T_i reads the initial value of Q, then in schedule S' also transaction T_i must read the initial value of Q.
2. If in schedule S transaction T_i executes **read**(Q), and that value was produced by transaction T_j (if any), then in schedule S' also transaction T_i must read the value of Q that was produced by the same **write**(Q) operation of transaction T_j.
3. The transaction (if any) that performs the final **write**(Q) operation in schedule S must also perform the final **write**(Q) operation in schedule S'.

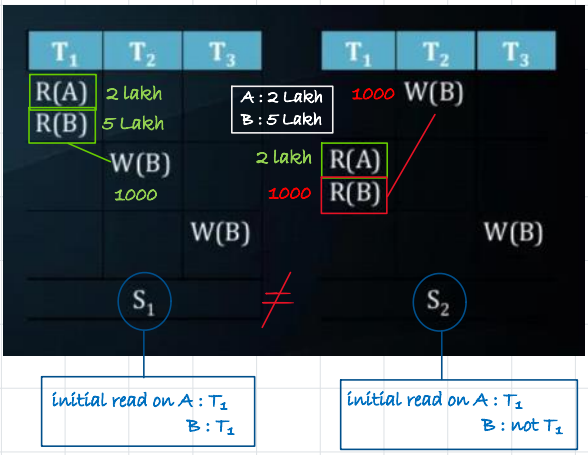
→ for each data item
→ (i) Initial read
→ (ii) updated read [write- read-sequence]
→ (iii) Final write



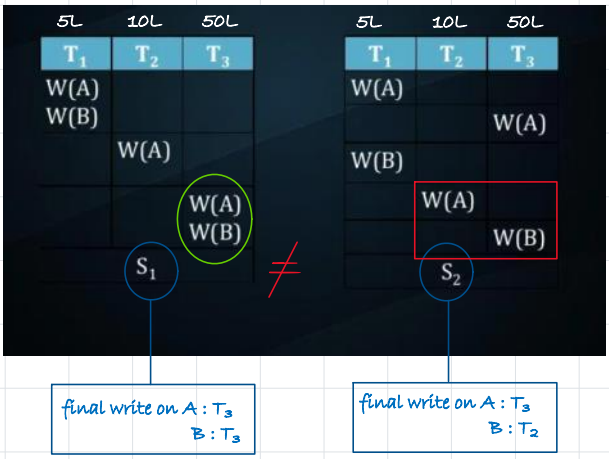
(i) Initial read (ii) final write

same write - read sequence

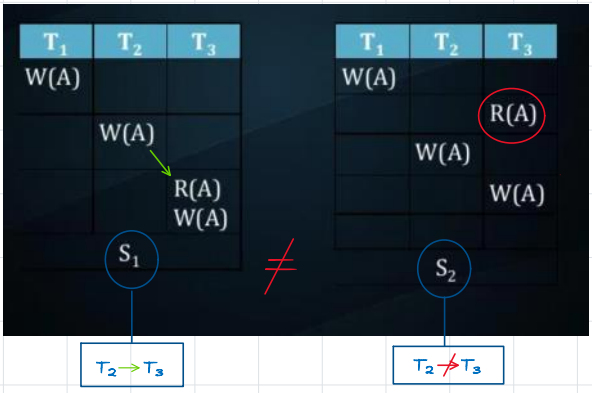
(i) Initial read



(ii) final write



(iii) updated read should be same



Que.

T_1	T_2
R(A)	
W(A)	
	R(A)
	W(A)
R(B)	
W(B)	
	R(B)
	W(B)

2 serial schedule

1. $<T_1, T_2>$ T_1 followed by T_2
2. $<T_2, T_1>$ T_2 followed by T_1

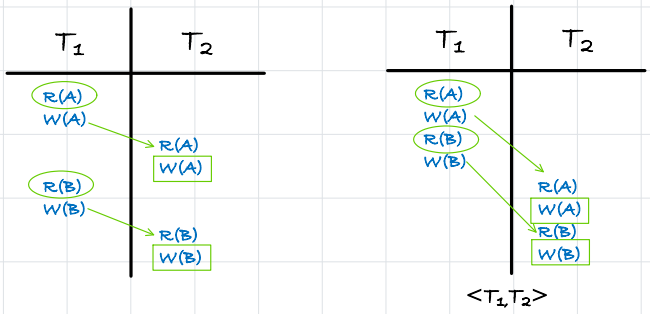
Dummy method : 1. Initial read
2. Final write

(A) checking for $<T_2, T_1>$ T_2 followed by T_1

Intial read : T_1

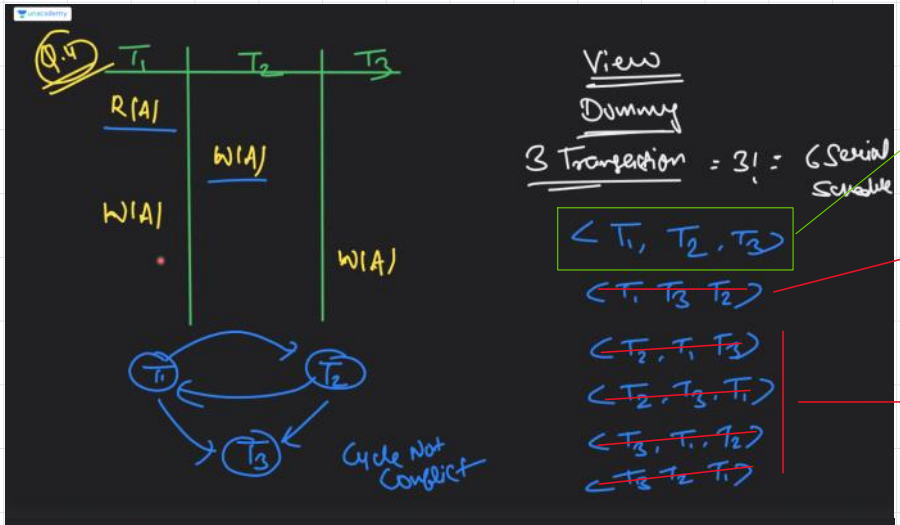
$<T_2, T_1>$ not possible

(A) checking for $<T_1,T_2>$ T_1 followed by T_2



Intial read : T_1
Final write : T_2
Write - read sequence : T_1 to T_2

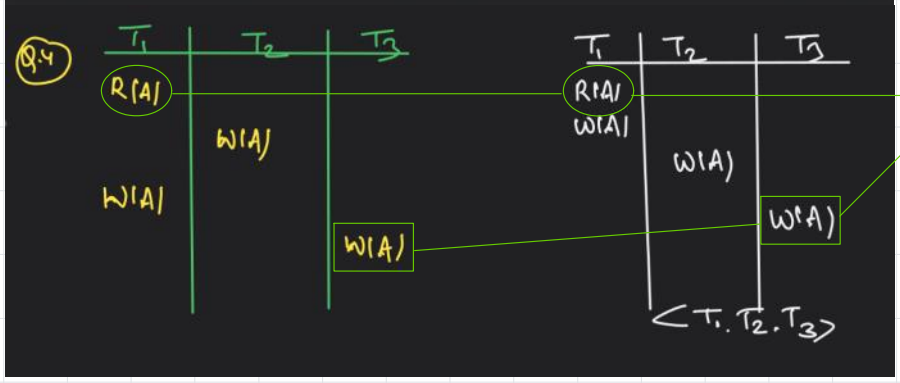
$<T_1,T_2>$ is view serializable



remaining

eliminated on the basis of final write W(A) - T_3

eliminated on the basis of initial read R(A) - T_1



Intial read : T_1
Final write : T_3
Write - read sequence : does not exist. (blind writes)

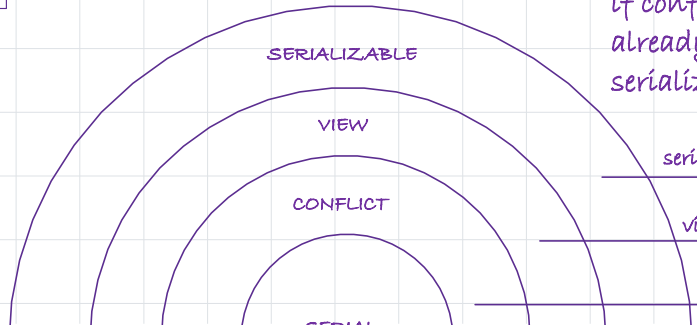
this schedule is not conflict serializable but it is view serializable.

concurrency subsystem

isolation

concurrent

consistent

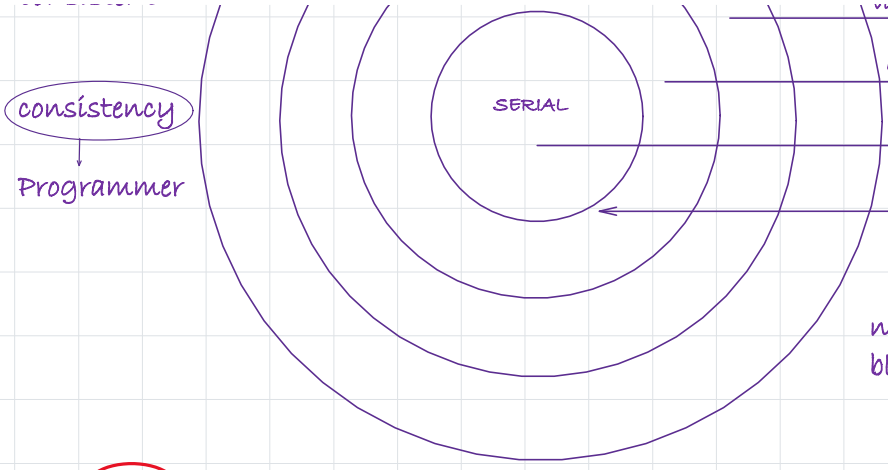


if conflict then already view serializable

serializable (not conflict and not view): conflict may or may not

view serializable : serialiable but ocnflict may or may not

conflict serializable : view serializable and also Serializable schedule



view serializable : serializable but deadlock may or may not

conflict serializable : view serializable and also Serializable schedule

serial schedule : conflict, view and serializable schedule

non serial schedule

not conflict but view serilizable : case of blind writes.

Note:

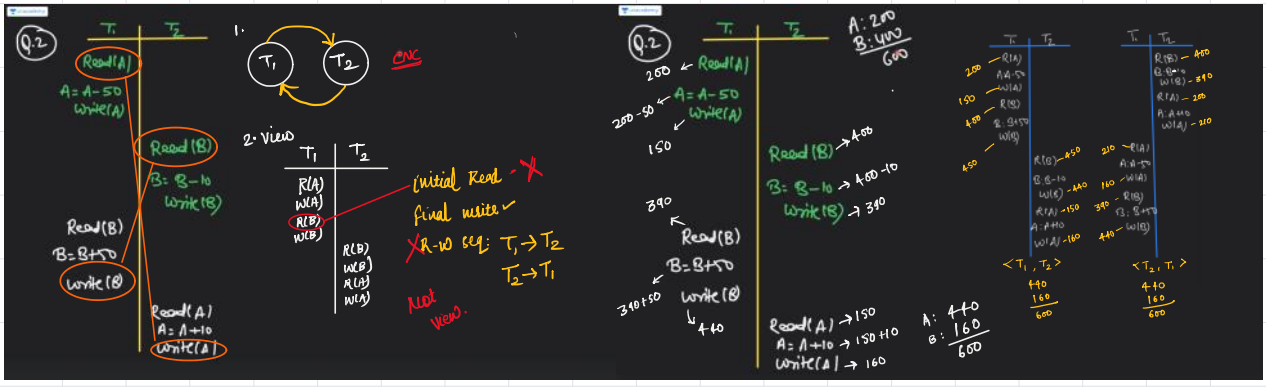
Practically -
A schedule is serializable if either it is conflict serializable (or) view serializable (or) both.
Theoritcally - In general implementation of 'only' serializability (neither conflict nor view) is not possible so practically implemented by conflict (or) view

1. Conflict seralizability : More strict, less time complexity (polynomial time) (its very easy to implement). Blind writes : may or may not.

2. View serilizability : Less strict, NP complete, exponential time complexity (hard to implement)

- not conflict + view serializable : there must be blind writes.
- if no blind writes then Conflict and also View
- If blind writes then view but conflict may or may not.

so we check conflict serializability (by testing method)



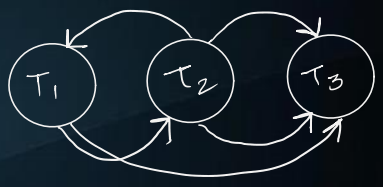
it is not conflict nor view serializable but it is serializable (C_4). here serializability is achieved based on computation not by R/W operations. but practically serializability is achieved by operations and not achieved by computations.

Questions -

Q.

T_1	T_2	T_3
	R(A)	
	R(B)	
W(B)		
		R(B)
W(A)		
	W(A)	
		W(A)

1. Conflict Serializable -



CNC

2. View serializable - Initial Read: T_2

Final write: T_3

$< T_2\ T_1\ T_3 >$

T_1	T_2	T_3
	R(A)	
	R(B)	
W(B)	W(A)	
W(A)		
		R(B)
		W(A)

Initial Read: ✓

Final write: ✓

write - Read seq: ✓

Not Conflict but View

Blind writes

Q.14 Consider the following schedule S of transactions T_1 and T_2:
Which of the following is TRUE about the schedule S? [2004: 2 Marks]

T_1	T_2
Read(A)	
A = A−10	
	Read(A)
	Temp = 0.2 * A
	Write(A)
	Read(B)
Write(A)	
Read(B)	
B = B +10	
Write(B)	
	B = B + Temp
	Write(B)

A. S is serializable only as T_1, T_2

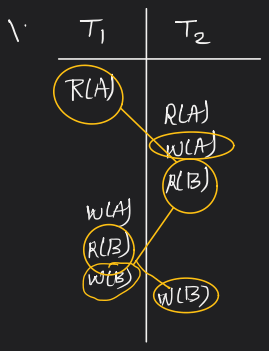
B. S is serializable only as T_2, T_1

C. S is serializable both as T_1, T_2 and T_2 T_1

D. ✓ S is not serializable either as T_1 or as T_2

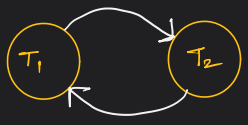
B = B + Temp
Write(B)

1.



2. View

Initial Read :
Final write :
write - Read :



CVC

unacademy

MCQ Q.13

Consider the following transactions with data items P and Q initialized to zero:

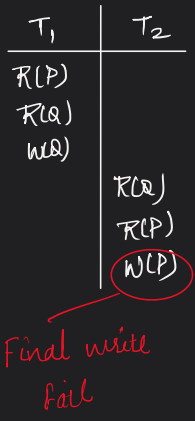
T_1: read (P);
read (Q);
if P = 0 then Q := Q + 1 ;
write (Q).

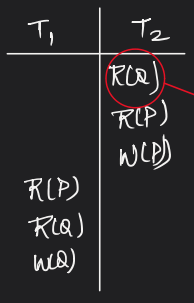
T_2: read (Q) ;
read (P);
if Q = 0 then P := P + 1 ;
write (P).

Any non-serial interleaving of T_1 and T_2 for concurrent execution leads to

[GATE-2012-CS: 1M]

Sol

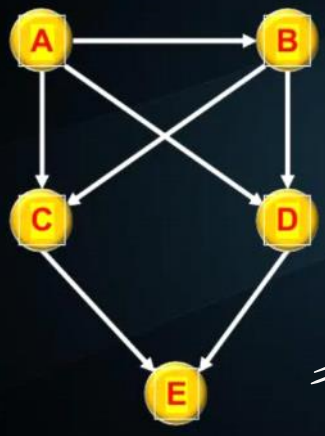


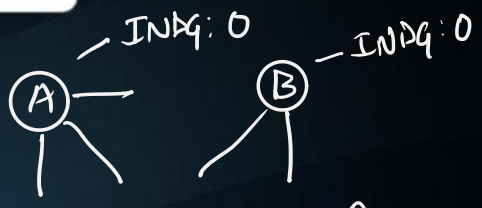


fail

Topological Sorting

Q.1



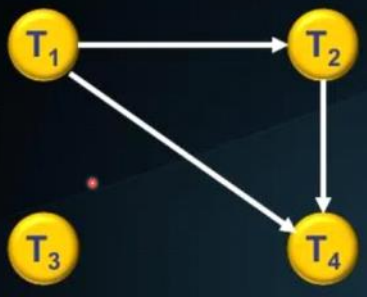


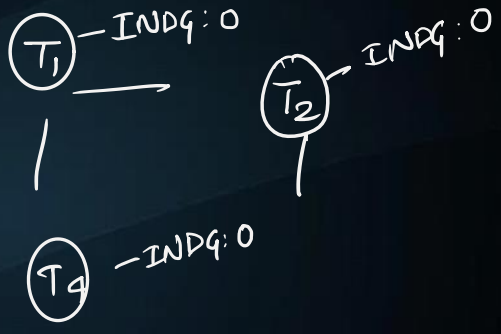
C and A — INDG: 0

A - B — C, D, E
D, C, E

Ans: (2)

Q.2



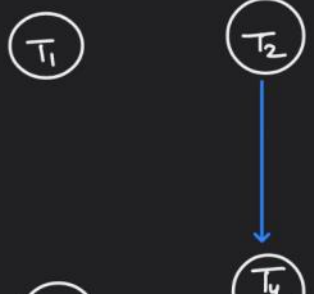


T4 — INDG: 0

Sol:

T1 - T2 - T4 - T3
T1 - T3 - T2 - T4
T3 - T1 - T2 - T4
T1 - T2 - T3 - T4

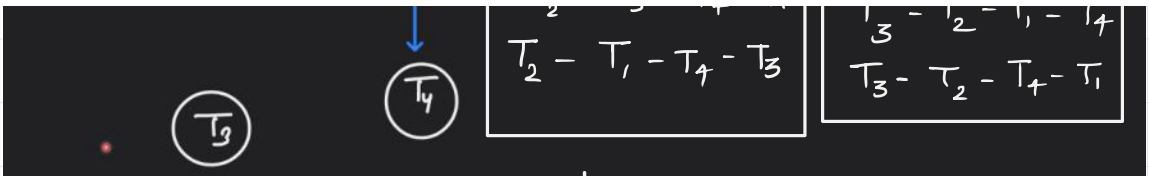
Q.



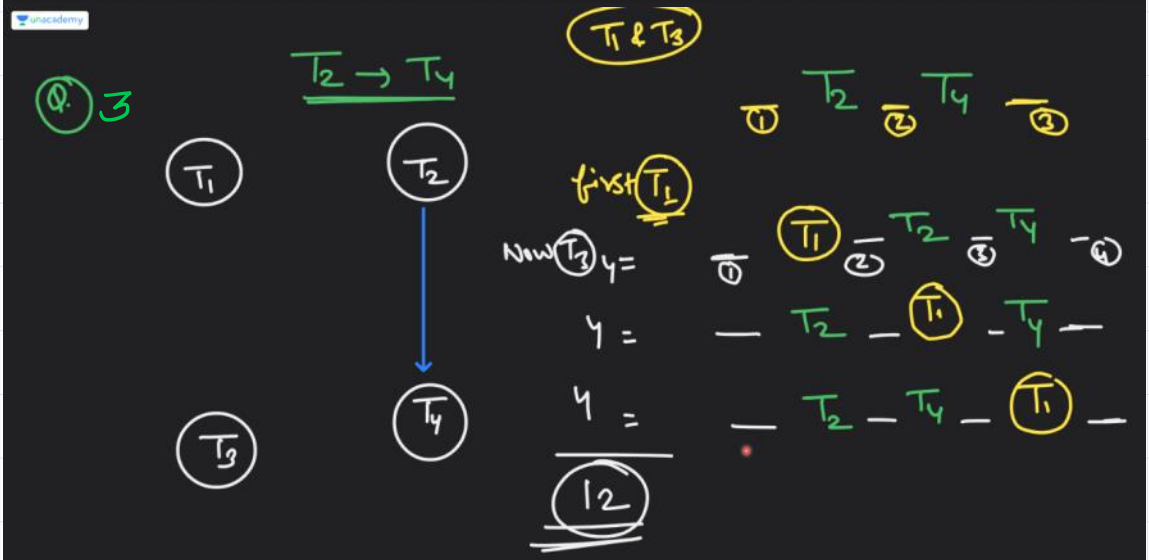
T2 - T1 - T3 - T4
T2 - T4 - T1 - T3
T2 - T4 - T3 - T1
T2 - T3 - T1 - T4
T2 - T3 - T4 - T1
T2 - T1 - T4 - T3

T1 - T2 - T3 - T4
T1 - T3 - T2 - T4
T1 - T2 - T4 - T3

T3 - T1 - T2 - T4
T3 - T2 - T1 - T4
T3 - T2 - T4 - T1



Total - 12



Locked: $T_2 \rightarrow T_4$

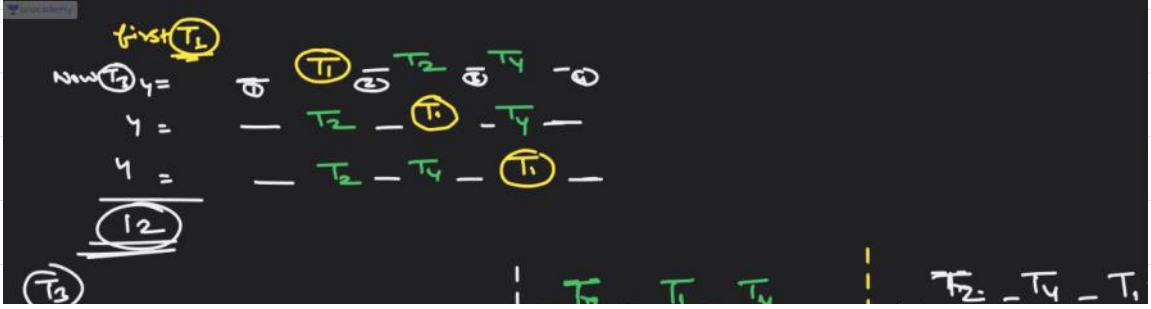
T_1 and T_3 : _ T_2 _ T_4 _

$T_1 - T_3 - T_2 - T_4$
$T_2 - T_1 - T_3 - T_4$
$T_2 - T_4 - T_1 - T_3$

T_1: T_1 T_2 _ T_4 _
_ T_2 T_1 T_4 _
_ T_2 _ T_4 T_1

T_3:
4 _ T_1 _ T_2 _ T_4 _
4 _ _ T_2 _ T_1 _ T_4 _
4 _ _ T_2 _ T_4 _ T_1 _

Total : 12 PLACES.



first T_1

Now T_2 y = ① T_1 ② T_2 ③ T_4 ④

y = — T_2 — T_1 — T_4 —

y = — T_2 — T_4 — T_1 —

12

T_3

— T_1 — T_2 — T_4 —

T_3 T_1 T_2 T_4

T_1 T_3 T_2 T_4

T_1 T_2 T_3 T_4

T_1 T_2 T_4 T_3

— T_2 — T_1 — T_4 —

T_3 T_2 T_1 T_4

T_2 T_3 T_1 T_4

T_2 T_1 T_3 T_4

T_2 T_1 T_4 T_3

— T_2 — T_4 — T_1

T_3 T_2 T_4 T_1

T_2 T_3 T_4 T_1

T_2 T_4 T_3 T_1

T_2 T_4 T_1 T_3

Q.4 $R_4(x)$ $R_2(x)$ $R_3(x)$ $W_1(y)$ $W_2(x)$ $R_3(y)$ $W_2(y)$

T_1	T_2	T_3	T_4
			R(x)
	R(x)		
W(y)		R(y)	
	W(x)		
		R(y)	
	W(y)		

T_4 and T_1 — INDG: 0

T_3 — INDG: 0

T_2 - INDG: 0

T_3 - T_2

— T_3 — T_2 —

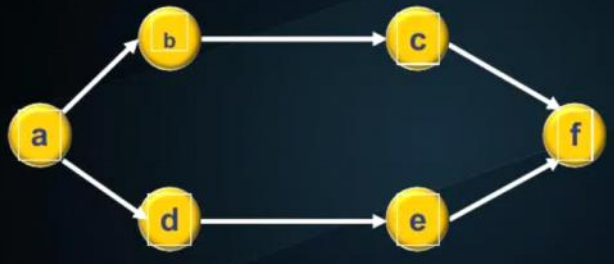
T_1 T_4 T_3 T_2

T_4 T_1 T_3 T_2

T_1 T_3 T_4 T_2

3 choices.

Q. Consider the following directed graph:



The number of different topological ordering of the vertices of the graph is _____.

[MCQ: 2016]

a - INDG: 0

b and d - INDG: 0

C and e - INDG: 0

f - INDG: 0

abdcef

afcdef

abdecf

adbcef

6 choices

f - INDG : 0

adbcef
adbecf
adebcf

6 choices

Q.46 Consider the following read-write schedule S over three transactions T_1, T_2, and T_3, where the subscripts in the schedule indicate transaction IDs:

$$S: r_1(z);\ w_1(z);\ r_2(x);\ r_3(y);\ w_3(y);\ r_2(y);\ w_2(x);\ w_2(y);$$

Which of the following transaction schedules is/are conflict equivalent to S ?

(A) $T_1T_2T_3$

(B) $T_1T_3T_2$

(C) $T_3T_2T_1$

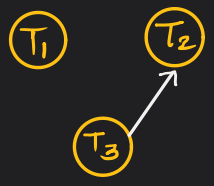
(D) $T_3T_1T_2$

T_1	T_2	T_3
R(Z) W(Z)		
	R(X)	
		R(Y) W(Y)
	R(Y) W(X) W(Y)	

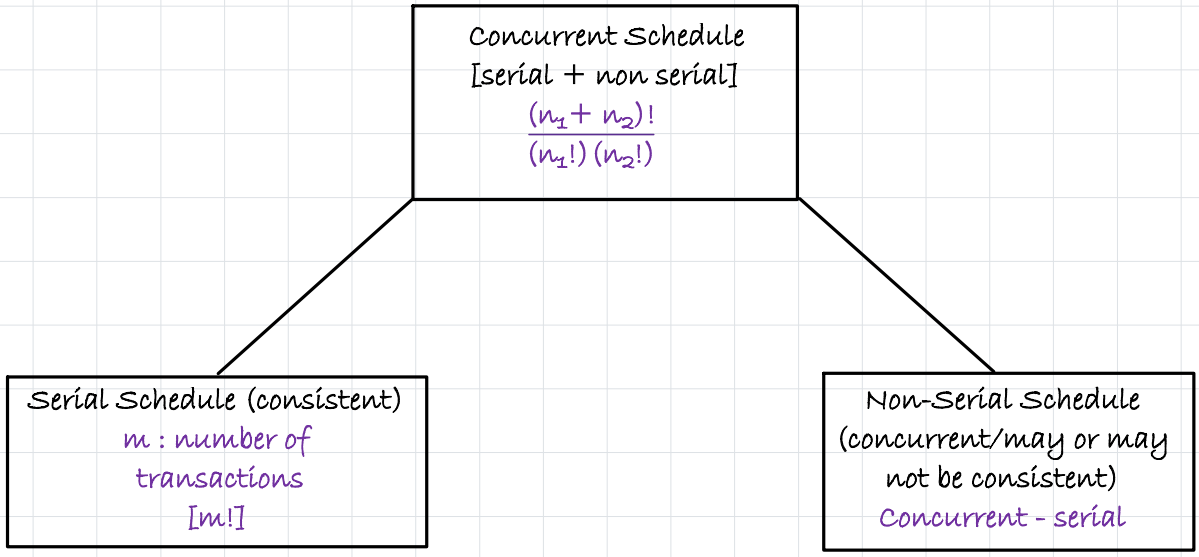
$T_1\ T_3\ T_2$

$T_3\ T_1\ T_2$

$T_3\ T_2\ T_1$



TOPIC : finding total number of schedule.



1. Total number of concurrent schedule : $\frac{(n_1+n_2)!}{(n_1!)(n_2!)}$ $\frac{(2+2)!}{(2!)(2!)}$

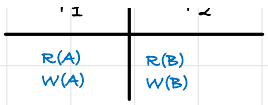
T_1 = n_1 operation
2 operation
T_2 = n_2 operation
2 operation

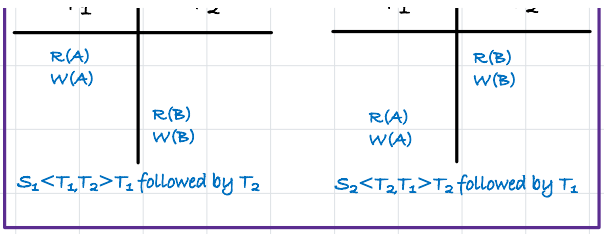
T_1	T_2
R(A) W(A)	R(B) W(B)

serial schedule

T_1	T_2
R(A) W(A)	

T_1	T_2
	R(B) W(B)

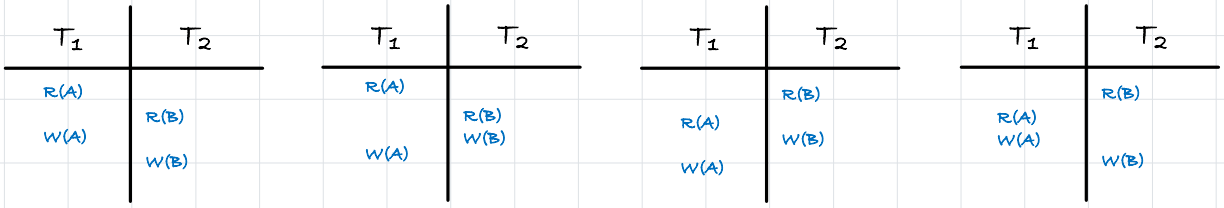




$S_1 < T_1 T_2 > T_1$ followed by T_2

$S_2 < T_2 T_1 > T_2$ followed by T_1

non serial schedule



The Number of Concurrent schedule that can be formed
Over m transaction having n_1 n_2 n_3 ....n_m operation respectively

Total # of Concurrent Schedule $= \frac{(n_1+n_2+n_3+\cdots n_m)!}{(n_1!)(n_2!)(n_2!)\ldots(n_m!)}$

Total # of Non Serial Schedule $= \frac{(n_1+n_2+n_3+\cdots n_m)!}{(n_1!)(n_2!)(n_2!)\ldots(n_m!)} - m!$

serial schedule

example -

Q) T_1 : 2 operation
T_2 : 3 operation
T_3 : 4 operation

(i) Total # Concurrent Schedule ?
(ii) Total # Serial Schedule ?
(iii) Total Non Serial Schedule ?

① Total concurrent : $\frac{(2+3+4)!}{(2!)(3!)(4!)} = \frac{9!}{2 \times 6 \times 24} = \frac{362880}{288}$

$= 1260.$

② Total serial : $m! = 3! = 6$

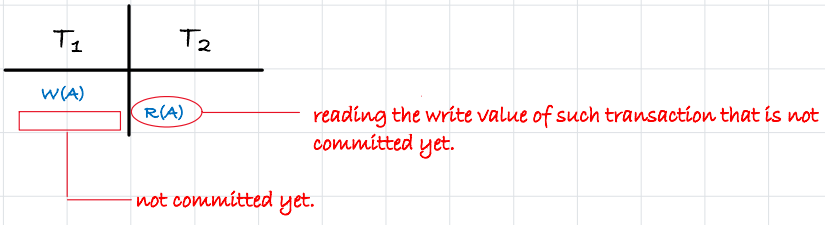
③ Total Non-serial : Concurrent - Serial $= 1260 - 6$

$= 1254.$

TOPIC : Problems due to concurrent execution

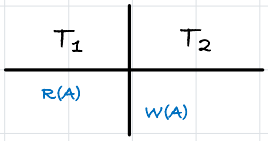
1. WR (write-read) problem / dirty read / uncommited read problem.
2. RW (read-write) problem / un(non)-repeatable read problem.
3. WW (write-write) problem / lost update problem.
4. Phantom tuple problem.

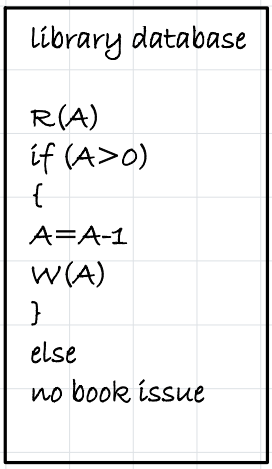
1. WR (write-read) problem / dirty read / uncommited read problem : modification done by one transaction (uncommitted transaction) and read by another transaction is called uncommitted read.

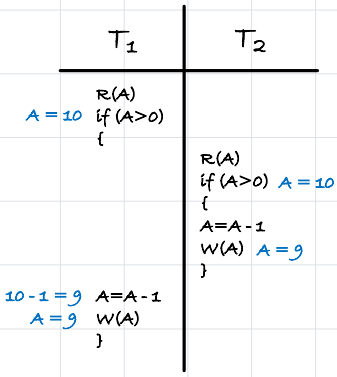


if T_1 fails it have to rollback and so T_2 also have to rollback.

2. RW (read-write) problem / un(non)-repeatable read problem :



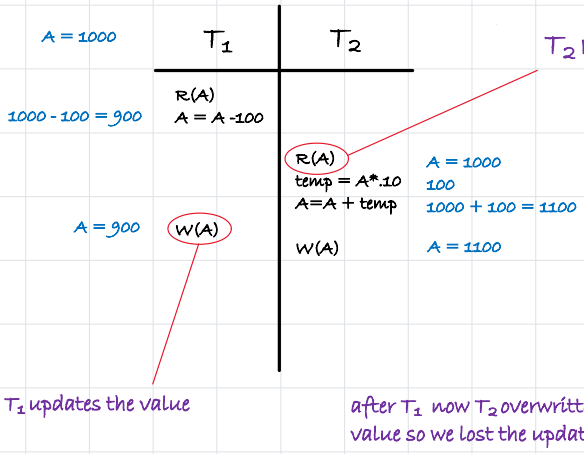




korth book copies : 10
book issued : 2

9 (book left) + 2 (book issued) 11 (total), inconsistent due to concurrent execution.

3. WW (write-write)/lost update problem : if there is two operation of different transaction and between them there is no read operation second write operation overwritten the first write value, lost of update problem.



T_2 reads the value of account before T_1 update

T_1 updates the value

after T_1 now T_2 overwritten the value so we lost the update here.

TOPIC : Dependency

(A)

T_1	T_2
W(A) . . . Commit	R(A)

Dependency : YES

uncommitted read : T_2 read the value of data item "A" which is written by uncommitted transaction T_1

T_2 depends on T_1

(B)

T_1	T_2
W(A) Commit	R(A) - -

Dependency : NO

committed : T_1 transaction is being committed and finished. T_2 reads the value that is committed and cannot rollback.

(C)

T_1	T_2
W(A) rollback	R(A)

Dependency : NO

Transaction failed : 'rollback', T_1 value goes back to the original value stored in database and T_2 read the same value.

(D)

T_1	T_2
W(A)	W(A)

Dependency : NO

No read : T_2 is not reading the value which is written by T_1

(E)

T_1	T_2
W(A)	W(A) R(A)

Dependency : NO

Not uncommitted read : T_1 is done and then T_2 is reading the value of its own transaction.

(F)

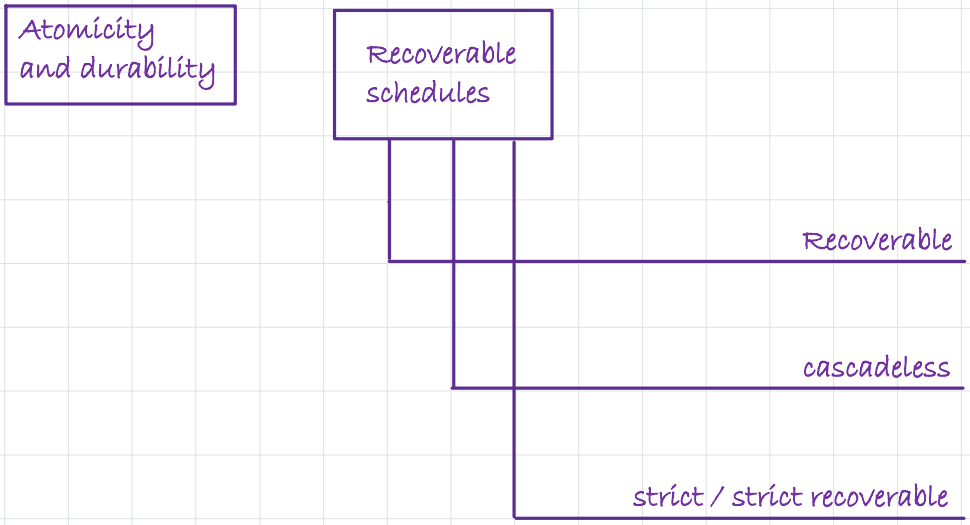
T_1	T_2
W(A)	
	W(B)
	R(A)
R(B)	

Dependency : YES

Depends on each others : $T_{2(R(A))}$ depends on $T_{1(W(A))}$

$T_{1(R(B))}$ depends on $T_{2(W(B))}$

TOPIC : Recoverable schedules.



1. Recoverable schedule: A recoverable schedule is one where each pair of transaction where T_i and T_j such that, T_j read data item that was previously written by T_i, then commit of T_i must appear before commit of T_j

(OR)

If T_2 depends on T_1 (T_2 read a value which is written by T_1) then commit of T_2 must be delayed untill commit/rollback of T_1

T_1	T_2
W(A)	
	R(A)
	-
	-
	commit

If transaction fails : not recoverable

T_1	T_2
W(A)	
	R(A)
commit/ rollback	-
	-
	commit

If transaction fails : recoverable
if "write then read" (W-R) then the commit or rollback of 'write' must appear before 'read' commit.

Need to address the effect of transaction failures on concurrently running transactions.

- Recoverable schedule — if a transaction T_j reads a data item previously written by a transaction T_i, then the commit operation of T_i appears before the commit operation of T_j.
- The following schedule is not recoverable

value in database : 10,000
T_8 transaction occurs.

50,000

50,000

abort occurs, goes to original value 10,000

T_8	T_9
read(A)	
write(A)	Read(A)
failure	commit
read(B)	

value shown to user who is executing transaction T9 : still 50,000

- If T_8 should abort, T_9 would have read (and possibly shown to the user) an inconsistent database state. Hence, database must ensure that schedules are recoverable.

Examples

not recoverable

(1) T_1	T_2
w(A)	$R_1(A)$
Rollback	C_2(commit)

recoverable

(3) T_1	T_2
w(A)	
C|R	R(A)

(rollback) recoverable

(5) T_1	T_2
w(A)	R|A
C|R	rollback

safe not recoverable

(2) T_1	T_2
w(A)	
	R(A)
C_1	C_2

(no dependency) recoverable

(4) T_1	T_2
w(A)	
	w(A)
	R(A)
Commit/ Rollback	commit

recoverable

(6) T_1	T_2
w(A)	
	R(A)
C|R	C|R

Note:

Recoverable schedule may (or) may not be free from
1. (i) WR problem (ii) WW problem (iii) RW problem
2. Cascading rollback.

T_1	T_2
R(A)	
	W(A)
	commit
commit	

recoverable but RW problem

T_1	T_2
W(A)	
	W(A)
	commit
commit	

recoverable but WW problem

T_1	T_2
R|A	
W(A)	
	R(A)
W(B)	
	R(B)
commit	
	commit

recoverable but WR problem

2. Cascading rollback : a single transaction failure leads to a series of transaction rollbacks.

T_1	T_2	T_3	T_4
W(A)			
	R(A)		
		R(A)	
			R(A)

none of the transactions has committed.

here T_2, T_3, T_4 depends on T_1

if T_1 fail then rollback due to dependency T_2, T_3, T_4 also rollback (cascading rollback)

2. 1. Cascadeless schedule : A cascadeless schedule is one where each pair of transaction where T_i and T_j such that, T_j read data item that was previously written by T_i , then commit of T_i must appear before Read of T_j

T_i	T_j
W(A)	
commit/ rollback	
	R(A)

(OR)

not allowed uncommitted (dirty) read

Note:

1. No WR / No uncommitted read.
2. No Cascading rollback.

may (or) may not be free from
(i) RW problem.
(ii) WW (lost update) problem.

3. Strict recoverable schedule : A strict recoverable schedule is one where each pair of transaction where Ti and T_j such that, T_j read data item that was previously written by T_i , then commit of T_i must appear before Read and write of T_j

T_i	T_j
W(A)	
commit/ rollback	
	R(A) / W(A)

Note:

No WR problem and No WW problem only RW problem exists.

3 cases to check :

T_1	T_2
W(Q)	
	R(Q)
commit/ rollback	
	commit

recoverable schedule - commit/rollback before commit of read

-RW/WR/WW problem

T_1	T_2
W(Q)	
commit/ rollback	
	R(Q)

cascadeless schedule - commit/rollback before read

-no RW problem.

T_1	T_2
W(Q)	
commit/ rollback	
	R(Q) / W(Q)

strict / strict recoverable schedule - commit/rollback before read and write

-no WR/WW problem.
-no cascading rollback
-only RW problem.

Note:

1. every strict recoverable is cascadeless and recoverable schedule but vice versa not always true.
2. every cascadeless schedule is recoverable schedule but every recoverable shcedule may or may not be cascadeless

examples :

T_i	T_j
W(A)	
	R(A)
W(B)	
	R(A)
Commit	
	Commit

recoverable but not cascadeless and not strict also

T_i	T_j
W(A)	
	W(A)
W(B)	
	W(B)
Commit	
	Commit

recoverable and cascadeless but not strict

T_i	T_j
W(A)	
	W(A)
W(B)	
	W(B)
Commit	
	Commit

recoverable and cascadeless but not strict

T_i	T_j
W(A)	
	W(C)
W(B)	
	W(D)
Commit	
	Commit

recoverable and cascadeless and strict but not serial schedule

T_i	T_j
R(A)	
	W(A)
	R(B)
W(B)	
Commit	
	commit

commit but no read and write

because no commit after write and there is no read write.

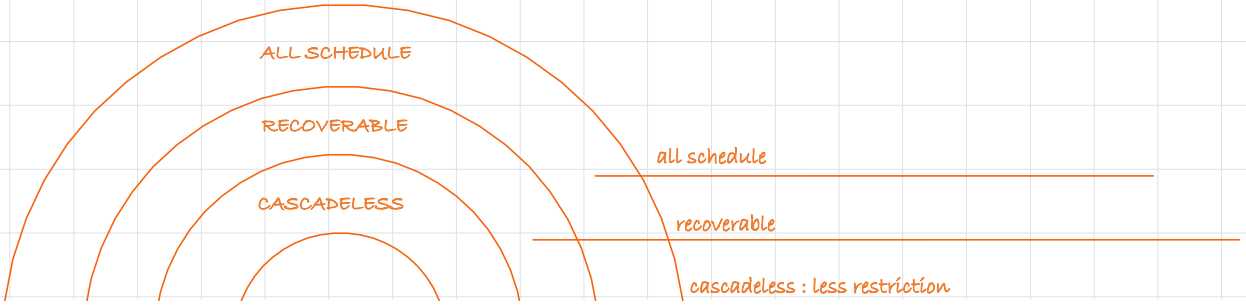
recoverable and cascadeless and strict but not conflict serializable

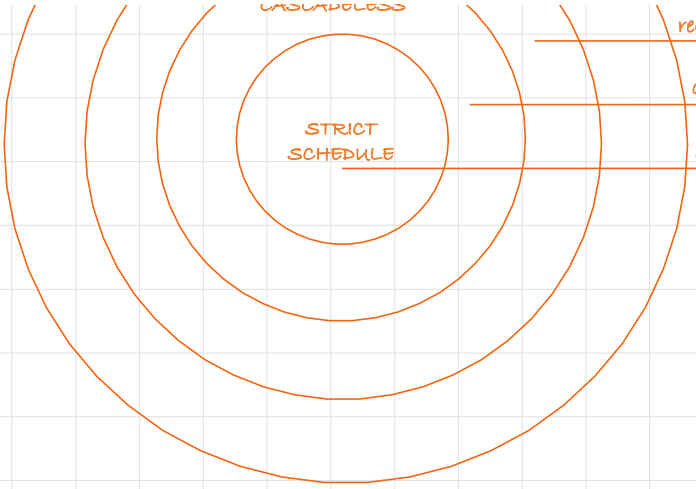
T_1	T_2
W(A)	
	R(B)
	W(A)
W(B)	
	commit
commit	

because no commit after write and there is another write

recoverable (no WR/ uncommited read) and cascadeless but not strict recoverable

Atomicity and Durability





recoverable

cascadeless : less restriction

strict schedule : more restriction

Q.1 $r_1(x)r_2(z)r_1(z)r_3(x)r_3(y)w_1(x)w_3(y)r_2(y)w_2(z)w_2(y)\ C_1\ C_2\ C_3$

↓ Commit 3

T_1	T_2	T_3
R(x)		
	R(z)	
R(z)		
		R(x)
		R(y)
W(x)		
		W(y)
	R(y)	
	W(z)	
	W(y)	
Commit		
	Commit	
		Commit

Not recoverable

Q.2 $r_3(x)\ r_1(x)w_3(x)r_2(x)w_1(y)\ r_2(y)w_2(x)C_3C_1\ C_2$

T_1	T_2	T_3
		R(x)
R(x)		
		W(x)
	R(x)	
W(y)		
	R(y)	
	W(x)	
		Commit
Commit		
	Commit	

Recoverable ✓

Cascadeless ✗ NO

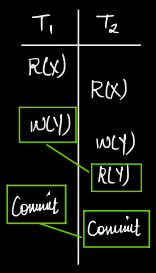
(∵ Having Uncommitted (Dirty) Read)

Only Recoverable.

Q.3 $r_1(x)\ r_2(x)w_1(y)w_2(y)r_2(y)\ C_1\ C_2$

T_1	T_2
R(x)	

① Recoverable ✓



① Recoverable ✓

② Cascadeless (NO uncommitted Read)

Q. Consider the following database schedule with two transactions, T_1 and T_2.

$S = r_2(X); r_1(X); r_2(Y); w_1(X); r_1(Y); w_2(X); a_1; a_2$

where $r_i(Z)$ denotes a read operation by transaction T_i on a variable Z, $w_i(Z)$ denotes a write operation by T_i on a variable Z and a_i denotes an abort by transaction T_i

Which one of the following statements about the above schedule is TRUE?

[MCQ:2016–2M]

A. S is non-recoverable

B. S is recoverable, but has a cascading abort

C. S does not have a cascading abort

D. S is strict

Sol.

T_1	T_2
	R(X)
R(X)	
	R(Y)
W(X)	
R(Y)	
	W(X)
abort	
	abort

① Recoverable

② Cascadeless - NO uncommitted Read.

Q. Let S be the following schedule of operations of three transactions T_1, T_2 and T_3 in a relational database system:

$R_2(Y), R_1(X), R_3(Z), R_1(Y), W_1(X), R_2(Z), W_2(Y), R_3(X), W_3(Z)$

Consider the statements P and Q below:

P: S is conflict-serializable.

Q: If T_3 commits before T_1 finishes, then S is recoverable.

Which one of the following choices is correct?

[MCQ: 2021-2M]

A. Both P and Q are true.

B. P is true and Q is false.

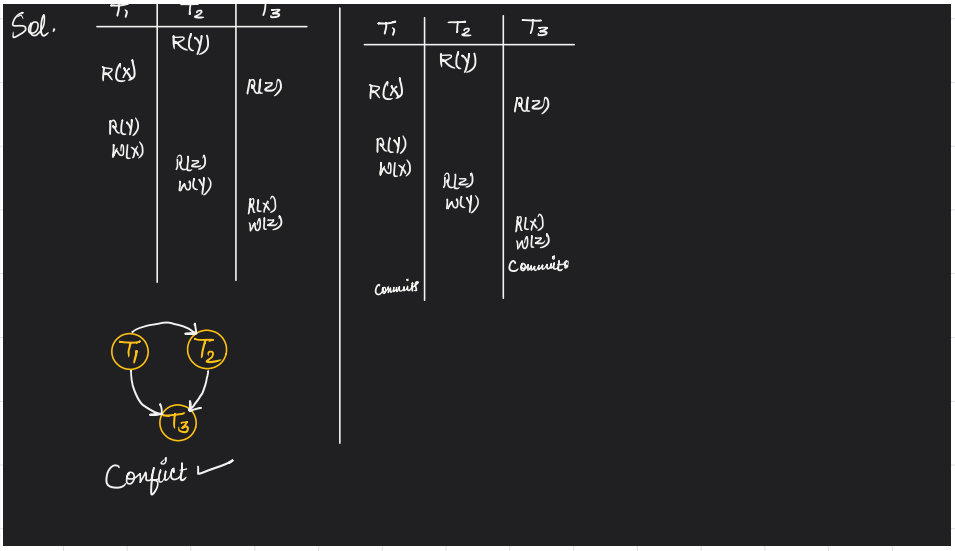
C. P is false and Q is true.

D. Both P and Q are false.

Sol.

T_1	T_2	T_3
	R(Y)	
R(X)		
		R(Z)

T_1	T_2	T_3
	R(Y)	
R(X)		



TOPIC :finding number of conflict serializable.

1. Write down all operation of the following (later) transaction.

$T_2 \longrightarrow (T_1)$ following / later transaction

2. Start from last operation of the first transaction and put at the correct place such that conflict operation order must be maintained same as transaction order.

$T_1 : R_1(X)\ W_1(X)\ R_1(Y)\ W_1(Y)$
$T_2 : R_2(Y)\ W_2(Y)\ R_2(Z)\ W_2(Z)$

(A) T_1 followed by T_2
step 1 : Write down all operation of the following (later) transaction.

_____$R_2(Y)$_____$W_2(Y)$_____$R_2(Z)$_____$W_2(Z)$_____

step 2 : Start from last operation of the first transaction and put at the correct place such that conflict operation order must be maintained same as transaction order.

(i) $W_1(Y)$

$W_1(Y)$ _____$R_2(Y)$ $W_1(Y)$ _____$W_2(Y)$ $W_1(Y)$ _____$R_2(Z)$ $W_1(Y)$ _____$W_2(Z)$ $W_1(Y)$ _____

transaction order
will change (T_2 to T_1)

rest of the operations are attached before $W_1(Y)$
eg. $R_1(X)\ W_1(X)\ R_1(Y)\ W_1(Y)\ R_2(Y)\ W_2(Y)\ R_2(Z)\ W_2(Z)$

(B) T_2 followed by T_1

step 1 : Write down all operation of the following (later) transaction.

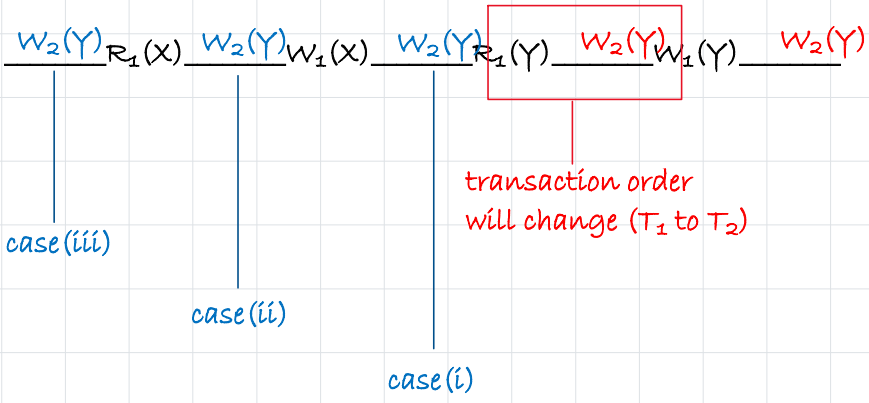
_____$R_1(X)$_____$W_1(X)$_____$R_1(Y)$_____$W_1(Y)$_____

step 2 : Start from last operation of the first transaction and put at the correct place such that conflict operation order must be maintained same as transaction order.

(i) $R_2(Z)$ $W_2(Z)$

$R_2(Z)$ $W_2(Z)$ can be placed anywhere (different data item) but only after $W_2(Y)$

(ii) $W_2(Y)$

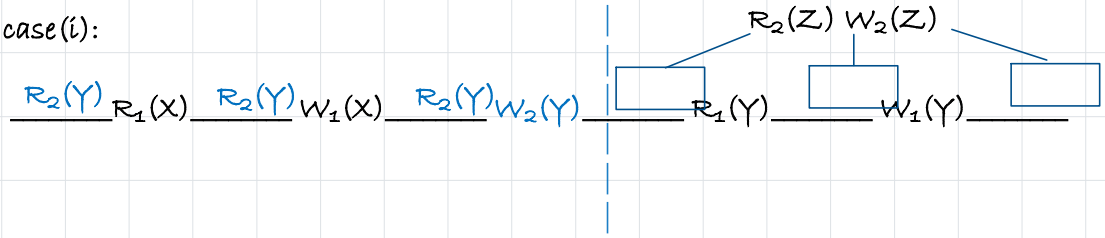


case (i): $R_1(X)$ $W_1(X)$ $W_2(Y)$ $R_1(Y)$ $W_1(Y)$

case (ii): $R_1(X)$ $W_2(Y)$ $W_1(X)$ $R_1(Y)$ $W_1(Y)$

case (iii): $W_2(Y)$ $R_1(X)$ $W_1(X)$ $R_1(Y)$ $W_1(Y)$

case (i):



- 3 ways to put $R_2(Y)$

3

- for $R_2(Z)$ $W_2(Z)$ we have 3 positions / places

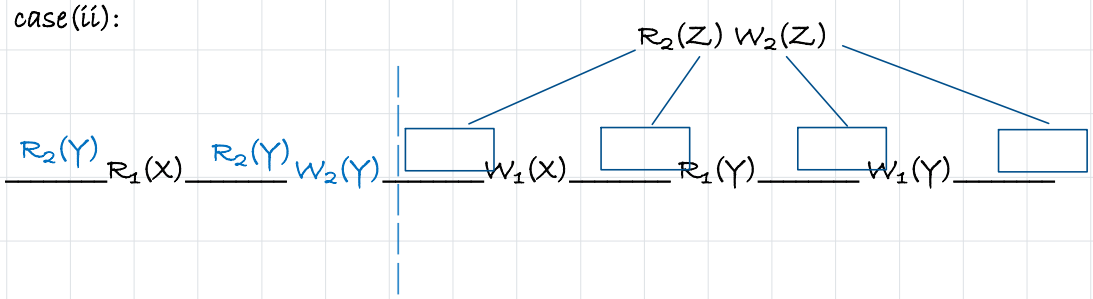
out of 3 places put them together $[^3C_1]$ (dono ko 1 room mein 3 mai se)

(OR)

out of 3 places put them separately $[^3C_2]$ (dono ko 2 alag room mein 3 mai se)

total : $3 \times [{}^3C_1 + {}^3C_2]$
: $3 \times [3 + 3]$
: 18 ways

case(ii):



- 2 ways to put $R_2(Y)$

2

- for $R_2(Z)$ $W_2(Z)$ we have 4 positions / places

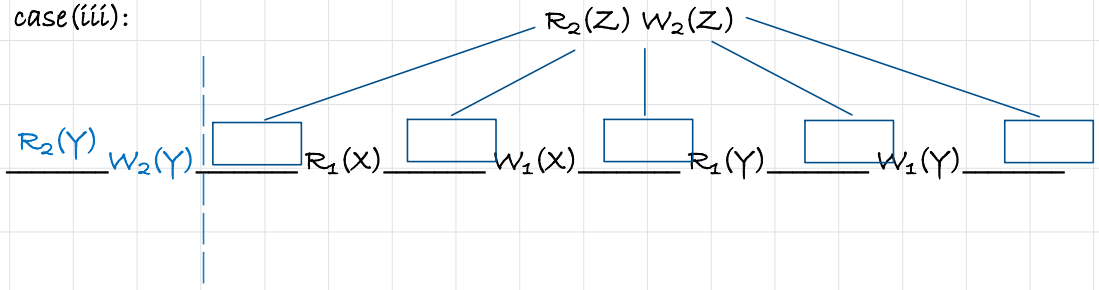
out of 4 places put them together $[{}^4C_1]$

(OR)

out of 4 places put them separately $[{}^4C_2]$

total : $2 \times [{}^4C_1 + {}^4C_2]$
: $2 \times [4 + 6]$
: 20 ways

case(iii):



- 1 way to put $R_2(Y)$

1

- for $R_2(Z)$ $W_2(Z)$ we have 5 positions / places

out of 5 places put them together $[{}^5C_1]$

(OR)

out of 5 places put them separately $[{}^5C_2]$

total : $1 \times [{}^5C_1 + {}^5C_2]$
: $1 \times [5 + 10]$
: 15 ways

Total cases in T_2 followed by T_1 : Case(i): 18 ways
Case(ii): 20 ways
Case(iii): 15 ways
total : 53 ways.

Total cases in T_1 followed by T_2 : Case(i): 1 way

Total number of conflict serializable : 54.

Q.2 [NAT]

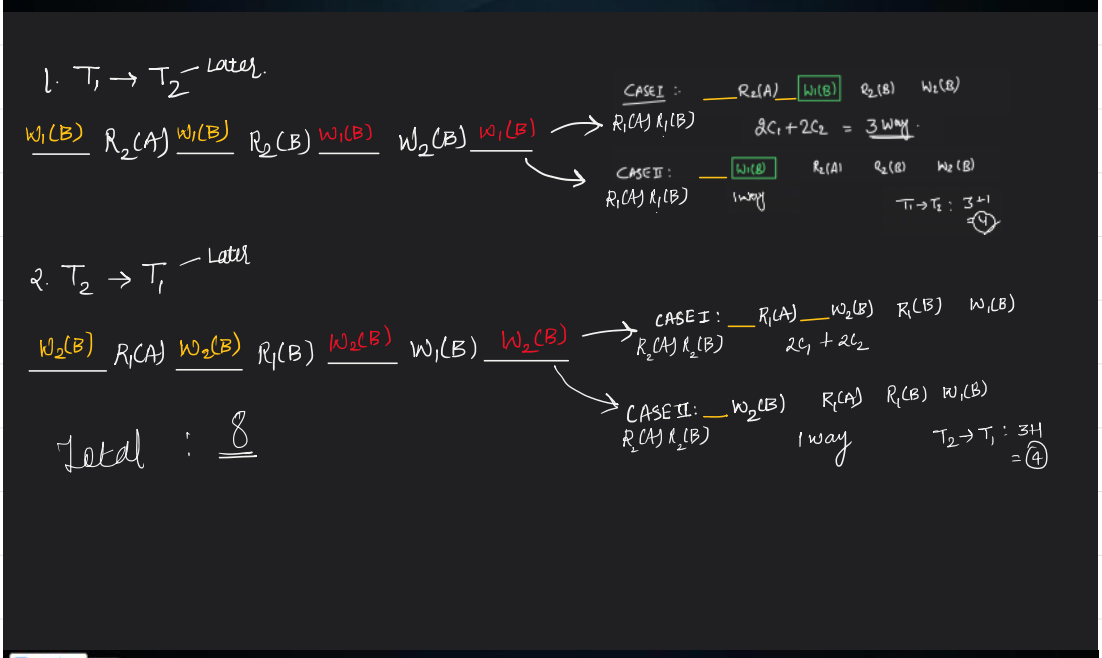
Consider the transaction T_1 and T_2 given below:

T_1 : $R_1(A)$ $R_1(B)$ $W_1(B)$

T_2 : $R_2(A)$ $R_2(B)$ $W_2(B)$

Where $R_i(A)$ denote a read operation by transaction T_i on a Data Item (A) W_i (B) Denote a write operation by transaction T_i on a Data Item B.

The Total number of conflict serializable schedule is 8.



Q.3 [NAT]

Consider the transaction T_1 and T_2 given below:

T_1 : $R_1(A)$ $R_1(B)$ $W_1(B)$

T_2 : $R_2(B)$ $R_2(A)$ $W_2(B)$

Where $R_i(A)$ denote a read operation by transaction T_i on a Data Item (A) W_i (B) Denote a write operation by transaction T_i on a Data Item B.

- schedule is not given.
- it is asking for conflict serializable.

Data Item (A) W_i (B) Denote a write operation by transaction T_i on a Data Item B.

The Total number of conflict serializable schedule is 5.

Sol. ① $T_1 \to T_2$ → LATER

$W_1(B)$ _____ $R_2(B)$ _____ $R_2(A)$ _____ $W_2(B)$ _____

└ 1 Place only.

② $T_2 \to T_1$ → Later

$W_2(B)$ _____ $R_1(A)$ $W_2(B)$ _____ $R_1(B)$ _____ $W_1(B)$ _____

2 Case: – (i) _____ $R_1(A)$ _____ $W_2(B)$ $R_1(B)$ $W_1(B)$

$2C_1 + 2C_2 = (3)$

(ii) _____ $W_2(B)$ $R_1(A)$ $R_1(B)$ $W_1(B)$

└ 1 PLACE ONLY. (1)

Total : $3+1=4$

unacademy

Q.4 Consider given schedule:

S: $r_1(x), r_2(y), w_3(y), r_4(x), w_4(z), w_3(y)$

How many conflict serializable schedules exists for the above schedule S? – 12

Sol

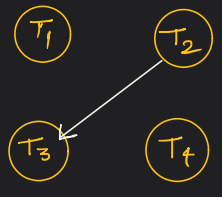
T_1	T_2	T_3	T_4
r(x)			
	r(Y)		
		w(Y)	
			r(x)
			w(z)
		w(Y)	

- schedule is given.
- it is asking for conflict serializable and not confict equivalent.
- we make cycle in conflict serializable.

1. Initial write : X : T_1 2. Final write : T_3 $[T_1 \to T_3]$

Y : T_2

3. WR sequence : No WR sequence



_ T_2 _ T_3 _

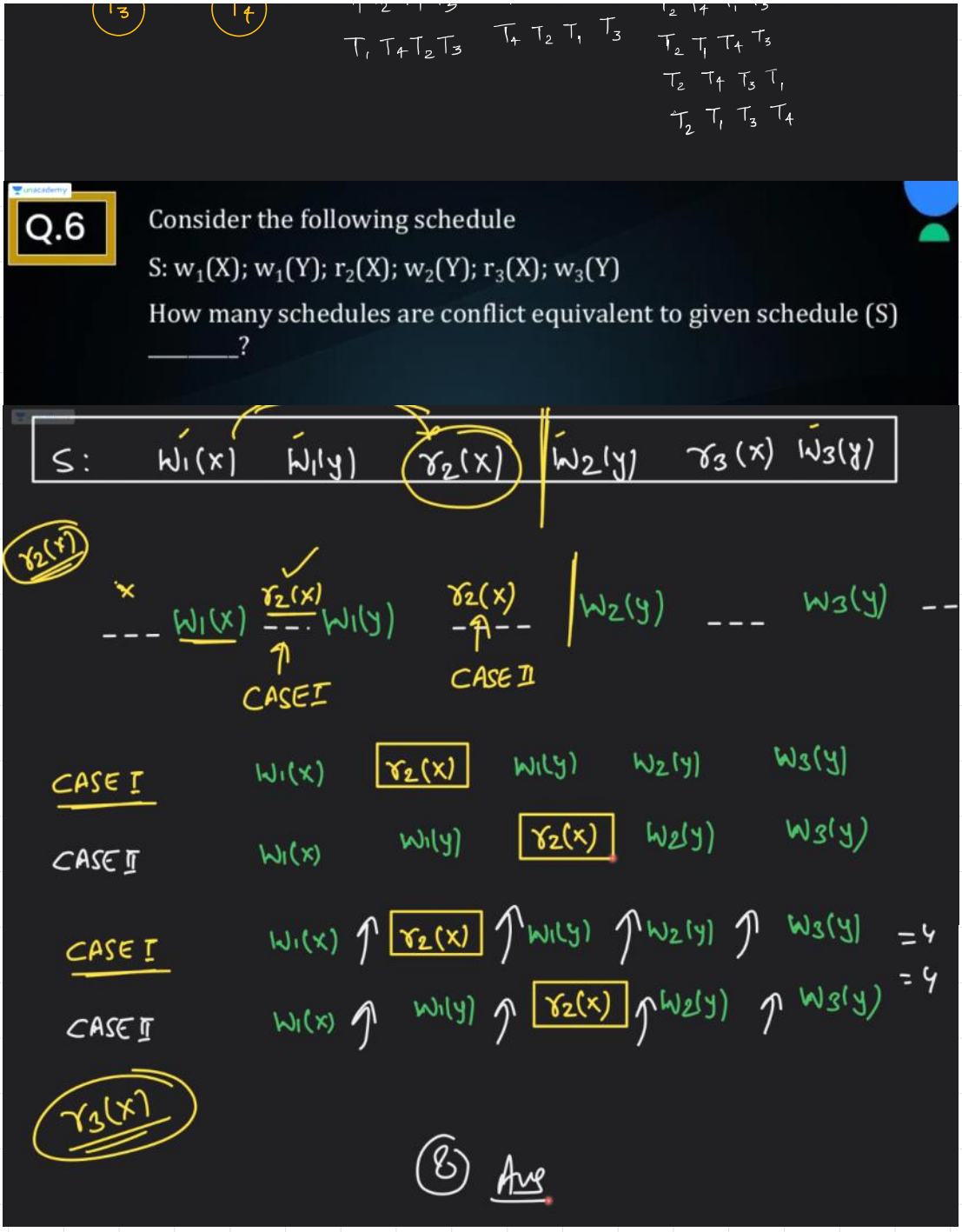
$T_1 T_2 T_3 T_4$ $T_4 T_2 T_3 T_1$ $T_2 T_3 T_1 T_4$

$T_1 T_2 T_4 T_3$ $T_4 T_1 T_2 T_3$ $T_2 T_3 T_4 T_1$

$T_1 T_4 T_2 T_3$ $T_4 T_2 T_1 T_3$ $T_2 T_4 T_1 T_3$

$T_2 T_1 T_4 T_3$

$T_2 T_4 T_3 T_1$



- schedule is given not transactions.
- it is asking for confict equivalent.

TOPIC :finding number of view equivalent schedule.

S : $r_1(P)$ $r_3(S)$ $w_1(Q)$ $r_2(Q)$ $r_4(Q)$ $w_2(R)$ $r_5(R)$ $w_4(T)$ $r_5(T)$ $w_5(Q)$

T_1	T_2	T_3	T_4	T_5
r(P)		r(S)		
w(Q)	r(Q)		r(Q)	
	w(R)			r(R)
			w(T)	r(T)
				w(Q)

1. Initial read : P : T_1
   S : T_3

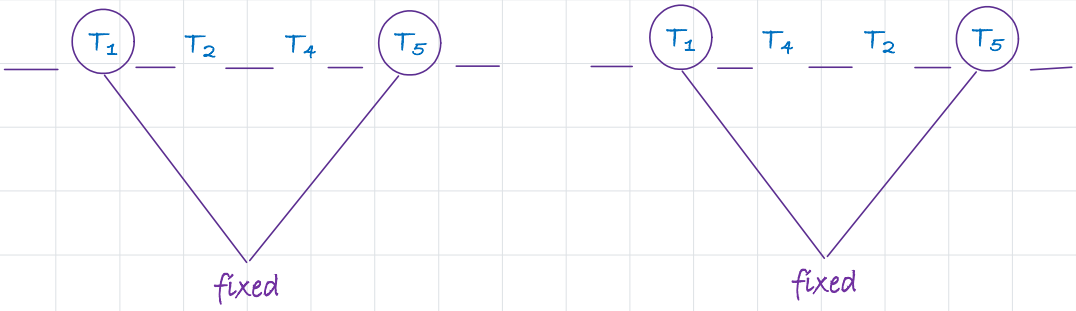
1. Final write : T_5 [T_1 to T_5]

3. Write read sequence : (i) T_1 to T_2
   (ii) T_1 to T_4
   (iii) T_2 to T_5
   (iv) T_4 to T_5

   T_1 appear before T_2 and T_4

   T_5 appear after T_2 and T_4

For T_3 : 10 places.



total possible serial schedules that will be view equal to s : 10

TOPIC : implementationof concurrency control scheme

(comes under isolation) (all transaction follow these rules)

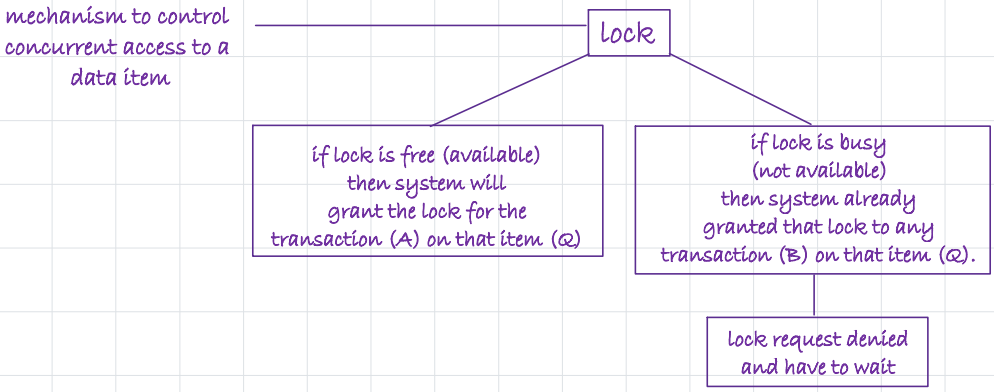
one way to ensure isolation is to require the data item to be accessed in a mutually exculsive manner.

- isolation : lots of people working concurrently on same bank account but their transactions are hidden or isolated from each other

- mutually exculsive : when one transaction access the data item then no other transaction can update/modify that item simuntaneously (at that time)

how to implement this process : we allow a transaction to access a data item only if transaction is getting a lock on that data item.
- with the help of lock based protocol.

1. Lock based protocol : before using any data item transaction request for a lock for that data item.



to implement lock based protocol we need two locks

1. Shared lock ≡ Shared mode on : S(A)
   Data item (A)
   S-lock is requested using lock-s instruction.

- use : when we want to perform only read operation.

- Syntax : S(A)
  .
  .
  R(A)
  unlock_S(A) (released after work is done)

2. Exclusive lock ≡ Exclusive mode on : X(A)
   Data item (A)

- use : when we want to perform only write ~~write(A)~~ ~~Read(A)~~ operation.
  ~~Read(A)~~ ~~write(A)~~

  X-lock is requested using lock-x instruction.

- Syntax : X(A)
  W(A) (or) R(A) (or) W(A)
  W(A) (or) R(A)

Lock compatible matrix :

T_j

T_i	data item (A)	S	X
	S	YES	NO
	X	NO	NO

T_i	T_j
s(A)	
	s(A)

granted
(because shared only read-read)

T_i	T_j
s(A)	
	x(A)

denied and transaction has to wait

T_i	T_j
x(A)	
	s(A)

denied and transaction has to wait

T_i	T_j
x(A)	
	x(A)

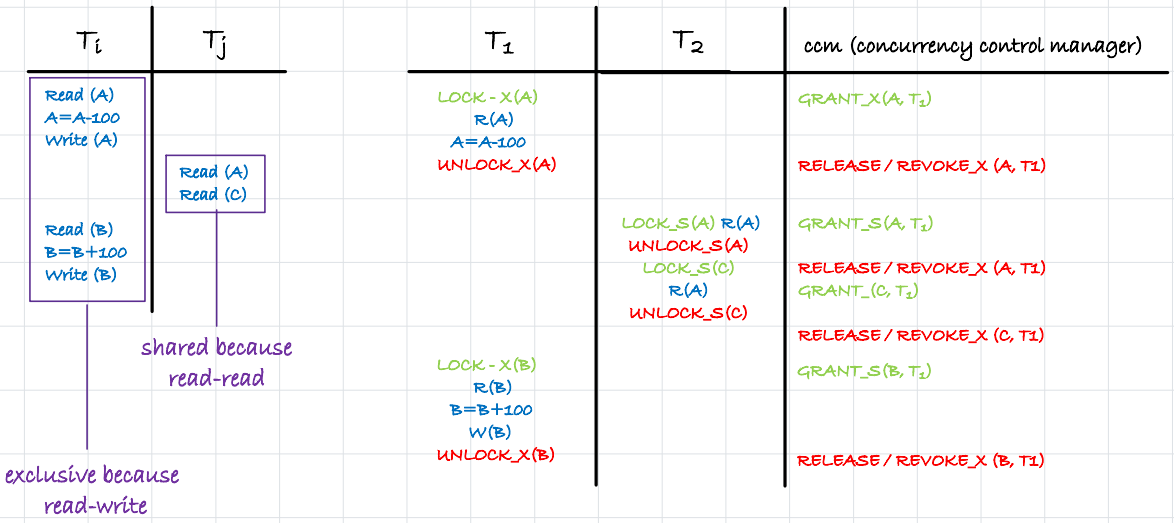
denied and transaction has to wait

a locking protocol is a set of rules followed by all transactions while requestions and releasing locks.

why use locks?
locking protocol enforce serializability by restricting the set of possible schedules.

T_1	T_2	Concurrency-control manager
lock-X(B)		grant-X(B, T_1)
read(B)		
B:=B – 50		
write(B)		
unlock(B)		
	lock-S(A, T_2)	grant-S(A, T_2)
	read(A)	
	unlock(A)	
	lock-S(B)	grant-S(B, T_2)
	read(B)	
	unlock(B)	
	display(A+B)	
lock-X(A)		grant-X(A, T_1)
read(A)		
A:=A + 50		
write(A)		
unlock(A)		

maintains / record everything



these locking protocol ensure serializability so we will go for 2PL (two phase locking protocol)

1. 2PL (two phase locking protocol)

Each transition issue lock and unlock request in two phase.

(A) Growing phase (acquire the lock) : Each transacation may

(i) transaction may obtain locks.

(ii) transactiin may not release locks.

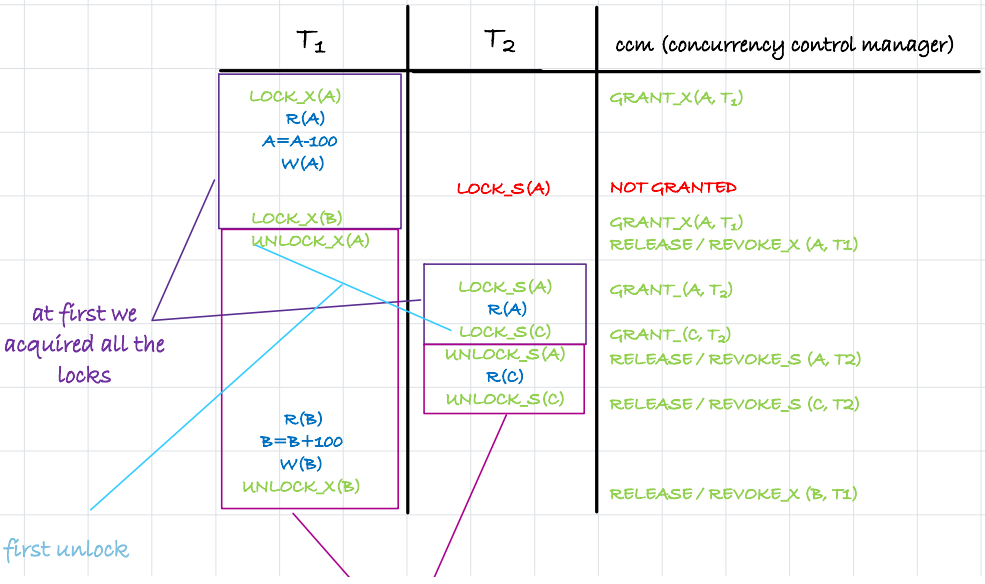
(B) Shrinking phase (release the lock) : Each transacation may

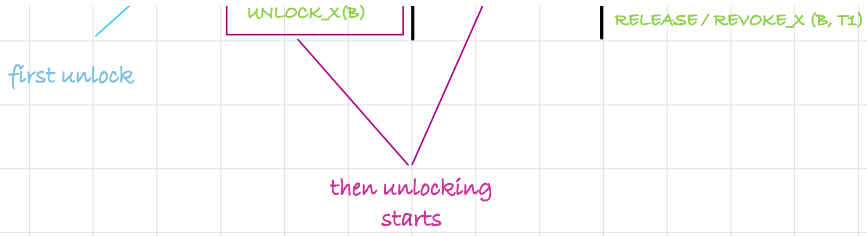
(i) transaction may release locks.

(ii) transaction may not obtain locks.

Note:

each transaction first complete its growing phase then start shrinking phase. once shrinking phase start transaction cannot get any new lock.





2PL(two phase locking protocol) important points :

1. 2PL ensure serializability (conflict/view serializability)

2. If a schedule is followed (allowed) by 2PL then it ensure conflict serializabile schedule.

3. Serializability order determined by the LOCK POINT.

Lock point : lock point is a point where shrinking phase of the transaction starts

(OR)

Position of the last lock (OR) first unlock

T_1	T_2	T_3
	*	
*		
		*

serializability order $<T_2T_1T_3>$

4. 2PL does not ensure recoverability.

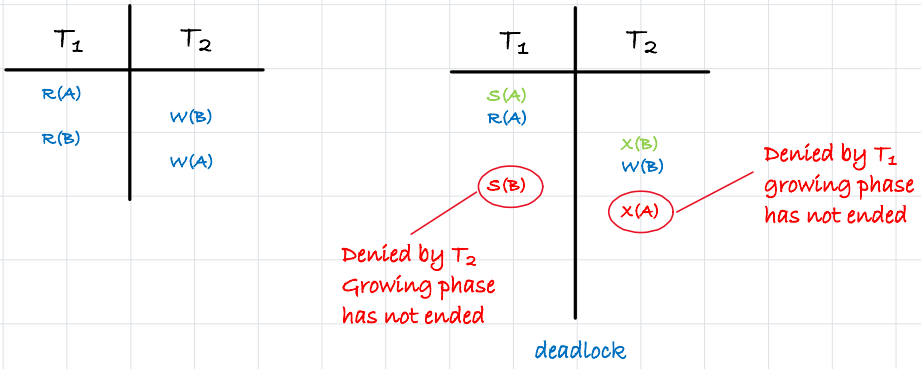
T_1	T_2
w(A)	
	R(A)
	commit
commit	

irrecoverable schedule

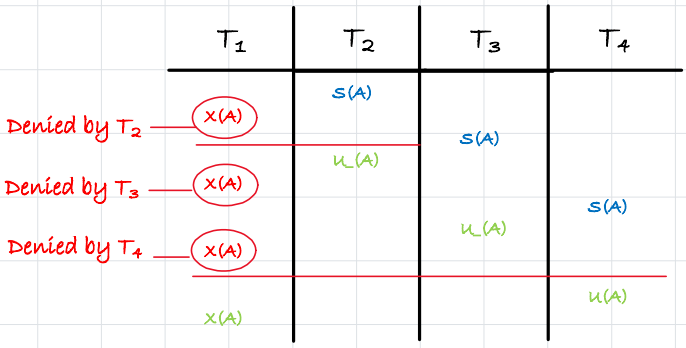
T_1	T_2
X(A)	
w(A)	
U(A)	
	S(A)
	R(A)
	U(A)
	commit
commit	

irrecoverable schedule but (allowed) followed by 2PL

5. 2PL suffers from deadlock (infinte waiting).



6. 2PL suffers from starvation.

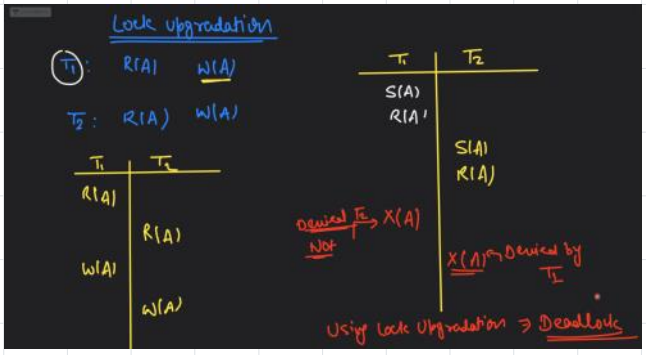


- Lock upgradation technique (not in GATE) :

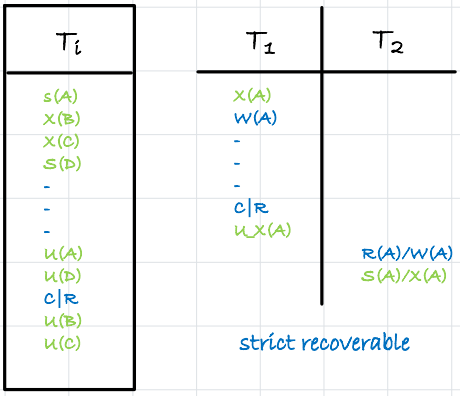
SHARED $\xrightarrow{\text{upgrade}}$ EXCLUSIVE : upgrade

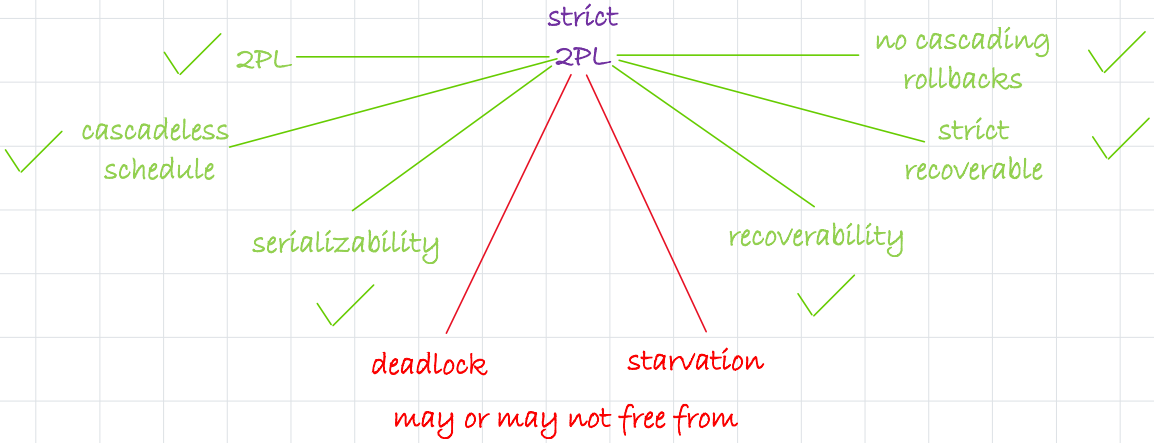
EXCLUSIVE $\xrightarrow{\text{downgrade}}$ SHARED : downgrade

using upgradation : still deadlock.



1.1 Strict 2PL : 2PL + All exclusive lock [X-lock] taken by transaction will be released only after commit or rollback. (this ensures recoverability and avoids cascading rollbacks)





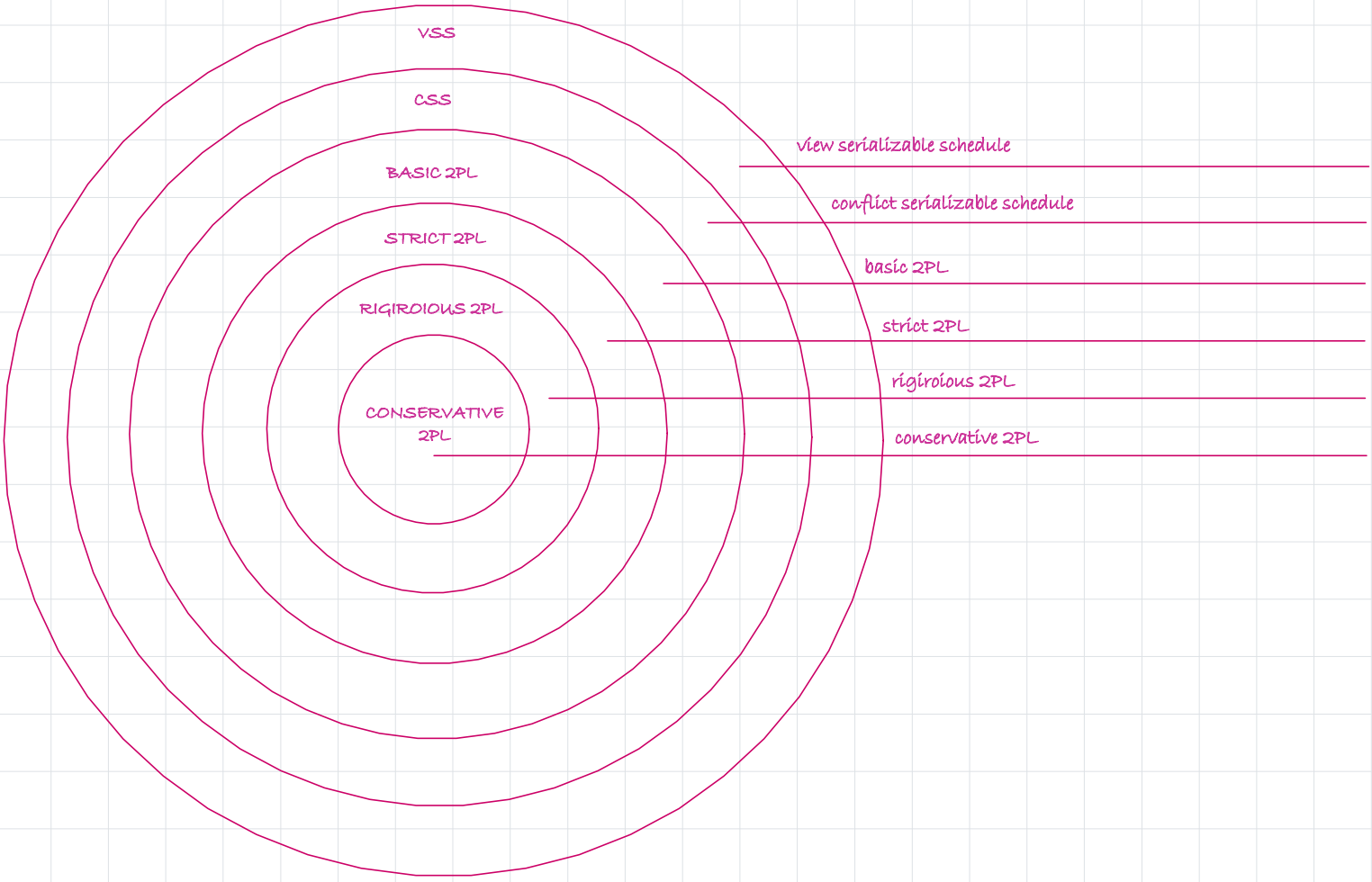
1.2. Rigourious 2PL : 2PL + All locks [shared and exclusive] will be released only after commit or rollback.

- Serializability order determined by the order of commit operation of the transaction.

- Rigorous 2PL also suffers from deadlock and starvation.

1.3. Conservative 2PL : 2PL + each transaction acquire the lock before the start of the execution (in the beginning of transaction)

(ensures no deadlock but starvation still exists)



Basic 2PL	Strict 2PL	Rigiroious 2PL	Conservative 2PL	not 2PL
1	2	3	4	5
Lock-S(A)	Lock-S(A)	Lock-S(A)	Lock-S(A)	Lock-S(A)
R(A)	R(A)	R(A)	Lock-X(B)	R(A)
Lock-X(B)	Lock-X(B)	Lock-X(B)	R(A)	Unlock(A)
R(B)	Unlock(A)	R(B)	R(B)	Lock-X(B)
Unlock(A)	R(B)	W(B)	W(B)	R(B)
W(B)	W(B)	Commit	Commit	W(B)
Unlock(B)	Commit	Unlock(A)	Unlock(A)	Unlock(B)
Commit	Unlock(B)	Unlock(B)	Unlock(B)	Commit

TOPIC : Timestamp based concurrency control

Timestamp protocol : a unique timestamp value assigned to each transaction when they arrive in the system. (by dbms software)

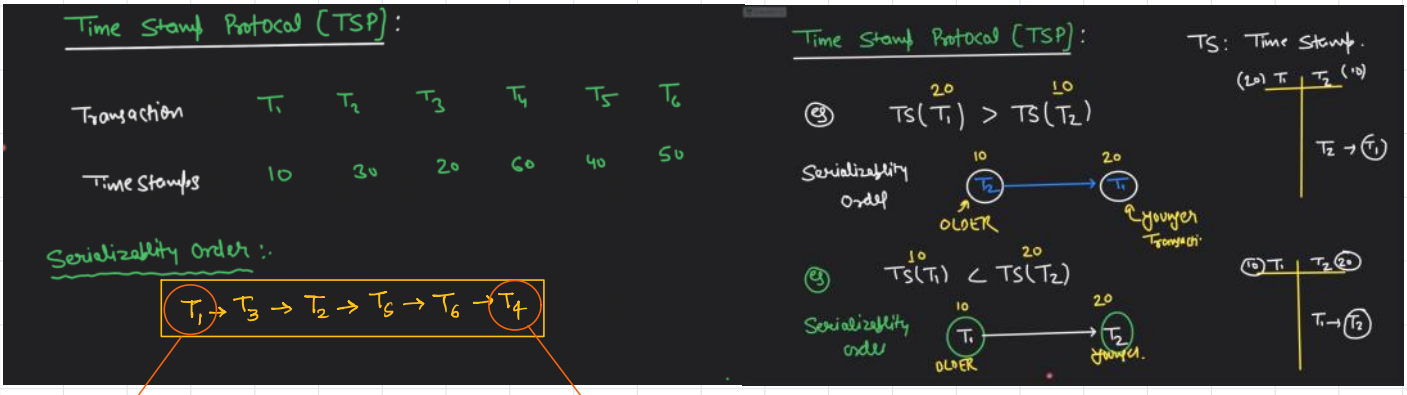
- based on their timestamp we determine the serializability order (serializability order pre-defined based on their timestamp)

Timestamp of T_1 : 10

Timestamp of T_2 : 20

serializability order : $T_1 \longrightarrow T_2$

T_1 followed by T_2



older transaction

younger transaction

- Each transaction T_i is issued a timestamp $TS(T_i)$ when it enters the system.
  - Each transaction has a unique timestamp
  - Newer transaction have timestamp strictly greater than earlier ones
  - Timestamp could be based on a logical counter
    - Real time may not be unique
    - Can use (wall-clock time, logical counter) to ensure
- Timestamp-based protocols manage concurrent execution such that **time-stamp order = serializability order**
- Several alternative protocols based on timestamps

types of timestamp :

1. Transaction timestamp [TS(ti)] : fixed.

2. Timestamp of data item [data item time stamp] : variable
   - read_timestamp (Q) RTS(Q)
   - write_timestamp (Q) WTS(Q)

(A) Read_timestamp (Q) : RTS(Q)

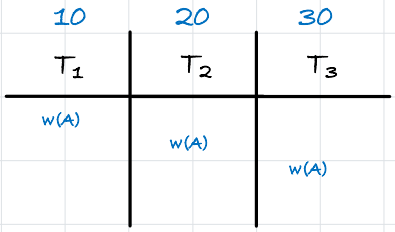
denotes the highest (largest) transaction timestamp that performers Read(Q) operation successfully

initially RTS = 0
RTS = 10 [max(0,10)]
RTS = 20 [max(10,20)]
RTS = 30 [max(20,30)]

RTS (A) = 30

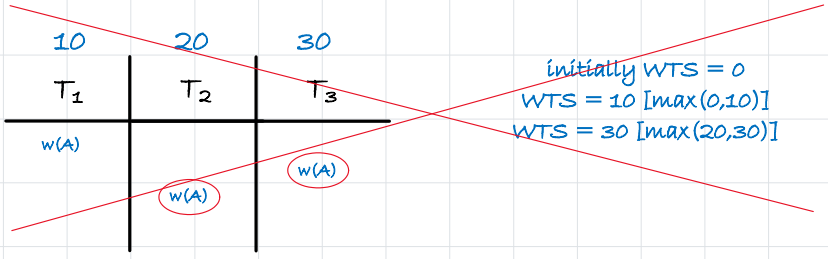
(B) write_timestamp (Q) : RTS(Q)

denotes the highest (largest) transaction timestamp that performers write(Q) operation successfully



initially WTS = 0
WTS = 10 [max(0,10)]
WTS = 20 [max(10,20)]
WTS = 30 [max(20,30)]

WTS (A) = 30



initially WTS = 0
WTS = 10 [max(0,10)]
WTS = 30 [max(20,30)]

not allowed

Conflict operations / Instructions	non-conflict operations
Read(Q) - write(Q) Write(Q) - Read(Q) Write(Q) - write(Q) same data item (Q)	Read(Q) - write(S) Write(S) - Read(P) Write(S) - write(P) same data item R(Q) - R(Q)

1. if transaction T_i wants to issue / performs Read (Q)

T_i : Read (Q)

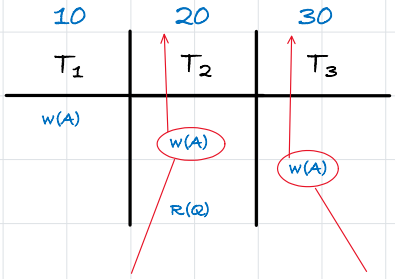
Check : (i) if TS(T_i) < WTS(Q) (less than the time stamp of data item Q)

Read operation reject and T_i rollback (will not work)

(ii) if $TS(T_i) > WTS(Q)$ (more than the time stamp of data item Q)

Read operation allowed

and set $RTS(Q) = max[TS(T_i), RTS(Q)]$



$TS(T_i) = 20$

$WTS(Q) = 30$

$TS(T_i) < WTS(Q)$

not allowed

reading old data that has been overwritten

data is updated.

2. if transaction T_i wants to issue / performs Write (Q)

T_i : Write (Q)

Check : (i) if $TS(T_i) < RTS(Q)$ (less than the time stamp of data item Q)

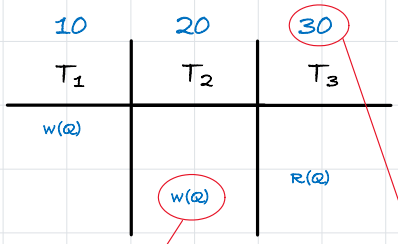
Write operation reject and T_i rollback (will not work)

(ii) if $TS(T_i) < WTS(Q)$ (less than the time stamp of data item Q)

Write operation reject and T_i rollback (will not work)

(iii) otherwis write operation is allowed

and set $RTS(Q) = max[TS(T_i), WTS(Q)]$



$TS(T_2) = 20$

$TS(T_i) = 20$

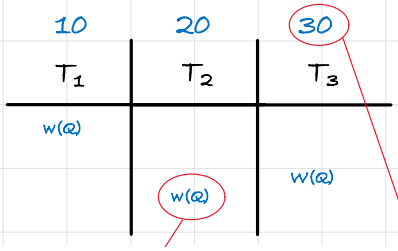
$RTS(Q) = 30$

$TS(T_i) < RTS(Q)$

not allowed

writing the old data that has been overwritten

data is updated.



$TS(T_2) = 20$

$TS(T_i) = 20$

$WTS(Q) = 30$

$TS(T_i) < WTS(Q)$ (obselete write : writing the old value)

writing the old data that has been overwritten

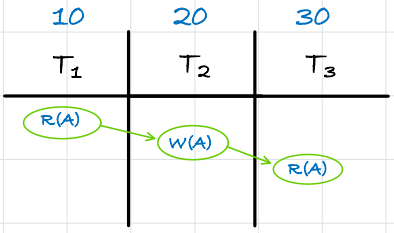
data is updated.

$TS(T_i) < WTS(Q)$ (obselete write : writing the old value) not allowed

final and best approach for Timestamp based protocols -
impose rules on read and write operations to ensure that :
(A) any conflicting operations are executed in timestamp order
(B) out of order operations cause transaction rollback

note :

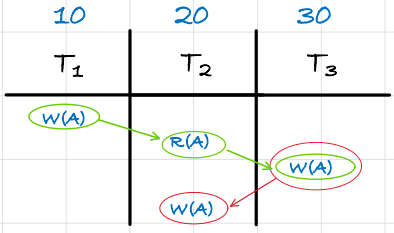
If serializability order is (based on timestamp) T_1 to T_2 [T_1 followed by T_2] then all the conflict operations order must be same [T_1 to T_2] as transaction serializability order. if not then transaction reject and rollback.



T_1 to T_2 to T_3

its TSP

timestamp order = serializability order



T_1 to T_2 to T_3

but conflict operation
$W_3(A)$ - $W_2(A)$
T_3 to T_2
not allowed under TSP

T_2 rollback and restart with new (higher/latest) timestamp
new $TS(T_i) > 30$

T_i : Write(Q)
(i) $TS(Ti) < RTS(Q)$
(ii) $TS(Ti) < WTS(Q)$
otherwise write operation allowed.

write operation reject and T_i rollback and restart with new (higher/latest) timestamp.
rollback : reverse arrow.

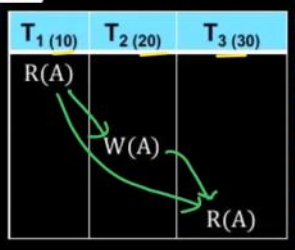
note:

all the conflicting operation order must be same as transaction timestamp ordering

note:

if, transaction timestamp ordering = conflict operation ordering then under TSP.

1



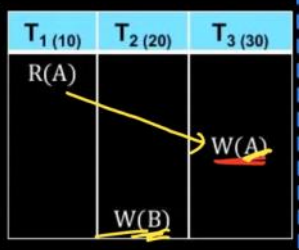
Serializability order

$T_1 \rightarrow T_2 \rightarrow T_3$

$< T_1, T_2, T_3 >$

YES its allowed Under TSP.

2

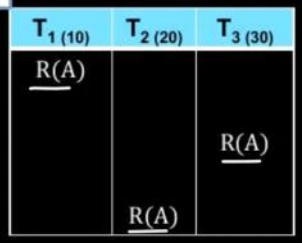


$< T_1, T_2, T_3 >$

$R_1(A) - W_3(A)$: $T_1 \rightarrow T_3$

YES its allowed Under TSP

3

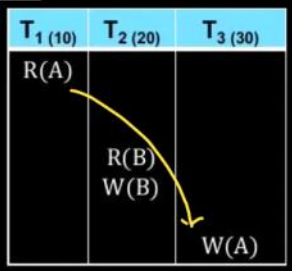


$< T_1, T_2, T_3 >$?

No Conflict operation

YES its allowed Under TSP.

4

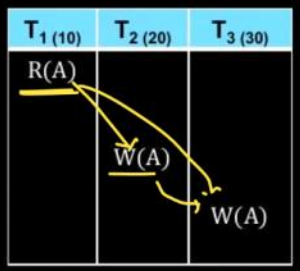


$T_1 \rightarrow T_2 \rightarrow T_3$

$R_1(A) - W_3(A)$: $T_1 \rightarrow T_3$ ✓

YES it allowed Under TSP.

5



$T_1 \rightarrow T_2 \rightarrow T_3$

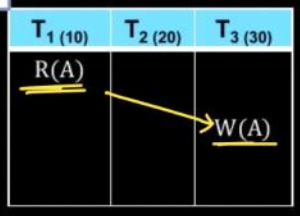
$R_1(A) - W_2(A)$: $T_1 \rightarrow T_2$

$R_1(A) - W_3(A)$: $T_1 \rightarrow T_3$

$W_2(A) - W_3(A)$: $T_2 \rightarrow T_3$

YES its allowed Under TSP.

6

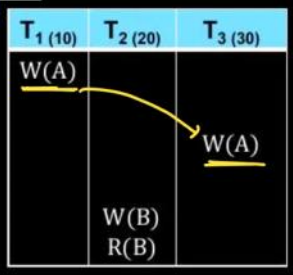


$T_1 \rightarrow T_2 \rightarrow T_3$

$R_1(A) - W_3(A)$: $T_1 \rightarrow T_3$

YES its allowed. Under TSP.

7

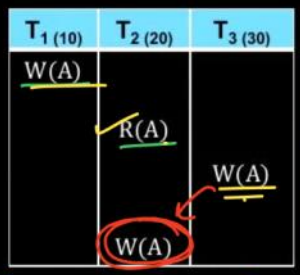


$T_1 \rightarrow T_2 \rightarrow T_3$

$W_1(A) - W_3(A)$: $T_1 \rightarrow T_3$

YES its allowed Under TSP

8



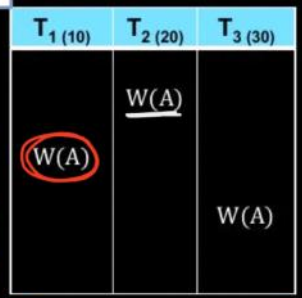
$T_1 \rightarrow T_2 \rightarrow T_3$

$W_3(A) - W_2(A)$: $T_3 \rightarrow T_2$

Not allowed Under TSP.

& T_2 Rollback.

9



$T_1 \rightarrow T_2 \rightarrow T_3$

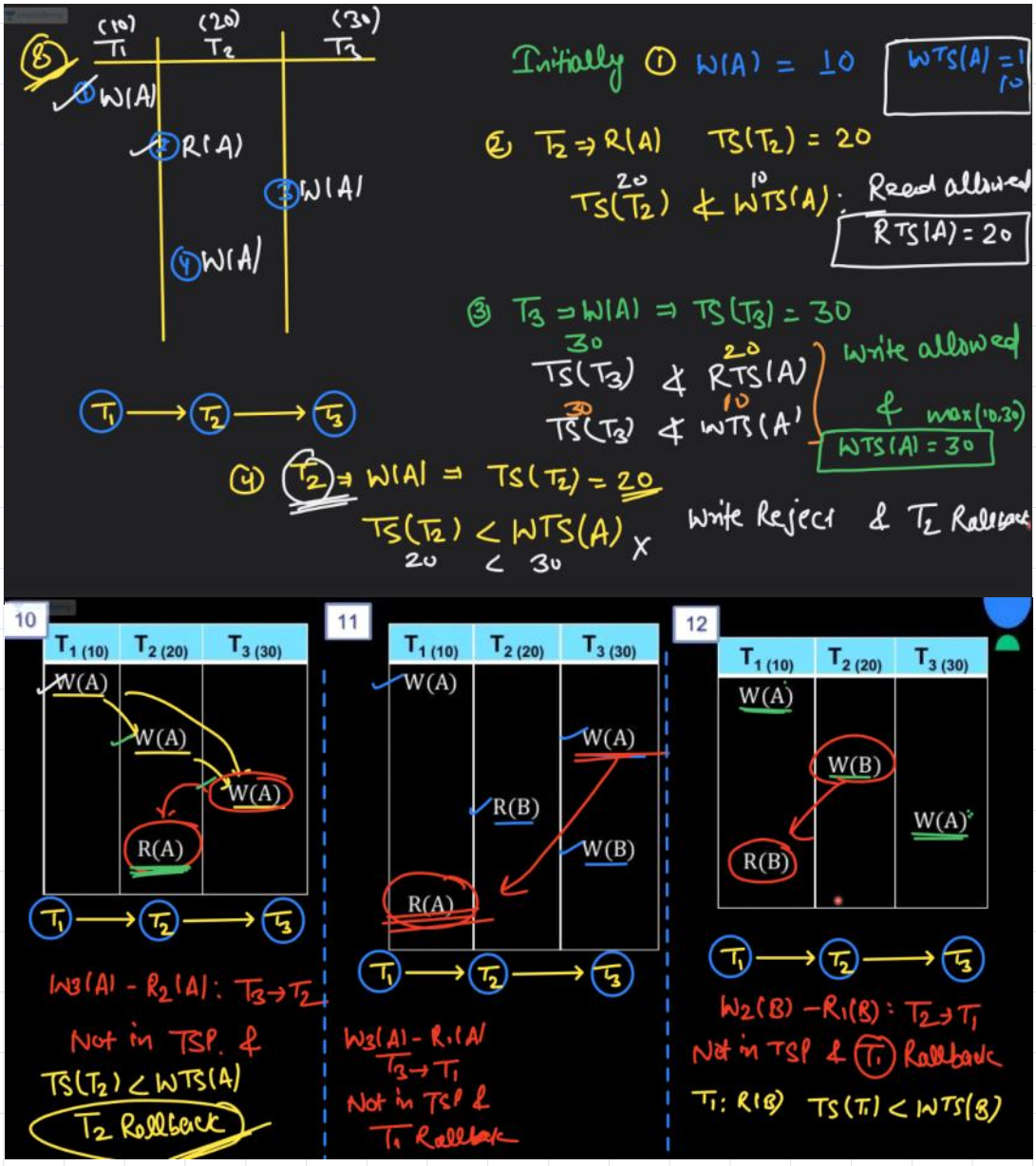
$W_2(A) - W_1(A) = 1$ $T_2 \rightarrow T_1$

Not allowed Under TSP.

T_1 Rollback

8

Initially ① $W(A) = 10$ $WTS(A) = 1$

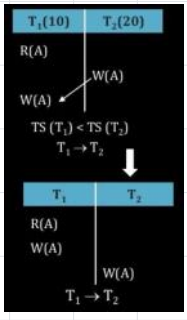


Thomas write rule (helps to achieve view serializability)

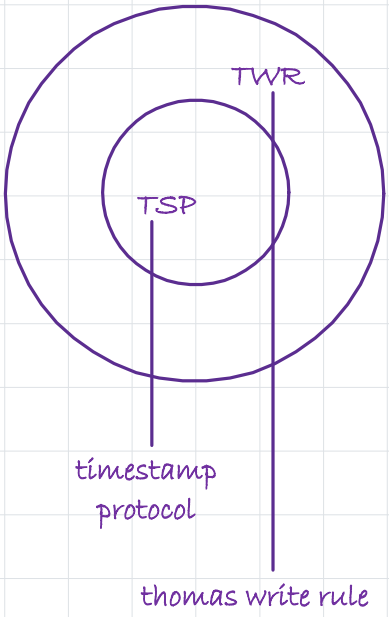
some little modification in timestamp protocol
TWR allows obselete write but obselete write is not allowed under TSP

1. T_i: Read(Q)
(i) $TS(T_i) < WTS(Q)$: reject and T_i rollback
(same as tsp)

2. T_i: write(Q)
(i) $TS(T_i) < WTS(Q)$: write ignored and no rollback
rest same as TSP.



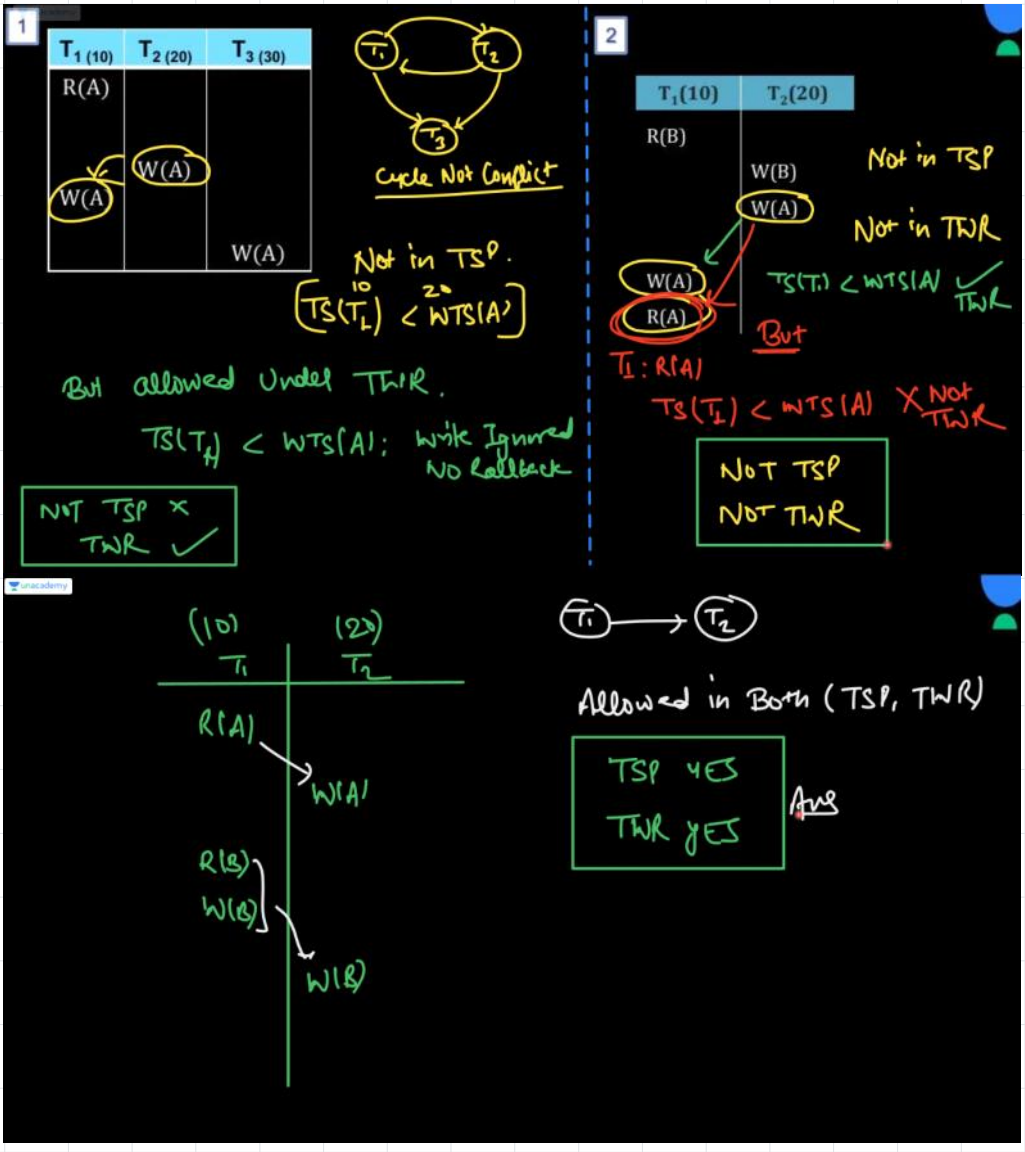
not in TSP but
allowed under TWR



1. If schedule is followed by TSP then already follwed by TWR.
2. If schedule is not followed by TSP, then it may allowed by Thomas write rule (TWR)

note:

there are some schedule which is not allowed by TSP but allowed by TWR



Strict time stamp ordering : timestamp protocol + strict schedule

- Ensure serializability
- Ensure recoverability
- Cascadeless
- Avoid cascading rollback
- Strict recoverable

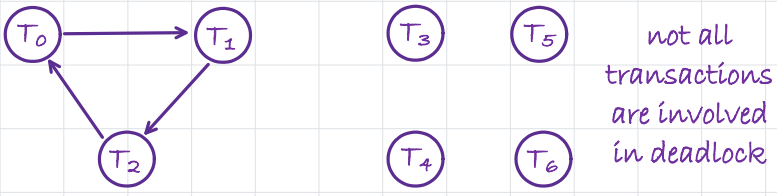
T_1	T_2
W(A)	
C|R	
	R(A)/W(A)

Topic : Deadlock prevention algorithm

Deadlock : a system is in a deadlock if there exist some set of transactions such that every transaction in a set is waiting for another transaction in a set.

eg. $T_0, T_1, T_2, T_3, T_4 \ldots\ldots\ldots\ldots T_{100}, T_{101}$
but here deadlock can occur some set of transactions and not all transactions.

$T_0, T_1, T_2, T_3, T_4, T_5, T_6$



deadlock : T_0, T_1, T_2

here, circular wait
T_0 request data item (lock) which is hold by T_1
T_1 request data item (lock) which is hold by T_2
T_2 request data item (lock) which is hold by T_3

deadlock by some set of transactions

note:

if we say that system is in deadlock it means some set of transactions are in a deadlock and it does not mean that all the transactions in system are in deadlock.

- Deadlock prevention (wait-die and wound-wait)
- Deadlock detection (wait for graph)

Deadlock prevention strategy : if we implement deadlock prevention strategy then deadlock will not occur.

approaches :
1. Conservative 2PL
2. Partial ordering

approaches :
1. Conservative 2PL
2. Partial ordering

not study because of problems.

- Conservative 2PL : acquire all the locks before the start of execution hence no deadlock but problem is implementation issue because in the beginning its difficult to know which operation is going to be perfomed by that transaction in future. also there is a starvation issue.
- Partial ordering : numbering assign to each transaction.

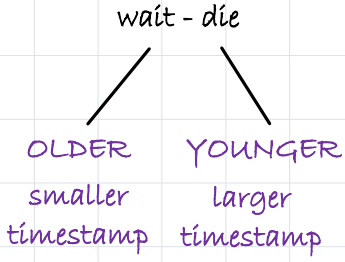
Q.1. which one is older transaction?
transaction which having smaller timestamp (came early in the system is older transaction.

Q.2. which one is younger transaction?
transaction which having higher (larger/greater) timestamp is younger transaction.

Q.3. Which transaction get more priority?
older get more priority because arrived in the system early and already did more work or computation as compared to the younger transaction.

3. Based on timestamp

1. wait - die : (T_i is requesting a data item (lock) that is currently being hold by T_j
(non pre-emptive) T_i is allowed to wait only if its (T_i) timestamp is smaller than that of T_j
otherwise T_i rollback (abort/die)



Data item hold by T_j and requesting by T_i
- if T_i is OLDER (smaller time stamp) : T_i will wait.
- if T_i is YOUNGER (larger time stamp) : T_i will rollback (abort/die).

ex. T_i (10) Data item (A)
T_j (20)

wait - die : T_i will wait

ex. T_i (10) Data item (A) (lock)
T_j (20)

wait - die : $TS(T_i)$ will wait.
$TS(T_i) < TS(T_j)$

ex. T_i (30) Data item (A) (lock)
T_j (20)

wait - die : $TS(T_i)$ will die. (rollback)
$TS(T_i) > TS(T_j)$

2. wound wait : T_i is requesting a data item(lock)that is currently being hold by T_j
(pre-emptive) T_i is allowed to wait only if its (T_i) timestamp is larger than that of T_j
otherwise T_j rollback (T_j is wound (abort by) T_i)

older wound the
younger transaction

T_j is holding a data item[lock(A)] and T_i is requesting that item(A)/ Lock(A)

- If $TS(T_i) < TS(T_j)$
then T_i (older)transaction wound (abort) younger T_j and take the lock (forcefully) from T_j

- If $TS(T_i) > TS(T_j)$
then T_i (younger) will wait T_j for to release.

note:

when transaction abort.die then restart with the original(same) timestamp, do not restart with higher timestamp because if we restart with higher transaction then that transaction wil become younger and we will again face rollback/abort (starvation)

- older transaction gets priority

ER model

Topic : Entity relationship model

1. Entity : Object. Physical : book

logical : account

1.1. Entity set : Collection / group of similar entities

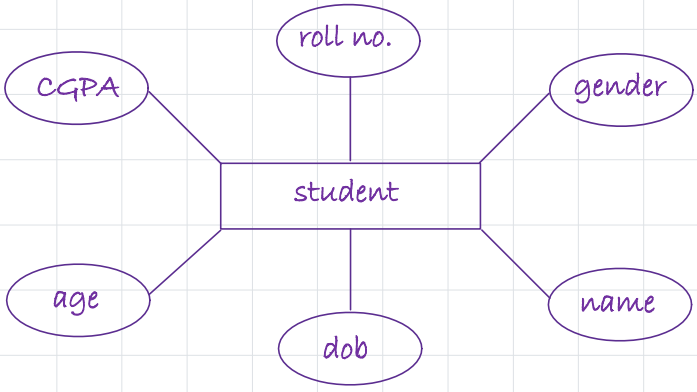
for ex: STUDENT

roll no.	name	gender
1	aryan	male
2	iram	female

entity set (schema)

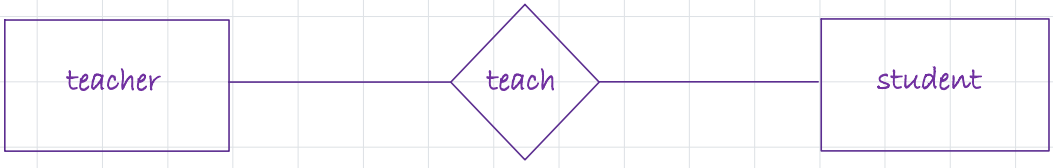
entities (instances)

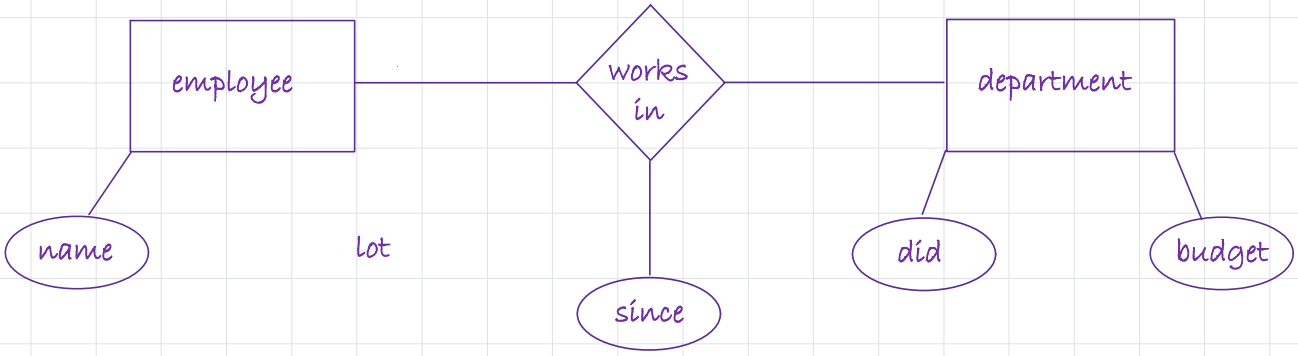
it is a set of entities of same type denoted by a rectangular box in ER diagram. entity can be defined by a list of attributes which are placed in ovals.



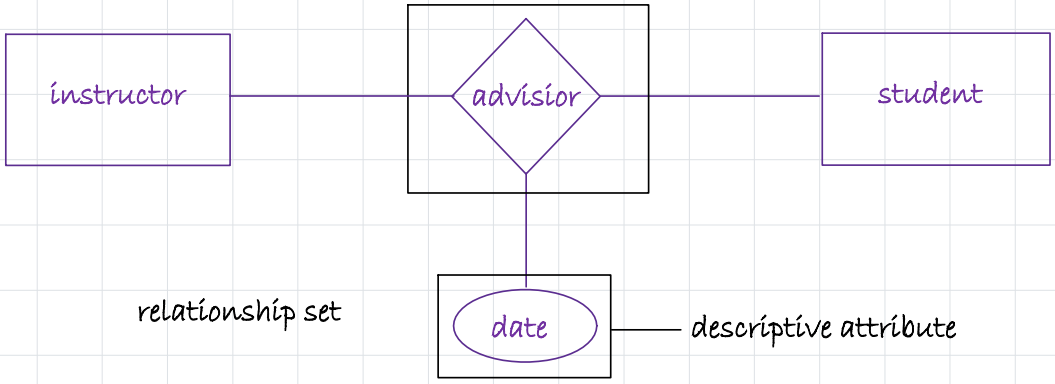
2. Relationship : assosication among the entities
(verb)

2.1. Relationship set : collection of the relationships among the entities
- represented by a diamond.

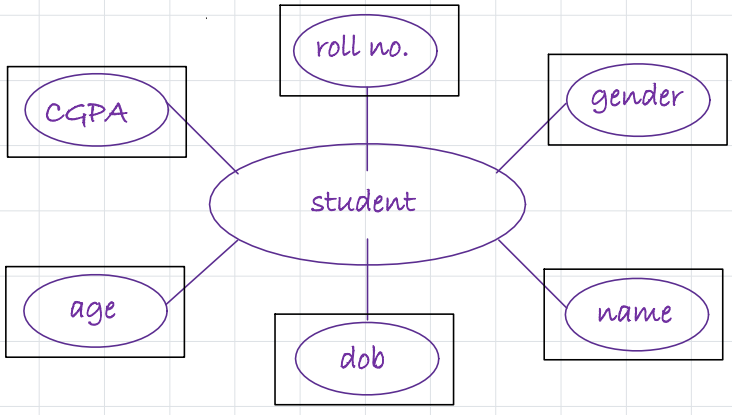




an attribute can also be property of a relationship set.



3. Attribute : attribute are the properties which describe the entity



Attributes

3.1. Attribute types :

(i) simple and composite attributes.

3.1. Attribute types :

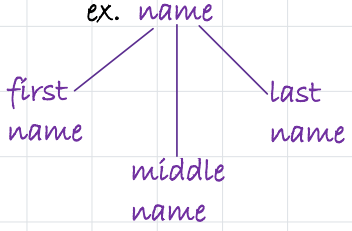
(i) simple and composite attributes.
(ii) single valued and multivalued attributes.
(iii) stored and derived attributes.
(iv) key attributes.
(v) descriptive attributes.
(vi) complex attributes.

(i) simple attribute

which cannot be divided further
ex. roll number

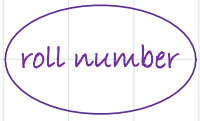
composite attribute

which can be divided further
ex. name



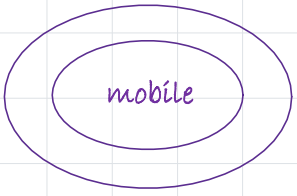
(ii) single valued attribute

which takes one value per entity
ex. roll number



multivalued attribute

which takes more than one value per entity
ex. mobile



(iii) stored attribute

which does not require any updation
ex. date of birth (d.o.b.)

derived attribute

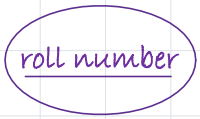
the value of attribute is derived from other attribute
ex. age

d.o.b : 11/04/2000
age :
2024 : 24 years
2060 : 60 years

2090 : 90 years

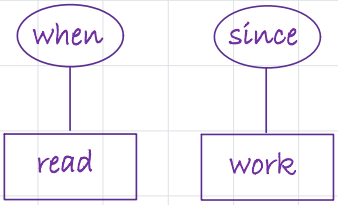
(iv) key attribute.

which uniquely identify an entity in the entity set
ex. roll number



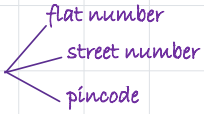
(v) descriptive attributes.

which gives information about the relationship set
ex.



(vi) complex attributes.

multivalued attribute
+ composite attribute



composite

ex. Address

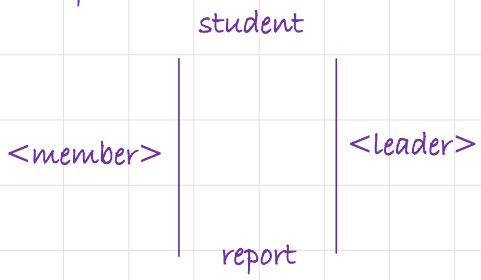
multivalued

2 house

4. Degree of relationship set : how many number of entity set are participating in a relationship set.

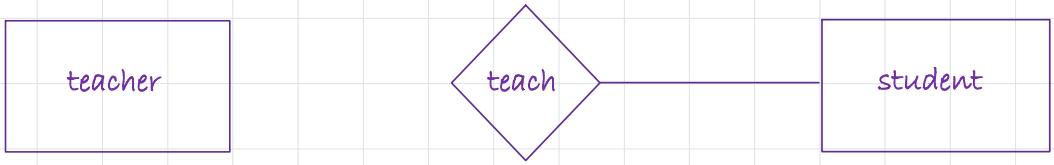
(i) Unary : if one entity set is participating. relationship among two entities of the same entity set

(recursive relationship set)



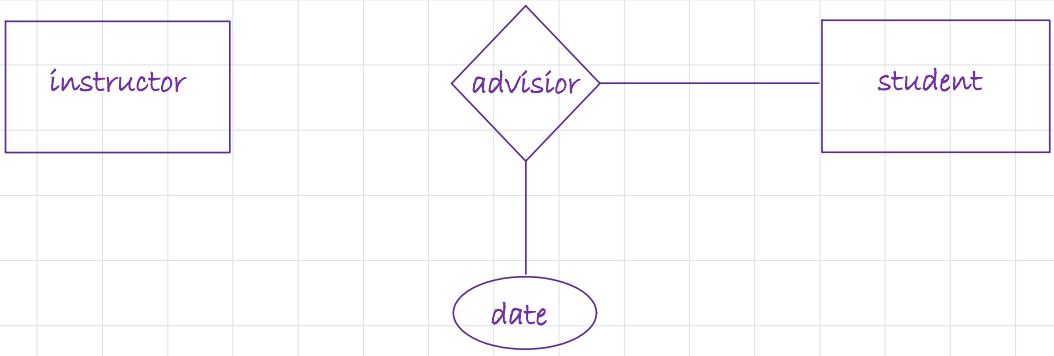
student (member) reports to student (leader)

(ii) Binary : if two entity set are participating.



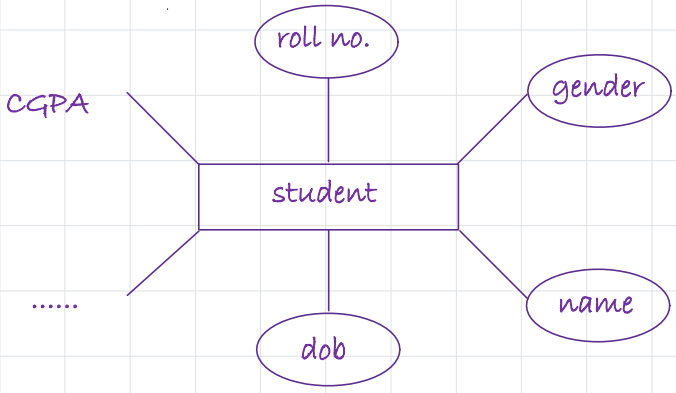
teacher teach student.

(iii) Ternary : if three entity set are participating.



x insctructor became advisior of y student on z date

(iv) N-ary : if n-entity set are participating.



RDBMS

Degree : number of attributes

cardinality : number of tuples

ER model

Degree of relationship set : how many number of entity set are participating

Degree : number of attributes

cardinality : number of tuples

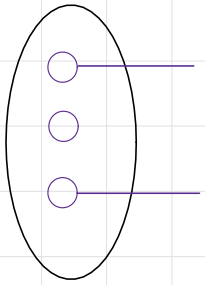
Degree of relationship set : how many number of entity set are participating

mapping cardinality : how entities are participating

1:1
1:M
M:1
M:N

5. Participation constraint ——————————— partial participation

represented by single line
- partial : not 100% participation

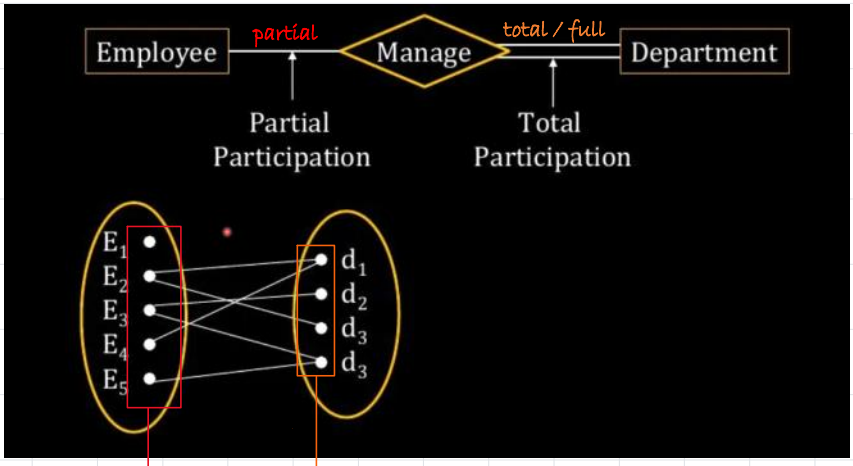


not every entity in the set participates in a relationship set.

total participation / full participation

represented by double line.
- total / full : 100% participation.

every entity in the set participates in a relationship set.



total / full participation from department

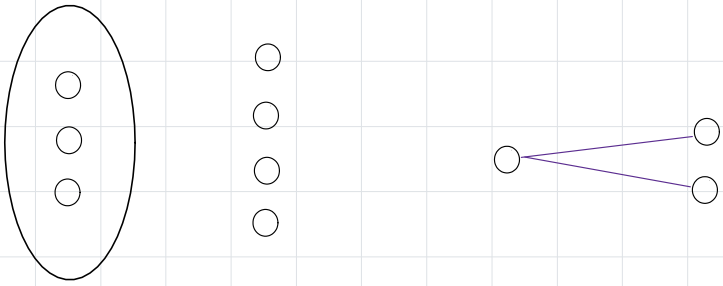
partial participation from employee

6. Mapping cardinality : the number of entities to which another entity can be associated via a relationship set.

(i) 1:1 - one to one

0. [one mapping : atmost one (0 or 1)]
[many mapping : 0 or more (0.....*)]

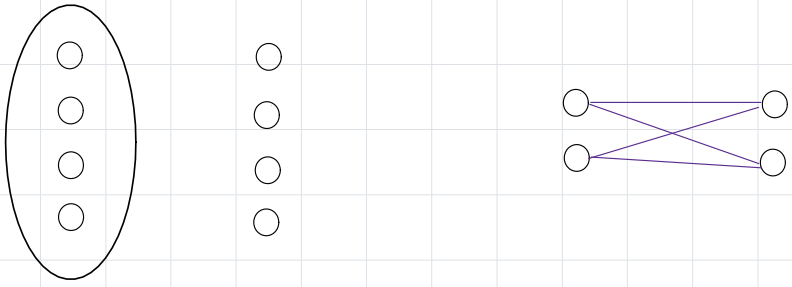
(ii) 1:M - one to many



(iii) M:1 - many to one

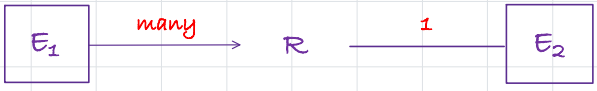
(iii) M:1 - many to one

(iv) M:N - many to many



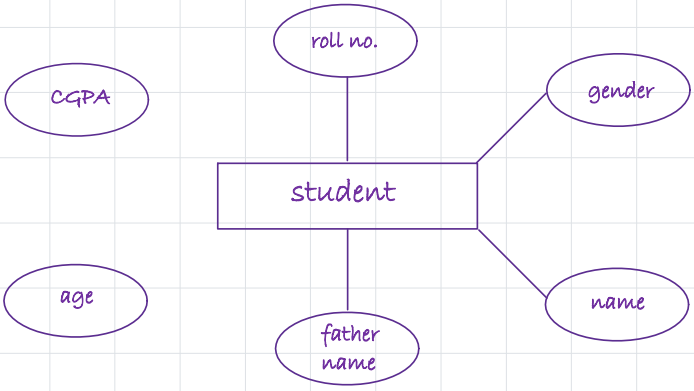
note :
some elements in A and B may not be mapped to any elements in other set

note :
Direction of arrow : one



note :
no arrows in many to many.

(7) strong entity set : Entity set having a key attribute



here student entity set is strong entity set, it has roll number as a key attribute.
it can uniquely identify students through roll number.

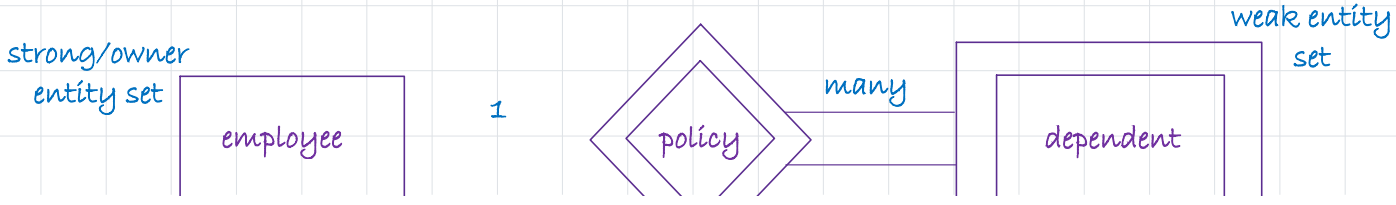
(8) weak entity set : Entity set that does not have any key attribute to differentiate each entity in entity set.
(cannot differentiate or uniquely identify students)

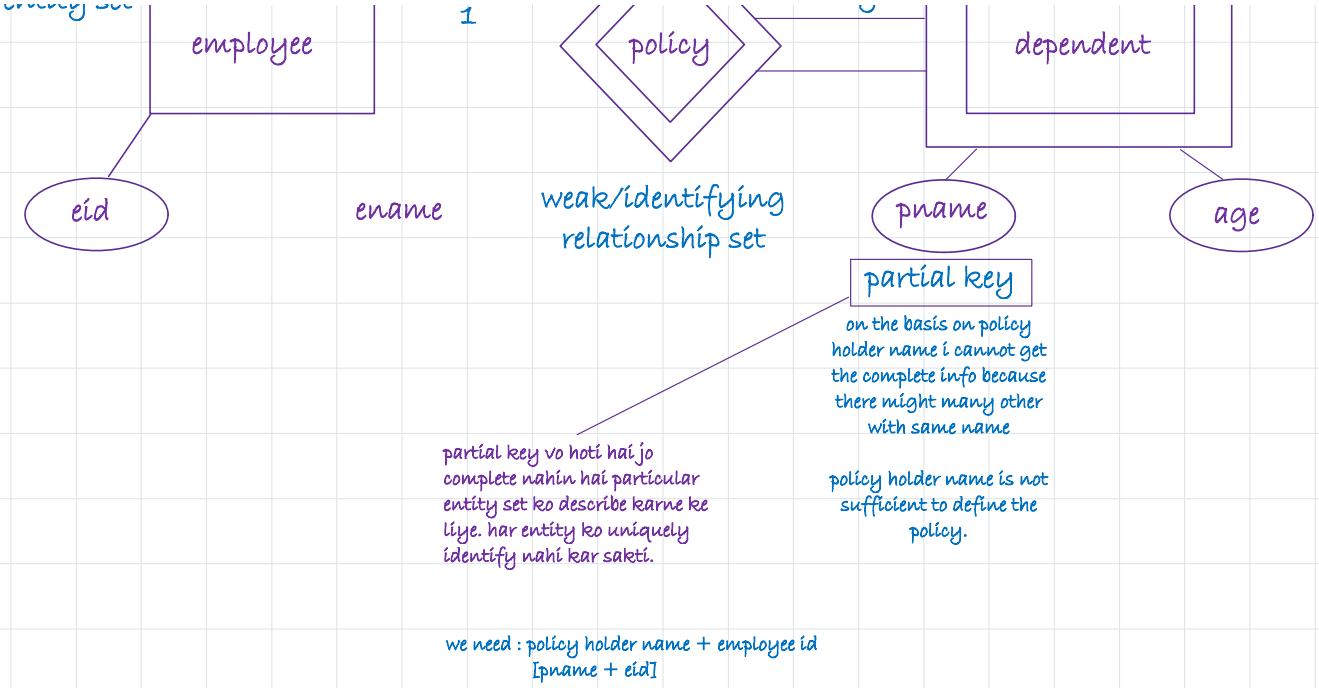
(or)

Entity set which having partial key

Representation :

weak entity participates in an idenitifying relationship type with an owner or identifying entity type.





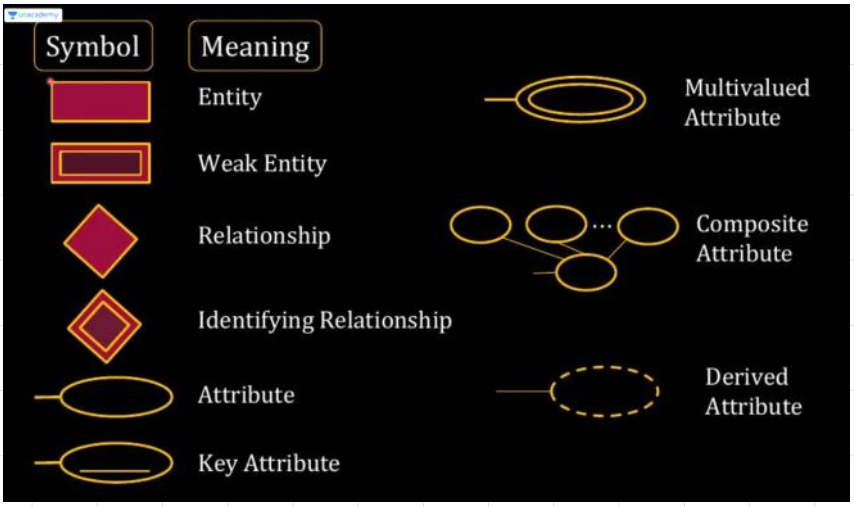
participation : 1 (strong) to many (weak)

weak entity set and relationship set : total (full participation)

weak entity set idenitfying : (partial key + key attribute of strong entity set)

- a weak entity can be identified uniquely only by considering the primary key of another (owner) entity. for each weak entity set there must be owner entity set, which is strong entity set.
- Relationship set between weak entity set and idenitifer entity set is also "weak relationship set"
- owner entity set and weak entity set must participate in a one-to-many relationship set (one owner, many weak entities)
- weak entity must have total participation in this identifying relationship set.

note :
weak entity set and multivalued attributes allowed to represent in ER diagram but not allowed in RDBMS table.



Q.1 Given the basic ER and relational models, which of the following is INCORRECT? [GATE-2012 : 1 Mark]

ER model - multivalued allowed

(A) An attribute of an entity can have more than one value.

ER model - composite allowed

(B) An attribute of an entity can be composite.

in RDBMS multivalued attributes are not allowed.

(C) ✓ In a row of a relational table, an attribute can have more than one value.

in RDBMS attribute can have exactly one value or NULL value

(D) In a row of a relational table, an attribute can have exactly one value or a NULL value.

Q.2 Which one of the following is used to represent the supporting many-one relationships of a weak entity set in an entity-relationship diagram? [2020:1 Mark]

(A) Rectangles with double/bold border

(B) Ovals with double/bold border

(C) Ovals that contain underlined identifiers

(D) ✓ Diamonds with double/bold border

Null value

- unknown value
- Unexisted
- non zero
- not applicable

In RDBMS there are two type of exception

here DBMS assign NULL value.

randomly ASCII values assigned by DBMS software such that two NULL values are not equal

not applicable

eg.name

first name
aryan

middle name

last name
k

aryan k. middle name is not applicable for that particular student.

unknown

eg. passport number

some student dont have passport

Topic : Integrity constraints (conditions/restrictions)

constraints that are used on database in order to maintain the integrity of database.

1. Domain constraint (atomic)

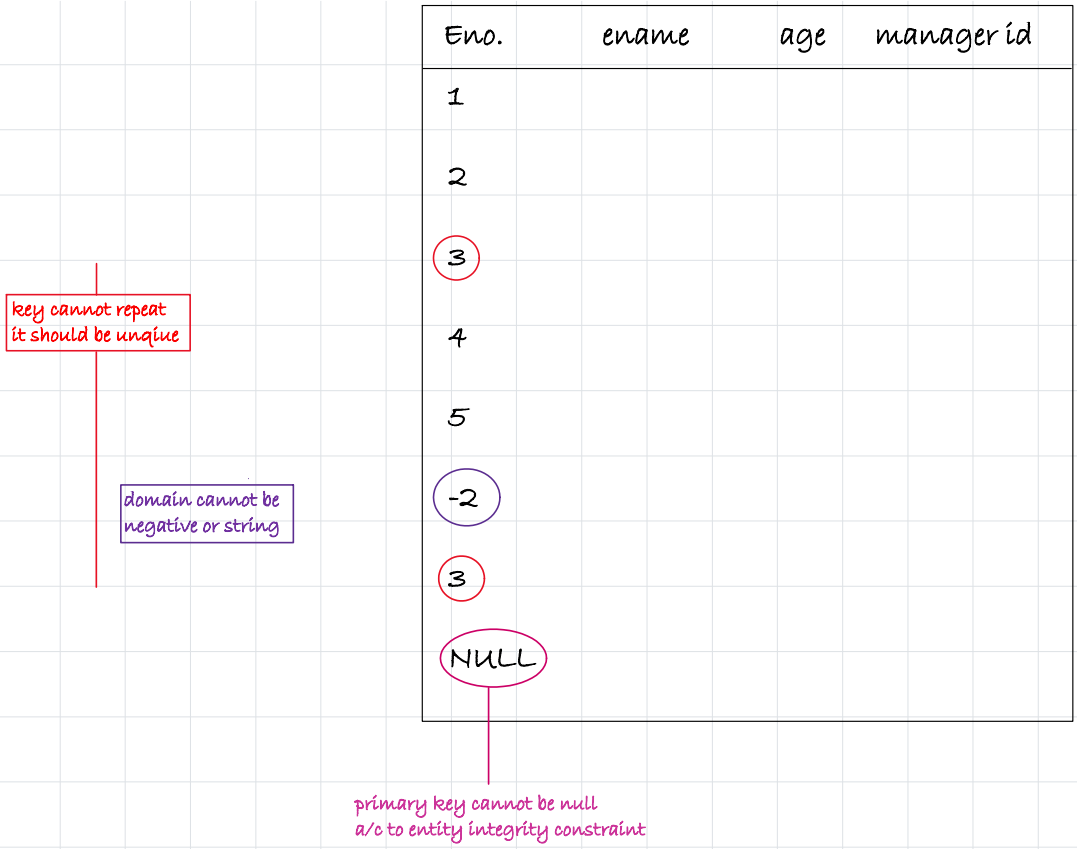
2. Key constraint (unique)

3. Entity integrity constraint (p.k not null)

4. Referential integrity constraint. (foreign key concept)

5. Functional dependency constraint. (we separately deal with it)

Employee table



Topic : foreign key

foreign key is a set of attributes which reference to the primay key (or) alternative key of same table (or) some other table.

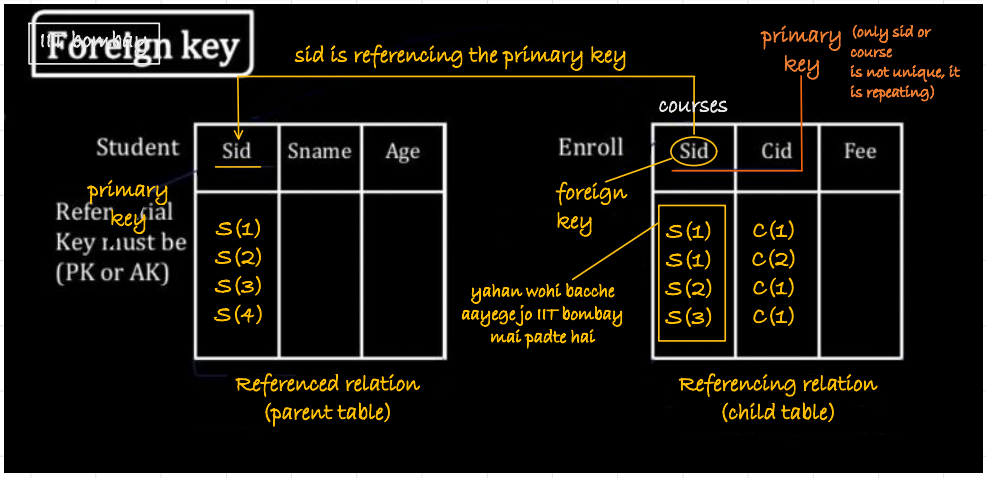
note :

ing : foreign key (going)

Referencing relation : Table which contains the foreign key.

(child table)

Referenced relation : Table which is referenced by foreign key.
(parent table)



1.
enrolled

foreign key(sid) reference STUDENT

yahan table ka naam isliye likha because at most one primary key
(when sid is primary key)

note :

by default foreign key reference to the primary key of referenced relation and atmost one primary key per table (relation)

2.
enrolled

foreign key(sid) reference STUDENT(sid)

assume login id is primary key and sid is alternative key then along with table name, mention the attribute name also because more than one alternative key possible.
(because many alternative key possible)

note :

by default foreign key reference to the primary key of referenced relation and atmost one primary key per table (relation)

note :

sometimes same table contains primary key and foreign key both. (foreign key with different name)

note : foreign key constraint

- the value present in the foreign key must be present in the primary key of referenced relation.

- foreign key may contain duplicates and null value.

because you cannot put anything you want in database, there are constraints

Integrity constraint

constraint : restriction

Integrity constraint is a condition specified on a database schema and restrict the data that can be stored an instance of database.

vo condition jo pure schema par apply hogi aur agar apply nahi kar paa raha then database me us data ko insert karne se rok dena.

if data is following the constraints/restrictions then data will be allowed in database oterhwise it will not allow data to be stored in database hence illegal instances.

- Legal instances

the instances which satisfy all the integrity constraints specified on database schema

- Illegal instances

the instances which does not satisfy all the integrity constraint

Types of integrity constraints :

1. Domain constraints (atomic)
2. Key constraints (unique)
3. Entity integrity constraint (p.k. not null)
4. referential integrity constraint (foreign key constraint)
5. Functional dependency constraint.

- when we have to check all these integrity constraint whether integrity constraint violates or not?

at the time of (i) insertion
(ii) updation
(iii) deletion

(a) At the time of insertion/updation, check : (i) domain
(ii) key constraint
(iii) entity integrity
(iv) referential integrity constraint

(b) At the time of deletion, check : (i) only referential integrity constraint

1. Domain constraint : all the attribute must follow the domain.

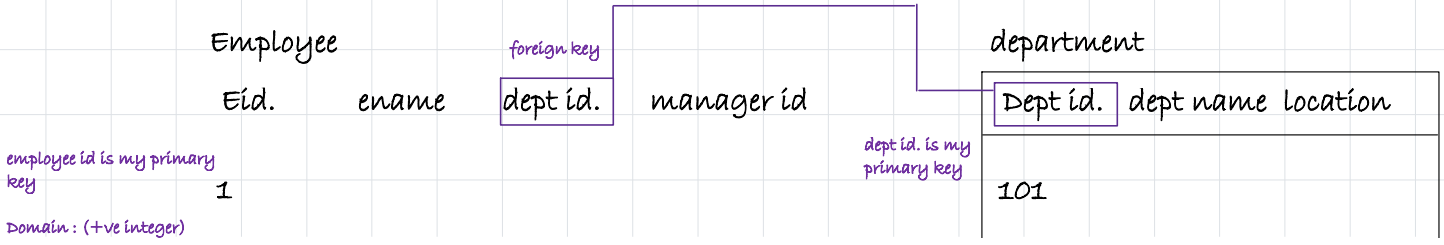
Ex. Roll no [+ve integer]
domain
no student having roll number as a string or (-ve) number

2. Key constraint : uniqueness [key should not repeat]
key attribute must be unique.

3. Entity integrity constraint : Primary key not null

4. Referential integrity constraint : foreign key constraints.

- how all the these constraints are being checked? whether they violate the constraints or not?



employee id is my primary key

Domain : (+ve integer)

1
2
3
4
5
6

dept id. is my primary key

101
102
103
104
105
106

here dept id. is the foreign key in emp table reference to department table.

check all the constraints and when they violate

(i) Domain constrains : if any employee id is -ve number (or) strings then domain constraint violates.

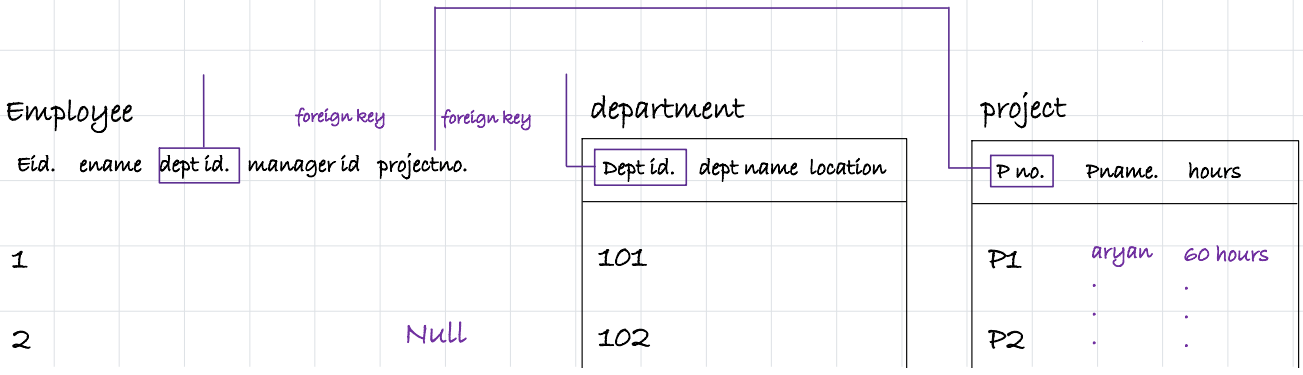
(ii) Key constraint : Eid is my key (unique and not null)
- if any employee id repeats then key constraints violates.

(iii) Entity integrity constraint : if primary key (eid) is null then entity integrity constraint violates.

(iv) Referential integrity constraint : if in the employee table we insert dept id. = '500' (does not exist) then referntial integrity violation occur,

In this example, dept id. in the employee table must be between 101 to 107 (or) repeat (or) null.

note :
the value present in foregin key must be present in primary key of referenced relation.



2	Null	102
3		103
4		104
5		105
6		106

employee id is my primary key

Domain : (+ve integer)

dept id. is my primary key

P2
P3
P4
P5
P6

- here dept id. in employee table is foreign key reference to primary key of department table.

- here pno. (projectno.) in employee table is foreign key reference to primary key of project table.

- here 2 foreign key in employee table.

Q. when foreign key contain the NULL?

- if any employee (emp id. 2) not working in any of these project in the company (e_2 is on bench) then e_2 contain null

Important point about the foreign key :

(i) foreign key may contain duplicates and NULL values.

(ii) The value present in foreign key must be present in primary key.

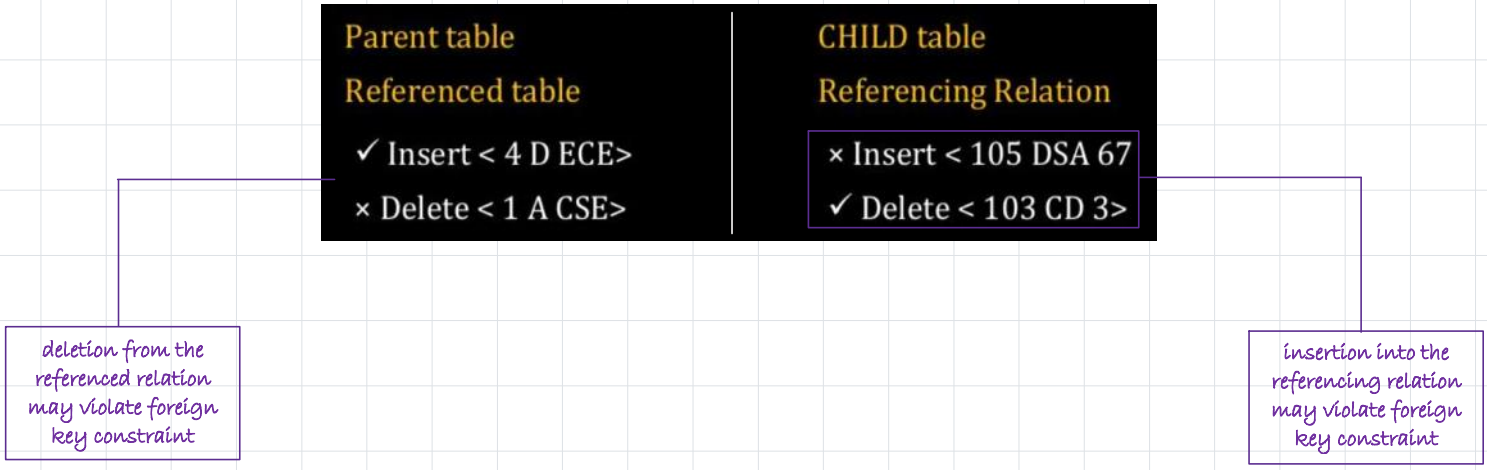
(iii) sometimes same table contain both primary key and foreign key with different.

(iv) in referencing relation we can have more than one foreign key.

(v) you cannot perform the blind deletion of primary key that is used as foreign key but i can insert.

note :
at the time of deletion we check referntial integrity (foreign key) constraint and does not check domain, key and entity integrity because during insertion we already insert only those tuples (entries) which satisfy the constraints

and during deletion if primary key is deleted then foreign key (that primary key is using) we take care about that foreign key.



note :
a relation can act as a parents and child. relation may contain a primary and a foreign key that refer to the same relation

Q. Consider a relation R(AB) where 'A' is a primary key and 'B' is a foreign key reference to the primary key A of the same table then which of the following is correct according to the constraint ?

(or)

which one of the following row sequence successfully inserted into R?

(a) (1,2) (2,3) (3,2) (4,5) (5,1) (2,5) (6,null) (primary key (2) repeats, it must be unique)

(b) (1,2) (2,2) (3,1) (4,2) (5,7) (6,5) (7,8) (primary key till 7 but foreing key =8)

(c) (1,5) (2,4) (3,1) (4,3) (5,1) (6,2) (null,5) (primary key cannot be null)

(d) (1,2) (2,3) (3,4) (4,5) (5,6) (6,9) (7,2) (primary key till 7 but foreing key =9)

✓(e) (1,2) (2,3) (3,null) (4,1) (5,null) (6,2) (7,4)

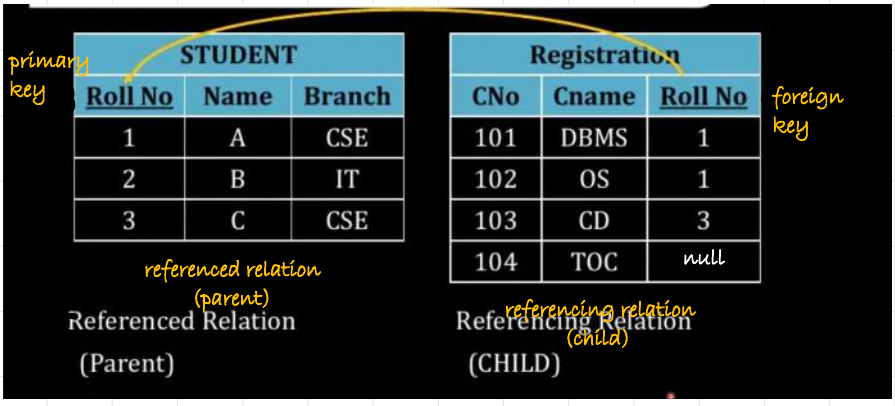
A	B
1	2
2	3
3	null
4	1
5	null
6	2
7	4

Referenced relation -

(i) Insertion : no violation
(ii) deletion : may cause violation if primary key is used by referencing relation

(a) On delete no action : means if it cause problem on delete then deletion is not allowed on that table

foreign key (sid) reference Student (sid)
On delete no action



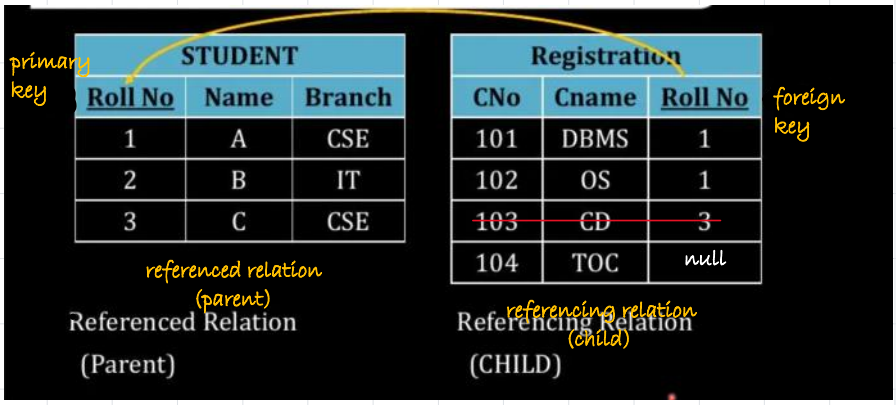
foreign key
Problem : 1 ya 3 delete karne par, vo use aa rahi
allowed : on 2

on delete no action
if i want to delete 3 from parent table, deletion is not allowed.

(b) On delete cascade : if we want to delete primary key value from referenced table then it will delete that value from referencing table also cascadlly.

foreign key (sid) reference Student (sid)

On delete cascade



on delete cascade
if i want to delete 3 from parent table, toh uske sath foreign key se bhi 3 delete kiya jayega and its tuple.

note :
on delete cascade is not suggestable when one (same) table contain both primary key and foreign key both.

primary key — manager is the employee of same company — foreign key

Employee table	Eid.	ename	dept.	manager id
	E(1)	A	x	E(2)
	E(2)	B	y	null
	E(3)	C	z	E(2)
	E(4)	D	x	E(3)
	E(5)	E	y	E(4)
	E(6)	F	x	E(5)

here eid is primary key and manager id is foreign key which reference to primary key (eid) of same employee table

Q. If we want to delete (E_2.....null) with on delete cascade, then how many tuple additionally deleted to preserve referential integrity constraint?

primary key

manager is the employee of same company

Employee table	Eid.	ename	dept.	manager id	
	E(1)	A	X	E(2)	additional deletion of Primary key (E1) caused by (E2)
	E(2)	B	Y	null	original (cause E(2)) deletion.
	E(3)	C	Z	E(2)	additional deletion of primary key (E3) caused by (E2)
	E(4)	D	X	E(3)	additional deletion of primary key (E4) caused by (E3)
	E(5)	E	Y	E(4)	additional deletion of primary key (E5) caused by (E4)
	E(6)	F	X	E(5)	additional deletion of primary key (E6) caused deletion of (E5)

foreign key

ans : 6

Q.

The following table has two attributes A and C where A is the primary key and C is the foreign key referencing A with on-delete cascade.

The set of all tuples that must be additionally deleted to preserve referential integrity when the tuple (2, 4) is deleted is:

(GATE: PSU 2 Marks)

A (,3 4) and (6, 4)

B (5, 2) and (7, 2)

C (5, 2), (7, 2) and (9, 5)

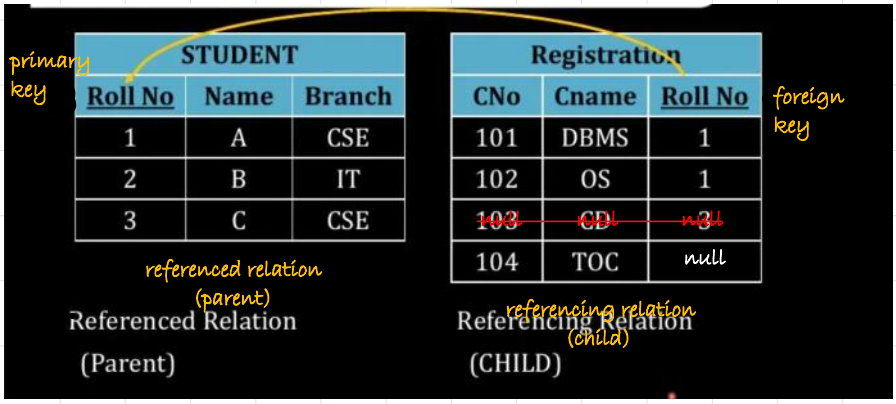
D 1

A	C
~~2~~	~~4~~
3	4
4	3
5	2
~~7~~	~~2~~
~~9~~	~~5~~
6	4

(c) On delete set null : if we want to delete primary key value from the referenced table then it will try to set null values in place of that value in referencing table.

foreign key (sid) reference Student (sid)

On delete set null



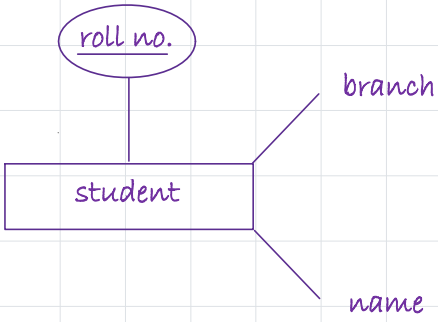
on delete set null
if i want to delete 3 from parent table, toh uske sath foreign key se 3 ki place par null place kiya jayega.

Topic : ER model to RDBMS conversion

ER model

RDBMS

(i) simple attribute



STUDENT

roll no.	name	branch	CGPA

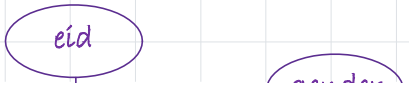
entity set

key attribute

relation (table)

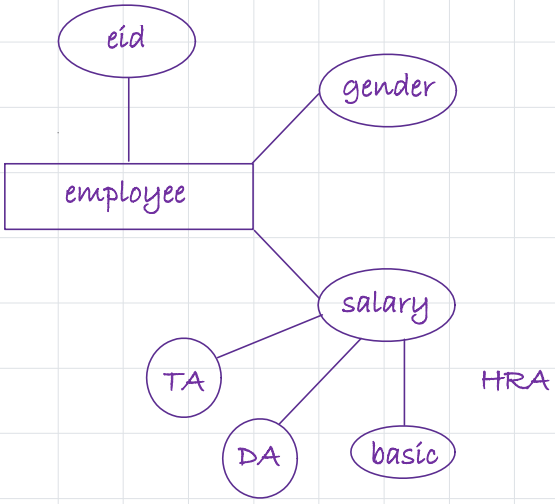
primary key

(ii) composite attribute



employee

eid	gender	Basic	TA	DA	HRA



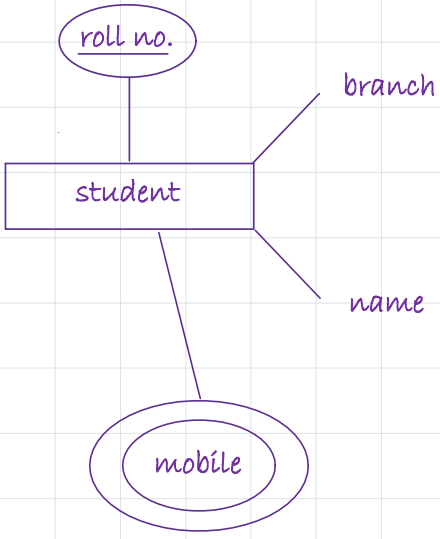
composite attribute

employee

eid	gender	Basic	TA	DA	HRA

set of simple component

(iii) multivalued attribute



multivalued attribute

STUDENT

roll no.	name	branch

STUDENT

roll no.	mobile no.

separate table for multivalued attribute

conclusion :

entity set	relation (table)
key attribute	primary key
composite attribute	set of simple component
multivalued attribute	separate table for multivalued attribute (child table)

(i) 1 to many

(ii) many to 1

(iii) many to many

(iv) 1 to 1

how many tables required?

(i) 1 to many (1:M) : 2 tables required and 1 foreign key (relationship merge with many side)

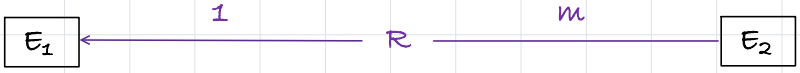
2 table :

$E_1 (A_1 A_2 A_3)$

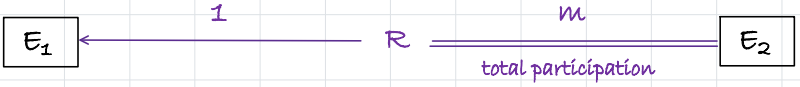
$RE_2 (B_1 B_2 B_3 A_1)$ — foreign key

Partial participation : 2 table (RDBMS)

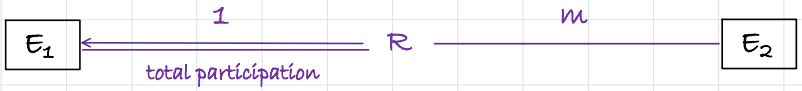
Total participation :



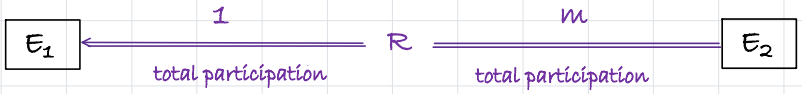
2 table (RDBMS)



2 table (RDBMS)



1 table (RDBMS)



1 table (RDBMS)

(ii) many to 1 (M:1) : 2 tables required and 1 foreign key (relationship merge with many side)

2 table :

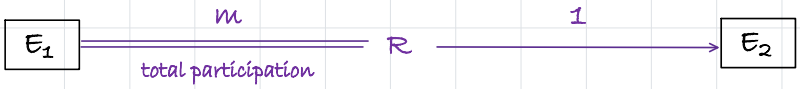
$E_2 (B_1\ B_2\ B_3)$ ——— foreign key

$RE_1 (A_1\ A_2\ A_3\ B_1)$

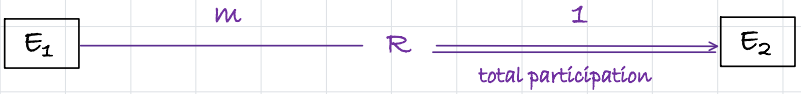
Partial participation : 2 table (RDBMS)

Total participation :

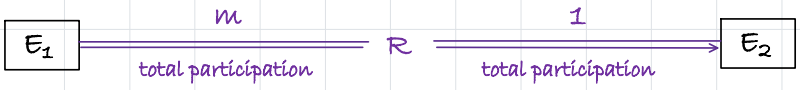
2 table (RDBMS)



2 table (RDBMS)



1 table (RDBMS)



1 table (RDBMS)

(iii) many to many (M:N) : 3 tables required and 2 foreign key (separate table created for relationship set)

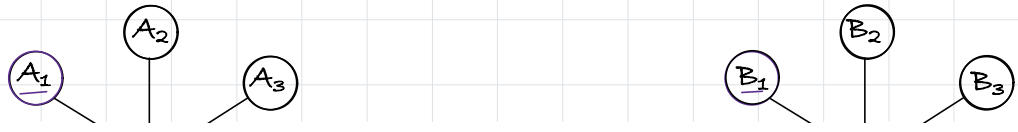
2 table :

E_1 (A_1 A_2 A_3) — foreign key of E_2

R (A_1 B_1) — foreign key of E_1 — (2 foreign keys)

E_2 (B_1 B_2 B_3)

Partial participation : 3 table (RDBMS)

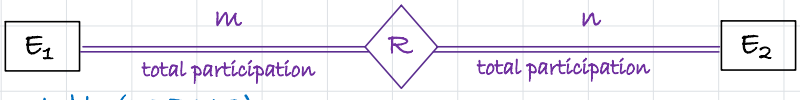


Total participation :

3 table (RDBMS)

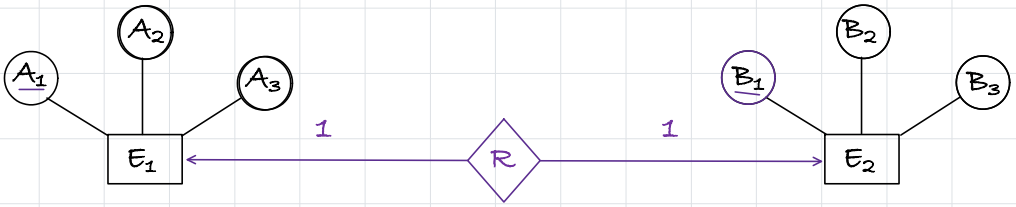
2 table (RDBMS)

2 table (RDBMS)



1 table (RDBMS)

(iv) one to one (1:1) : 2 tables required and 1 foreign key (relationship merge with either left side or right side)



2 table : merge with left side (or)
$E_1R\ (A_1 A_2 A_3 B_1)$

2 table : merge with right side
$E_1(A_1 A_2 A_3)$

2 table : merge with left side (or)

$E_1R\ (A_1\ A_2\ A_3\ B_1)$

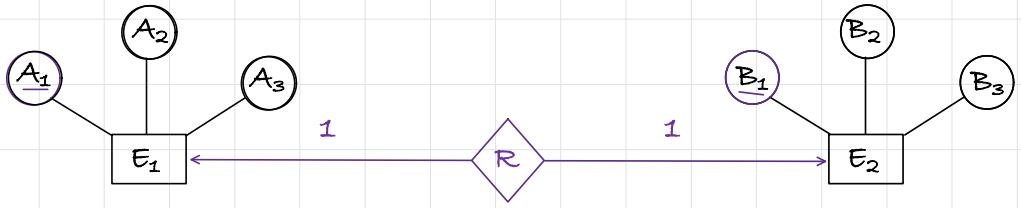
$E_2(B_1\ B_2\ B_3)$

2 table : merge with right side

$E_1(A_1\ A_2\ A_3)$

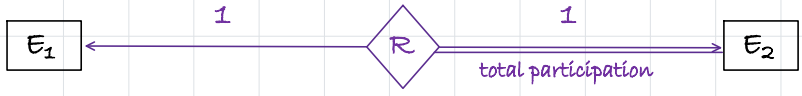
$RE_2(B_1\ B_2\ B_3\ A_1)$

Partial participation : 2 table (RDBMS)

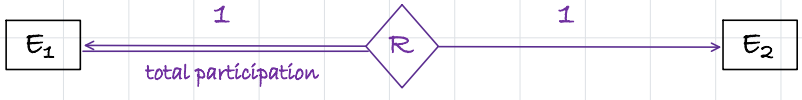


Total participation :

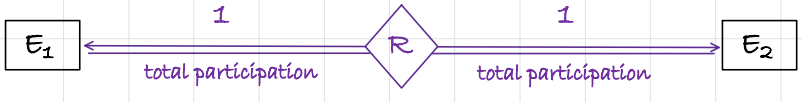
2 table (RDBMS)



1 table (RDBMS)



1 table (RDBMS)

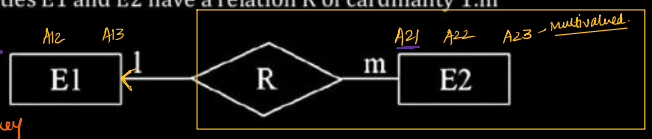


1 table (RDBMS)

note :

- If total participation (=) is at both side (1:M, M:1, M:N and 1:1) then only 1 table required.
- If total (full) participation (=) is at one side (in 1:M, 1:1 and M:1) then only 1 table required.
- If in M:N total participation is at any one [left (or) right] then only 2 table required.

Q. Consider the following entity relationship diagram(ERD), where two entities E1 and E2 have a relation R of cardinality 1:m

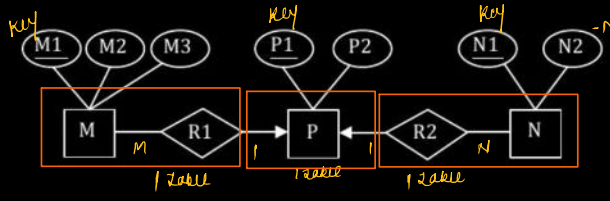


The attributes of E1 are A11, A12 and A13 where A11 is the key attribute. The attributes of E2 are A21, A22, A23 where A21 is the key attribute and A23 is a multi-valued attribute. Relation R does not have any attribute. A relational database containing minimum number of tables with each tables satisfying the requirements of the third normal from (3NF) is designed form the above ERD. The number of tables in the database is [GATE-2004 : 2 Marks]

(A) 2 (B) 3 (C) 5 (D) 4

Q. Common Data for Question

Consider the following ER Diagram



(i) The minimum number of tables needed to represent M, N P, R1, R2 is [GATE-2008 : 2 Marks]

(A) 2 (B) 3 (C) 4 (D) 5

(ii) Which of the following is a correct attribute set for one of the table for the correct answer to the above question? GATE-2008 : 2 Marks]

(A) {M1, M2, M3, P1}
(B) {M1, P1, N1, N2}
(C) {M1, P1, N1}
(D) {M1, P1}

ER model :
entity relationship diagrams used to represent diagrammatic design (high level design) of databases.

database design steps :

1. Requirement : what type of data stored and what operation required, etc.]
2. Conceptual and logical : [ER diagram]

high level design

3. ER diagram to RDBMS table design and apply normalization
4. Physical DB design (indexing design)
5. User interface design and security design.

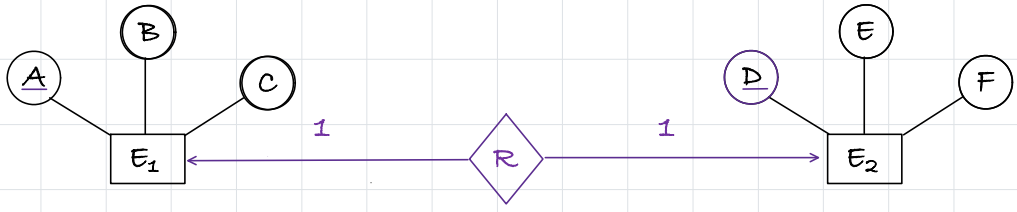
low level design

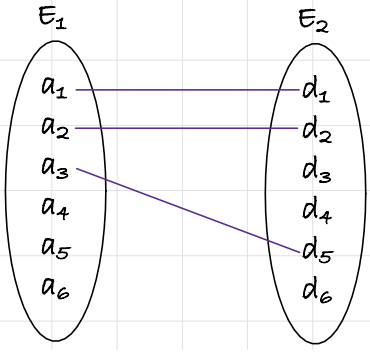
table to Fd then attribute closure, finding candidate key and then extra FD. finding minimal cover and then applied normal forms and we also did decomposition.

Q. how to find candidate key of relationship set?
Q. Why 1:M (1 to many) relationship set merge with many side?
Q. How to check normal forms while conversion?
Q. All reason (all logic) of conversion from ER to RDBMS?

Topic : candidate key of relationship set[R]

(i) one to one





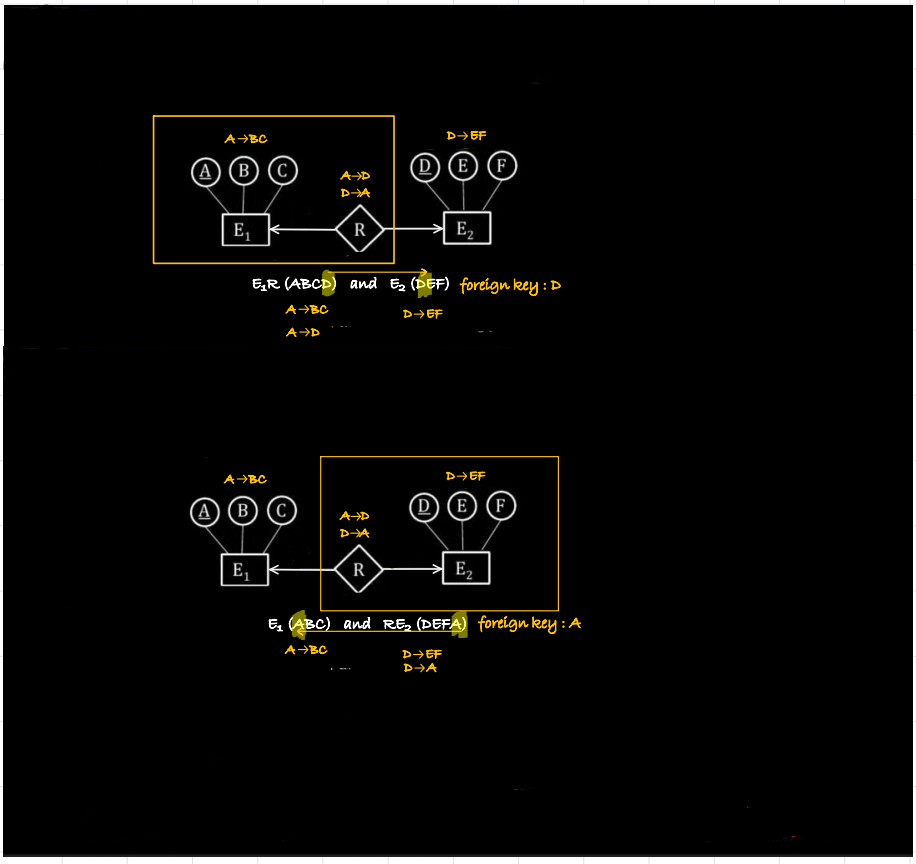
R

A	D
a_1	d_1
a_2	d_2
a_3	d_5

candidate key banane ke lie unique hona chaiye

hence, candidate key : A (or) D

Fd's : A to BC and D to EF
Fd's of relationship set : A to D and D to A



Q. what happen if we merge into single table in 1:1 participation and having partial participation?

a_2	-	-	d_3	-	-
a_3	-	-	d_4	-	-
a_4	-	-	Null	Null	Null
Null	NULL		d_1	-	-

getting candidate (primary) key because we are getting null value due to partial participation

Candidate key = {A, D}

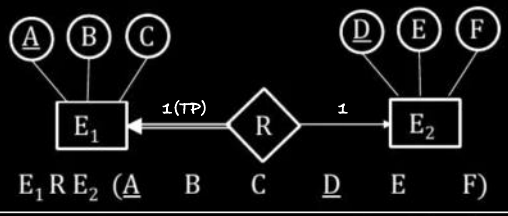
No Candidate key is present, & No primary key

A relational table in which no attribute having "NOT NULL" values are "NOT" allowed in RDBMS . So,

- minimum 2 Relation Table Required and 1 Foreign key.

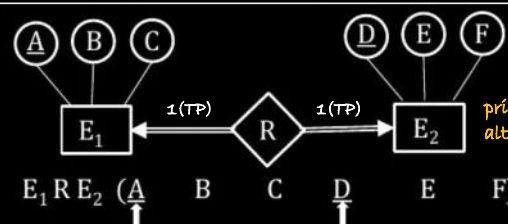
E_1 E_2 Entity set R Rel set between E_1E_2

With 1:1 mapping and at least one end having total participation.



primary key : A
alternative key : D

E_1 R E_2 (A B C D E F)



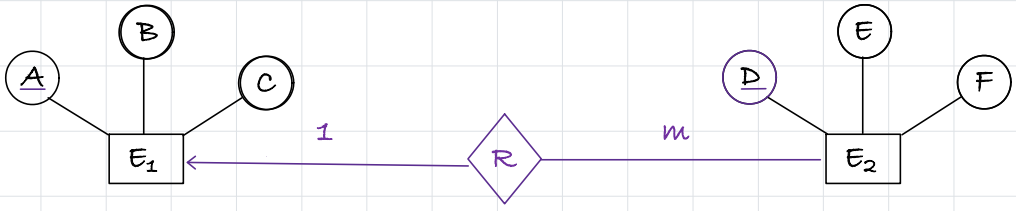
primary key : A
alternative key (primary key also) : D

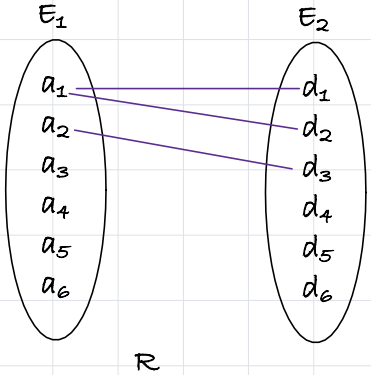
E_1 R E_2 (A B C D E F)

Primary Key

UNIQUE & NOT NULL (Alternative Key)

(ii) one to many





R

Fd's : A to BC
and D to EF

A	D
a_1	d_1
a_1	d_2
a_2	d_3

unique

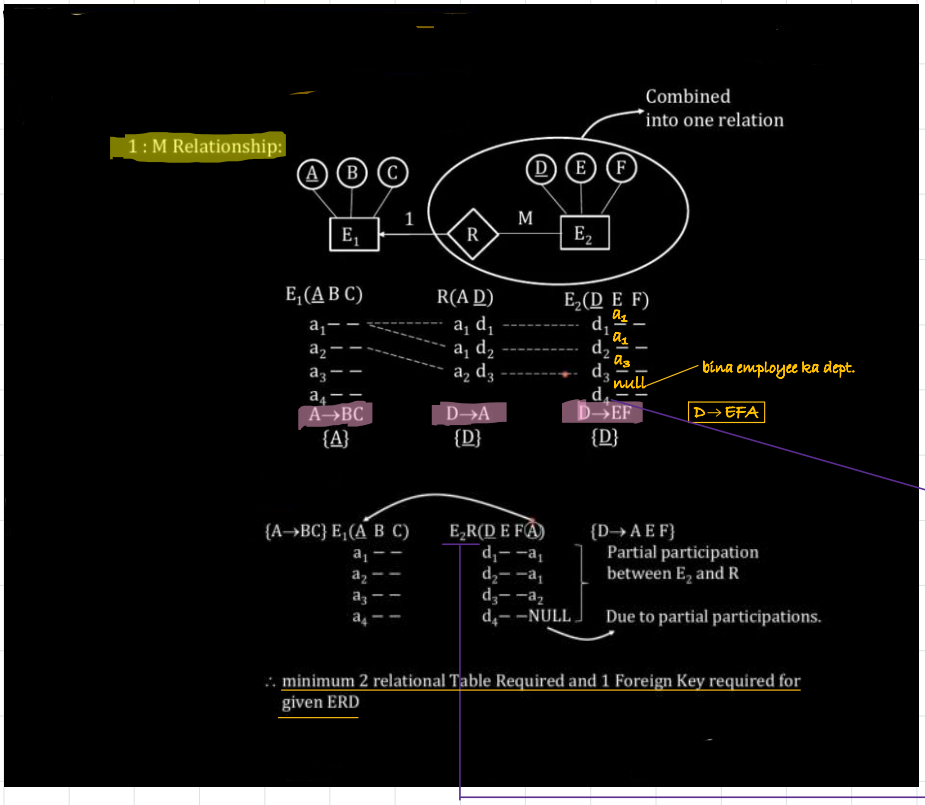
candidate key banane ke lie unique hona chaiye

hence, candidate key : D

Fd's : A to BC and D to EF

Fd's of relationship set : D to A

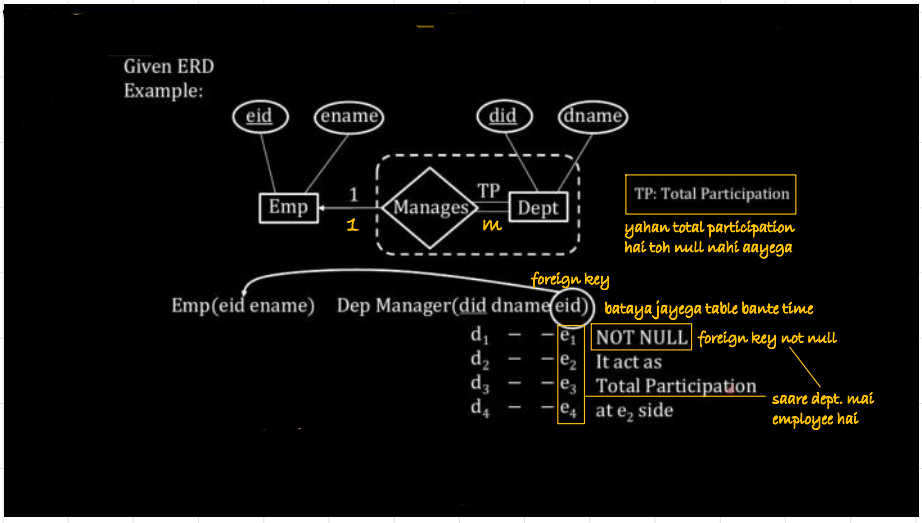
Q. Why 1:M (1 to many) relationship set merge with many side?



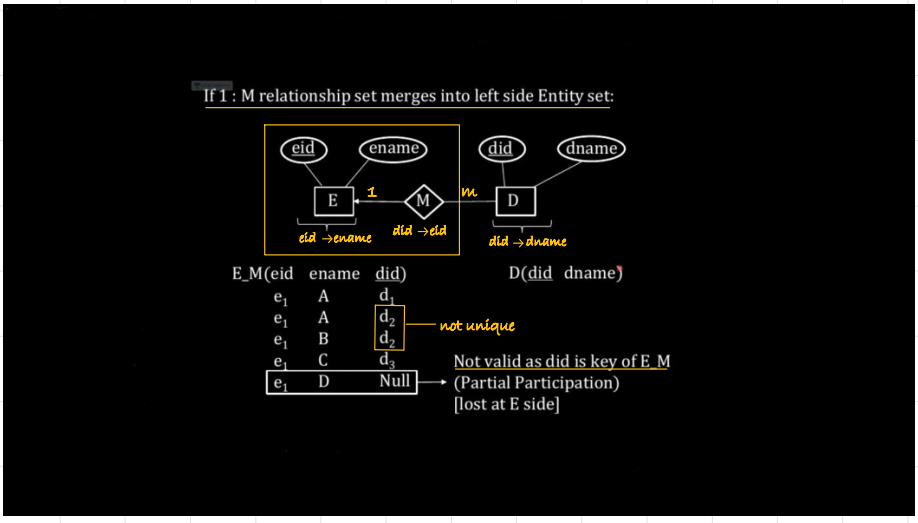
d_4 ke corresponding koi participate nahi kar paa raha because of partial participation E_2 and R ki wajah se

islie E_2R

Q. Why 1:M (1 to many) relationship set merge with many side?

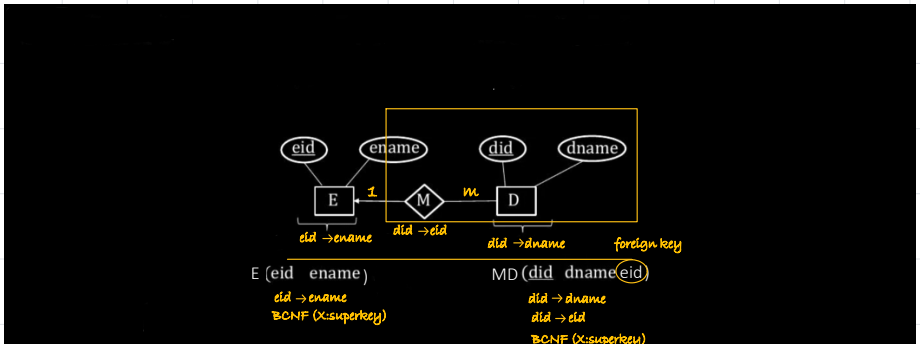


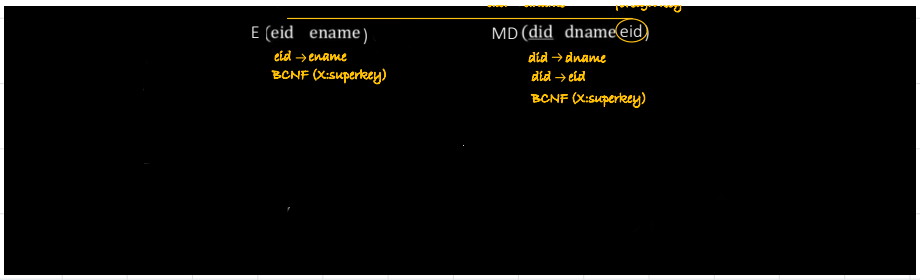
Q. what happen if we merge at one side in 1 to many relationship?



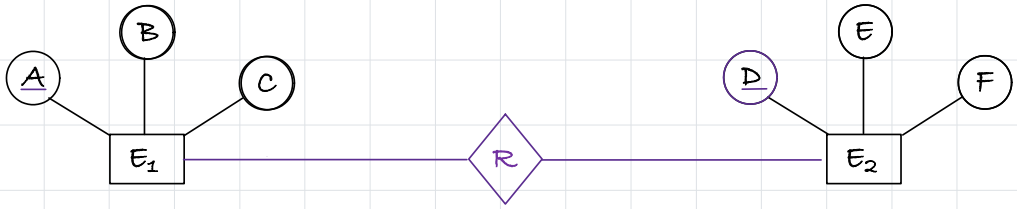
Disadvantages : 1) data redundancy
2) not superkey
3) partial participation will be lost.

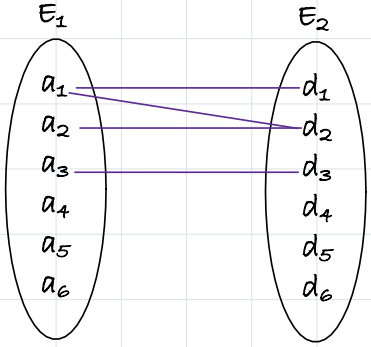
Q. This is 1:M relationship set merges into right side.
- How to check normal forms while conversion?





(iii) many to many





Fd's : A to BC
and D to EF

R

A	D
a_1	d_1
a_1	d_2
a_2	d_2
a_3	d_3

candidate key banane ke lie unique hona chaiye but both not unique.

hence, candidate key : AD

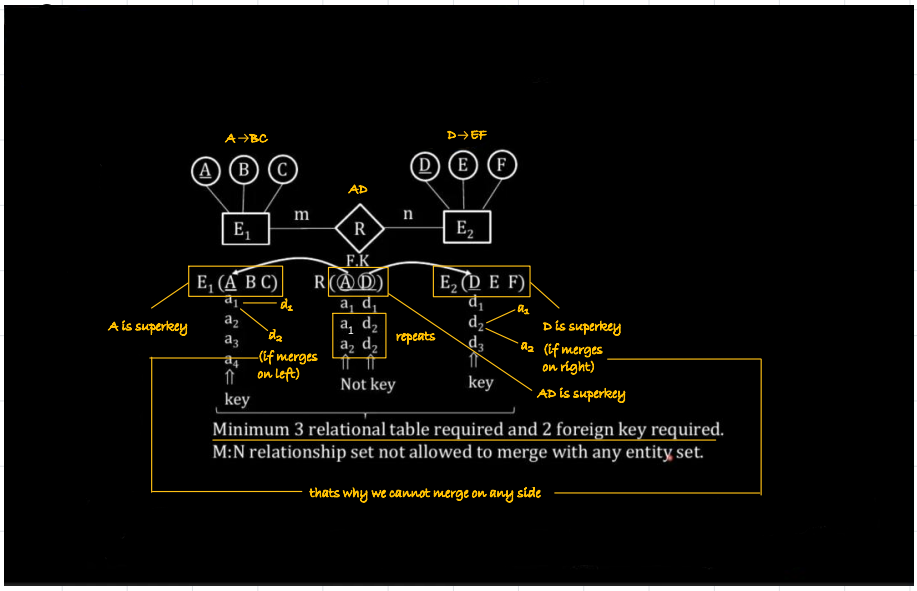
Fd's : A to BC and D to EF

Fd's of relationship set : AD

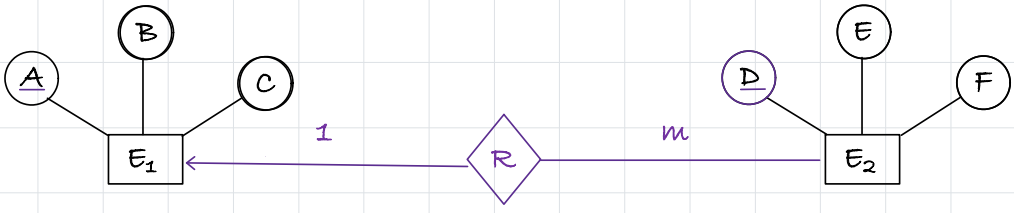
Q. Why in many to many three tables are required?

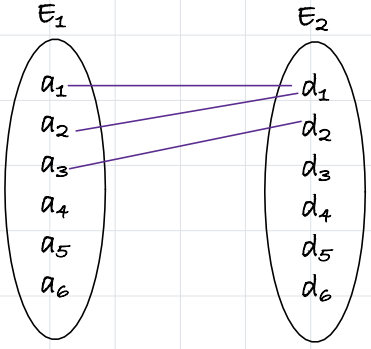
(or)

Why separate table is created for relationship set?



(iv) many to one





Fd's : A to BC
and D to EF

R

A	D
a_1	d_1
a_2	d_1
a_3	d_2

unique

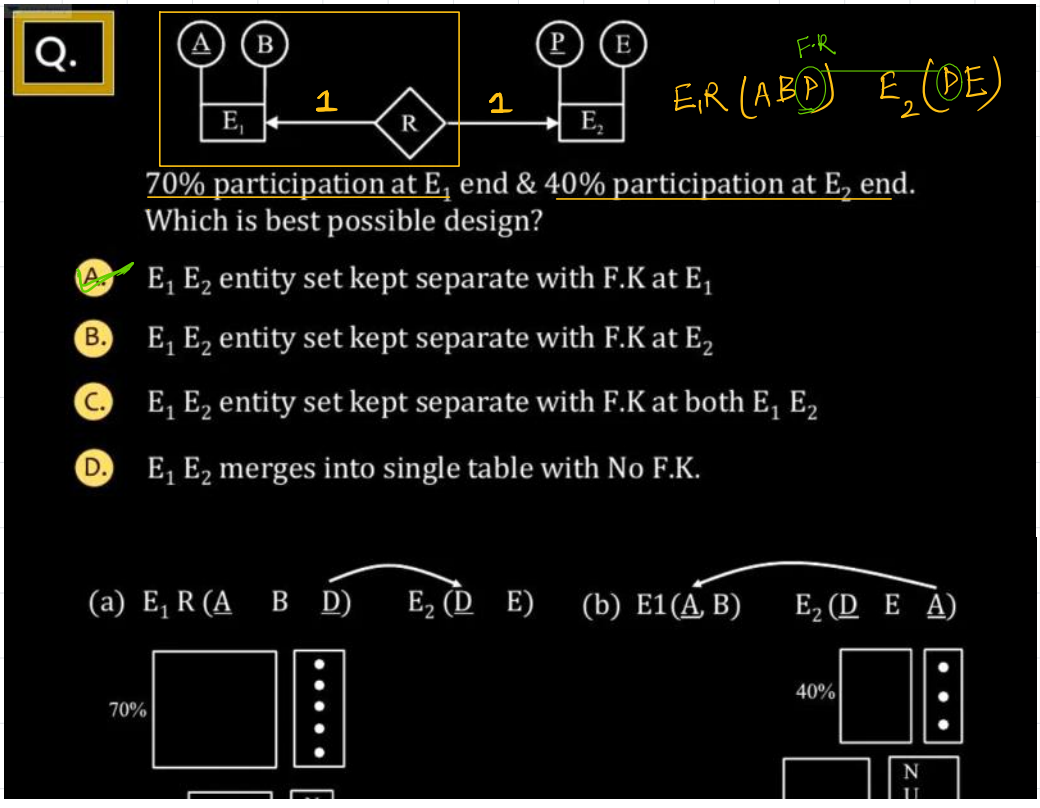
candidate key banane ke lie unique hona chaiye

hence, candidate key : D

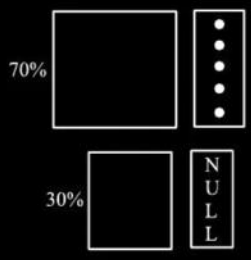
Fd's : A to BC and D to EF

Fd's of relationship set : D to A

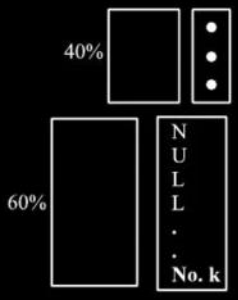
	candidate key of relationship set[R]
1 to many	D (candidate key from many side)
many to 1	A (candidate key from many side)
1 to 1	A (or) D
many to many	AD



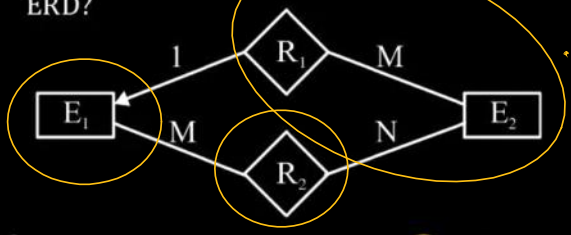
(a) E_1 R (A B D) E_2 (D E)



(b) E1(A, B) E_2 (D E A)



Q. E_1 E_2 entity set R_1 R_2 Relationship set related between E_1 and E_2 with 1:M and M : N mapping min Relational Table required in ERD?



- E_1 – ✓
- R_1 E_2 – MERGES AT MANY SIDE.
- R_2 –

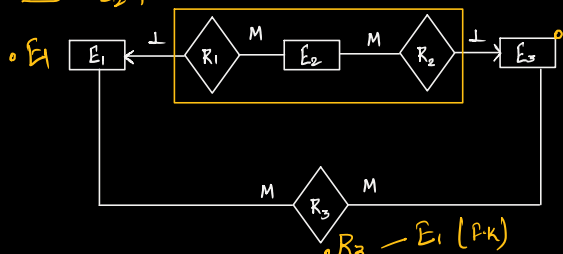
A. 2 B. 3 C. 4 D. 5

Q. $E_1E_2E_3$ entity set R_1 Relation between E_1 E_2 with 1 : M, R_2 Relations between E_2 E_3 with M : 1, R_3 Relationship between E_1 & E_3 with M : N

How many minimum relational tables required for given ERD?

A. 2 B. 3 C. 4 D. 5

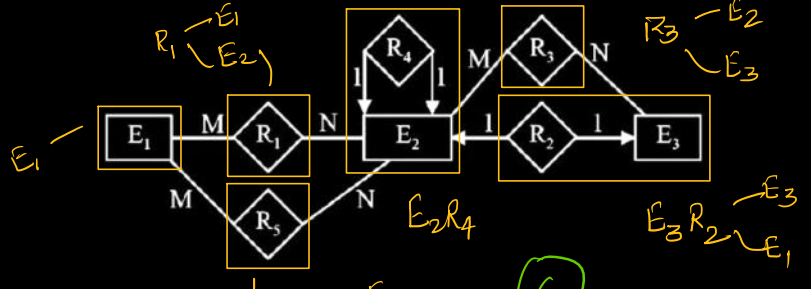
Sol. E_2R_1 E_2R_2



- E_1
- E_3
- R_1 E_2 R_2 (2 F.K.)
  - E_1 – P.K
  - E_3 – P.K
- R_3 – E_1 (F.K)
  - E_3 (F.K)

Q. Consider the following ER diagram [NAT]

Total number of RDBMS table in the above diagram?

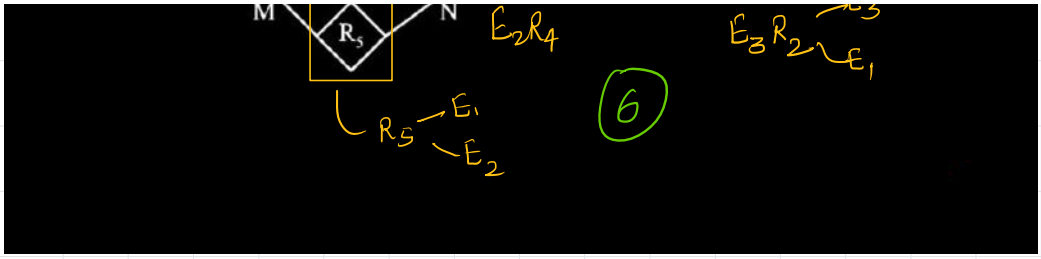


R_1 – E_1 E_2

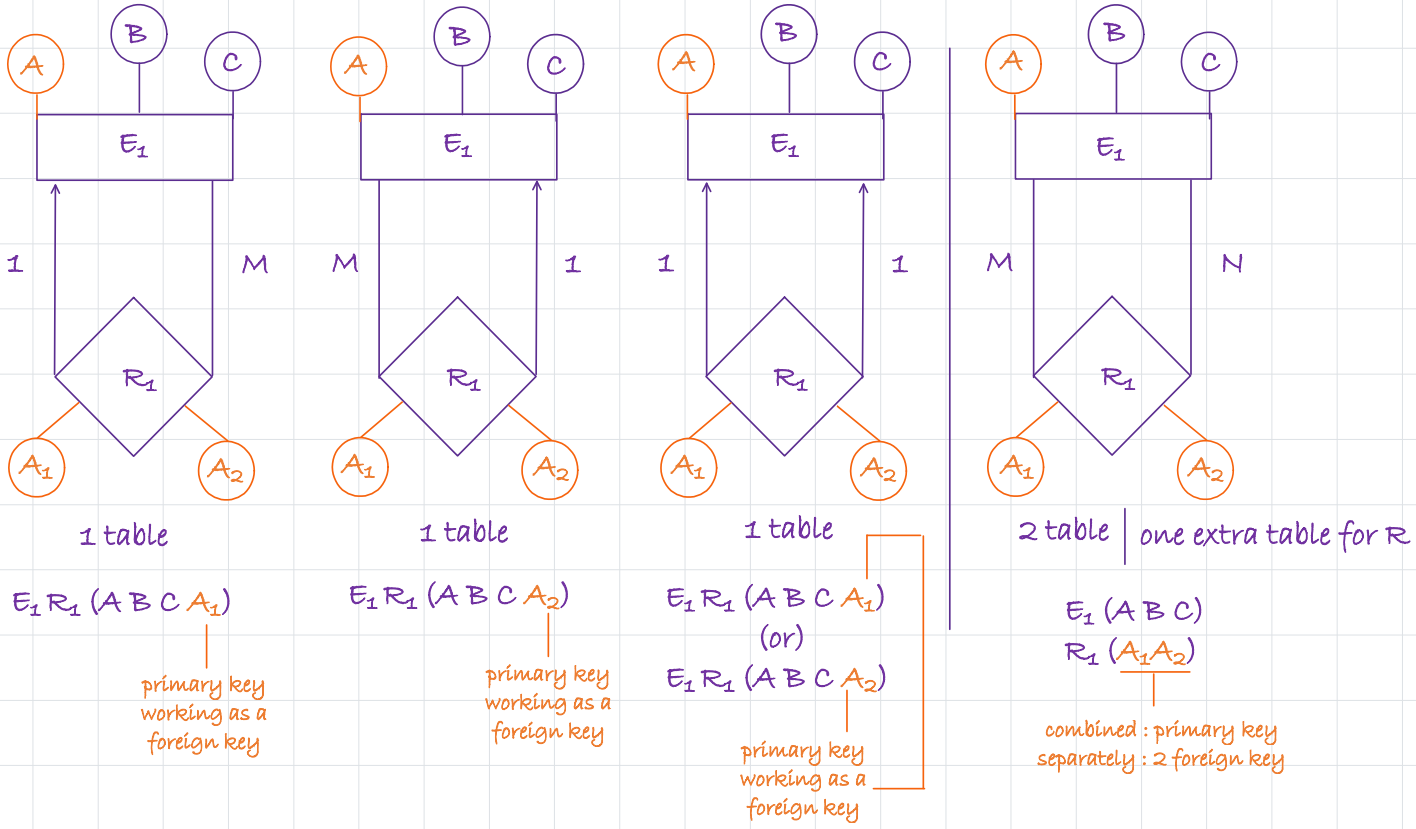
R_3 – E_2 E_3

E_2R_4

E_3R_2 – E_3 E_1



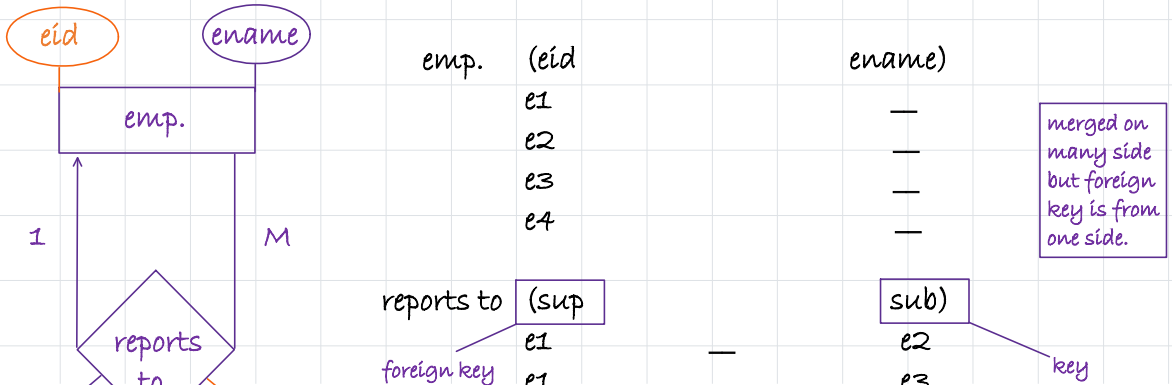
Self referential relationship set (unary relationship)

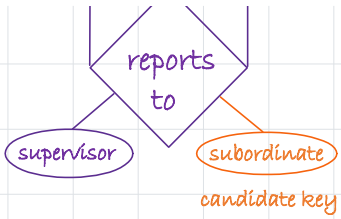


through employee entity set : reports to relationship set related between supervisor and subordinate

(i) one to many / many to one

each supervisor can supervise many subordinates and each subordinate reports to one supervisor





emp. reports to

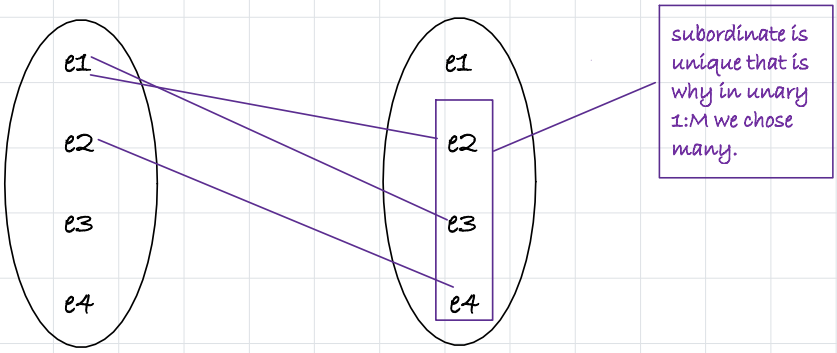
(eid	ename	sup)
e1	→	NULL
e2	→	e1
e3	→	e1
e4	→	e2

reports to (sup — sub)

foreign key

e1	—	e2
e1	—	e3
e2	—	e4

key

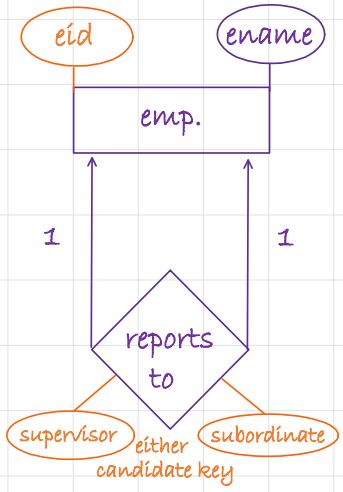


subordinate is unique that is why in unary 1:M we chose many.

minimum 1 table and 1 foreign key refer to same table

(ii) one to one

each supervisor can supervise one subordinate and one subordinate reports to one supervisor



emp. reports to — foreign key

(eid	ename	sup)
e1	→	NULL
e2	→	e1
e3	→	e2
e4	→	e3

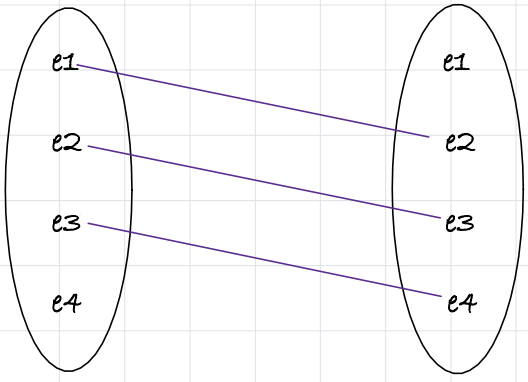
(OR)

emp. (eid — ename)

e1	—
e2	—
e3	—
e4	—

reports to (sup — sub)

e1	—	e2
e2	—	e3
e3	—	e4

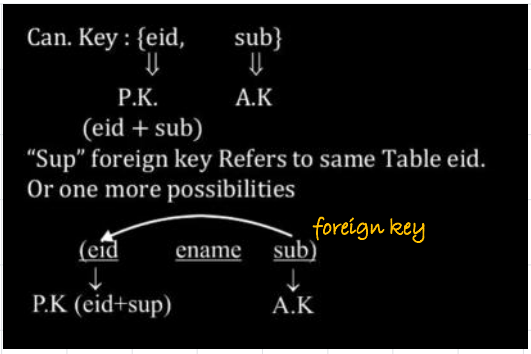


Can. Key : {eid, sub}

P.K. (eid + sub) — A.K

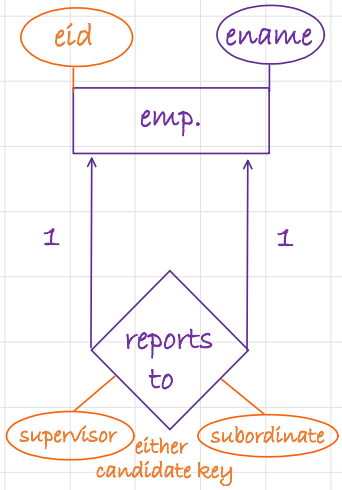
"Sup" foreign key Refers to same Table eid.

Or one more possibilities



(iii) many to many

each supervisor can supervise many subordinate and each subordinate reports to many supervisor



emp. (eid ename)

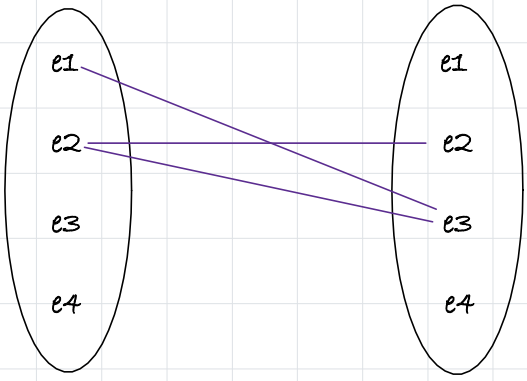
e1	—
e2	—
e3	—
e4	—

reports to (sup sub)

e2	—	e2
e1	—	e3
e2	—	e3

emp. reports to (foreign key)

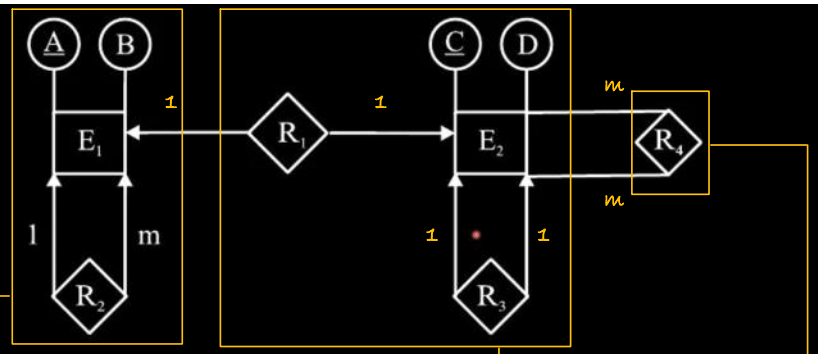
(eid	ename	sup)
e1	→	NULL
e2	→	e2
e3	→	e2,e1
e4	→	NULL



minimum 2 table and 2 foreign key

Questions :

Q.



(I) $E_1\ R_2\ (\underline{A}\quad B\quad A)$ refers to E_1

3 key (II) $E_2\ R_1\ R_3\ (\underline{C}\quad D\quad A\quad C)$

(III) $R_4\ C_3\ C_4$

foreign key

foreign key

Q. Consider the following ER diagram: [MCQ]

How many total attributes required for the minimized relations of the above ER diagram?

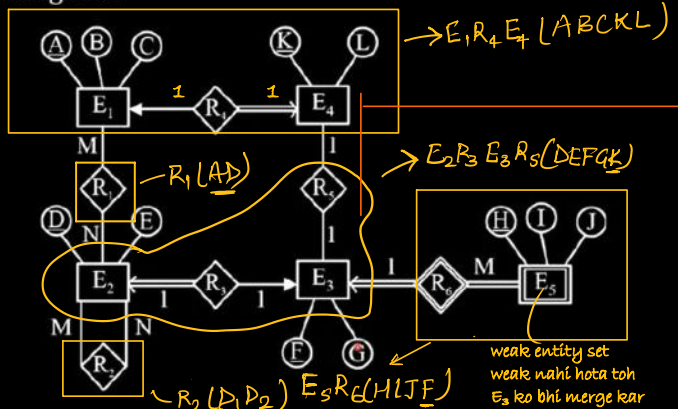
A. 14

B. 15

C. 18

D. None of these

F·K = 6



$\rightarrow E_1R_4E_4\ (ABCKL)$

$R_1\ (\underline{AD})$

$\rightarrow E_2R_3\ E_3R_5(DEFGK)$

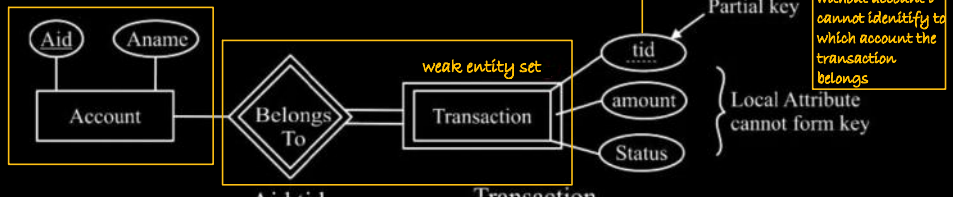
$R_2\ (\underline{D_1P_2})$ $E_5R_6(HIJF)$

1 to 1 hai isko upar merge kardo ya neeche humne neeche kiya hence K as a foreign key.

weak entity set weak nahi hota toh E_3 ko bhi merge kar lete

⇒ Entities of weak entity set are depending entities.

Ex.



cannot tell about complete transaction uniquely because without account i cannot identify to which account the transaction belongs

Aid tid

Transaction

Aid	Aname
A_1	
A_2	
A_3	
A_4	

Aid	tid
A_1	t_1
A_2	t_1
A_2	t_2
A_3	t_2

tid	amount	Status
t_1	5000	Debit
t_1	4000	Credit
t_2	5000	Debit
t_2	5000	Debit

[1: m] Ambiguity

not able to identify, to which account these transaction belongs

MCQ

In an Entity-Relationship (ER) model, suppose R is a many-to-one relationship from entity set E1 to entity set E2. Assume that E1 and E2 participate totally in R and that the cardinality of E1 is greater than the cardinality of E2

MCQ

In an Entity-Relationship (ER) model, suppose R is a many-to-one relationship from entity set E1 to entity set E2. Assume that E1 and E2 participate totally in R and that the cardinality of E1 is greater than the cardinality of E2.
Which one of the following is true about R?

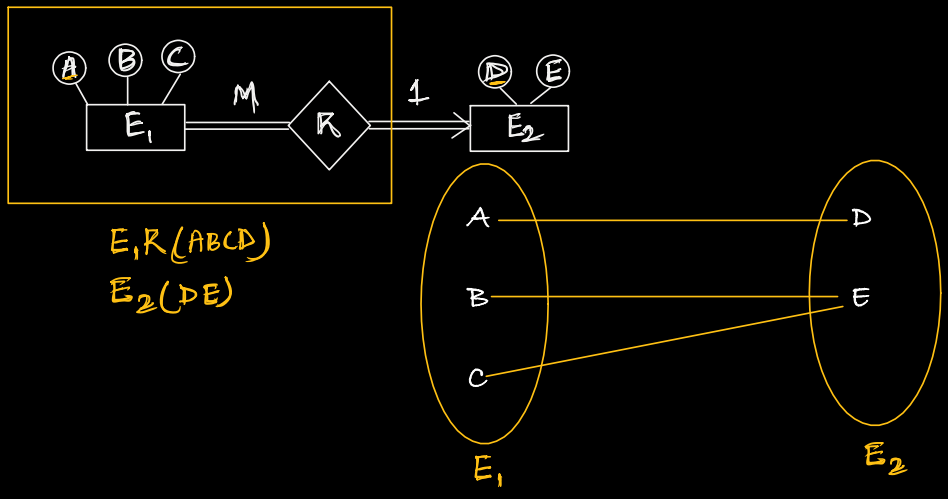
[GATE-2018-CS: 1M]

(A) Every entity in E1 is associated with exactly one entity in E2.

(B) Some entity in E1 is associated with more than one entity in E2.

(C) Every entity in E2 is associated with exactly one entity in E1.

(D) Every entity in E2 is associated with at most one entity in E1.



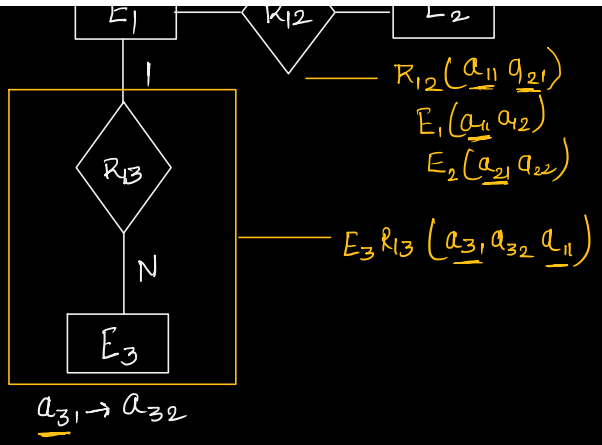
E1R(ABCD)
E2(DE)

NAT

Consider an Entity-Relationship (ER) model in which entity sets E_1 and E_2 are connected by an m: n relationship R_{12} . E_1 and E_3 are connected by a 1: n (1 on the side of E_1 and n on the side of E_3) relationship R_{13}.
E_1 has two single-valued attributes a_{11} and a_{12} of which a_{11} is the key attribute. E_2 has two single valued attributes a_{21} and a_{22} of which a_{21} is the key attribute. E_3 has two single valued attributes a_{31} and a_{32} of which a_{31} is the key attribute. The relationships do not have any attributes.
If a relational model is derived from the above ER model, then the minimum number of relations that would be generated if all the relations are in 3 NF is 4

[GATE-2015-CS: 2M]

3NF: X - Superkey.
Y : Prime.



MCQ

Consider the following statements S1 and S2 about the relational data model:

S1: A relation scheme can have at most one foreign key. M:N (2 Foreign key)

S2: A foreign key in a relation scheme R cannot be used to refer to tuples of R.

foreign key refer to primary key either it its own table or in any other table

Which one of the following choices is correct?

[GATE-2021-CS: 1M]

A Both S1 and S2 are true

B S1 is true and S2 is false

C S1 is false and S2 is true

D Both S1 and S2 are false

MCQ

Let E_1 and E_2 be two entities in an E-R diagram with simple single-valued attributes. R_1 and R_2 are two relationships between E_1 and E_2, where R_1 is one-to-many and R_2 is many-to-many. R_1 and R_2 do not have any attributes of their own. What is the minimum number of tables required to represent this situation in the relational model?

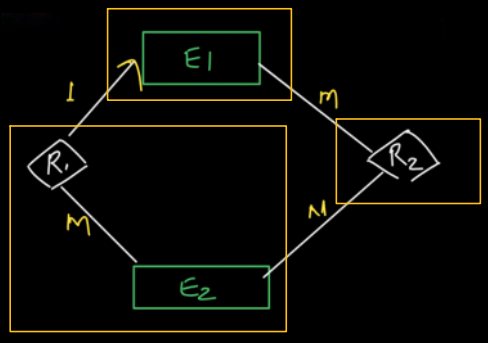
[GATE-2005]

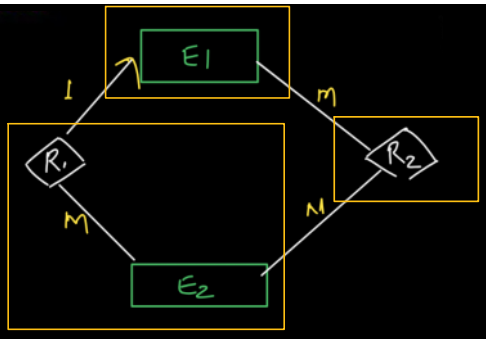
A 2

B 3

C 4

D 1





MCQ

Consider the entities 'hotel room', and 'person' with a many to many relationship 'lodging' as shown below:

If we wish to store information about the rent payment to be made by person (s) occupying different hotel rooms, then this information should appear as an attribute of

[GATE-2005]

A Person

B Hotel Room

C Lodging ✓

D None of these

NAT

Consider the following tables T1 and T2.

In table T1, **P** is the primary key and **Q** is the foreign key referencing **R** in table T2 with on-delete cascade and on-update cascade. In table T_2, **R** is the primary key and **S** is the foreign key referencing **P** in table T1 with on-delete set NULL and on-update cascade. In order to delete record ⟨3,8⟩ from table T1, the number of additional records that need to be deleted from table T1 is 0 Ans.

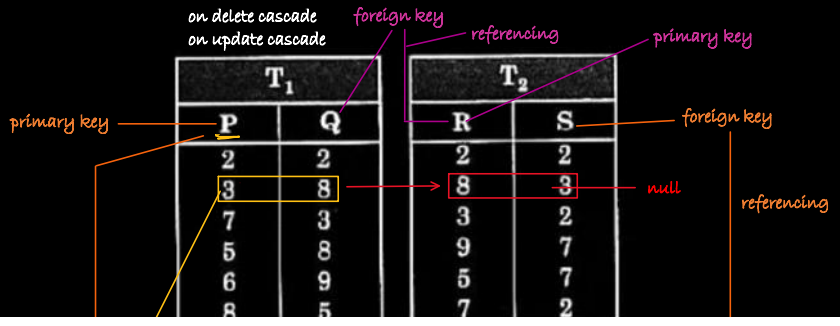
T_1

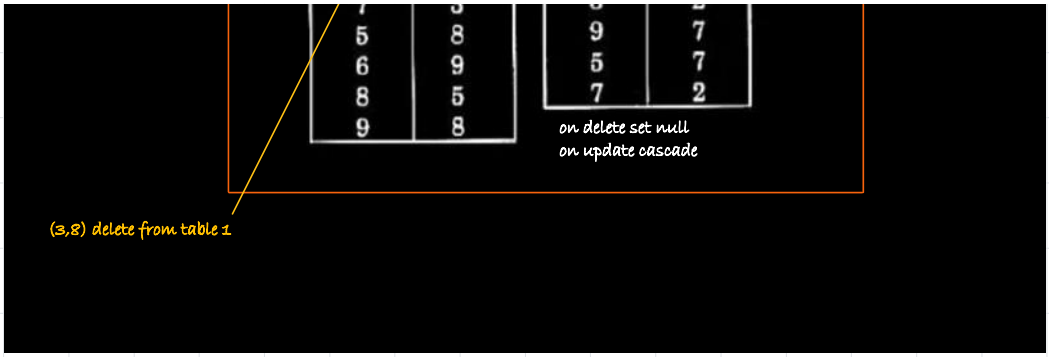
P	Q
2	2
3	8
7	3
5	8
6	9
8	5
9	8

T_2

R	S
2	2
8	3
3	2
9	7
5	7
7	2

[GATE-2017-CS: 1M]




5 8
6 9
8 5
9 8
9 7
5 7
7 2
on delete set null
on update cascade
(3,8) delete from table 1

Query language

Query language - invented by scientist E.F. codd
to retrieve the data from database

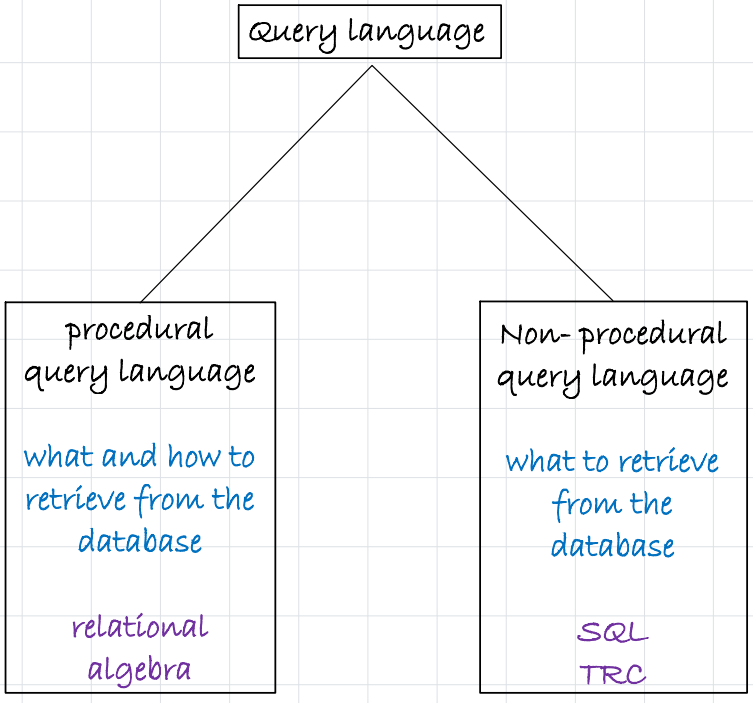
practical

SQL : structured query languaage

theoritical

- RA : relational algebra

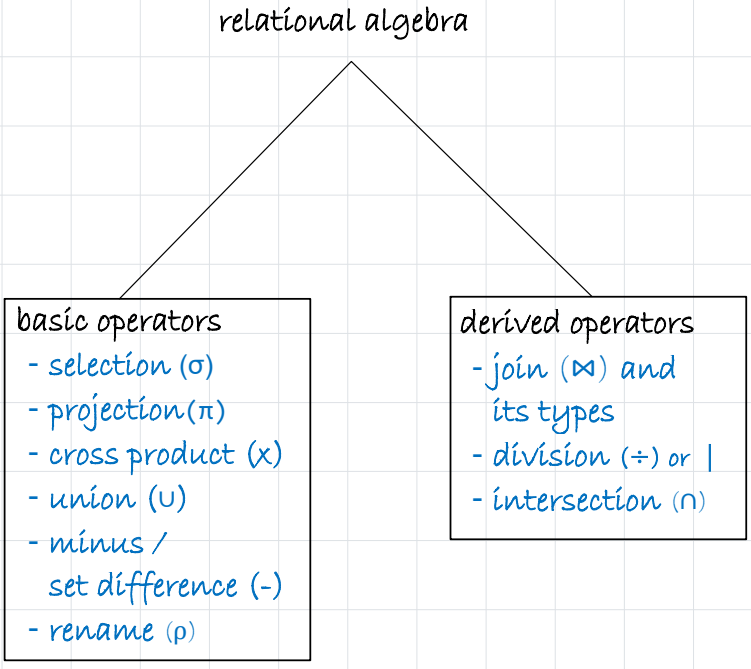
- RC : relational calculus



Difference between RA and SQL

Relational algebra	SQL
By default distinct value in output value does not repeat input : relation (table) output : relational (table)	By default distinct value retain value may repeat

Relational algebra : the basic idea of query language is that query executed on a database table (relation) tuple by tuple (row by row) one tuple at a time



1. selection (σ) : it selects a tuple (record/row) based on specified condition

syntax : - $\sigma_{condition}$ (relation)

ex: $\sigma_{CGPA>9}$ (student)

$\sigma_{state = 'WB'}$ (student)

2. projection(π) : it selects attribute (column/field) (or) attribute list from relation

syntax : - $\pi_{attribute}$ (relation)

- $\pi_{attribute\ list}$ (relation)

ex: π_{name} (student)

$\pi_{name\ contact}$ (student)

π_{ABCD} (R)

1. example :

student

roll no.	name	CGPA
1	Ritesh	9
2	suman	10
3	shweta	9
4	rashmi	10
5	nanda	9
6	aryan	10
7	shweta	9
8	aryan	10

$\pi_{roll\ no.}$ (student) : gave us the roll no. (attribute list)

roll no.
1
2
3
4
5
6
7
8

π_{name} (student) : gave us the name (attribute list)

name
ritesh
suman
shweta
rashmi
nanda
aryan

values does not repeat

student

roll no.	name	CGPA
1	Ritesh	9
2	suman	10
3	shweta	9
4	rashmi	10
5	nanda	10
6	aryan	10
7	shweta	10
8	aryan	9

$\sigma_{CGPA>9}$ (student) :

gave us the tuples based on condition specified

roll no.	name	CGPA
2	suman	10
4	rashmi	10
5	nanda	10
6	aryan	10
7	shweta	10

tuples with CGPA 9 and less than 9 will be deleted

q. write a query to find sname whose CGPA is greater than 9.

$\pi_{name}[\sigma_{CGPA>9}$ (student)] :

selection gave us the tuples based on condition specified and projection gave us the attributes according to the condition specified

name
suman
rashmi
nanda
aryan
shweta

2. example :

parts

Pid	Pname	color
1	aryan	Red
2	suman	Green
3	shweta	Red
4	rashmi	Green
5	nanda	Yellow
6	ritesh	Blue

write a query to find pname of Red color parts.

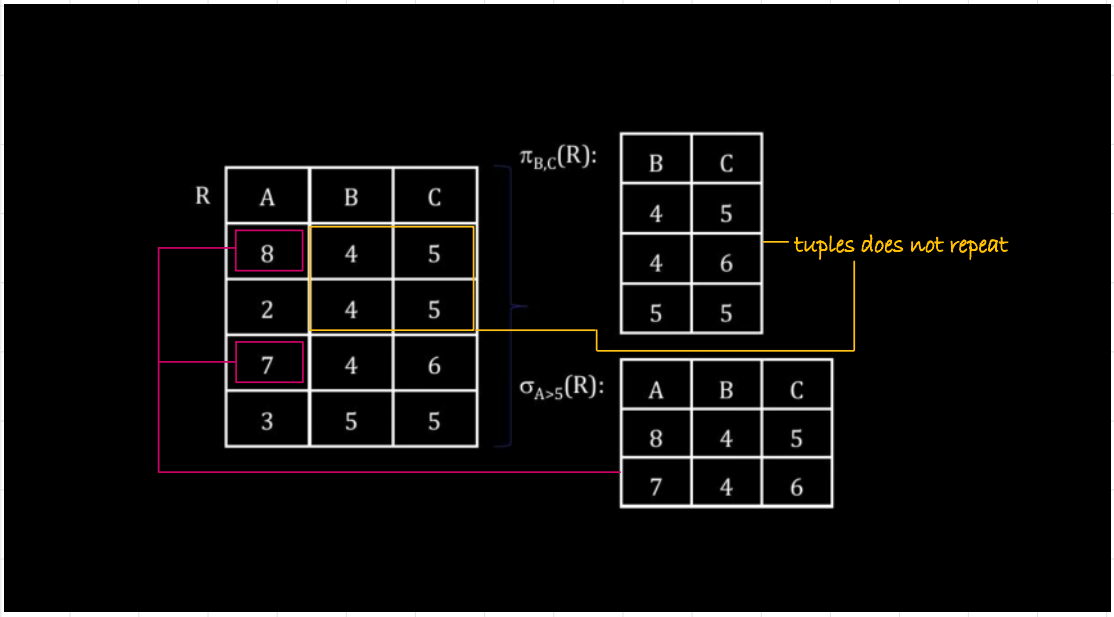
$\pi_{pname}[\sigma_{color = 'red'}$ (parts)]

$\sigma_{color = 'red'}$ (parts)

Pid	Pname	color
1	aryan	Red
3	shweta	Red

$\pi_{pname}[\sigma_{color = 'red'}$ (parts)]

Pname
aryan
shweta



note :

relation algebra always maintain the order

Reserves (R_1)

Sid	Bid	day
22	101	10/10/96
58	103	11/12/96

Sailors(S_1)

Sid	Sname	Rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

Sailors(S_2)

Sid	Sname	Rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

1. $\sigma_{rating>8}(S_2)$:

sid	sname	Rating	age
28	yuppy	9	35.0
58	rusty	10	35.0

2. $\pi_{sname, rating}[\sigma_{rating > 8}(S_2)]$:

$[\sigma_{rating > 8}(S_2)]$:

sid	sname	Rating	age
28	yuppy	9	35.0
58	rusty	10	35.0

$\pi_{sname, rating}[\sigma_{rating > 8}(S_2)]$:

sname	Rating
yuppy	9
rusty	10

3. $\pi_{age}(S_2)$:

age
35.0
55.5

values does not repeat.

some important points/conclusion

1. selection is commutative

$$\sigma_{c3}(\sigma_{c2}(\sigma_{c1}(R))) \equiv \sigma_{c1}(\sigma_{c3}(\sigma_{c2}(R)))$$

C : condition

$$\sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c1} \wedge \sigma_{c2}(R))$$

$$\sigma_{c1}(\sigma_{c2}(R)) \equiv \sigma_{c1}(\sigma_{c2}(R))$$

$$\sigma_{rating > 9}[\sigma_{age \geq 35}(S_1)) \equiv \sigma_{rating > 9} \wedge \sigma_{age \geq 35}(S1)$$

Sailors(S_1)

Sid	Sname	Rating	age
22	dustin	7	45.0

22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

C_1 : rating > 9
C_2 : age ≥ 35

$\sigma_{rating > 9}[\sigma_{age \geq 35}(S_1))$ ≡ $\sigma_{age \geq 35}[\sigma_{rating > 9}(S_1))$

sid	sname	Rating	age
5	rusty	10	35.0

rating greater than 9 and age greater than or equal to 35

sid	sname	Rating	age
5	rusty	10	35.0

age greater than or equal to 35 and rating greater than 9

2. both are not equivalent

$\pi_{A1}[\sigma_{C1}(R)]$ ≠ $\sigma_{C1}[\pi_{A1}(R)]$

C_1 : CGPA > 9
A_1 : name

student

roll no.	name	CGPA
1	Ritesh	9
2	suman	10
3	shweta	9
4	rashmi	10
5	nanda	9

4	rashmi	10
5	nanda	9
6	aryan	10
7	shweta	9
8	aryan	10

$\pi_{name}[\sigma_{CGPA>9}(student)]$

name
suman
rashmi
nanda
aryan
shweta

$\neq$

$\sigma_{CGPA>9}[\pi_{name}(student)]$

name	CGPA
ritesh	
suman	
shweta	
rashmi	
nanda	
aryan	

empty because we filterted the name already

3. L.H.S can be replaced by R.H.S but R.H.S. cannot be replaced by L.H.S in this example

$\sigma_{C1}[\pi_{A1}(R)] \implies \pi_{A1}[\sigma_{C1}(R)]$

because in this (L.H.S. part) firstly attribute A_1 is selected then condition applied only on A_1 attribute

ex.

A_1 [attribute] : CGPA
C_1 [condition]: CGPA > 9
then L.H.S. can be replaced by R.H.S.

applying on same condition !

4. only possible when a is lesser than b

$\pi_A[\pi_{AB}(R)] \equiv \pi_A(R)$

$\pi_A(R) \equiv \pi_A[\pi_B(R)]$

if and only if $a \leq b$

ex.

$\pi_{sid}(student) \equiv \pi_{sid}[\pi_{sid\ sname}(student)]$

a

a a b

5. projection is not commutative

$\pi_A(R) \not\equiv \pi_B(R)$

a [attribute] : name
b [attribute] : bid

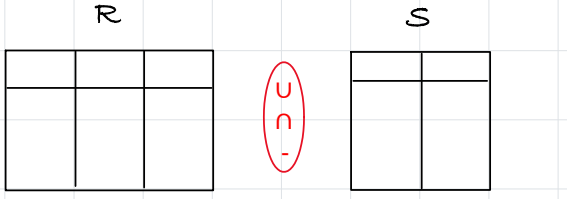
$\pi_{name}(S_1) \not\equiv \pi_{bid}(S_1)$

topic : set operators

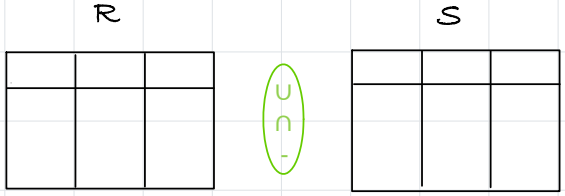
1. union (∪)
2. intersection (∩)
3. minus / set difference (-)

to apply set operators both relation must be union (type) compatible relation R (A_1 A_2 A_3 .....A_n) and Relation S (B_1 B_2 B_3 .....B_n) is union compatible (type compatible)

if and only if :
1. Arity [degree (number of attributes)] of R and S must be same.



arity is not same hence not possible to apply



arity is same hence possible to apply

2. Range (domain) of attribute must be same.

R

name	branch	city

$\cup$

S

age	CGPA	contact

domain is different hence not possible to apply

table : C language

R

name	branch	city

table : java language

S

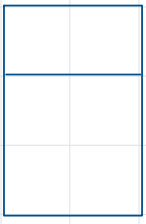
name	branch	city

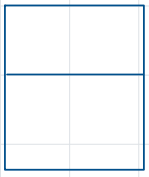
$\cup$

domain is same hence possible to apply

note :

same domain with different name is allowed





Relation R and Relation S

Relation R (A_1 A_2 A_3 ..... A_n) and Relation S (B_1 B_2 B_3 ..... B_n) is union (type) compatible

1. Arity of R $\equiv$ Arity of S
   - Arity of R(4) $\equiv$ Arity of S(4)

2. Domain (A_i) $\equiv$ Domain (B_i)
   - Domain (A_1) $\equiv$ Domain (B_1)
   - Domain (A_2) $\equiv$ Domain (B_2)
   - Domain (A_3) $\equiv$ Domain (B_3)

1. set operation

$\pi_{sid\ sname}(\ldots\ \ldots\ \ldots) \cap \pi_{sid\ sname}(\ldots\ \ldots\ \ldots)$

arity and domain are same so operations allowed

R

A
2
2
2
3

S

B
2
2
4

union : $R \cup S$: $\{X/X \in R \vee X \in S\}$: tuples that are either in R or in S

A
2
3
4

set difference : $R-S$: $\{X/X \in R \wedge X \notin S\}$: those present in R but not in S

A
3

intersection : $R \cap S$: $\{X/X \in R \wedge X \in S\}$: those belongs to R and S

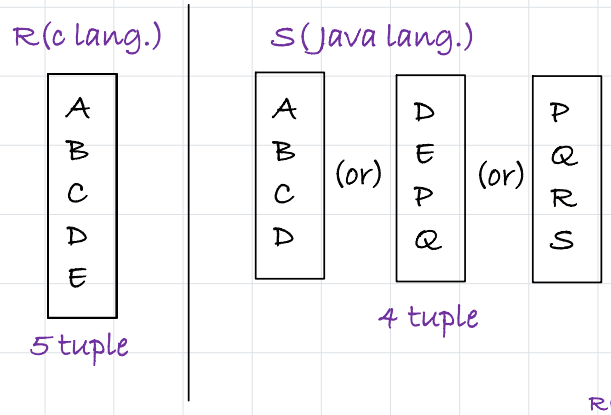
A
2

Assume relation R and relation S consists M and N tuple respectively

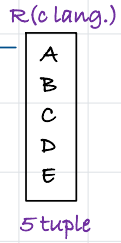
(i) Range of tuples in $R \cup S$ = max(M, N) to M + N
minimum maximum

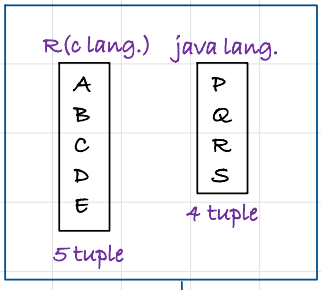
minimum = maximum of (M, N)

maximum = M + N

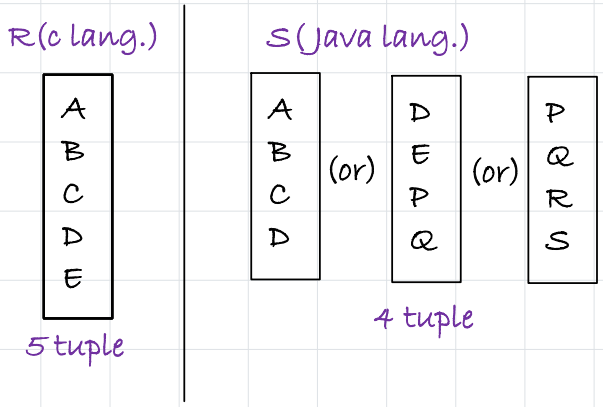


R∪S
- minimum = maximum of (M, N)
= 5, 7
- maximum = M + N
= 5 + 4
= 9

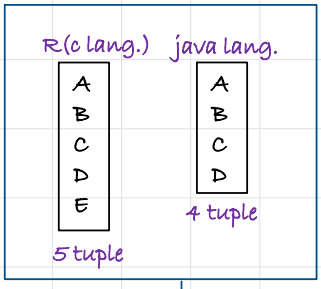




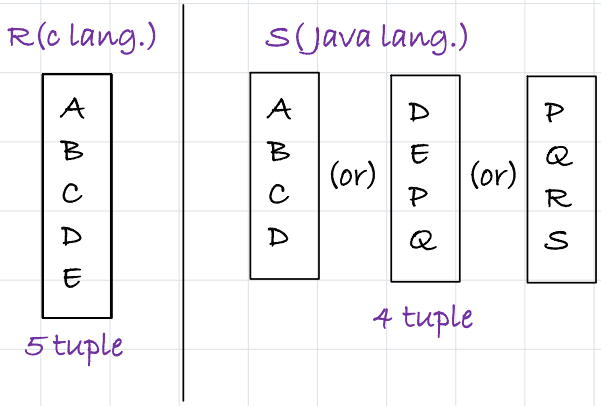
(ii) Range of tuples in R∩S = ∅ to min (M, N)



R∩S
- minimum = ∅ (nothing common)
= 0, 2
- maximum = min (M, N)
= 4 (common)

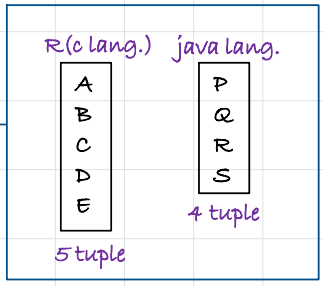


(iii) Range of tuples in R-S = ∅ to M

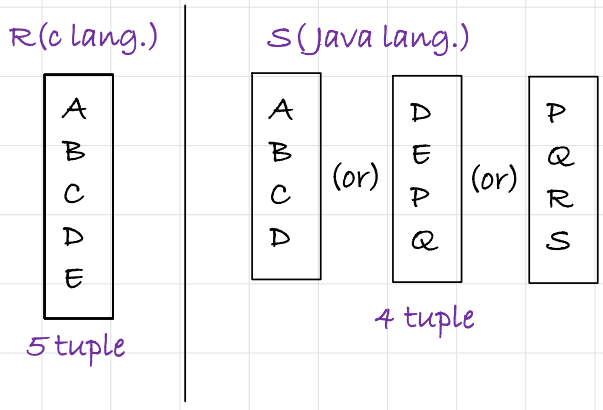


R-S

- minimum = ø (whole table cancelled)
  = 0, 1, 3
- maximum = (R-S) : M (5)

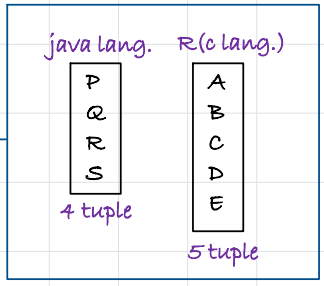


(iv) Range of tuples in S-R = ø to N



S-R

- minimum = ø (whole table cancelled)
  = 0, 2
- maximum = (S-R) : N (4)



Example: to find all courses taught in the Fall 2009 semester, but not in the Spring 2010 semester

$$\pi_{course_id}(\sigma_{semester = \text{"Fall"} \wedge year = 2009}(section)) - \pi_{course_id}(\sigma_{semester = \text{"Spring"} \wedge year = 2010}(section))$$

Sailors(S_1)

Sid	Sname	Rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

Sailors(S_2)

Sid	Sname	Rating	age
28	yuppy	9	35.0
31	lubber	8	55.5
44	guppy	5	35.0
58	rusty	10	35.0

$S_1 \cup S_2$

22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0
44	guppy	5	35.0
28	yuppy	9	35.0

$S_1 - S_2$

22	dustin	7	45.0

$S_1 \cap S_2$

31	lubber	8	55.5
58	rusty	10	35.0

$S_2 - S_1$

44	guppy	5	35.0
28	yuppy	9	35.0

@korth book example :

R

A	B
α	1
α	2
β	1

S

C	B
α	1
β	3

Union :

RUS :

A	B
α	1
α	2
β	1
β	3

SUR :

C	B
α	1
β	3
α	2
β	1

RUS ≡ SUR

note :

union is commutative

set difference/EXCEPT :

S - R :

C	B
β	3

R - S :

A	B
α	2
β	1

$R - S \neq S - R$

note :

minus / set difference is not commutative

intersection :

R∩S :

A	B
α	1

S∩R :

C	B
α	1

$R \cap S \equiv S \cap R$

note :

intersection is commutative

6. Cross product / cartesian product ($R \times S$)

it result all attributes of R followed by all attributes of S, and each record of R paired with every record of S.

R	S
n_1 tuple	n_2 tuple
c_1 attribute	c_2 attribute

$|(R \times S)| = |n_1| \times |n_2|$ tuple (cardinality : multiply)

Degree $(R \times S)$ = Degree(c_1) + Degree(c_2) (attribute, degree : add)

$R \times S \equiv S \times R$

note :

cross product is commutative

example :

R	S
3 tuple	2 tuple
2 attribute	3 attribute

R x S = 3 x 2 tuple = 6 tuple
2 + 3 attribute = 5 attribute

example :

R

A	B	C
1	2	3
4	5	6

S

D	E	F
7	8	9
10	11	16

R x S = 2 x 2 tuple = 4 tuple
3 + 3 attribute = 6 attribute

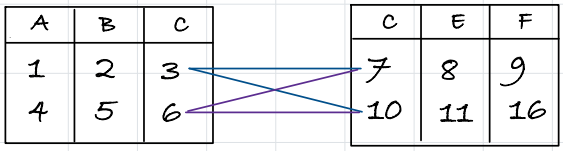
R x S =

A	B	C	D	E	F
1	2	3	7	8	9
1	2	3	10	11	16
4	5	6	7	8	9
4	5	6	10	11	16

variation : if in relation R and relation S attribute having same name (common name)

R(ABC)
S(CEF)

R S



A	B	C
1	2	3
4	5	6

C	E	F
7	8	9
10	11	16

common attribute, we can write in 3 ways :

TYPE 1 :

R x S =

A	B	R.C	S.C	E	F
1	2	3	7	8	9
1	2	3	10	11	16
4	5	6	7	8	9
4	5	6	10	11	16

TYPE 2 :

R x S =

A	B	3	4	E	F
1	2	3	7	8	9
1	2	3	10	11	16
4	5	6	7	8	9
4	5	6	10	11	16

TYPE 3 : best approach

R x S =

R.A	R.B	R.C	S.C	S.E	S.F
1	2	3	7	8	9
1	2	3	10	11	16
4	5	6	7	8	9
4	5	6	10	11	16

note :

- for set operation both relation must be union (type) compatible

- for cross (cartesian) product and join operation both relation does not require union (type) compatible

note :

- relation R with n tuples and relation S with 0 tuples then
- number of tuples in R x S = 0 tuples

- relation R with n tuples and relation S with 0 tuples then
- number of tuples in R x S = 0 tuples

if in R and S any table is empty then R x S = empty

Reserves (R_1)

Sid	bid	day
22	101	10/10/96
58	103	11/12/96

Sailors (S_1)

Sid	Sname	Rating	age
22	dustin	7	45.0
31	lubber	8	55.5
58	rusty	10	35.0

R_1 : 2 Tuple, 3 Attribute

S_1 : 3 Tuple, 4 Attribute

$R_1 \times S_1 = 2 \times 3 = 6$ Tuple

$3 + 4 = 7$ Attribute

R_1.sid	R_1.bid	R_1.day	S_1.sid	S_1.name	S_1.rating	S_1.age
22	101	10/10/96	22	dustin	7	45.0
22	101	10/10/96	31	lubber	8	55.5
22	101	10/10/96	58	rusty	10	35.0
58	103	11/12/96	22	dustin	7	45.0
58	103	11/12/96	31	lubber	8	55.5
58	103	11/12/96	58	rusty	10	35.0

1. why cross product is required?

for performing the JOIN operation (inner and outer join).

2. But if we dont want to perform join operation then why cross product is required?

if we require something like R.C < S.D (comparison)

R.B < S.C
R.C < S.B

R

A	B	C
1	2	4
4	5	6

S

B	C	D
7	8	8
10	11	4

if we require something like
R.C < S.D
then it is not possible in query language directly because at a time it fetch a tuple either from relation R (or) from relation S at a time so it is not possible to compare that is why do the cross products of R and S (R x S) to convert them into one tuple to make comparison possible.

R x S = 2 x 2 tuple = 4 tuple
3 + 3 attribute = 6 attribute

R x S=

R.A	R.B	R.C	S.B	S.C	S.D
1	2	4	7	8	8
1	2	4	10	11	4
4	5	6	7	8	8
4	5	6	10	11	4

R.C < S.D =

R.A	R.B	R.C	S.B	S.C	S.D
1	2	4	7	8	8
4	5	6	7	8	8

but
R : 500 tuple
S : 400 tuple

R x S = 500 x 400 tuple = 2,00,000 tuple

cross product is not meaningfull until and unless we do not apply condition [selection] and not selecting [project] any attribute, so we have to study join.

1. Join operations ($\bowtie$) : it is used to combine information from two (or) more relation (table)

(A) Inner join
  (i) Conditional (theta) join ($\bowtie_c$)
  (ii) Equi join
  (iii) natural join

(B) Outer join
  (i) Left outer join ($\overset{\cdot}{\bowtie}$) (ulman($\overset{\cdot}{\bowtie}_L$))
  (ii) Right outer join ($\overset{\cdot}{\bowtie}$) (ulman($\overset{\cdot}{\bowtie}_R$))
  (iii) Full outer join ($\overset{\cdot}{\bowtie}$) $\equiv$ outer join ($\overset{\cdot}{\bowtie}$) (ulman($\overset{\cdot}{\bowtie}$))

(C) Cross join ($R \times S$)

(i) natural join ($R \bowtie S$) : it is derived operator performed in 3 steps.

step 1 : cross (cartesian) product of R and S.

step 2 : select the tuples which satisfy equality condition on all common attributes (from $R \times S$)

Step 3 : projection of distinct attribute.

$$R \bowtie S = \pi_{\text{distinct attribute}} [\sigma_{\text{equality condition on all common attributes}} (R \times S)]$$

- Step 3 : projection of distinct attribute.
- step 2 : select the tuples which satisfy equality condition on all common attributes (from $R \times S$)
- step 1 : cross (cartesian) product of R and S.

example :
R(ABC) and S(BCD)

$$R \bowtie S = \pi_{ABCD} [\sigma_{R.B = S.B \wedge R.C = S.C} (R \times S)]$$

distinct attribute (b was repeating)

$R \bowtie S =$

A	B	C	D

(b was repeating)

$R \bowtie S =$

A	B	C	D

(ii) Conditional (theta) join ($R \bowtie_c S$) : it is derived operator performed in 3 steps.

step 1 : cross (cartesian) product of R and S.

step 2 : select the tuples which satisfy given condition on all common attributes (from R X S)

Step 3 : projection of all attribute.

$$R \bowtie_c S = \pi_{\text{all attribute}} [\sigma_{\text{given condition on all common attributes}} (R \times S)]$$

- π : Step 3 : projection of all attribute.
- σ : step 2 : select the tuples which satisfy given condition on all common attributes (from R X S)
- $(R \times S)$: step 1 : cross (cartesian) product of R and S.

example :
R(ABC) and S(BCD)

$$R \bowtie_c S = \pi_{ABCBCD} [\sigma_{\text{given condition}} (R \times S)]$$

distinct attribute (b was repeating)

$R \bowtie_c S =$

R.A	R.B	R.C	S.B	S.C	S.D

condition :

$S1 \bowtie_{S1.sid < R1.sid} R1$

$R \bowtie_c S = \sigma_c (R \times S)$

S_1 Sid

R_1 Sid

(Sid)	Sname	Rating	age	(Sid)	bid	day
22	dustin	7	45.0	22	101	10/10/96
22	dustin	7	45.0	58	103	11/12/96
31	lubber	8	55.5	22	101	10/10/96
31	lubber	8	55.5	58	103	11/12/96
58	rusty	10	35.0	22	101	10/10/96
58	rusty	10	35.0	58	103	11/12/96

$S_1 \times R_1$

$S1 \bowtie_{S1.sid < R1.sid} R1$

output

(Sid)	Sname	Rating	age	(Sid)	bid	day
22	dustin	7	45.0	58	103	11/12/96
31	lubber	8	55.5	58	103	11/12/96

$S1 \bowtie_{S1.sid < R1.sid} R1$

- Result schema same as that of cross-product. 4 + 3 = 7 attribute
- Fewer tuples than cross – product, might be able to compute more efficiently. — many tuples in cross product, but in conditional there will be less tuples according to condition.
- Sometimes called a theta –join.

(iii) Equi join ($R \bowtie_c S$) : it is a subset or similar to conditional join but here only equality condition is applied. (a case of condition join where the condition c contains only equalities.)

step 1 : cross (cartesian) product of R and S.

step 2 : select the tuples which satisfy equality condition on all attributes (from R X S)

Step 3 : projection of attribute (jo attribute par ho rahi use ek hi baar lenge).

$R \bowtie S = \pi_{\text{attribute}} [\sigma_{\text{equality condition on all attributes}} (R \times S)]$

Step 3 : projection of attribute.

step 2 : select the tuples which satisfy equality condition on all attributes (from R X S)

step 1 : cross (cartesian) product of R and S.

condition : applied on sid.

$S1 \bowtie_{sid} R1$

$S_1 \times R_1$

S_1 R_1

(Sid)	Sname	Rating	age	(Sid)	bid	day
22	dustin	7	45.0	22	101	10/10/96
22	dustin	7	45.0	58	103	11/12/96
31	lubber	8	55.5	22	101	10/10/96
31	lubber	8	55.5	58	103	11/12/96
58	rusty	10	35.0	22	101	10/10/96
58	rusty	10	35.0	58	103	11/12/96

Sid	Sname	Rating	age	bid	day
22	dustin	7	45.0	101	10/10/96
58	rusty	10	35.0	103	11/12/96

- Result schema similar to cross – product, but only one copy of fields for which equality is specified.
- Natural join: Equijoin on all common fields.

duplicates hai 'sid' unhe ek hi baar likhna hai

R(ABC) and S(BCD)

1. Equi join :

$(R \bowtie_{R.B = S.B} S)$

$\pi_{Attribute}[\sigma_{R.B = S.B}(R \times S)]$

R(ABC) S(BCD)

A	B	R.C	S.C	D

condition applied on B
B will not be repeating

2. natural join :

$(R \bowtie S)$

$\pi_{distinct\ attribute}[\sigma_{R.B = S.B \wedge R.C = S.C}(R \times S)]$

R(ABC) S(BCD)

A	B	C	D

condition applied on all common attribute
hence, B and C will not be repeating

natural join is like a equi join in which equality condition applied on all common attributes of R and S

example :

1. T_1 (ABC) and T_2 (BCDE)

$(T_1 \bowtie T_2) = \pi_{ABCDE}[\sigma_{T1.B = T2.B \cap T1.C = T2.C}(T_1 X T_2)]$

when nothing is common

2. T_1 (AB) and T_2 (CD)

$(T_1 \bowtie T_2) \equiv T_1 X T_2 = \pi_{ABCD}(T_1 X T_2)$

is condition mai jab common na ho toh 'join' equal ho jata hai 'cross product' ke because there is no tuple that satisfy the equality condition

example : $(R \bowtie S)$

$\pi_{ABCD}[\sigma_{R.B = S.B \wedge R.C = S.C}(R \times S)]$

R

A	B	C
1	2	4
3	2	6

dangling tuple of R

S

B	C	D
2	4	8
2	7	4

dangling tuple of S

$R \times S$ = 2 x 2 tuple = 4 tuple

3 + 3 attribute = 6 attribute

Step 1 : $R \times S$ =

R.A	R.B	R.C	S.B	S.C	S.D
1	2	4	2	4	8
1	2	4	2	7	4
3	2	6	2	4	8
3	2	6	2	7	4

Step 2 : $R.B = S.B \wedge R.C = S.C$

R.A	R.B	R.C	S.B	S.C	S.D
1	2	4	2	4	8
1	2	4	2	7	4
3	2	6	2	4	8
3	2	6	2	7	4

only one tuple satisfied the condition

Step 3 : $R.B = S.B \wedge R.C = S.C$

R.A	R.B	R.C	S.B	S.C	S.D
1	2	4	2	4	8

Step 4 : $R \bowtie S$

A	B	C	D
1	2	4	8

two types of tuple :

#Dangling tuple : Dangling tuple (row) is a tuple that fail to match (satisfy) any tuple of other relation in common attribute (or) given condition.

#Spurious tuple : extra tuple in join (lossy join)

Q. how to check dangling tuple directly?

$(R \bowtie_{R.B < S.D} S)$

R(ABC)

A	B	C
1	3	5
4	6	7
7	9	11

S(DEFG)

D	E	F	G
4	6	7	8
7	7	8	9

7	9	11

$(R \bowtie_{R.B < S.D} S)$

3<4 true
3<7 true
6<4 false
6<7 true
9<4 false
9<7 false

dangling tuple
- failed to match
with other relation.

Q. how to check dangling tuple step by step?

$(R \bowtie_{R.B < S.D} S)$

R(ABC)

A	B	C
1	3	5
4	6	7
7	9	11

S(DEFG)

D	E	F	G
4	6	7	8
7	7	8	9

		R.A	R.B	R.C	S.D	S.E	S.F	S.G
3<4	true	1	3	5	4	6	7	8
3<7	true	1	3	5	7	7	8	9
6<4	false	4	6	7	4	6	7	8
6<7	true	4	6	7	7	7	8	9
9<4	false	7	9	11	4	6	7	8
9<7	false	7	9	11	7	7	8	9

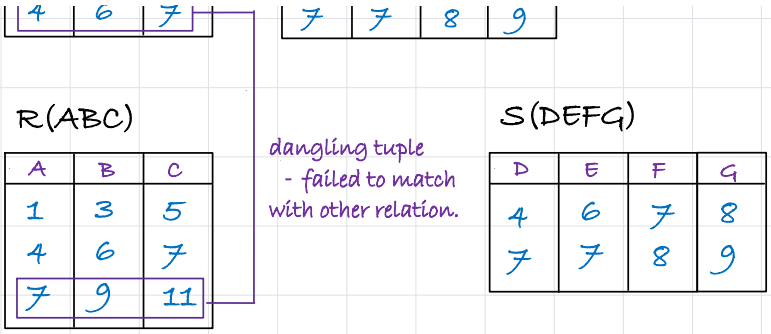
output :

R.A	R.B	R.C	S.D	S.E	S.F	S.G
1	3	5	4	6	7	8
1	3	5	4	7	8	9
4	6	7	7	7	8	9

output :

A	B	C
1	3	5
1	3	5
4	6	7

D	E	F	G
4	6	7	8
4	7	8	9
7	7	8	9



original:

topic : outer join

(i) Left outer join ($\bowtie$) (ulman($\dot{\bowtie}_L$))
(ii) Right outer join ($\bowtie$) (ulman($\dot{\bowtie}_R$))
(iii) Full outer join ($\bowtie$) ≡ outer join ($\bowtie$) (ulman($\bowtie$))

why we need outer join?
because, in inner join some tuple failed to satisfy the join condition (dangling tuples) that caused data loss (not receiving complete table data).

by default,

outer join = natural join + dangling tuple

outer join = inner join + dangling tuple

(i) Left outer join ($R \bowtie S$) (ulman($R \dot{\bowtie}_L S$))

($R \bowtie S$) or ($R \dot{\bowtie}_L S$) = $R \bowtie S$ and includes the tuple from left side relation R, those tuples who failed to satisfy join condition (dangling tuples)

($R \bowtie S$) or ($R \dot{\bowtie}_L S$) = $R \bowtie S$ + includes dangling tuple of left side relation R.

(i.1) Left conditional outer join ($R ?_c S$) (ulman($R(\dot{\bowtie}_L)_c S$))

($R ?_c S$) or ($R(\dot{\bowtie}_L)_c S$) = $R \bowtie_c S$ and includes the tuple from left side relation R, those tuples who failed to satisfy join condition

$(R ⟕_c S)$ or $(R(⟕_L)_c S)$ = $R ⟕_c S$ + includes dangling tuple of left side relation R.

example :

R

A	B	C
1	2	4
3	2	6

S

B	C	D
2	4	8
2	7	4

$R ⋈ S$

A	B	C	D
1	2	4	8

Left outer join $(R ⟕ S)$:

A	B	C	D
1	2	4	8
3	2	6	n

n → null

(ii) right outer join $(R ⟖ S)$ (ulman $(R ⟖_R S)$)

$(R ⟖ S)$ or $(R ⟖_R S)$ = $R ⋈ S$ and includes the tuple from right side relation R, those tuples who failed to satisfy join condition

$(R ⟖ S)$ or $(R ⟖_R S)$ = $R ⋈ S$ + includes dangling tuple of right side relation R.

(ii.1) right conditional outer join $(R ⟖_c S)$ (ulman $(R(⟖_R)_c S)$)

$(R ⟖_c S)$ or $(R(⟖_R)_c S)$ = $R ⋈_c S$ and includes the tuple from right side relation R, those tuples who failed to satisfy join condition

$(R ⟖_c S)$ or = $R ⋈_c S$ + includes dangling tuple of right side relation R.

$(R \bowtie_c S)$ or $(R(\dot{\bowtie}_R)_c S)$ = $R \bowtie_c S$ + includes dangling tuple of right side relation R.

example :

R

A	B	C
1	2	4
3	2	6

S

B	C	D
2	4	8
2	7	4

$R \bowtie S$

A	B	C	D
1	2	4	8

right outer join $(R \bowtie S)$:

A	B	C	D
1	2	4	8
n	2	7	4

null → n

(iii) full outer join $(R \bowtie S)$ (ulman $(R \dot{\bowtie}_R S)$)

$(R \bowtie S)$ or $(R \dot{\bowtie} S)$ = $R \bowtie S \cup R \bowtie S$

$(R \bowtie S)$ or $(R \dot{\bowtie} S)$ = $R \bowtie S$ + includes dangling tuple of R and S.

(iii.1) full conditional outer join $(R \bowtie_c S)$ (ulman $(R(\dot{\bowtie})_c S)$)

$(R \bowtie_c S)$ or $(R(\dot{\bowtie})_c S)$ = $R \bowtie_c S \cup R \bowtie_c S$

$(R \bowtie_c S)$ or = $R \bowtie_c S$ + includes dangling tuple of R and S.

$(R \bowtie_c S)$

or $(R(\bowtie)_c S)$ = $R \bowtie_c S$ + includes dangling tuple of R and S.

example :

R

A	B	C
1	2	4
3	2	6

S

B	C	D
2	4	8
2	7	4

$R \bowtie S$

A	B	C	D
1	2	4	8

full outer join $(R \bowtie S)$:

A	B	C	D
1	2	4	8
3	2	6	n
n	2	7	4

null (for the n in A) — null (for the n in D)

Q. Let R and S be two relations with the following schema

R(P, Q, R1, R2, R3)

S(P, Q, S1, S2)

Where {P, Q} is the key for both schemas. Which of the following queries are equivalent?

I. $\pi_P (R \bowtie S)$

II. $\pi_P(R) \bowtie \pi_P(S)$

III. $\pi_P(\pi_{P,Q} (R) \cap \pi_{P,Q}(S))$

IV. $\pi_P(\pi_{P,Q}(R) - (\pi_{P,Q} (R) - \pi_{P,Q} (S)))$

A Only I and II

B Only I and III

C Only I, II and III

D Only I, III and IV

R

S

R

P	Q	R_1	R_2	R_3
1	2	-	-	-
3	4	-	-	-
5	6	-	-	-
7	8	-	-	-

S

P	Q	S_1	S_2
1	2	-	-
3	4	-	-
5	6	-	-
7	8	-	-

$\pi_{attribute}[\sigma_{R.P = S.P \wedge R.Q = S.Q}(R \times S)]$

$R \bowtie S =$

P	Q
1	2
5	6

I. $\pi_P(R \bowtie S)$ $\pi_P =$

P
1
5

II. $\pi_P(R) \bowtie \pi_P(S)$ $\pi_P =$ (R)

P
1
3
5
7

$\pi_P =$ (S)

P
1
3
5
7

1
3
5
7

III. $\pi_P(\pi_{P,Q}(R) \cap \pi_{P,Q}(S))$

1. $(\pi_{P,Q}(R) \cap \pi_{P,Q}(S))$

P	Q
1	2
5	6

$\cap$

P	Q
1	2
5	6

P	Q
1	2
5	6

$\pi_P =$

P
1
5

IV. $\pi_P(\pi_{P,Q}(R) - (\pi_{P,Q}(R) - \pi_{P,Q}(S)))$

1. $(\pi_{P,Q}(R) - \pi_{P,Q}(S))$

P	Q
1	2
5	6

$-$

P	Q
1	2
5	6

$= 0$

2. $(\pi_{P,Q}(R) - (\pi_{P,Q}(R) - \pi_{P,Q}(S)))$

P	Q
1	2
5	6

$- 0 =$

P	Q
1	2
5	6

P	Q
1	2
5	6

$\pi_P =$

P
1
5

6.rename operator

STUDENT (sid, sname, age)

operations i can do with rename operator here :

(i) table rename

(ii) all attribute rename

(iii) particular attribute rename

(iv) table + attribute rename

(i) table rename

STUDENT (sid, sname, age) —table renaming→ temp

ρ (temp, STUDENT)

sid	sname	age

ab yeh table temp naam se hai rather than student

(ii) all attributes rename

STUDENT (sid, sname, age) —all attributes renaming→ STUDENT

$\rho_{(S, N, A)}$ student

S	N	A

ab yeh table ke saare attribute rename kardiye

(iii) particular attribute rename

STUDENT (sid, sname, age) —particular attribute renaming→ STUDENT

$\rho_{(1 \to s)\ (2 \to N)}$ student

S	N	Age

ab yeh table ke particular attribute rename kardiye

7.assignment operator [⟶]

temp ← any result

Consider the following table named Student in a relational database. The primary key of this table is rollNum.

Student

Roll Num	Name	Gender	Marks
1	Naman	M	62
2	Aliya	F	70
3	Aliya	F	80
4	James	M	82
5	Swati	F	65

The SQL query below is executed on this database.

```
SELECT *
FROM Student
WHERE gender = 'F' AND marks > 65;
```

The number of rows returned by the query is

8.divison operator [/] : it is used to retrieve attributes value of R which has paired with every attribute of other relation S.

(derived operator)

$$\frac{\pi_{AB}(R)}{\pi_{B}(S)} = \text{Quotient } (A);$$

getting those value of 'A' which pair with every value of B of S in relation R.

course — enrolled

it will retrieve values of attribute 'A' from R for which there must be pairing 'B' value for every 'B' of S.

Enrolled

Sid	Cid
S_1	C_1
S_1	C_2
S_1	C_3
S_2	C_1
S_2	C_3
S_3	C_1

Course

Cid
C_1
C_2
C_3

Q. write a query to find sid who enrolled any course (or) some course (or) atleast one course?

π_{sid} (enrolled) output: S_1

S_2

S_3

Q. write a query to find sid who enrolled every course?

$\pi_{sid,cid}$ (enrolled) / π_{cid} (course) output: S_1

jo condition lagate hai vo dusra attribute

jo cheez chaiye hoti hai vo pehla attribute

$\pi_{sid,cid}$ (enrolled) / π_{cid} (course) = getting sid who enrolled every course

$\pi_{sid,cid}$ (enrolled) / π_{cid} (course)

doosra attribute same hona chaiye

step 1 : i assumed that every student enrolled every course

π_{sid} (enrolled) × π_{cid} (course)

sid
S_1
S_2
S_3

×

cid
C_1
C_2
C_3

sid	cid
S_1	C_1
S_1	C_2
S_1	C_3
S_2	C_1
S_2	C_2
S_2	C_3
S_3	C_1
S_3	C_2
S_3	C_3

universal result : every student enrolled every course.

step 2 : subract the actual enrolled (attendance sheet) students

π_{sid}(enrolled) X π_{cid} (course) - enrolled = didnt enroll

all enrolled

sid	cid
S_1	C_1
S_1	C_2
S_1	C_3
S_2	C_1
S_2	C_2
S_2	C_3
S_3	C_1
S_3	C_2
S_3	C_3

-

actually enrolled

sid	cid
S_1	C_1
S_1	C_2
S_1	C_3
S_2	C_1
S_2	C_3
S_3	C_1

=

so so students didnt enrolled into so so courses

sid	cid
S_2	C_2
S_3	C_2
S_3	C_3

Q. write a query to find STUDENT who attend every class?

π_{sid}(enrolled) - π_{sid}[π_{sid}(enrolled) X π_{cid} (course) - enrolled]

sid	cid
S_2	C_2
S_3	C_2
S_3	C_3

- π_{sid}[π_{sid}(enrolled) X π_{cid} (course) - enrolled]

sid
S_2
S_3

- π_{sid}(enrolled) - π_{sid}[π_{sid}(enrolled) X π_{cid} (course) - enrolled]

sid
S_1
S_2
S_3

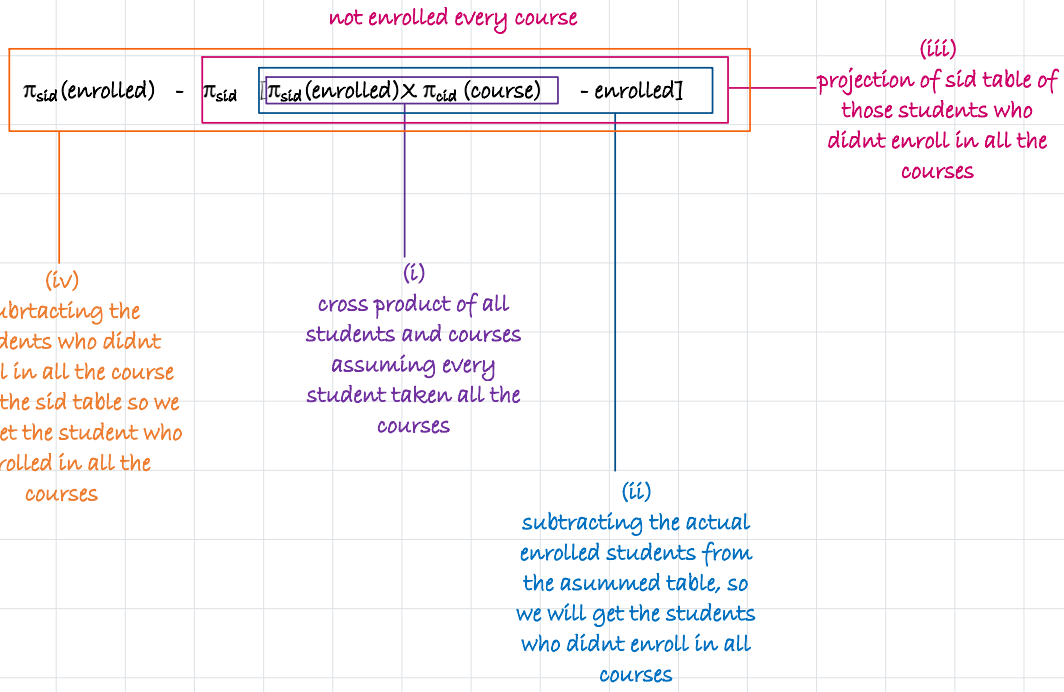
=

sid
S_2
S_3

=

sid
S_1

student who enrolled every course



in terms of A and B:

$$\frac{\pi_{AB}(R)}{\pi_{B}(S)} = \pi_{A}(R) - \pi_{A}[\pi_{A}(R) \times \pi_{B}(S) - R]$$

$$\frac{\pi_{AB}(R)}{\pi_{B}(S)} = \pi_{A}(R) - \pi_{A}[\pi_{A}(R) \times S - R]$$

we can do this for 2 variable and 2 condition

$$\frac{\pi_{ABCD}(R)}{\pi_{CD}(S)} = \pi_{AB} - \pi_{AB}[\pi_{AB}(R) \times \pi_{CD}(s) - R]$$

another variation :

$$\frac{\pi_{AB}(R)}{\pi_{A}(S)} = \text{Quotient (B)};$$ getting those value of 'B' which pair with every value of A of S in relation R.

course — enrolled

$$\frac{\pi_{AB}(R)}{\pi_{A}(S)} = \pi_{B}(R) - \pi_{B}[\pi_{B}(R) \times \pi_{A}(S) - R]$$

Consider the two relation Suppliers and Parts are given below.

Suppliers

S_{no}	P_{no}
S_1	P_1
S_1	P_2
S_1	P_3
S_1	P_4
S_2	P_1
S_2	P_2
S_3	P_2
S_4	P_2
S_4	P_4

Parts

P_{no}
P_2
P_4

$\pi_{S_{no}, P_{no}}$ (Suppliers) / $\pi_{P_{no}}$ (Parts)

The number of tuples are there in the result when the above relational algebra query executes is _____.

$$\frac{\pi_{sno,\ pno}(S)}{\pi_{pno}(P)} = \pi_{pno}(P) - \pi_{pno}[\pi_{pno}(P) \times \pi_{sno}(S) - P]$$

another way to solve numerical step by step :

step 1 : dividing supplier by P_2

Suppliers

S_{no}	P_{no}
S_1	P_1
S_1	P_2
S_1	P_3
S_1	P_4
S_2	P_1
S_2	P_2
S_3	P_2
S_4	P_2
S_4	P_4

S_1
S_2
S_3
S_4

jahan jahan P_2 hai woh likhne hai

step 2 : dividing supplier by P_4

Suppliers

S_{no}	P_{no}
S_1	P_1
S_1	P_2
S_1	P_3
S_1	P_4
S_2	P_1
S_2	P_2
S_3	P_2
S_4	P_2
S_4	P_4

P_2 ko consider karte hue krna hai

S_1
S_2
S_3
S_4

S_1
S_4

jahan jahan P_4 hai woh likhne hai

Q. Retrieve all student who are Enrolled every course?

Enrolled | Course

Q. Retrieve all student who are Enrolled every course?

Enrolled	
Sid	Cid
S_1	C_1
S_1	C_2
S_1	C_3
S_2	C_1
S_2	C_3
S_3	C_1

Course
Cid
C_1
C_2
C_3

Solution

$\Pi_{Sid,Cid}$ (Enrolled) / Π_{Cid} (Course)

Find

2nd attribute must be same.

step 1 : dividing enrolled by C_1

S_1	C_1
S_1	C_2
S_1	C_3
S_2	C_1
S_2	C_3
S_3	C_1

S_1
S_2
S_3

jahan jahan C_1 hai woh likhne hai

step 2 : dividing enrolled by C_1C_2

C_1 ko consider karte hue krna hai

S_1	C_1
S_1	C_2
S_1	C_3
S_2	C_1
S_2	C_3
S_3	C_1

S_1
S_2
S_3

S_1

jahan jahan C_2 hai woh likhne hai

step 3 : dividing enrolled by $C_1C_2C_3$

C_3 ko consider karte hue krna hai

S_1	C_1
S_1	C_2
S_1	C_3
S_2	C_1
S_2	C_3
S_3	C_1

S_1 S_1

jahan jahan C_3 hai woh likhne hai

NAT

Consider a database that has the relation schema CR(StudentName, CourseName). An instance of the schema CR is as given below:

The following query is made on the database.

$T1 \leftarrow \pi_{CourseName}(\sigma_{StudentName='SA'}(CR))$

$T2 \leftarrow CR \div T1;$

CR	
Student Name	Course Name
SA	CA
SA	CB
SA	CC
SB	CB
SB	CC

Student Name	Course Name
SD	CC
SD	CD
SE	CD
SE	CA
SE	CB
SF	CA

NAT

Consider a database that has the relation schema CR(StudentName, CourseName). An instance of the schema CR is as given below:

The following query is made on the database.

$T1 \leftarrow \pi_{CourseName}(\sigma_{StudentName='SA'}(CR))$

$T2 \leftarrow CR \div T1;$

The number of rows in T2 is _____.

[GATE-2017-CS: 2M]

CR	
Student Name	Course Name
SA	CA
SA	CB
SA	CC
SB	CB
SB	CC
SC	CA
SC	CB
SC	CC
SD	CA
SD	CB

Student Name	Course Name
SD	CC
SD	CD
SE	CD
SE	CA
SE	CB
SF	CA
SF	CB
SF	CC

step 1 : dividing CR by CA

CR	
Student Name	Course Name
SA	CA
SA	CB
SA	CC
SB	CB
SB	CC
SC	CA
SC	CB
SC	CC
SD	CA
SD	CB

Student Name	Course Name
SD	CC
SD	CD
SE	CD
SE	CA
SE	CB
SF	CA
SF	CB
SF	CC

SA
SC
SD
SE
SF

jahan jahan CA hai woh likhne hai

step 2 : dividing CR by CA CB

CA ko consider karte hue krna hai

CR	
Student Name	Course Name
SA	CA
SA	CB
SA	CC
SB	CB
SB	CC
SC	CA
SC	CB
SC	CC
SD	CA
SD	CB

Student Name	Course Name
SD	CC
SD	CD
SE	CD
SE	CA
SE	CB
SF	CA
SF	CB
SF	CC

SA
SC
SD
SE
SF

SA
SC
SD
SE
SF

jahan jahan CB hai woh likhne hai

step 3 : dividing CR by CA CB CC

CA CB ko consider karte hue krna hai

CR	
Student Name	Course Name
SA	CA
SA	CB
SA	CC
SB	CB
SB	CC
SC	CA
SC	CB
SC	CC
SD	CA
SD	CB

Student Name	Course Name
SD	CC
SD	CD
SE	CD
SE	CA
SE	CB
SF	CA
SF	CB
SF	CC

SA
SC
SD
SE
SF

SA
SC
SD
SF

jahan jahan CC hai woh likhne hai

step 4: dividing CR by CA CB CC CD (additionally given by sir)

CA CB CC ko consider karte hue krna hai

CR	
Student Name	Course Name
SA	CA
SA	CB
SA	CC

Student Name	Course Name
SD	CC
SD	CD
SE	CD

SA
SC

kaafi kuch likha hai

CR

Student Name	Course Name
SA	CA
SA	CB
SA	CC
SB	CB
SB	CC
SC	CA
SC	CB
SC	CC
SD	CA
SD	CB

Student Name	Course Name
SD	CC
SD	CD
SE	CD
SE	CA
SE	CB
SF	CA
SF	CB
SF	CC

SA
SC
SD
SF

SD

jahan jahan CD hai woh likhne hai

Q. Consider the following three relations in a relational database:

Employee (eId, Name), Brand (bId, bName), Own(eId, bId)

Which of the following relational algebra expressions return the set of eIds who own all the brands? [GATE: 2022]

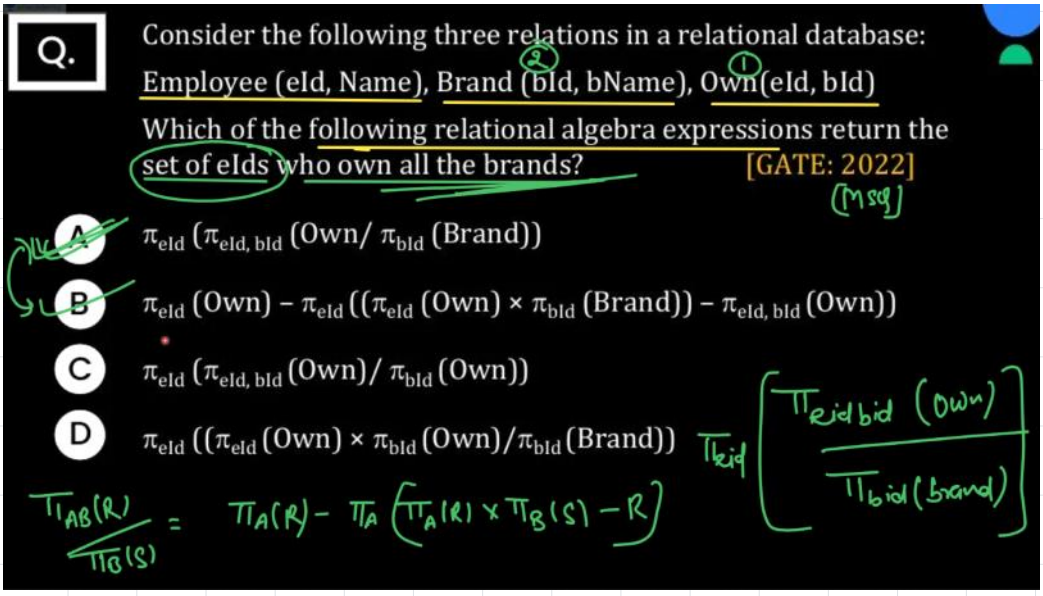
A $\pi_{eId}(\pi_{eId, bId}(Own / \pi_{bId}(Brand))$

B $\pi_{eId}(Own) - \pi_{eId}((\pi_{eId}(Own) \times \pi_{bId}(Brand)) - \pi_{eId, bId}(Own))$

C $\pi_{eId}(\pi_{eId, bId}(Own) / \pi_{bId}(Own))$

D $\pi_{eId}((\pi_{eId}(Own) \times \pi_{bId}(Own) / \pi_{bId}(Brand))$

Q. Consider the following three relations in a relational database:



q. how to write queries?

consider the database with relations :

S supplier (sid, sname, rating)
P parts (pid, pname, color)
C catalog (sid pid, cost)

q.1 find the sid of supplier whose rating is greater than 9?

$\pi_{sid}[\sigma_{rating > 9}(supplier)]$

q.2 find the pid of red color parts?

$\pi_{pid}[\sigma_{color = red}(parts)]$

q.3 retrieve the sid of supplier whose cost is greater than 20,000?

$\pi_{sid}[\sigma_{cost > 20,000}(catalog)]$

q.4 retrieve the sid of supplier who supplied some red color parts?

$\pi_{sid}[\sigma_{p.pid = c.pid \wedge p.color = red}(catalog \times parts)]$

sid of red color parts ?

(i) why supplier and parts? x

color in parts table but no common attribute between supplier and parts table so supplier and parts table not required.

(ii) why supplier and catalog? x

color attribute present in parts table so part table is mandatory

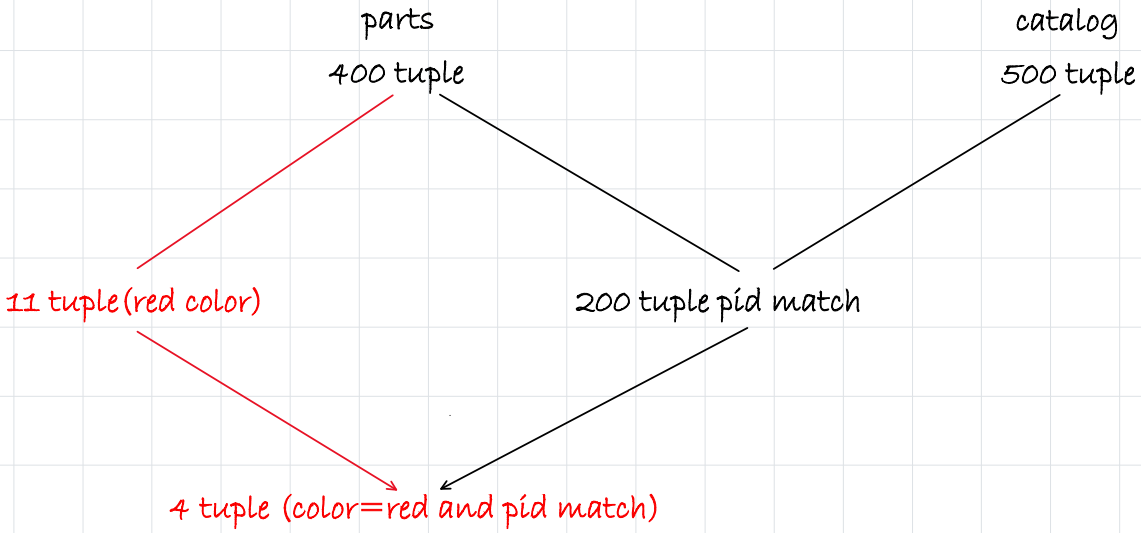
(iii) why parts and catalog?

color red applied on parts table then getting pid, that pid match with catalog pid so we fetch (retrive) sid of red color parts. so parts and catalog table required.

q.4 retrieve the sname of the supplier who supplied red color parts?

$\pi_{sname}[\sigma_{p.pid = c.pid \wedge c.sid = s.sid \wedge p.color = red}(supplier \times catalog \times parts)]$

how to write optimise query and analysis of queries.



output : 4

q. retrieve the sid of supplier who supplied some red color parts?

query (i): $\pi_{sid}[\sigma_{p.pid = c.pid \wedge p.color = red}(catalog \times parts)]$

query (ii): $\pi_{sid}[\sigma_{p.pid = c.pid}(\sigma_{color = red}(parts) \times catalog]$

query (ii): $\pi_{sid}[\sigma_{p.pid = c.pid}(\sigma_{color = red}(parts) \times catalog]$

query (iii): $\pi_{sid}[\sigma_{p.color = red}(parts \bowtie catalog)]$

query (iv): $\pi_{sid}[\sigma_{p.color = red} \bowtie catalog)]$

all these four queries will give the same answer but
which one is the most efficient/optimise query and why?

query (i): $\pi_{sid}[\sigma_{p.pid = c.pid \wedge p.color = red}(catalog \times parts)]$

400 x 500
2lakh.

query (ii): $\pi_{sid}[\sigma_{p.pid = c.pid}(\sigma_{color = red}(parts) \times catalog]$

11 x 500
5500

query (iii): $\pi_{sid}[\sigma_{p.color = red}(parts \bowtie catalog)]$

200

query (iv): $\pi_{sid}[\sigma_{p.color = red} \bowtie catalog)]$

11 ⋈ ___

so, query (iv) is the most optimised query.

note : most efficient query

(i) let an attribute A belongs to R only then

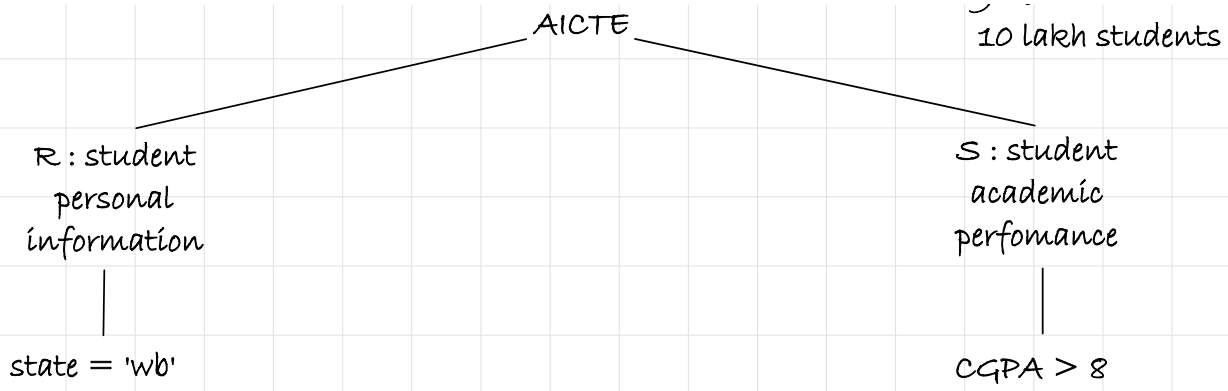
$\sigma_{'A' = 'a'}(R) \bowtie S$

(ii) let an attribute A belongs to R only and attribute B belongs to S only then

$\sigma_{'A' = 'a'}(R) \bowtie \sigma_{'B' = 'b'}(S)$

25 state
9 UT
10 lakh students

AICTE



$\pi_{name}[\sigma_{state = 'wb'} \wedge_{CGPA > 8}(R \bowtie S)]$ or $\pi_{name}[\sigma_{state = 'wb'}(R) \wedge \sigma_{CGPA > 8}(S)]$

25 state
9 ut

CGPA
0 to 8, 9 10

0-8 not included.

Q. Consider the following relation schemas:

b-Schema = (b-name, b-city, assets)

a-Schema = (a-num, b-name, bal)

d-Schema = (c-name, a-number)

Let branch, account and depositor be respectively instances of the above schemas. Assume that account and depositor relations are much bigger than the branch relation.

Consider the following query:

$\Pi_{c\text{-name}}(\sigma_{b\text{-city} = \text{"agra"} \wedge bal < 0}(\text{branch} \bowtie \text{account} \bowtie \text{depositor}))$

Which one of the following queries is the most efficient version of the above query? [GATE-2007: 2 Marks]

A. $\Pi_{c\text{-name}}(\sigma_{bal < 0}(\sigma_{b\text{-city} = \text{"Agra"}}\ \text{branch} \bowtie \text{account}) \bowtie \text{depositor})$

B. $\Pi_{c\text{-name}}(\sigma_{b\text{-city} = \text{"Agra"}}\ \text{branch} \bowtie (\sigma_{bal < 0}\ \text{account}) \bowtie \text{depositor})$

C. $\Pi_{c\text{-name}}(\sigma_{b\text{-city} = \text{"Agra"}}\ \text{branch} \bowtie \sigma_{b\text{-city} = \text{"Agra"} \wedge bal < 0}\ \text{account} \bowtie \text{depositor})$

D. $\Pi_{c\text{-name}}(\sigma_{b\text{-city} = \text{"Agra"}}\ \text{branch} \bowtie (\sigma_{b\text{-city} = \text{"Agra"} \wedge bal < 0}\ \text{account} \bowtie \text{depositor}))$

structured query language

(i) DDL (data definition language) : modification allowed at schema (definition) level

- create
- alter
- drop table

(ii) DML (data manipulation language) : modification allowed at data level

- insert
- update
- delete

(iii) DCL (data control language) : control transactional operation.

- commit
- abort

(iv) DQL (data query language) : used to retrieve the data from DB

- select
- from
- where

optional

SQL		RA
SELECT [distinct] $A_1, A_2, A_3 \ldots A_n$	≡	projection
FROM $R_1, R_2, R_3 \ldots R_m$	≡	cross product
WHERE condition (predicate)	≡	selection

relational algebra : $\pi_{A1,A2,A3.....An}[\sigma_{condition}(R_1 \times R_2 \times R_3 .... \times R_n)]$

a. difference between SQL and RA

(i) relational algebra is a procedural query language and SQL is a non procedural query language

(ii) relational algebra by default eliminate the duplicates but in SQL duplicate retains

b. is SQL case sensitive (keywords)

(i) SQL is not case sensitive
SELECT, SELECT, SELECT..

but strings are case sensitive

branch : IT

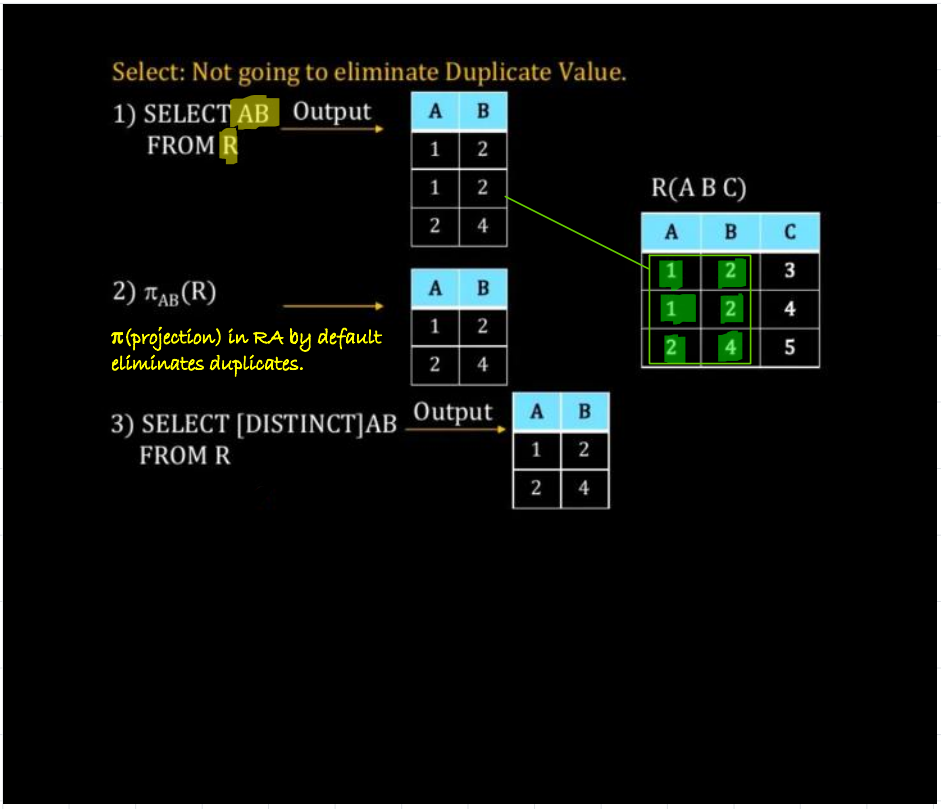
Branch : 'IT'
Branch : 'it' x

c. difference between SELECT (in SQL) and π in RA

(i) SELECT in SQL retain duplicates
but π(projection) in RA by default eliminates duplicates.

d. how many mandatory clauses in SQL?

(a) 1
(b) 2 select, from
(c) 3
(d) 4



q. SQL is non procedural, duplicated retain and keyword is not case sensitive then still why SQL is most widely used practically?

relational algebra : works on 'set theory'
SQL : works on multi set [bag] (duplicates allowed)

set : [1,2,3,4]
multi set : [1,1,1,2,2,2,3,3,3,3,3,4,4,4]

commercial database works on multi set (bag) because ,

- if we want to perform union of two table (relation) then SQL just simply takes one relation and simply add the data of another relation (its fastest) but in relational algebra first we eliminate the duplicates then perfoms the operations (very high time consuming)

- in RA duplicates remove so we cannot find average marks but in SQL average marks : 9+10+9+10+9+8/6=48.3333
removing duplicates is very expensive.

sid	marks
S_1	9
S_2	10
S_3	9
S_4	10
S_5	9
S_6	8

- in SQL aggregate function (sum,count,min,max,average) are allowed but they are not allowed in RA

- SQL have lost of features that are not supported by RA.

S supplier (sid, sname, rating)
P parts (pid, pname, color)
C catalog (sid pid, cost)

topic : rename operator

keywords : 'AS' '_' (space)

from supplier AS S, parts _ P

q.1 find the sid of red color parts?

RA : $\pi_{sid}[\sigma_{p.pid = c.pid \wedge p.color = red}(catalog \times parts)]$

```
SQL :  SELECT   sid
       FROM     catalog   parts
       WHERE    catalog.pid = parts.pid
       AND      parts.color = 'red'
```

now using rename operator.

```
SQL :  SELECT   sid
       FROM     catalog AS C,   parts as P
       WHERE    C.pid = P.pid
       AND      P.color = 'red'
```

q.2 find the sname of red color parts?

RA : $\pi_{sname}[\sigma_{p.pid = c.pid \wedge c.sid = s.sid \wedge color = red}(supplier \times catalog \times parts)]$

RA : $\pi_{sname}[\sigma_{p.pid = c.pid \wedge c.sid = s.sid \wedge color = red}(supplier \times catalog \times parts)]$

now using rename operator.

SQL :
```
SELECT  sname
FROM    catalog AS C,  parts as P,  suplier as S
WHERE   C.pid = P.pid ∧
        C.sid = S.pid ∧
        P.color = 'red'
```

q. why alisaing/renaming is used?

roll no.	game id.
1	G_1
1	G_2
1	G_3
2	G_1
3	G_2

(i) find the student who participate at-least one game/some game/any game.

$\pi_{roll\ no.}(game)$

roll no.
1
2
3

(ii) find the student who participate at-least two game/some game/any game.

$\rho_{(t1,\ game)}$ $\rho_{(t2,game)}$

$\pi_{t1.roll\ no.}(\sigma_{t1.roll\ no. = t2.roll\ no. \wedge\ t1.game\ id \neq t2.game\ id}(t_1 \times t_2))$

(iii) find the student who participate at-least three game/some game/any game.

$\rho_{(t1,\ game)}$ $\rho_{(t2,game)}$ $\rho_{(t3,game)}$

$\pi_{t1.roll\ no.}(\sigma_{t1.roll\ no. = t2.roll\ no. = t3.roll\ no. \wedge\ t1.game\ id \neq t2.game\ id \wedge t2.game\ id \neq t3.game\ id \wedge t3.game\ id \neq t1.game\ id}(t_1 \times t_2 \times t_3))$

important points :

(i) this SELECT will give sname column

```
SELECT sname
FROM supplier
```

output : sname

(ii) this SELECT * will give all attribute of that relational schema

```
SELECT* sname
FROM supplier
```

output : sid sname rating

*: all attribute of that particular schema in the output

```
SELECT*
FROM parts
WHERE color = 'red'
```

output : pid pname color
those tuples having red color

topic - aggregate function

aggregate function are the function that takes collection of values (set (or) multi set) as a input and produce the single output.

(i) COUNT
(ii) SUM
(iii) AVG
(iv) MIN
(v) MAX

important points about aggregate function:

(i) aggregate operator always discard the null values.

null value :
- unknown value

- the value that does not exist
- two null values are not equal
- random ASCII charatcer assigned by the DBMS software.
- not zero.

(ii) arithmatic operation with NULL gives results as null.
null (+) any value = null (output)

(iii) comparison operation with NULL will give unknown as a result
comparision operator :=, <, >, <>,etc

0<null : unknown
5<null : unknown
5<>null : unknown

(iv) while performing the aggregate operation then first discard/ignore/remove the NULL and then perform the calculation (or) computation.

q.

sid	(branch)	marks
S_1	CS	90
S_2	IT	70
S_3	CS	70
S_4	EC	56
S_5	CS	null

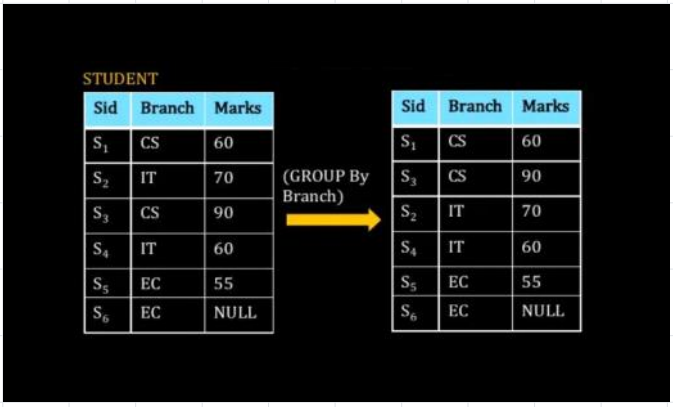
(i) count (marks) : 4
(ii) count (*) : 5 (no.of tuples)
(iii) count ([DISTINCT]marks): 3
(iv)SUM(marks): 286
(v)SUM([DISTINCT]marks): 216
(vi)AVG(marks): 71.5
(vii)AVG([DISTINCT]marks): 72
(viii)count ([DISTINCT]*) : not allowed
(ix)count (*[DISTINCT]) : not allowed

SQL clauses :

(i) FROM : it is the first executble clause. it is simply relation (or) cross product of two or more relation.

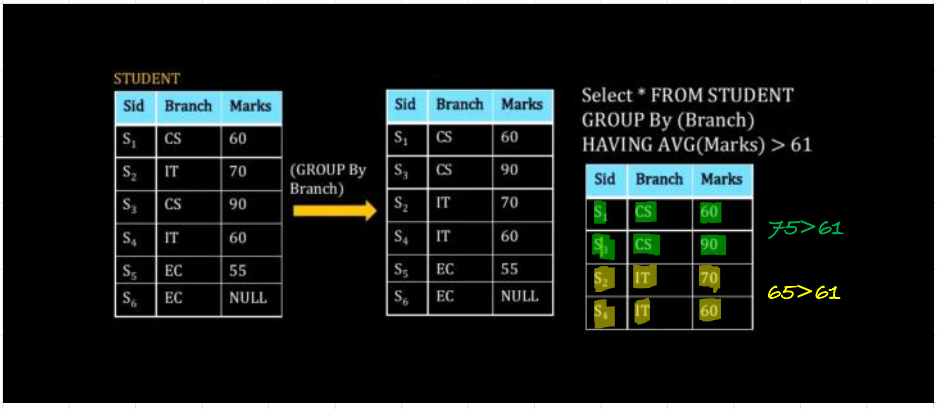
(ii) WHERE : it is the second executable clause. it selects the tuple based on specified condition.

(iii) GROUP BY : it is the third executable clause if used in the query. it groups the table based on the specified attributes.



(iv) HAVING : it is the fourth executable clause. it is used to select the group which satisfy the condition.

- latest DBMS software : HAVING can be used without GROUP BY clause.
- SQL standard (1992) [followed by GATE examiner] : HAVING always used with GROUP BY clause.



note : having clause always proceed by 'GROUP BY' clause.

q. SELECT min(marks), Branch
FROM student

(syntax error)

sid	(branch)	marks
S_1	CS	60
S_2	IT	70
S_3	CS	90
S_4	IT	60
S_5	EC	55
S_6	EC	null

marks	(branch)
55	CS
	IT
	CS
	IT
	EC
	EC

not allowed!

note :
when aggregrate operator and other attribute [normal (or) unaggregrate attribute] used in SELECT clause, it is only allowed only when other attribute [normal (or) unaggregrate attribute] is in GROUP BY clause.

```
SELECT min(marks), Branch
FROM student
GROUP BY (Branch)
```

q. SELECT min(marks), Branch
FROM student
GROUP BY (Branch)

sid	(branch)	marks
S_1	CS	60
S_2	IT	70
S_3	CS	90
S_4	IT	60
S_5	EC	55
S_6	EC	null

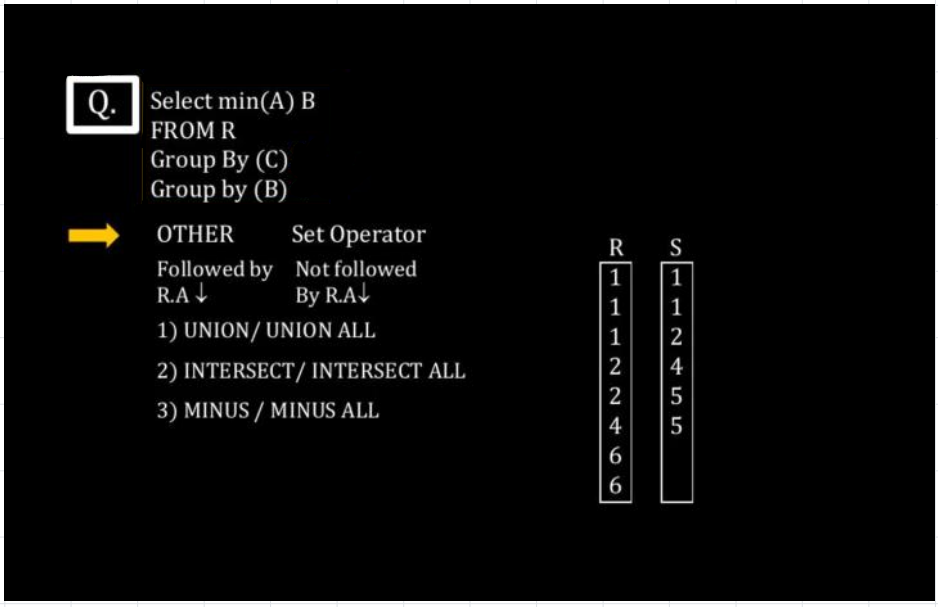
marks	(branch)
60	CS
60	IT
55	EC

(v) SELECT, (vi)DISTINCT, (vii)ORDER BY

topic : SQL set operators

(i) union and union all
(ii) intersect and intersect all
(iii) minus (except) and minus all (except all) [set difference]

followed by relational algebra not followed by relational algebra



- R union S :
  only distinct tuple - 1,2,4,5,6

- R union all S :
  complete R and complete S + duplicates - 1,1,1,2,2,4,6,6,1,1,2,4,5,5

- R intersect S :
  distinct common tuple from R and S - 1,2,4

- R intersect all S :
  how many maximum number of time common in both R and S - 1,1,2,4

- R minus (except) S :
  getting those values which are present in 'R' but not in 'S' - 6

- R minus all (except all) S :
  number of duplicates in R - number of duplicates in S - 1,2,6,6

other set operator

(i) IN/NOT IN
(ii) ANY
(iii) ALL
(iv) EXISTS/NOT EXISTS

(ii) ANY (OR) : compare a value with each value in a set and return true if any value is given condition satisfied for every/each value in the set.

example :
(a) X>ANY (11,21,51)
  (X>11) (or) (X>21) (or) (X>51)
  output : 12,13.....

(b) X<ANY (11,21,51)
  (X<11) (or) (X<21) (or) (X<51)
  output : 50,49,48.....

(iii) ALL (AND) : compare a value with each value in a set and return true if any value is compared according to given condition.

example :
(a) X>ALL (11,21,51)
  (X>11) (and) (X>21) (and) (X>51)
  output : 52,53,54.....

(b) X<ALL (11,21,51)
  (X<11) (and) (X<21) (and) (X<51)
  output : 10,9,8.....

Q.1 Find Name of Supplier whose turnover is better than the turnover of any (some) Supplier of Delhi?

```
Select    Sname
From      Supplier
Where
```

```
Select      Sname
From        Supplier
Where
            turnover > ANY (Select turnover
                            From Supplier
                            Where City = Delhi)
```

Supplier:

Sno	Sname	City	turnover
1	A	Delhi	4 Cr.
2	B	Bang.	5 Cr.
3	C	Delhi	6 Cr.
4.	D	Konchi	7 Cr.

X> ANY (4,6)
(X>4) (or) (X>6)

output : 5,6,7

sname
B
C
D

but, C is the supplier of delhi itself.

correct query :

```
Select      Sname
From        Supplier
Where       City <> Delhi AND   (city is not equal to delhi.)
            turnover > ANY (Select turnover
                            From Supplier
                            Where City = Delhi)
```

X> ANY (4,6)
(X>4) (or) (X>6)

output : 5,6,7
but city <> 'delhi'

sname
B
D

Q. Retrieve eid who get more salary than any employee of dept = 5?

EMP

Eid	dno	Salary
E_1	5	50 k
E_2	3	20 k
E_3	5	30 k
E_4	3	40 k
E_5	4	60 k

query (i) : with ANY

```
SELECT  eid
FROM    emp
WHERE   dno <>5

AND  salary > ANY (SELECT salary
                   FROM emp
```

```
                        FROM  emp
                        WHERE  dno = 5)
```

query (ii) : without ANY

```
SELECT  eid
FROM    emp
WHERE   dno <>5

AND  salary > (SELECT min(salary)
               FROM  emp
               WHERE  dno = 5)
```

output :

E_4

E_5

query (i) : with ALL

```
SELECT  eid
FROM    emp
WHERE   dno <>5

AND  salary > ALL (SELECT salary
                   FROM  emp
                   WHERE  dno = 5)
```

query (ii) : without ALL

```
SELECT  eid
FROM    emp
WHERE   dno <>5

AND  salary > (SELECT max(salary)
               FROM  emp
               WHERE  dno = 5)
```

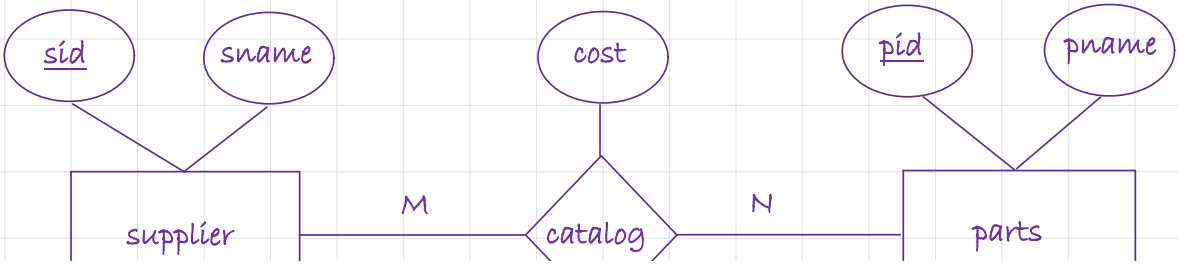
output :

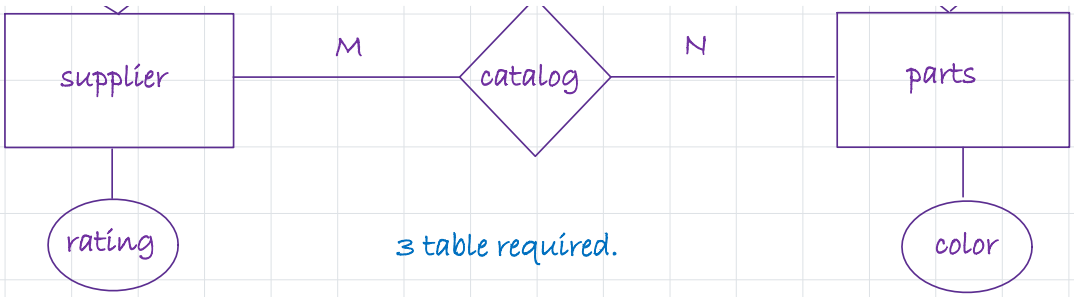
E_5

S supplier (sid, sname, rating)

P parts (pid, pname, color)

C catalog (sid pid, cost)





supplier

sid	sname	rating
S_1	snehil	9
S_2	pravin	10
S_3	abhi	9
S_4	ayushi	10
S_5	shivam	9

key : sid

catalog

sid	pid	cost
S_1	P_1	7k
S_1	P_2	8k
S_3	P_3	9k
S_4	P_4	10k

key : sid, pid

parts

pid	pname	color
P_1	A	red
P_2	B	green
P_3	C	red
P_4	D	yellow

key : pid

q(i) write a query to find sid of the supplier whose rating is greater than 9.

sol.(i)
```
SELECT   sid
FROM     supplier
WHERE    rating >9
```
output : S_4 and S_2

q(ii) write a query to find sname of the supplier whose rating is greater than 9.

sol.(ii)
```
SELECT   sname
FROM     supplier
WHERE    rating >9
```
output : pravin and ayushi

q(iii) retrieve sid whose parts costs greater than 7k.

sol.(iii)
```
SELECT   sid
FROM     catalog
WHERE    cost>7k
```
output : S_1, S_2 and S_4

q(iv) retrieve pid whose parts costs greater than 7k.

sol.(iv)
```
SELECT   pid
FROM     catalog
WHERE    cost>7k
```
output : P_2, P_3 and P_4

q(v) retrieve sname whose parts costs greater than 7k.

sol.(v) SELECT sname
FROM supplier S, catalog as P
WHERE S.Sid = C.sid AND C.cost>7k

output : snehil, pravin and ayushi

q(vi) retrieve pid of red color parts.

sol.(vi) SELECT pid
FROM parts
WHERE color = red

output : P_1 and P_3

q(vii) retrieve sid of supplier who supplied red color parts.

sol.(vii) SELECT sid
FROM catalog as C, Parts P
WHERE C.pid =P.pid AND P.color=red

output : S_1 and S_2

q(viii) retrieve sname of supplier who supplied red color parts.

sol.(viii) SELECT sname
FROM catalog as C, Parts P, supplier S
WHERE C.pid =P.pid AND C.sid = S.sid AND P.color=red

output : snehil and pravin

q(ix) retrieve sid of supplier who did not supplied red color parts.

sol.(ix) SELECT sid
FROM catalog as C, Parts P, supplier S
WHERE C.pid =P.pid AND C.sid = S.sid AND P.color<>red

output : S_1 and S_4

q(x) retrieve sname of supplier who did not supplied red color parts.

sol.(x) SELECT sname
FROM catalog as C, Parts P, supplier S
WHERE C.pid =P.pid AND C.sid = S.sid AND P.color<>red

output : snehil and ayushi

q(xi) retrieve sid of supplier who supplied red color parts.

sol.(xi) SELECT sid
FROM catalog as C, Parts P
WHERE C.pid =P.pid AND P.color=red

output : S_1 and S_2

note : directly one to many comparison is not allowed.

query (ii): (wrong query)

```
SELECT   sid
FROM   catalog
WHERE   pid =  (SELECT   Pid
                FROM   parts
                WHERE   color=red)
```

Pid : P_1 and P_3

note : query execute on DB table tuple by tuple (row by row) one tuple at a time

topic : IN and NOT IN operator

'IN' operator : like membership set

X in R : if R contains (belongs) to x then it is true.

q. R: [1,2,3,4,5]
X=4, is it available in R?
X IN R
4 IN [1,2,3,4,5] : true

q. S: [5,6,7,8,9]
X=4, is it available in R?
X IN R
4 IN [5,6,7,8,9] : false

'NOT IN' operator : like membership set

X NOT IN R : if R does not contains (does not belongs) to x then it is true.

q. R: [1,2,3,4,5]
X NOT IN R
4 NOT IN [1,2,3,4,5] : false

q. S: [5,6,7,8,9]
X NOT IN S
4 NOT IN [5,6,7,8,9] : true

query (ii):

```
SELECT   sid                SELECT   Pid
FROM   catalog     IN       FROM   parts
WHERE   pid                 WHERE   color=red
```

pid
P_1
P_3

query (ii): SELECT sid FROM catalog WHERE pid IN SELECT Pid FROM parts WHERE color=red

pid
P_1
P_3

P_1 IN [P_1P_3] : true
P_2 IN [P_1P_3] : false
P_3 IN [P_1P_3] : true
P_4 IN [P_1P_3] : false

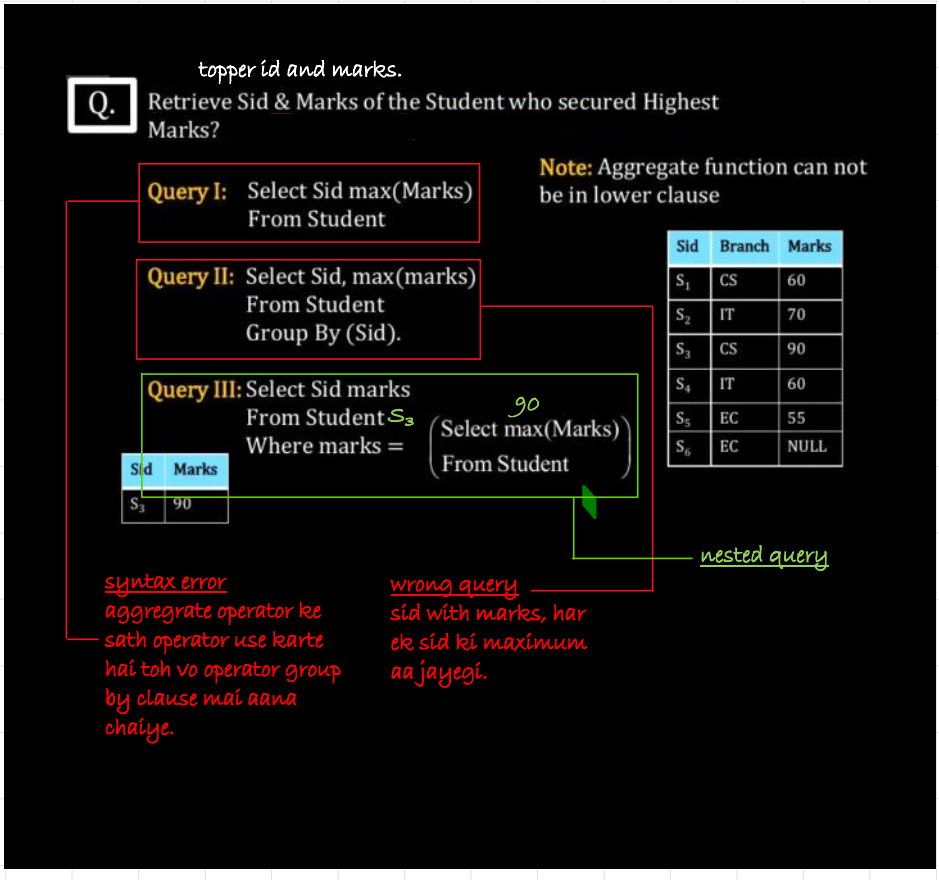
output : S_1 and S_2

query (ii): SELECT sid FROM catalog WHERE pid IN SELECT Pid FROM parts WHERE color<>red

pid
P_2
P_4

P_1 NOT IN [P_1P_3] : false
P_2 NOT IN [P_1P_3] : true
P_3 NOT IN [P_1P_3] : false
P_4 NOT IN [P_1P_3] : true

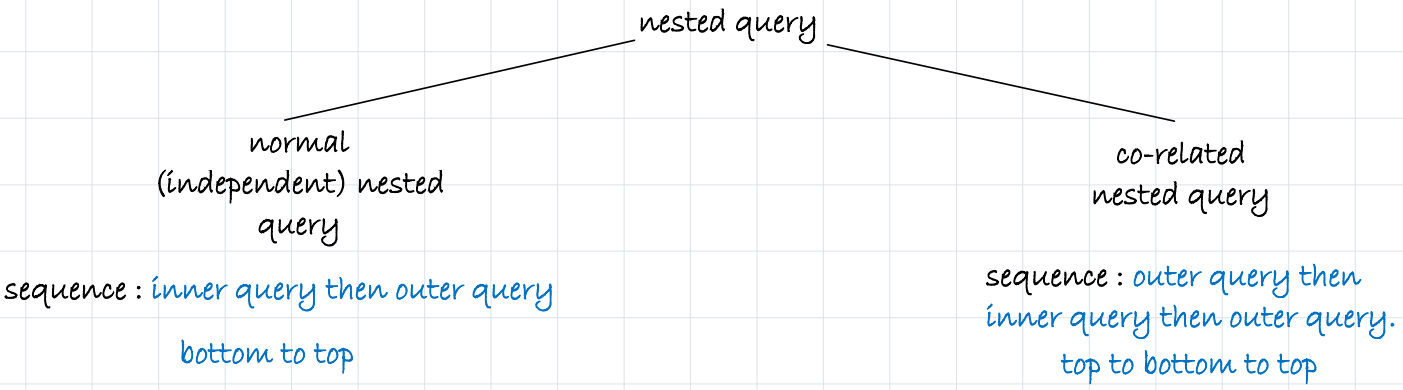
output : S_1 and S_4



topic - nested query

more than one query (query inside query)

example : (outer query (inner query (inner most query)))



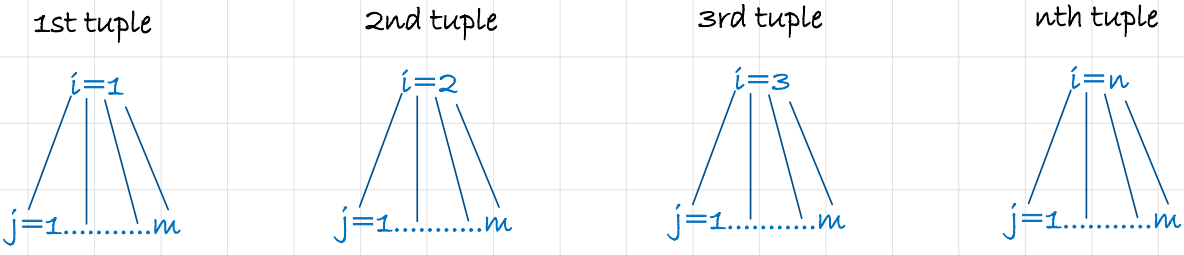
note : query executed on database table tuple by tuple (row by row) and one tuple at a time.

co-related nested query : inner query using attribute defined in outer query

sequence : outer query ⟶ inner query ⟶ outer query

top to bottom to top

```
for (i=1; i<=n; i++)
    for (j=1; j<=m; j++)
```



topic : exists and not exists

EXISTS : return true if inner query result is non-empty.

NOT EXISTS : return true if inner query result is empty.

supplier

sid	sname	rating
S_1	snehil	9
S_2	pravin	10
S_3	abhi	9
S_4	ayushi	10
S_5	shivam	9

key : sid

catalog

sid	pid	cost
S_1	P_1	7k
S_1	P_2	8k
S_3	P_3	9k
S_4	P_4	10k

key : sid, pid

parts

pid	pname	color
P_1	A	red
P_2	B	green
P_3	C	red
P_4	D	yellow

key : pid

```
SELECT  c.sid
FROM    catalog C
WHERE EXISTS  (SELECT*
               FROM   parts P
               WHERE P.pid = C.pid
               AND P.color = red)
```

inner query using attributes defined in the outer query

execution -

iteration (i) - catalog 1st tuple : compare with all tuple of parts table and return true if result is non empty.

catalog

sid	pid	cost
S_1	P_1	7k
S_1	P_2	8k
S_3	P_3	9k
S_4	P_4	10k

key : sid, pid

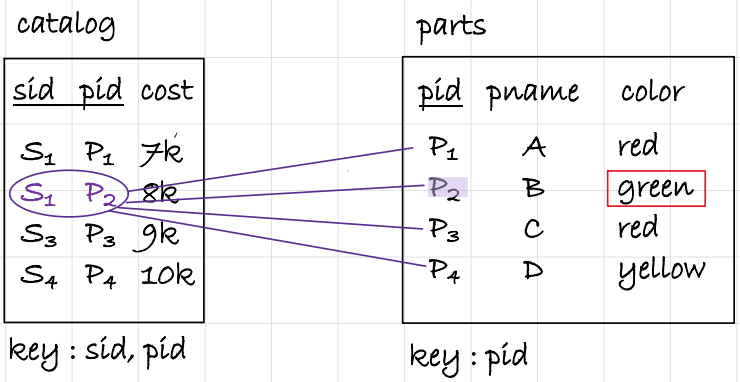
parts

pid	pname	color
P_1	A	red
P_2	B	green
P_3	C	red
P_4	D	yellow

key : pid

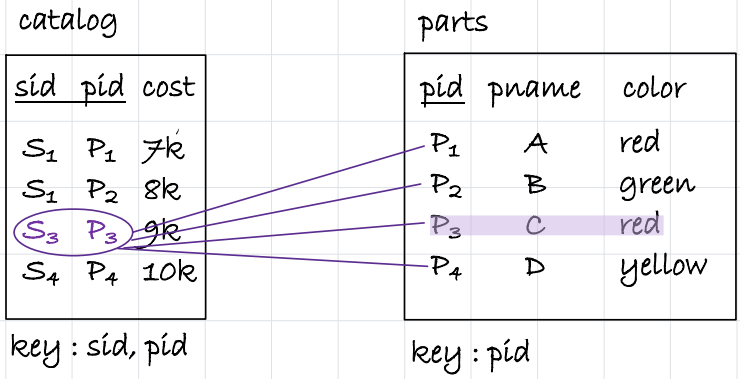
1 tuple return : P_1 A red
EXISTS : return true - S_1
NOT EXISTS : return fale

iteration (ii) - catalog 2nd tuple : compare with all tuple of parts table and return true if result is non empty.



0 tuple return : P_2 B green
pid matches but color is green
EXISTS : return false
NOT EXISTS : return true - S_1

iteration (iii) - catalog 3rd tuple : compare with all tuple of parts table and return true if result is non empty.



0 tuple return : P_3 C red
EXISTS : return true - S_3
NOT EXISTS : return false

iteration (iv) - catalog nth tuple : compare with all tuple of parts table and return true if result is non empty.

catalog parts

sid	pid	cost
S_1	P_1	7k
S_1	P_2	8k
S_3	P_3	9k
S_4	P_4	10k

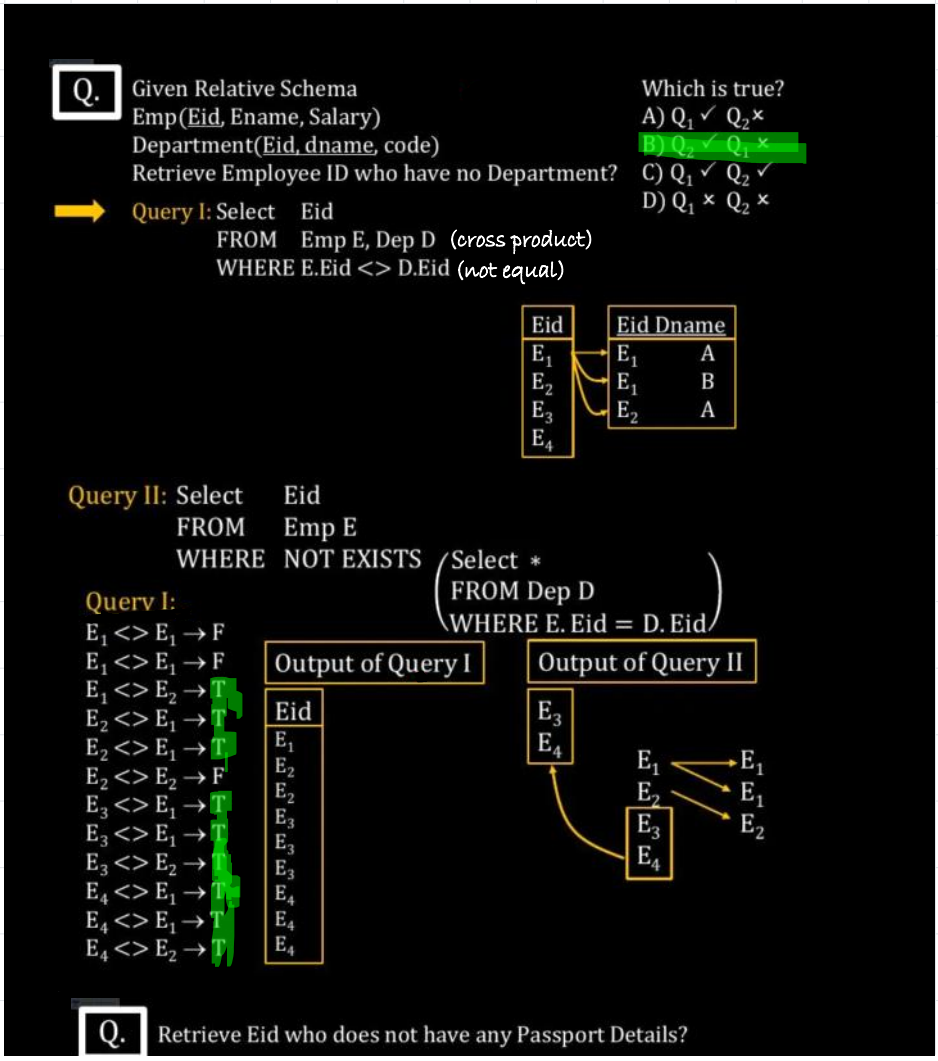
key : sid, pid

pid	pname	color
P_1	A	red
P_2	B	green
P_3	C	red
P_4	D	yellow

key : pid

0 tuple return : P_4 D yellow
EXISTS : return false
NOT EXISTS : return true - S_4

note :
- before EXIST and NOT EXIST, no attribute is required.
- before IN and NOT IN attribute is required.



Q. Retrieve Eid who does not have any Passport Details?

```
Select   Eid
FROM     Emp
WHERE PPro = NULL
```

Employee

eid	ename	PassPortNo
E1	A	20
E2	B	NULL
E3	B	16

NOTE: for comparison with NULL SQL support IS/ IS Not Clause

```
WHERE PPro IS NULL
```

Q. Retrieve Eid who are having some passport details?

```
Select   Eid
FROM     Emp
WHERE PPro IS NOT NULL
```
(having some passport detail)

Regular Expression:

%: Zero or More Character
_ : Exactly One character

Q. Retrieve Student whose Name Start with 'S' & end with M & at least 5 character?

'S _ _ _ % M'

```
Select *
FROM Student
WHERE Sname = 'S _ _ _ % M'
```

Output: Pattern → 'S _ _ _ % M'

LIKE: is used to compare to specify certain search Condition for a Pattern in a column.

```
Select *
FROM Student
WHERE Sname LIKE 'S _ _ _ % M'
```

Q. Retrieve all student whose Name NOT Start with 'C'?

```
Select   *
FROM     Student
WHERE Sname NOT LIKE 'C%'
```

```
Select   *
FROM     Student
WHERE Sname
```

LIKE A% → Starts with A

%J → end with J

%I% → Contain join I

'_ _ _ _' → All 4 length Name

'S _ _ _' → Starts with S & exactly 4 length character

'S _ _ _ %' → Starts with S & at least length

ORDER By: Order By Clause is used to sort the Rows.

Company (Name, invoice no)

Q. Display all the company in alphabetical order of their Name?

```
Query: Select  *
       FROM   Company
       Order by Name asc;
```

Q. If Reverse alphabetical order then

```
Order by Name Desc
```

topic : join with foreign key

foreign key :

(i) the value present in the foreign key must be present in the primary key of the referenced relation.

(ii) foreign key may contain duplicates and null value.

(iii) referencing relation : table which contain foreign key is called referencing (CHILD) table.

whenever two tables (relations) are joined (natural join) with respect to primary key and foreign key then maximum number of tuples in the resulting relation is equals to number of tuples in the referencing relation.

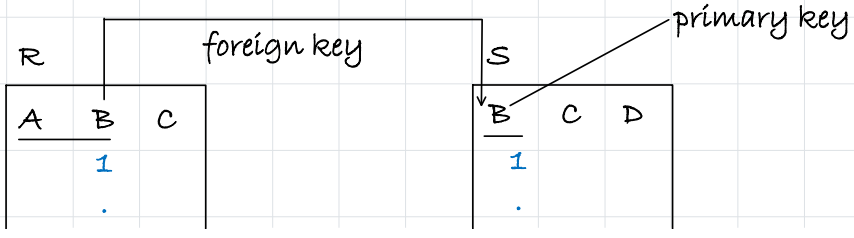
example : R(ABC) S(BCD)

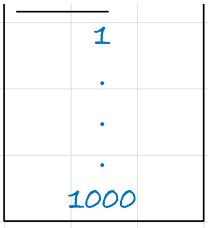
B is the foreign key (referencing relation) (referenced relation)

1000 tuples 5000 tuples

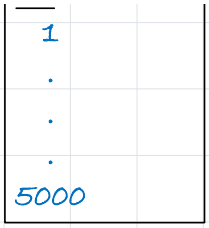
then, maximum number of tuples in RMS?

sol.





1000 tuple



5000 tuple

RMS = 1000

case (i)
1 - 100 times
2 - 100 times
.
.
9 - 100 times
10 - 100 times

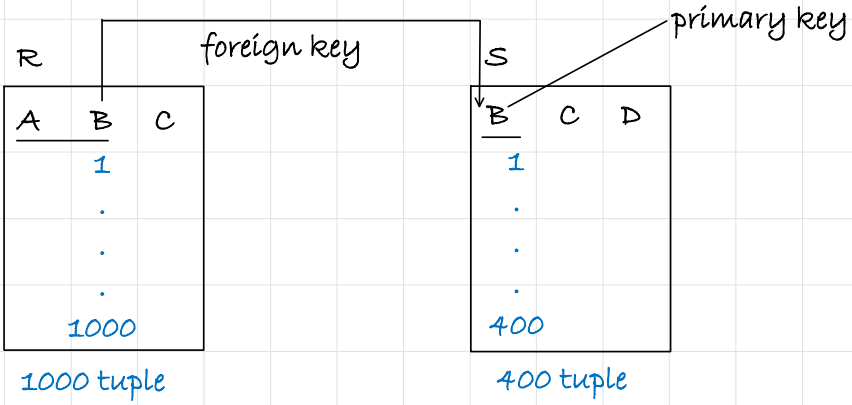
case (ii)
1 - 1000 times

(or)

2 - 1000 times

case (iii)
1,2,3...1000 m

maximum number of tuples : 1000



1000 tuple

400 tuple

RMS = 1000

case (i)
1 - 100 times
2 - 100 times
.
.
9 - 100 times

case (ii)
1 - 1000 times

(or)

2 - 1000 times

case (iii)
1,2,3...400
1,2,3....400
1,2,3....200

.
.
9 - 100 times
10 - 100 times

(or)

2 - 1000 times

1,2,3....200

maximum number of tuples : 1000

how to find minimum and its variations.

Q.1

Consider the relations:
Supply (SupplierID, Itemcode) with 1000 tuples
Inventory (Itemcode, color) with 2500 tuples
Let p and q be the number of maximum and minimum records in Supply JOIN Inventory, the value of p + q is 2000. (Itemcode is FK in Supply table).

referencing relation
supply (supplier ID, item code)

referenced relation
inventory (item code, color)

maximum number of tuple : 1000
minimum number of tuple : 1000

here item code in the supply table is foreign key that reference to primary key in inventory table and here item code is a key (it can be duplicate because it is not a single key, supplier ID is also a key) also in a supply table, so item code is a part of key and key cannot contain null.

- the value present in the foreign key must be present in the primary key of the referenced relation.
- foreign key may contain duplicates and null value.

Q. 2

Consider the relations:
Employee(Eid, Ename) with 2000 tuples
Department(Eid, Did) with 1000 tuples
Let p and q be the number of maximum and minimum records in Employee JOIN Department, the value of p + q is 2000. (Eid is FK in Department table).

referencing relation
department (Eid, Did)
maximum number of tuple : 1000
minimum number of tuple : 1000

referenced relation
employee (Eid, ename)

Q.3

Consider the relations:
R1(A, B,C) with n tuples
R2(B, D,E) with m tuples (B is FK in R1).

maximum tuples in $R1 \bowtie R2$
minimum tuples in $R1 \bowtie R2$

maximum tuples in $R1 \bowtie R2$
minimum tuples in $R1 \bowtie R2$

referencing relation — referenced relation
R1(A,B,C) — R2 (B,D,E)
maximum number of tuple : n
minimum number of tuple : n (if B is foreign key and not null)
0 (if B is foreign key and contain null)
(or)
B is not foreign key.

Q.5 Consider the join R ⋈ S between R (A B C) and S (A D E) with attribute A being the primary key in both relations and attribute A in S is a Foreign Key referencing to attribute A in R. R has m tuples and S has n tuples, maximum and minimum number of tuples in R ⋈ S respectively are?

(a) m, m + n
(b) m*n, 0
(c) min (m, n), min (m, n)
(d) max (m, n), min (m, n)

referencing relation — referenced relation
S(A, D, E) — R(A, B, C)
n tuples. — m tuples.

in S, A is primary and foreign key both
hence unique + not null.

and also A is also foreign key so the value present in R will be present in S also.

Q. 6 Consider the relations:
R1(A, B, C) with n tuples
R2(D, B, E) with m tuples
$\{A \rightarrow B,\ B \rightarrow C,\ D \rightarrow B,\ B \rightarrow E\}$
maximum tuples in $R1 \bowtie R2$
minimum tuples in $R1 \bowtie R2$

R1(A, B, C) — R2 (D, B, E)
$[A]^+ = ABC$ — $[D]^+ = DBE$
A is primary key — D is primary key

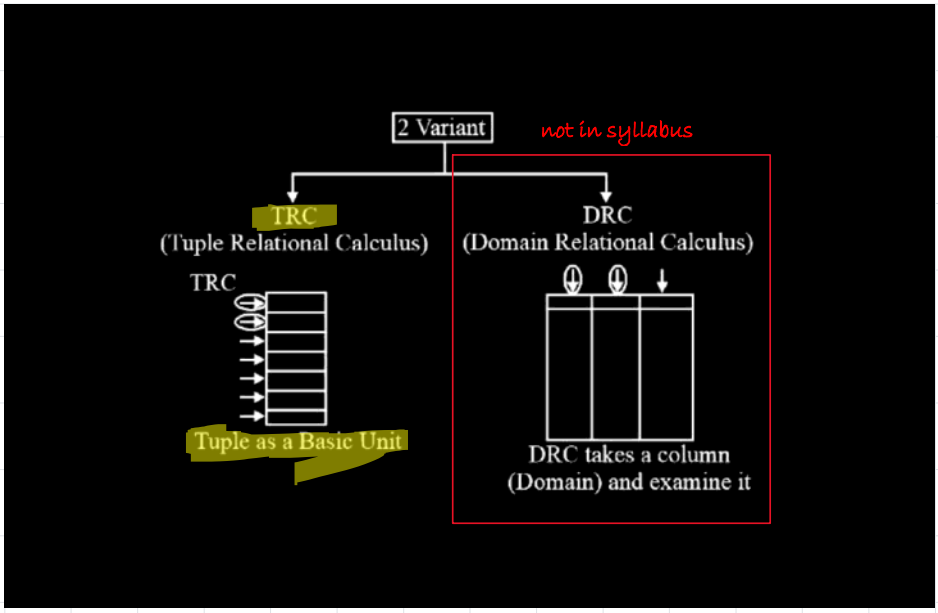
B is non-key attribute in both relation

maximum = n x m
minimum = 0

relational calculus : based on predicate calculus

(i) Tuple relational calculus (TRC)

(ii) domain relational calculus (DRC)



(i) Tuple relational calculus (TRC)

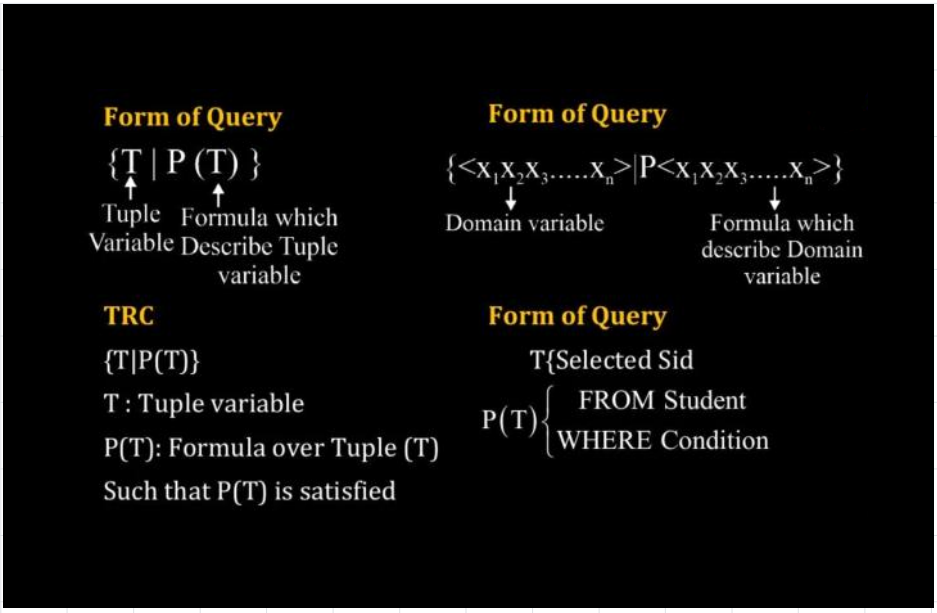
a tuple variable is a variable that takes on tuples of a particular relation schema as values that is every value assigned to a given tuple variable has the same number and type of fields.

- takes a tuple and examine it

- tuple as a basic unit

- it is variable that takes on tuples of a relational schema as value.

syntax : {t/p(t)}
t : tuple variable
p(t) : formula which describe tuple variable



R(ABC)

	A	B	C
t			
t			
t			

$t \in R$ means t is a tuple variable

(or) in a relation so t can take

$r(t)$ tuples of R

selection

q.(i) R(ABC)

R.A: $\sigma_{B>90(R)}$

TRC :

(i) $\{t/t \in R \wedge t.B=90\}$

(ii) $\{t/t \in R \wedge t[B]=90\}$

(iii) $\{t/t \in R \wedge B=90\}$

(iv) $\{t/R(t) \wedge t.B=90\}$

S supplier (sid, sname, rating)

P parts (pid, pname, color)

C catalog (sid pid, cost)

q(i) write a query to find the supplier whose rating is greater than 8.

sol.(i) SQL : SELECT *
FROM supplier
WHERE rating >8

R.A. : $\sigma_{rating > 8 (supplier)}$

TRC : {t/t ∈ supplier ⍰ t.rating>8}

q(ii) write a query to find parts whose color is red.

sol.(ii) TRC : {t/t ∈ parts ⍰ t.color = red}

q.(iii) R(ABC)

R.A: $\sigma_{B=90(R)}$

TRC :
(i) {t/t ∈ R ⍰ t.B =90}
(t is not empty here because we said t belongs to R)
(ii) {t/∃ X ∈ R (t.A = X.A ⍰ t.B = X.B ⍰ t.C = X.C ⍰ X.B = 90}
t is empty here

projection

q.(i) R(ABC)

R.A: $\pi_{AB(R)}$

TRC :
(i) {t.A, t.B/t ∈ R }
(ii) {t/∃ X ∈ R (t.A = X.A ⍰ t.B = X.B }

q.(ii) R(ABC)

R.A: $\pi_{A(R)}$

TRC :
(i) {t/∃ S ∈ R (t.A = S(A))}
some S belongs to R
us S ka A
t ke A mai assign kardo

(ii) {t/∃ S ∈ R (t.A = S.A}

selection and projection

q. (i) R(ABC)

- R.A: $\pi_A[\sigma_{B=90(R)}]$

TRC :
(i) {t.A/t ∈ R ∧ t.B=90}
(ii) {t.A/R(t) ∧ t.B=90}
(iii) {t.A/R(t) ∧ B=90}
(iv) {t/∃ X ∈ R [t.A = X.A ∧ X.B = 90]}
(v) {t/∃ X ∈ R [X.B = 90 ∧ t.A = X.A]}

- R.A: $\pi_{A,C}[\sigma_{B=90(R)}]$

TRC :
(i) {t.A, t.C/t ∈ R ∧ t.B=90}
(ii) {t.A, t.C/R(t) ∧ t.B=90}
(iii) {t.A, t.C/R(t) ∧ B=90}
(iv) {t/∃ X ∈ R [t.B = 90 ∧ t.A = X.A ∧ t.C = X.C]}

- R.A: $[\sigma_{B=90 \wedge C=100}(R)]$

TRC :
(i) {t/t ∈ R ∧ t.B=90 ∧ t.C=100}

- R.A: $\pi_A[\sigma_{B=90 \wedge C=100}(R)]$

TRC :
(i) {t.A/t ∈ R ∧ t.B=90 ∧ t.C=100}
(ii) {t/∃ X ∈ R [X.B = 90 ∧ X.C=100 ∧ t.A = X.A]}

- R.A: $\pi_{A, B, C}[\sigma_{B=90 \wedge C=100}(R)]$

TRC :
(i) {t.A, t.B, t.C /t ∈ R ∧ t.B=90 ∧ t.C=100}
(iv) {t/∃ X ∈ R [X.B = 90 ∧ X.C=100 ∧ t.A = X.A ∧ t.B = X.B ∧ t.C = X.C]}

Ques.

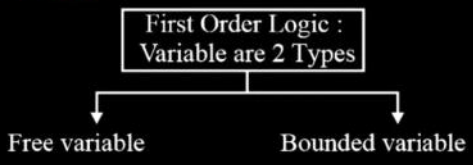
LOAN (Loanno, Branch name, Amount)

Q. Find Loan no of amount above 5000.

TRC: {T|T∈ Loan (Total amount > 50,000)}

DRC: {<Loanno., branch name, amount>| < Loan no, Branch, amount ∈ Loan (amount > 50000) }

NOTE: Formula of TRC is expressed using first order logic **(Predicate logic).**



Quantifiers

1. ∀ : For all (ALL)
2. ∃ : There exists (Any)

Bounded Variable: If tuple variable is preceded by the quantifier then it is Bounded variable.

Belong

1. ∀ ∈ Supplier
2. ∃ ∈ Supplier

Free variable: If tuple variable is not bounded by the quantifier then it is free variable.

NOTE: The result of TRC should be free variable.

{t/∃ X ∈ R [X.B = 90 ∧ t.A = X.A]}

X : bounded variable

t : free variable.

S supplier (sid, sname, rating)

P parts (pid, pname, color)

C catalog (sid pid, cost)

Q. Retrieve Sid of the supplier who supplied some Red color parts.

R.A:

$$\pi_{Sid}\left(\sigma_{C.Pid=P.Pid}\left(\text{Cata log} \times \sigma_{colour=Red}\left(\text{Parts}\right)\right)\right)$$

TRC:

$$\left(T \mid \exists C \in \text{Catalog}\left(\exists P \in \text{parts}\left(P.color = Red \wedge P.Pid = C.Pid\right) \wedge T = C.Sid\right)\right)$$

TRC: Unsafe operation occur

Unsafe operation: Student (t)

negation (t)

Unsafe operation: Student (t)

negation (t)

~(t): Not belongs to student table (infinite/Universe all tuple)

So, its unsafe operation.

Q. SELECT operation in SQL is equivalent to [MCQ: 2015-1M]

A. The selection operation in relational algebra.

B. The selection operation in relational algebra, except that SELECT in SQL retains duplicates.

C. The projection operation in relational algebra.

D. The projection operation in relational algebra, except that SELECT in SQL retains duplicates.

file organisation and indexing

file organisation

tells how database is stored
-in the form of blocks.

indexing

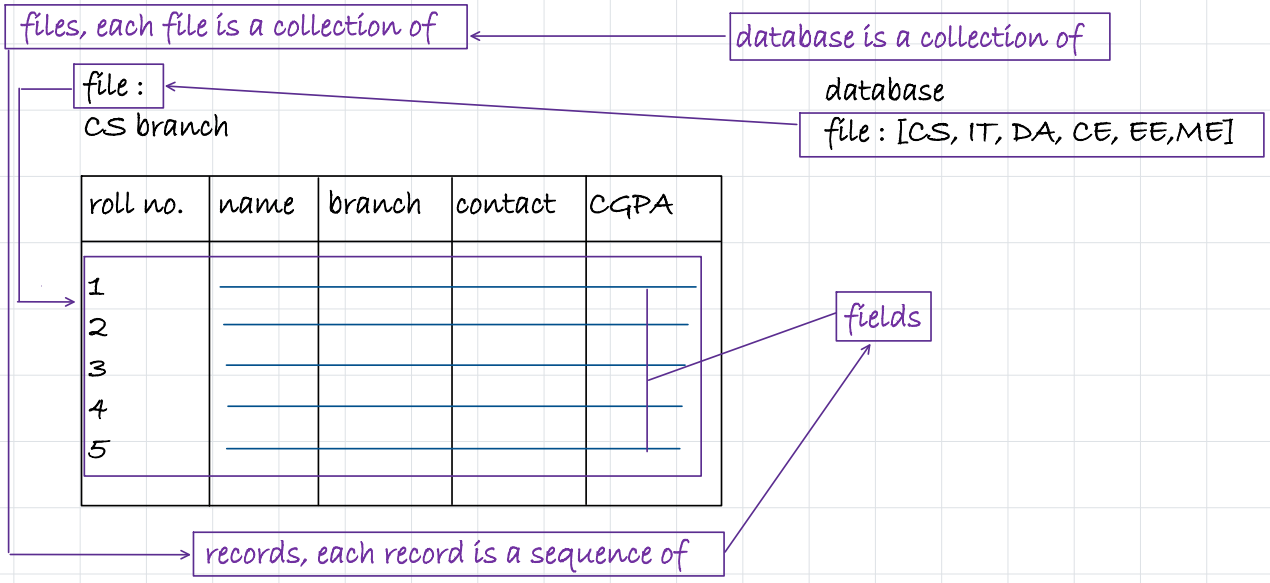
provides data structure for data access
in a efficient, easy and fast manner.

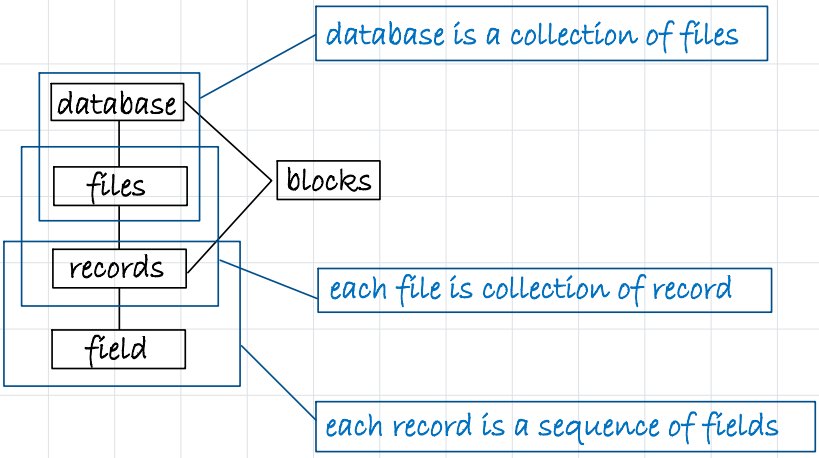
database stores in hard-disk aur main memory mai uski copy rakhege

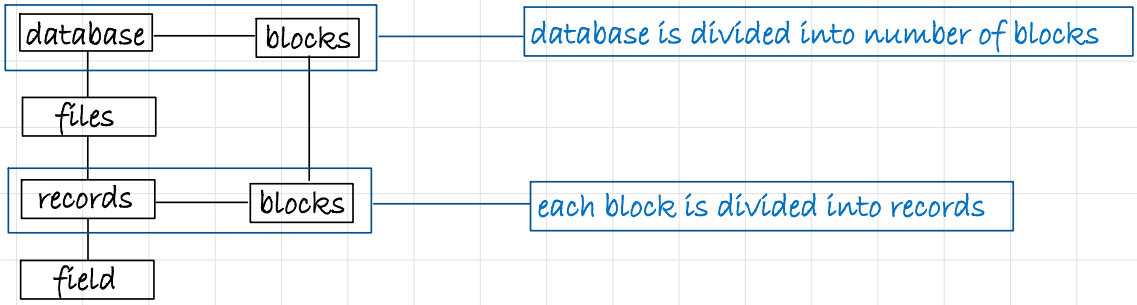
database → blocks

hard-disk → platter → surface → track → sector → DATA

blocks : group of consecutive sector.
in DBMS, basic unit of storage is block







Block size will be given (decided by the database and not us) 512 byte onwards

topic : file organization

data records can be

(i) fixed length records
(ii) variable length records

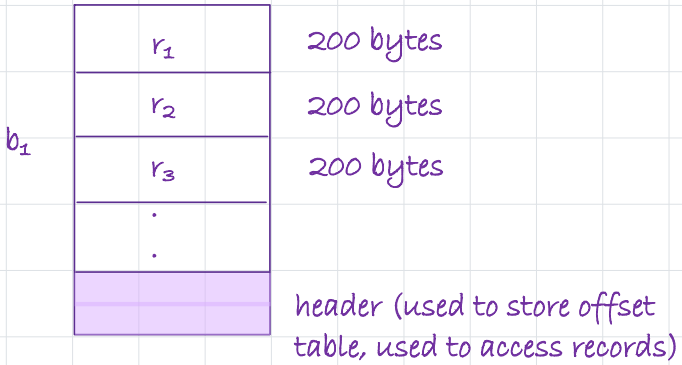
(i) fixed length records

1 record (tuple) size : 200bytes

c language -
name char (100) : 100bytes
address char (50) : 10bytes
branch char(50) : 50bytes

so we can calculate number of records per block stored.

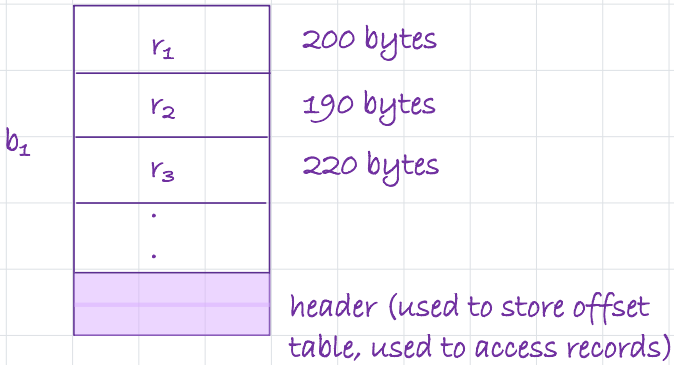
name	address	branch
100	50	50



(ii) variable length records

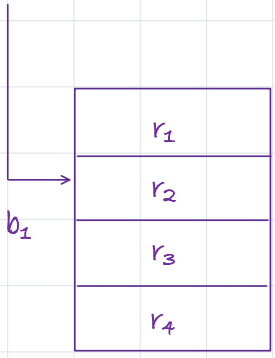
name char (100) : 100bytes
address char (50) : 50bytes
branch char(50) : not fixed

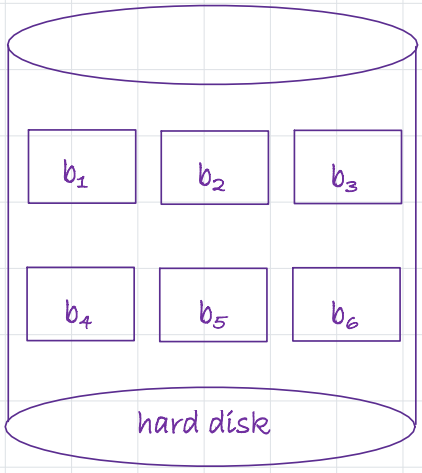
name	address	branch
100	50	not fixed



(i) blocking factor [b_{fr}]: average number of record per block.

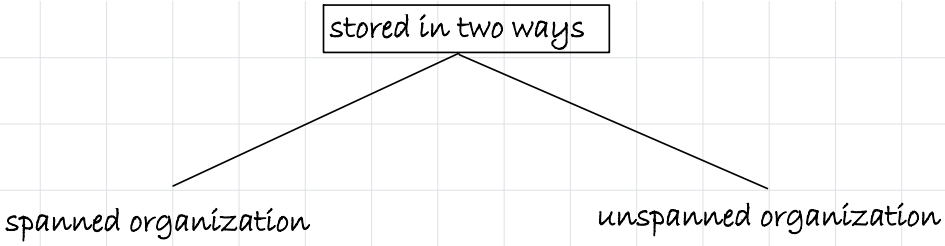
$b_{fr} = 4$
4 blocks per record





$$\text{blocking factor} = \frac{\text{block size}}{\text{record size}}$$

block size >>> record size



(i) spanned organisation : a record can be stored in more than one block.
(or)
it allows partial part of record stored in a block.

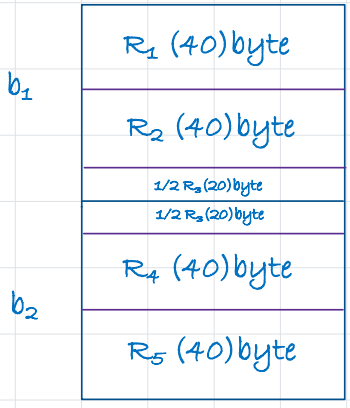
ex.
block size = 100 byte
record size = 40 byte

$$\text{blocking factor} = \frac{\text{block size}}{\text{record size}}$$

$$\text{blocking factor} = \frac{100}{40}$$

2.5 record per block

disadvantage : R_3 agar chaiye toh b_1 aur b_2 dono laana padega hard disk and hence block access increase



advantage : no memory wastage, no internal fragmentation.

disadvantage : block access increase, more access cost to access records, too complex to manage records

suitable : for variable length records.

(ii) uspanned organisation : no record can be stored in more than one block.
(default)
(or)
a record can be stored to a particular block
(does not allow partial storage).

ex.
block size = 100 byte

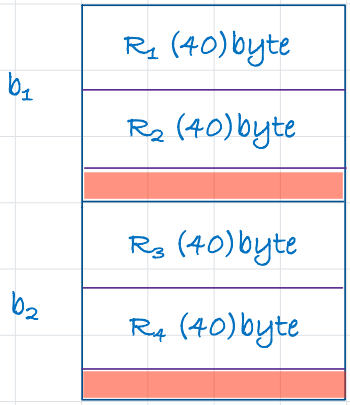
ex.
block size = 100 byte
record size = 40 byte

blocking factor of database file = $\left\lfloor \frac{\text{block size}}{\text{record size}} \right\rfloor$

blocking factor = $\left\lfloor \frac{100}{40} \right\rfloor$

2 record per block (floor value)

20bytes wastage.



advantage : block access reduce, access cost is less, easy to manage records

disadvantage : memory wastage (internal fragmentation)

suitable : for fixed length records.

what is i/o : input output cost (kitne blocks secondary memory se main memory mai transfer kiye jaa rahe hai kisi bhi record ko access karne ke liye)

what is search : attribute used to access the data from database.

organization of records in a file :

(i) ordered file organization : records sorted by ordering field. (called ordering key if ordering field is a key field) (key field ke according ordering kar dete hai)

advantage :
- reading records in order of ordering key value is extremely efficient (kisi bhi record ko easily access kar paoge) means searching easy
- finding next record
- binary search technique use karte hai

disadvantage :
- insertion is expensive due to re-organization of entire file.

lets assume you want to insert a student name that starts with letter 'M', abcdefghijkl(reading)...M...you have to shift the rest of the elements in order to insert 'M'

to access a record the average number of block access = $\lceil \log_2 B \rceil$ (B: data blocks)

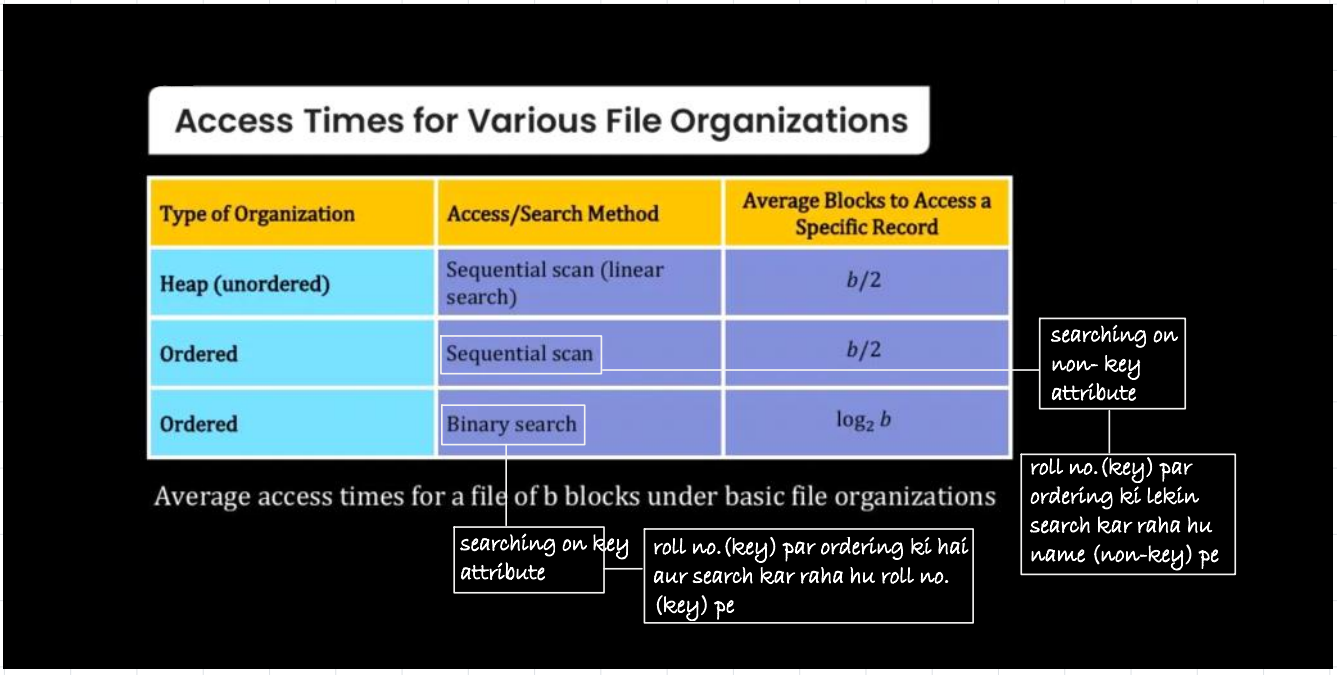
(ii) unordered file organization (heap files) : records placed in file in order of insertion.

- inserting a new record is difficult.

- searching for a record requires linear search.

- deletion techniques.

to access a record the average number of block access (avg.) = B/2 (B : data blocks)
worst case = B



note
default : unspanned

topic : indexing

- indexing is used to improve the searching efficiency .

- to reduce the i/o cost.

- indexing is ordered file.

indexing file

search key	pointer

block to a point where key is available

one index record size : size of search key (k) + size of pointer (p)

index file block size = DB file block size

$$\text{block factor of index file} = \frac{\text{block size}}{\text{one index record size}} = \left\lfloor \frac{B}{k+p} \right\rfloor$$

q. block size =100byte
key = 6byte
block pointer (B_p) = 4byte

one index record size : size of search key + size of pointer
one index record size : 6 + 4 = 10byte

$$\text{blocking factor} = \frac{\text{block size}}{\text{record size}}$$

$$\text{blocking factor} = \frac{100}{10}$$

in one block : 10 index entries.

note :
to access a record average number of block access = $\log_2 B_i + 1$

$\log_2 B_i$: index block access karne ke lie

1 : data block access (us box par gaye jahan hume record ko fetch karna hai)

note : block factor of index file will be more than block factor of data file
but number of index blocks will be less than number of database file block

example :
block size : 100B
record size : 40B
key : 13B
B_p: 7B

blocking factor of database file : $\left\lfloor \frac{100}{40} \right\rfloor$ = 2 record per block

block factor of index file = $\frac{\text{block size}}{\text{one index record size}}$ $\left\lfloor \frac{B-H}{k+p} \right\rfloor = \left\lfloor \frac{100-0}{13+7} \right\rfloor = \left\lfloor \frac{100}{20} \right\rfloor$ = 5 index record per block

- Suppose that:
  - record size R = 150 bytes, block size B = 512 bytes, r = 30000 records
- Then, we get:
  - blocking factor Bfr = blocking factor $= \frac{\text{block size}}{\text{record size}}$ $\left\lfloor \frac{512}{150} \right\rfloor$ = 3 records / block
  - number of file blocks $b = \frac{r}{bfr} = \frac{30000}{3}$ = 10000 data blocks

if we have 30002 records then number of data block?
number of data block = $\left\lceil \frac{30002}{3} \right\rceil$ 10001 data block

note : block factor nikalte time floor lena hai

Given the following data file
EMPLOYEE (NAME, SSN, ADDRESS, JOB, SAL, ...)
Suppose that:
- record size R=150 bytes, block size B=512 bytes r=30000 records
- For an index on the SSN field, assume the field size V_{SSN} =9 bytes, assume the record pointer size P_R=7 bytes. Then:

blocking factor $= \frac{\text{block size}}{\text{record size}}$ $\frac{512}{150}$ = 3 records / block

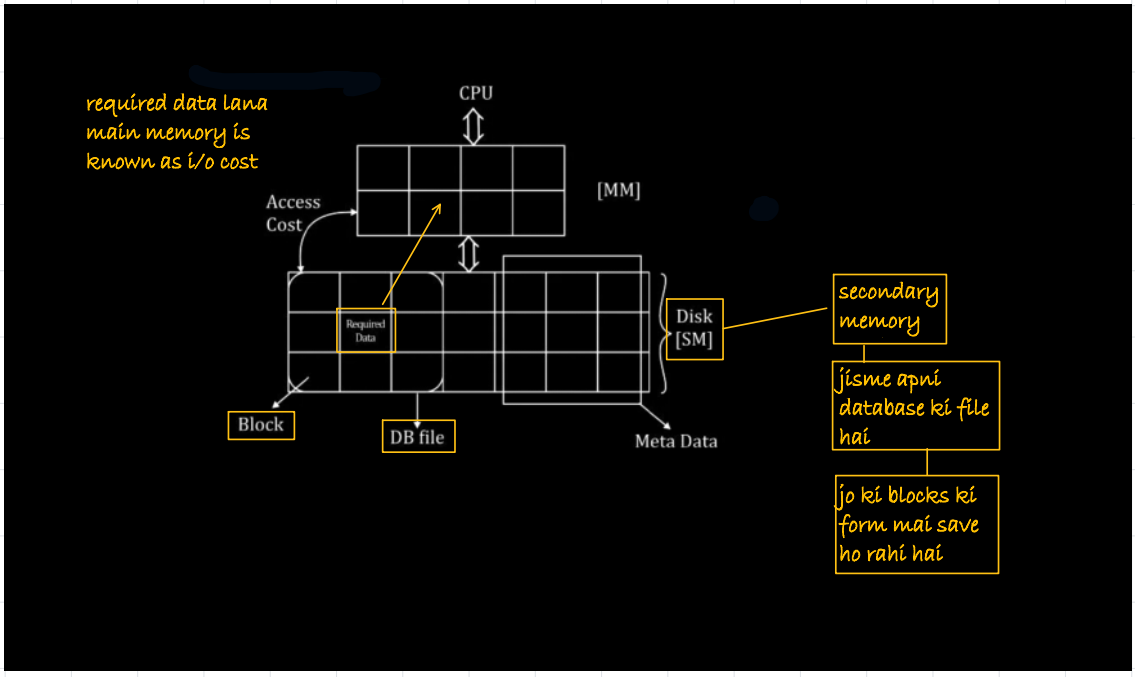
- index entry size $R_1=(V_{SSN}+P_R)=$ 9+7 = 16bytes
- index blocking factor Bfr =B div R = $\lfloor 512 \rfloor$

- index entry size $R_1=(V_{SSN}+P_R)=$ 9+7 = 16bytes
- index blocking factor $Bfr_1=B \text{ div } R_1=\left\lfloor \frac{512}{150} \right\rfloor$ = 32 index entries
- number of index blocks $b=(r/Bfr_1)=$ 30000/32 = 938 blocks (dense)
- binary search needs $\log_2 bI=\log_2 938$ =10 block accesses

to fetch a record using index
average number of block access = $\log_2 B_i + 1 = \log_2 938 + 1 = 10 + 1 = 11$

note :
(i) to organize DB file with fixed length record unspanned organization is preferred.

(ii) to organize DB file with variable length record spanned organization is preferred.



what is meta data?
data about data (data nahi bata raha but us data ke baare me baat rahe)

- record format
- field format
- number of blocks allocated for file
- number of record in file

- ordered field of file , etc.

types of entries :

(i) dense index files : index entry is created for every search key value

for each database record of database file there exist entry file in index file
number of index entries = number of database records

(ii) sparse index files : contains index records for only some search-key values (not for everyone).

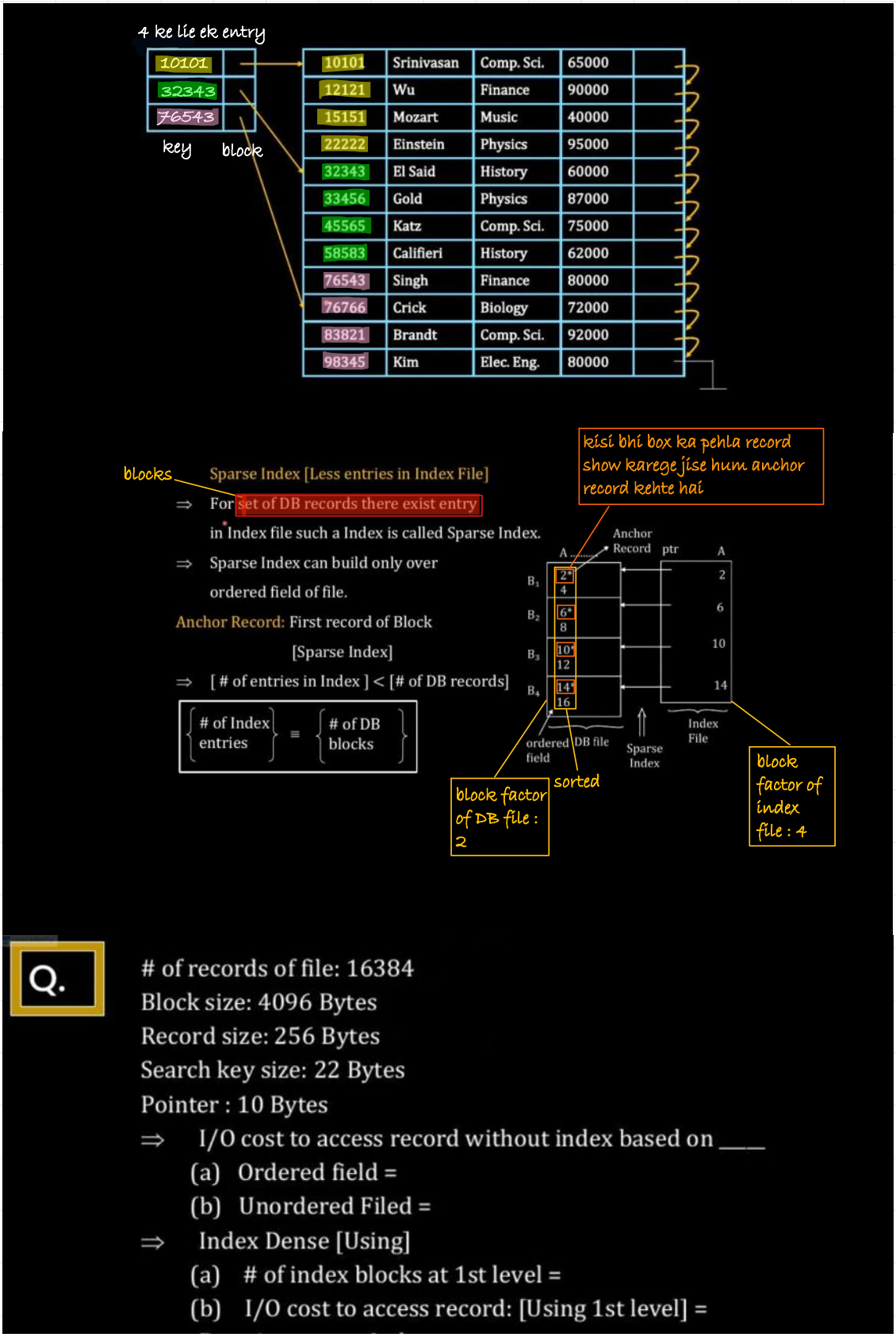
- applicable when records are sequentially ordered (sorted) on search key

to locate a record with a search key value K we:

(i) find index record with largest search-key value <K

(ii) search file sequentially starting at the record to which the index record points.

number of index entries = number of blocks



Q.

# of records of file: 16384
Block size: 4096 Bytes
Record size: 256 Bytes
Search key size: 22 Bytes
Pointer : 10 Bytes

⇒ I/O cost to access record without index based on ____

(a) Ordered field =

(b) Unordered Filed =

⇒ Index Dense [Using]

(a) # of index blocks at 1st level =

(b) I/O cost to access record: [Using 1st level] =

(a) # of index blocks at 1st level =

(b) I/O cost to access record: [Using 1st level] =

⇒ By using sparse Index:

(a) # of Index blocks at 1st level =

(b) I/O cost to access record: [Using 1st level] =

records = 16384 [2^{14}]
block size = 4096 byte [2^{12}byte]
record size = 256B [2^8 byte]
key = 22byte
B_p = 10 byte

without index :

Block factor of database file $= \frac{\text{block size}}{\text{record size}} = \frac{4096\ [2^{12}]}{256\ [2^8]} = 2^4 = 16$ record per block

number of data block [B] $= \frac{16384\ [2^{14}]}{16\ [2^4]} = 2^{10} = 1024$ data block

ordered file

$\lceil \log_2 B \rceil = \lceil \log_2 1024 \rceil = 10$
blocks access

unordered file

avg = b/2
1024/2 = 512

worst = B
B = 1024

with index :

one index record size = 22 + 10 = 32 byte

Block factor of index file $= \frac{\text{block size}}{\text{record size}} = \frac{4096\ [2^{12}]}{32\ [2^5]} = 2^7 = 128$ index entires per block

total number of index entries :

dense : 16384 [2^{14}] (number of database record)

total number of index block (b_i) : $\frac{16384\ (2^{14})}{128\ (2^7)} = 2^7 = 128$

to fetch a record using index average number of block access = $\lceil \log_2 B_i \rceil + 1$

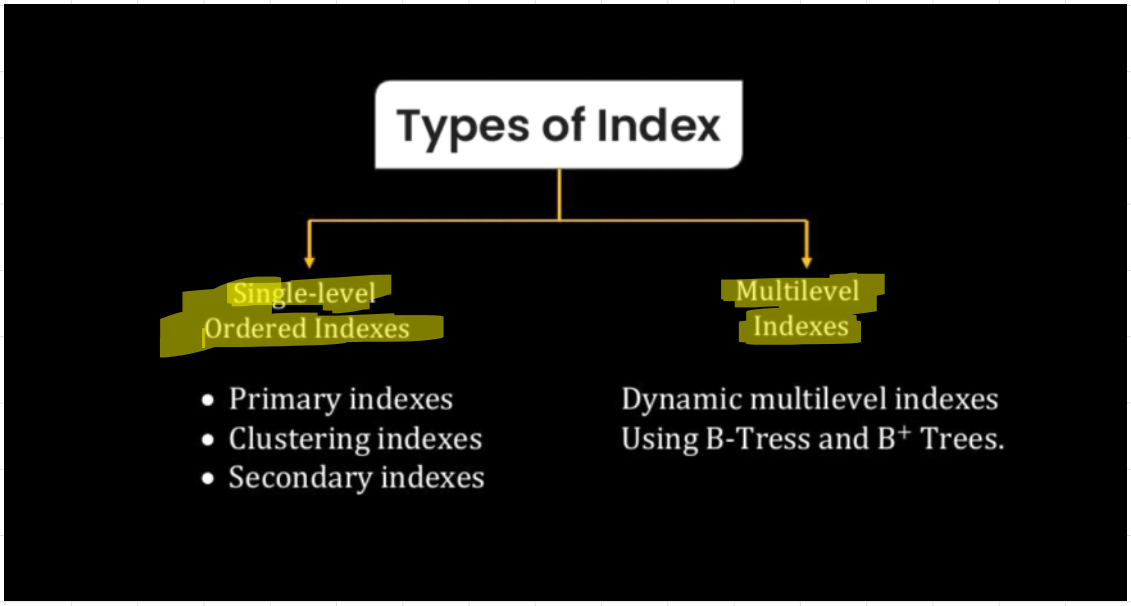
$\lceil \log_2 128 \rceil + 1$
7 + 1 = 8

sparse : 1024 [2^{10}] (number of database block)

total number of index block (b_i) : $\frac{1024\ (2^{10})}{128\ (2^7)} = 2^3 = 8$

to fetch a record using index average number of block access = $\lceil \log_2 B_i \rceil + 1$

$\lceil \log_2 8 \rceil + 1$
3 + 1 = 4

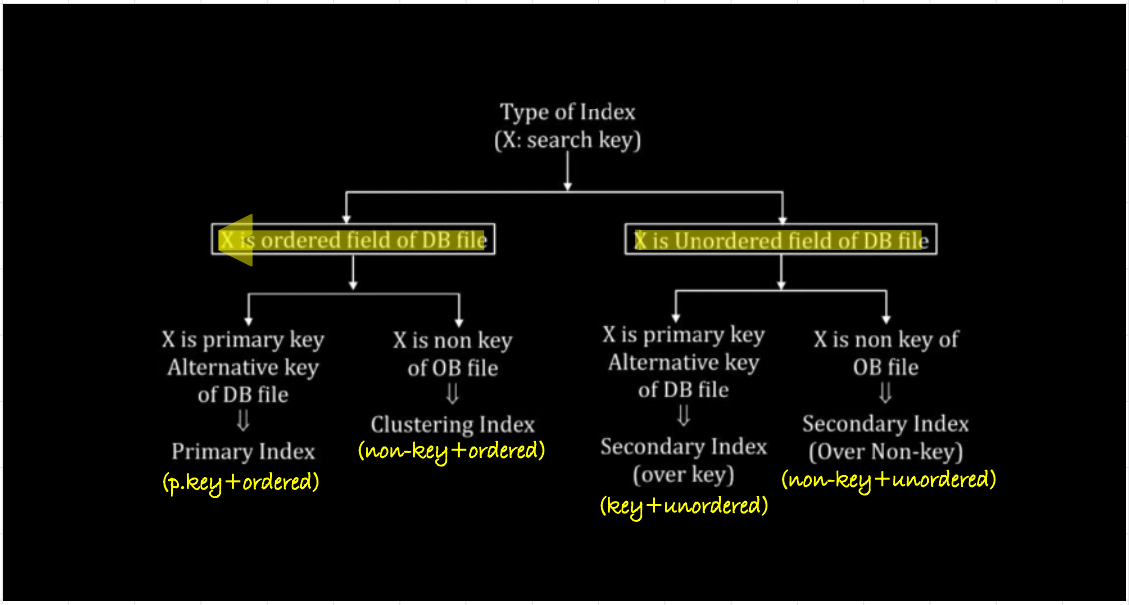


types of index :

(i) primary index : (search key is primary key + ordered file(DB file))

(ii) clustering index : (non-key + ordered file(DB file))

(iii) secondary index : (candidate key/non-key + unordered file(DB file))

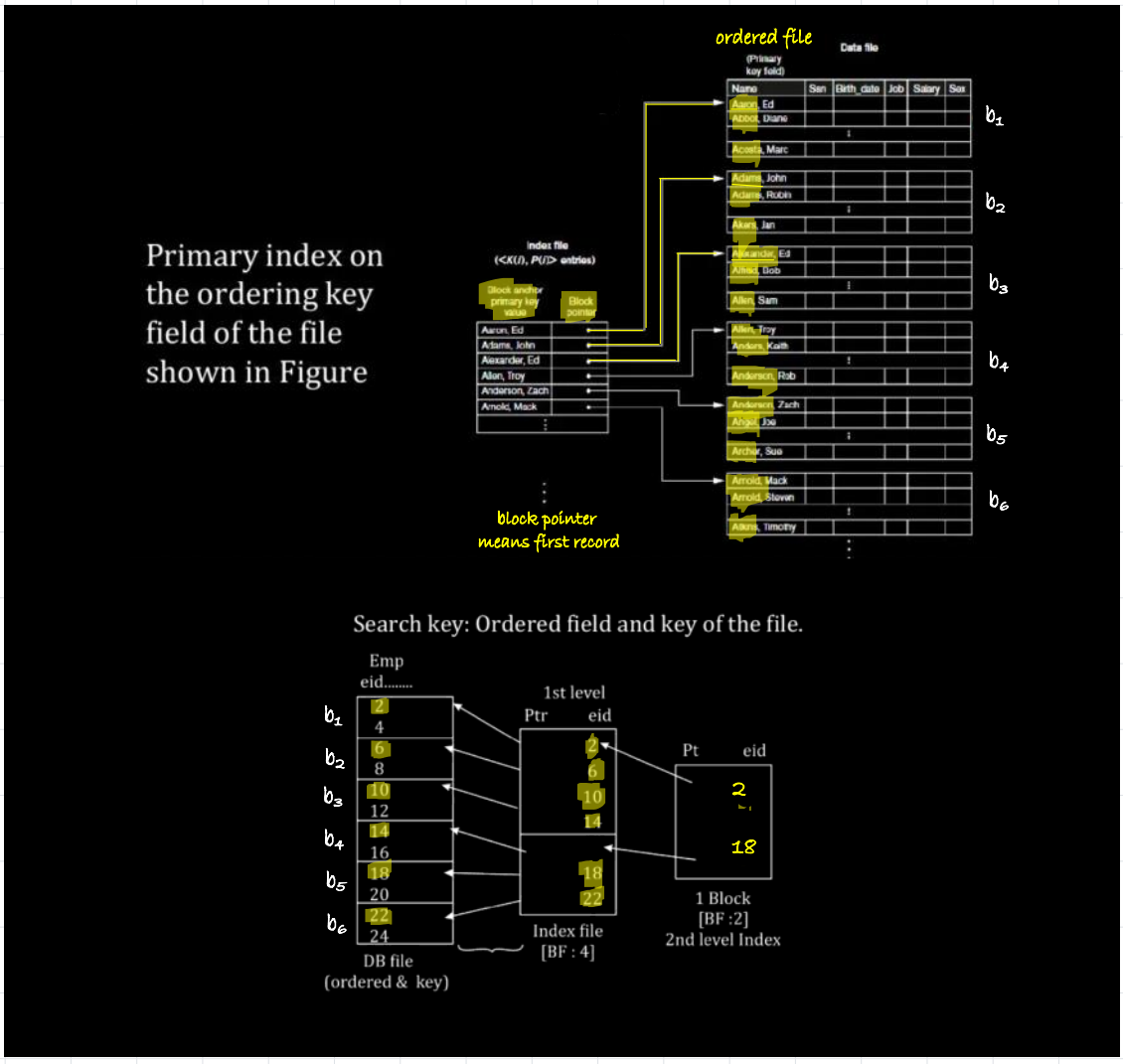


(i) primary index : (search key is primary key + ordered file(DB file))

ordered file with two fields -
(i)primary key,k(i)

(ii) pointer to disc block, P(i)

sparse (or) dense both are possible but mostly sparse.



Q.1 Suppose that we have Ordered file of 30,000 records, stored on a disk with Block Size 1024 Byte, file records are of fixed length & unspanned of size 100 Byte (Record size) and suppose that we Have created a primary index on the key field of the file of size 9 Byte and Block pointer of size 6 Byte then find the average number of Block Access to search for a record using with and Without Index ?

records = 30,000
block size = 1024 byte
record size = 100B
key = 9 byte
B_p = 6 byte

without index :

Block factor of database file $= \frac{\text{block size}}{\text{record size}} = \frac{1024}{100}$ = 10 record per block

number of data block [B] $= \frac{30000}{10}$ = 3000 data block

ordered file

$\lceil \log_2 B \rceil = \lceil \log_2 3000 \rceil = 12$ blocks access

unordered file

avg = b/2
3000/2 = 1500

worst = B
B = 3000

with index :

one index record size = 9 + 6 = 15 byte

Block factor of index file $= \left\lfloor \frac{\text{block size}}{\text{record size}} \right\rfloor = \left\lfloor \frac{1024}{15} \right\rfloor$ = 68 index entires per block

total number of index entries :

sparse : 3000 (number of database block)

total number of index block (b_i) : $\left\lceil \frac{3000}{68} \right\rceil = 45$

to access index block, average block access :

$\lceil \log_2 B_i \rceil$

$\lceil \log_2 45 \rceil$

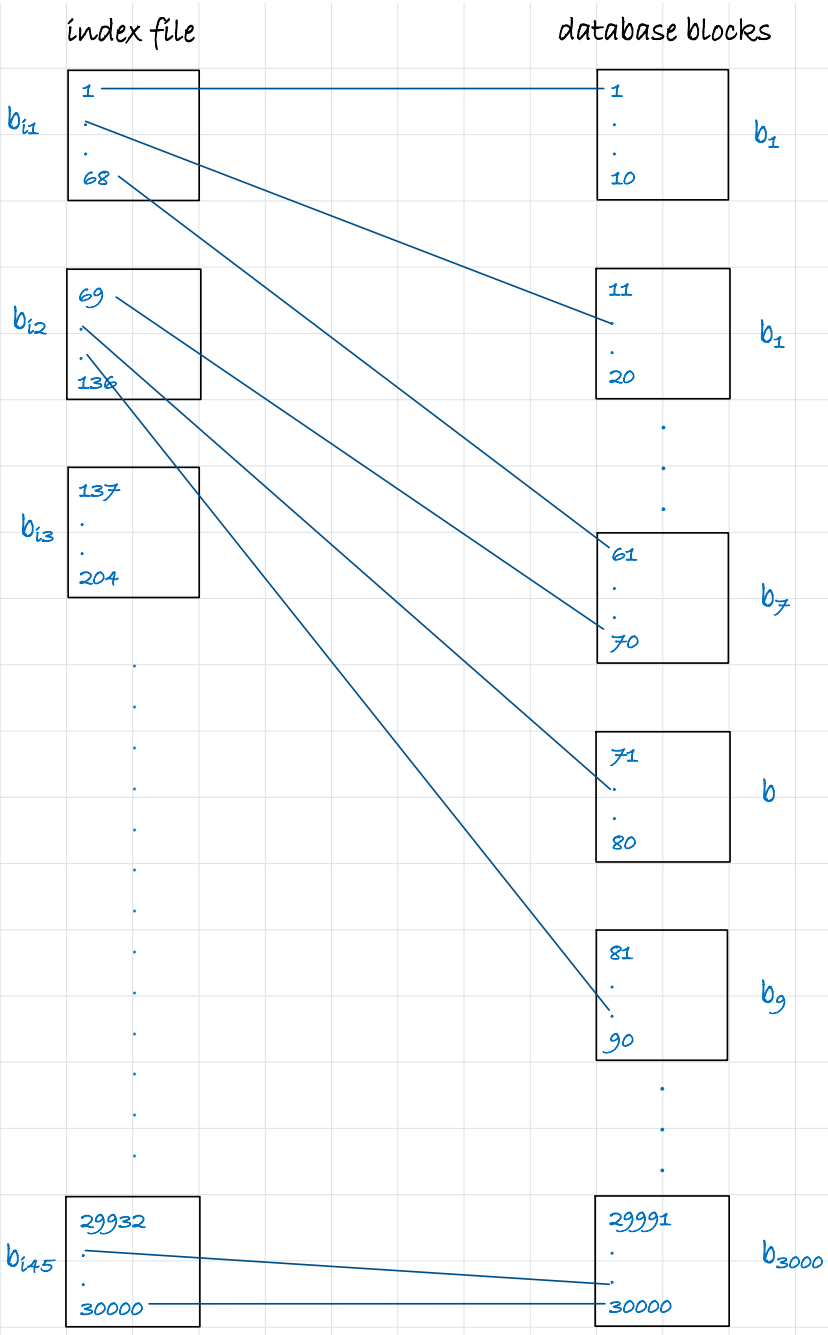
6

to access (fetch) a record using index average no. of block access : $\lceil \log_2 B_i \rceil + 1$

$\lceil \log_2 45 \rceil + 1$

6 + 1 = 7

Visualisation :

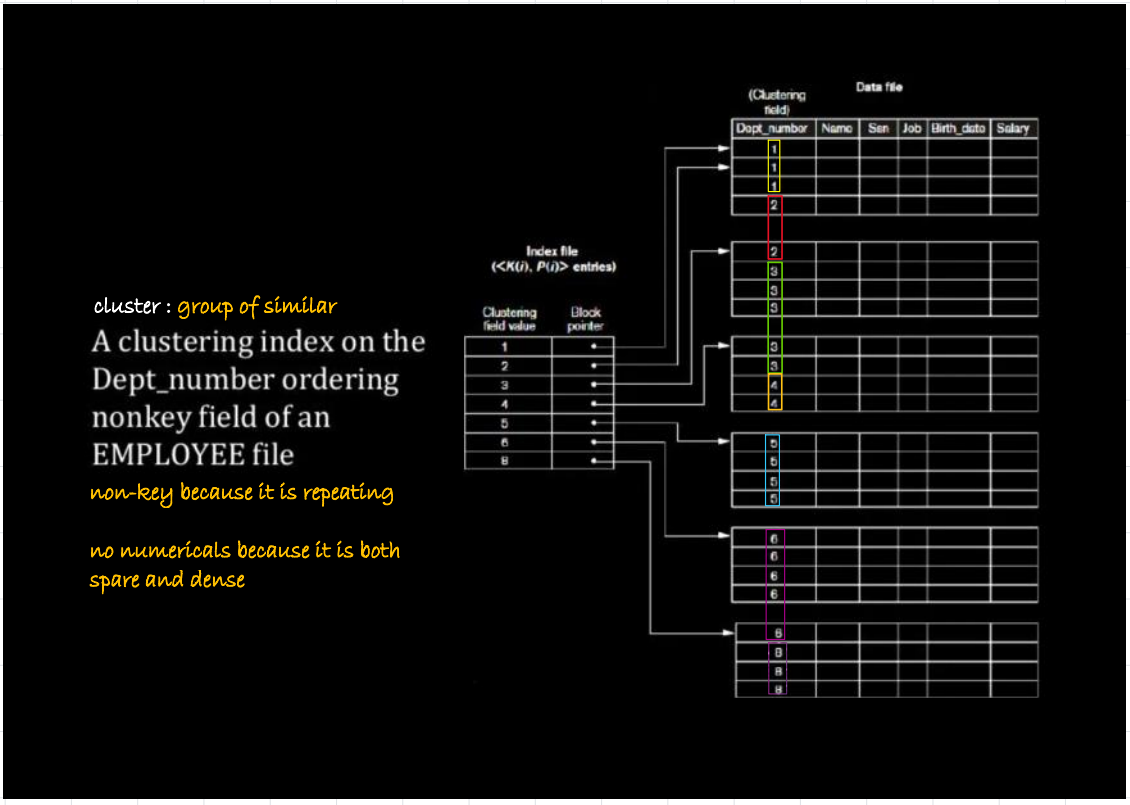


block factor of db file = 10r/b
total data block = 3000
bfi = 68 entries
total index block = 68

(ii) clustering index : (non-key + ordered file(DB file))

ordered file with two fields :
(i) same type as clustering field
(ii) disc block pointer

file records are physically stored on a non-key field without a distinct value for each record



for any DB relation can build either primary index (or) cluster index but not both

(iii) secondary index : (candidate key/non-key + unordered file(DB file))

the secondary index may be on a field which is a candidate key and has a unique value in every record, or a non-key with duplicate values.

ordered file with two fields :
(i) the first field is of the same data type as non-ordering field of the data file that is an indexing field.
(ii) the second field is either a block pointer (or) record pointer.
(iii) there can be many secondary indexed (and hence, indexing fields) for the same file.

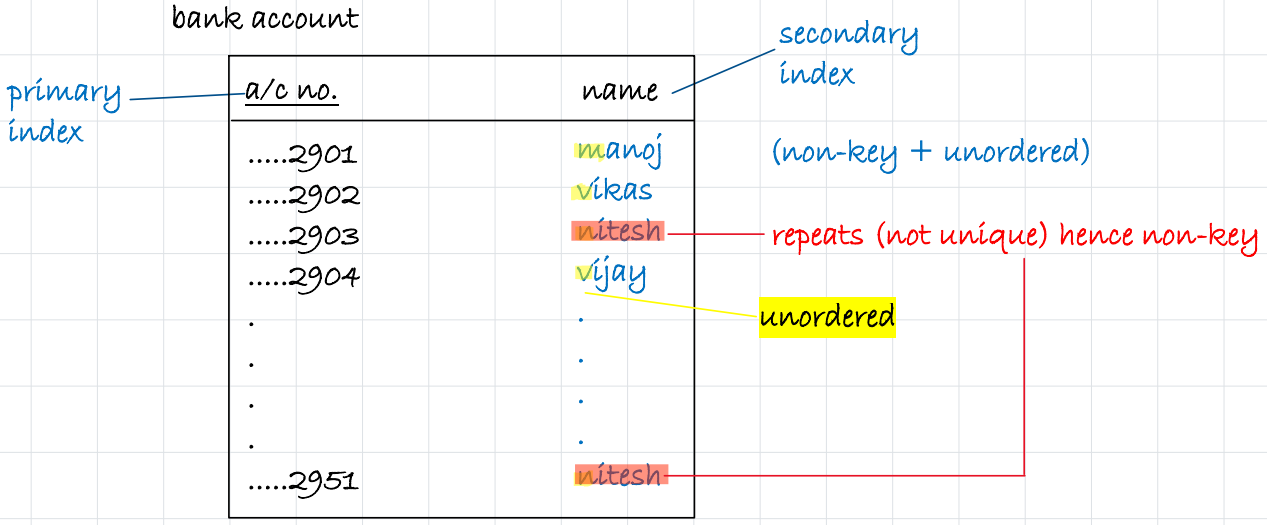
secondary index provides a way of accessing even if some primay index (or) clustering index already exists.

- it is dense.
- more than one secondary index possible.

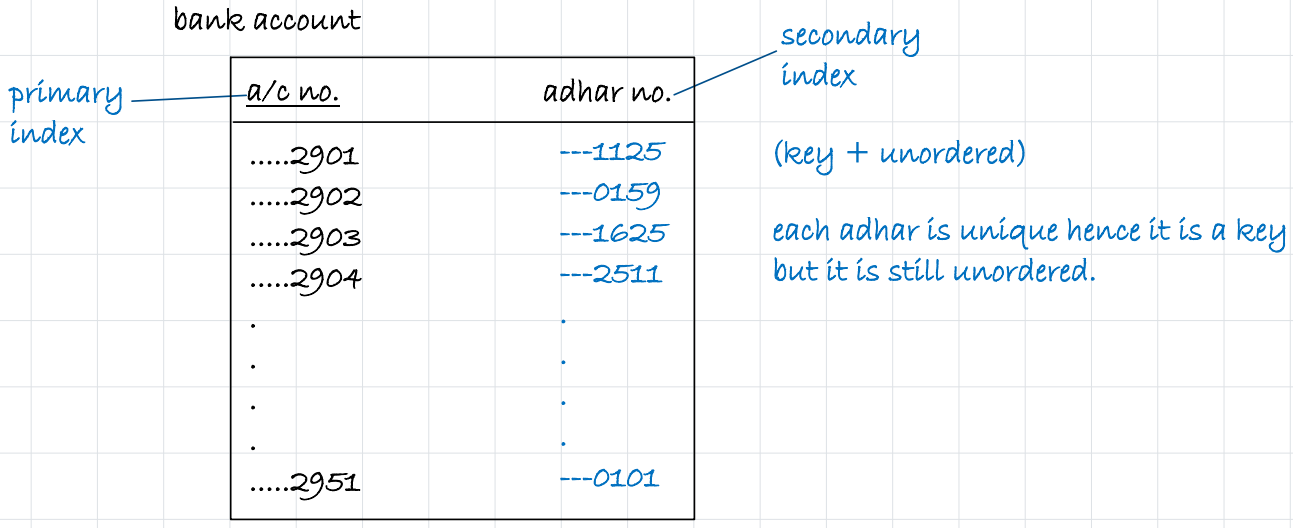
q. why in secondary index non-key/candidate key and why dense?

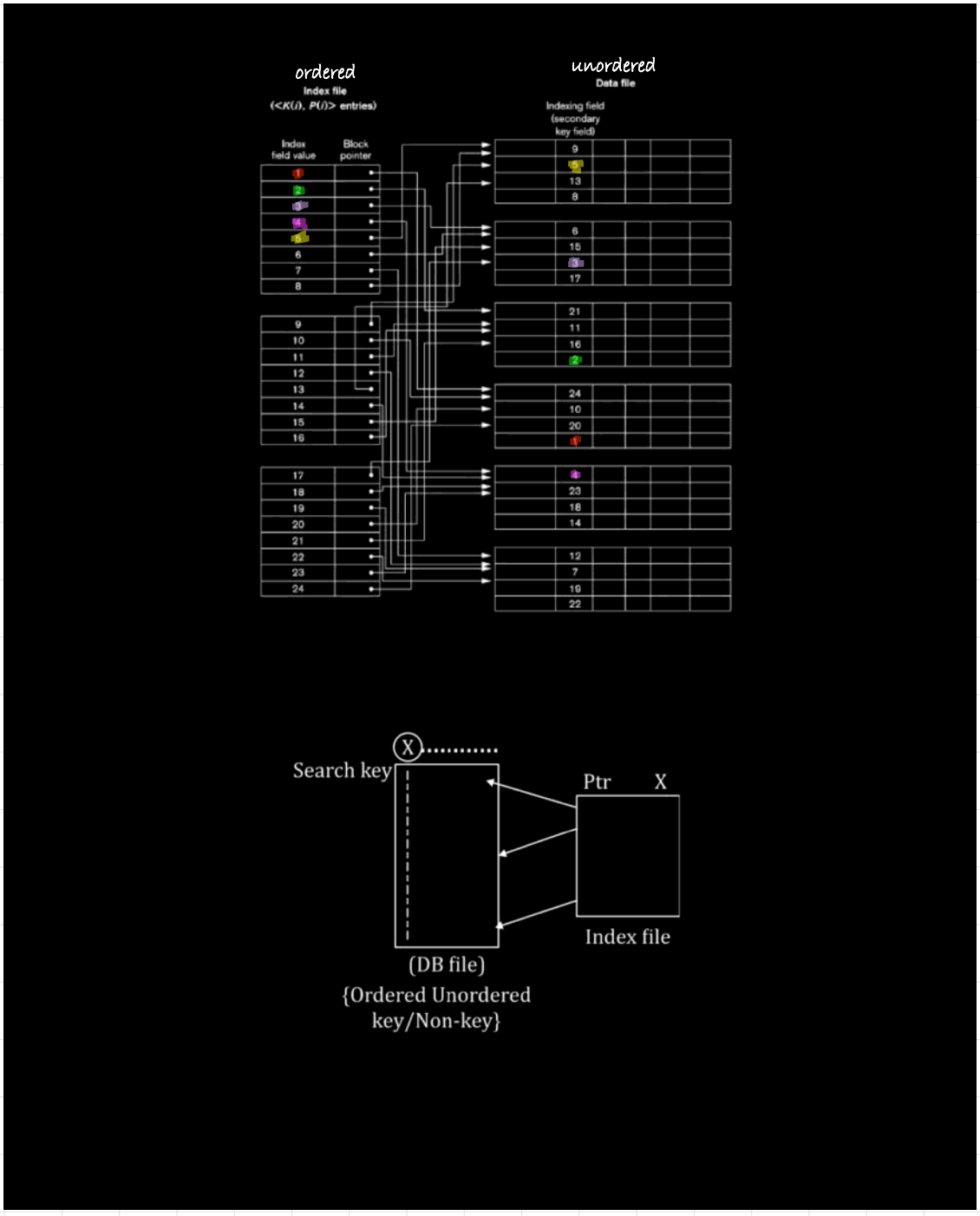
sol. secondary index applied already and primary index (or) clustering index already exist.

(i) non-key + unordered



(ii) key + unordered





multilevel index -

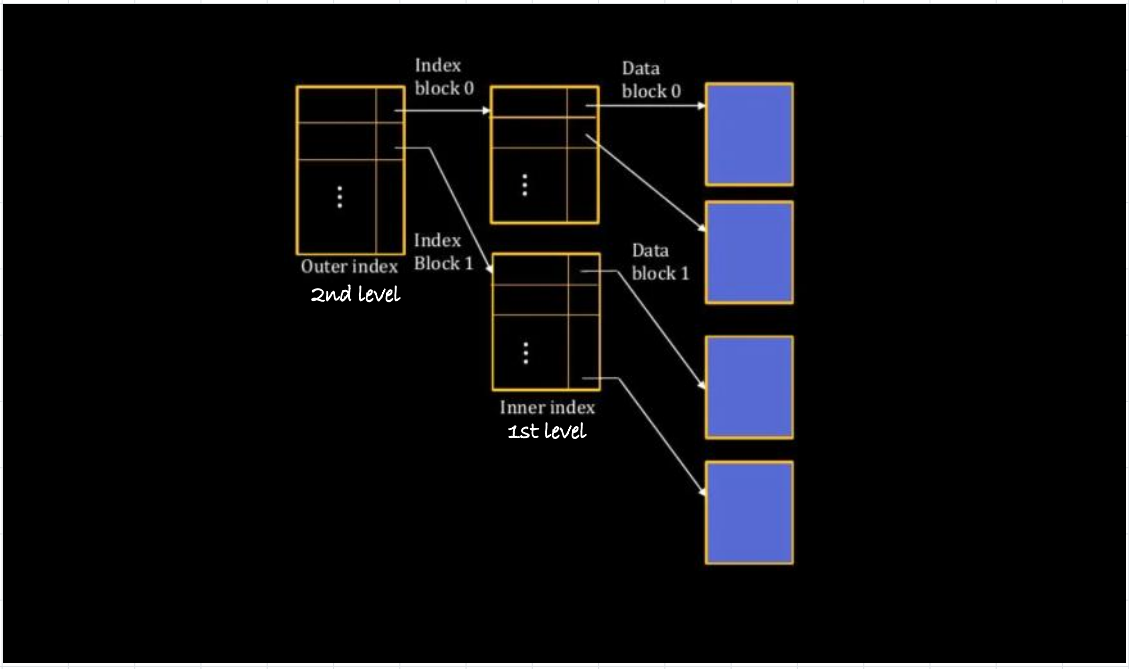
if index does not fit in memory, access become expensive.

solution - treat index kept on disk as a sequential file and construct a sparse index on it

- outer index : a sparse index of the basic index
- inner index : the basic index file

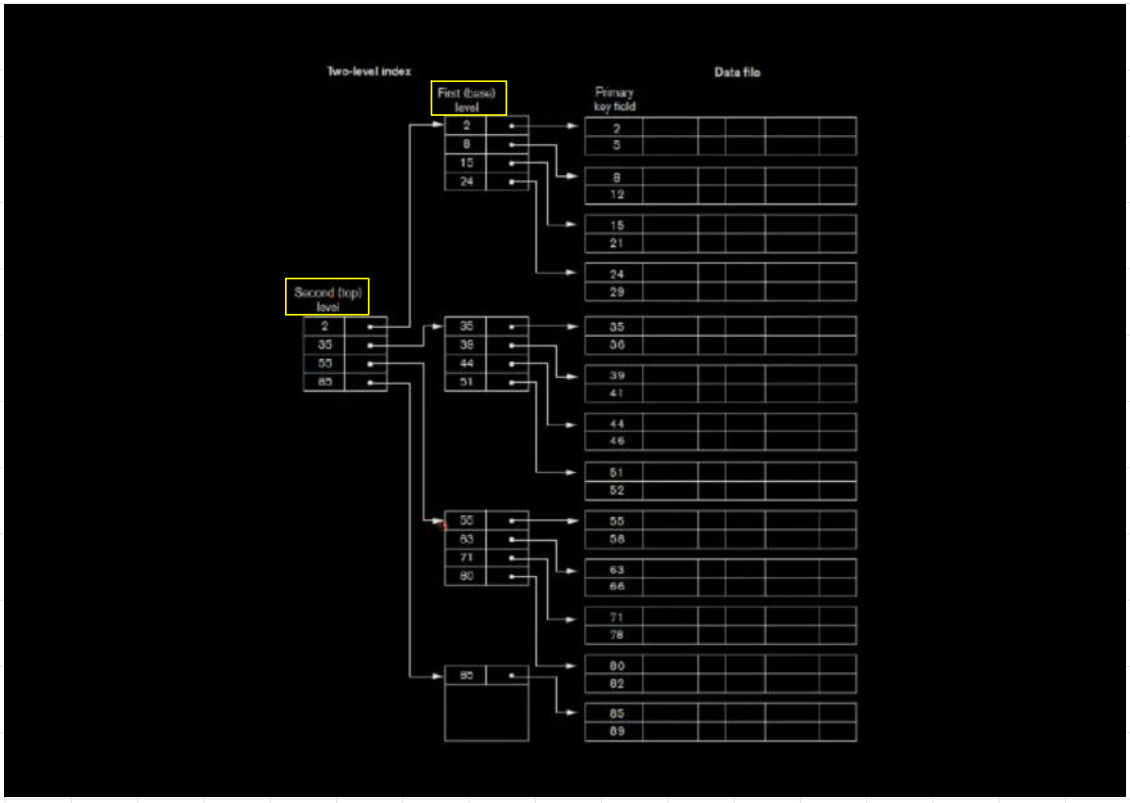
if outer index is too large to fit in main memory, yet another level of index can be created, and so on (jab tak ek dabbe mai na aajaye)

indices at all levels must be updated on insertion or deletion from the file



implementation :

- index file :
  (a) base level: applies multilevel index
  (b) second level : applies primary index to first level (base level)
  (c) third level : applies primary index to second level



we can repeat the above process until index entries fit into one block

if there are n level in multilevel index then the number of block access to search for a record = n+1
(at each level one index block + 1 data block)

Q.3 Find the average number of block access required to search for a record if multilevel Index is created on the Data file of Question 2.

Block factor of Index file = 68 Index entries per Block

I[st] Level: Total number of Index Block = 442 Index Block

II[nd] Level: number of Index Records (entries) = 442 (SPARSE number of I[st] level block) & Block factor = 68 Index entries for block

Total number Index Block = $\left\lceil \frac{442}{68} \right\rceil = 7$ Index Block

III[rd] Level: Number of Index Record = 7 (number of 2[nd] level block)

Total number of index Block = $\left\lceil \frac{7}{68} \right\rceil = 1$

Average Number of block Access = 1 + 1 + 1 + 1 = 4

Q.2 Consider a file of 16384 records. Each record is 32 bytes long and its key field is of size 6 bytes. The file is orderd on a non-key field, and the file organization is unspanned. The file is stored in a file system with block size 1024 bytes, and the size of block pointer is 10bytes. If the secondary index is built on the key field of the file, and a multilevel index scheme is used to store the secondary index, the number of first-level and second-level block in the multilevel index are respectively [GATE-2008 : 2 Marks]

(A) 8 and 0 (B) 128 and 6

(C) 256 and 4 (D) 512 and 5

records = 16384
block size = 1024 byte
record size = 132 byte
key = 6 byte
B_p = 10 byte

with index (dense) :

one index record size = 6 + 10 = 16 byte

Block factor of index file $= \left\lfloor \frac{\text{block size}}{\text{record size}} \right\rfloor = \left\lfloor \frac{1024}{16} \right\rfloor$ = 64 index entires per block

1st level

total number of index entries :

dense : 16384 [2^{14}] (number of database record)

total number of index block (b_i) : $\frac{16384\ (2^{14})}{64} = 2^8 = 256$

2nd level

total number of index entries :

dense : 256 (1st level index block)

total number of index block (b_i) : $\frac{256}{64} = 4$

total number of index block (b_i) : $\frac{256}{64} = 4$

3rd level

total number of index entries :

dense : 4 (2nd level index block)

total number of index block (b_i) : $\frac{4}{64} = 1$

total 3 level required.

to access a record using multilevel index average number of block access = n + 1
= 3 + 1
= 4

dynamic multilevel indexing

(i) B tree (balanced tree)
(ii) B+ tree

data structure pov

(i) B tree (balanced tree)

ORDER : p

minimum element : $\lceil p/2 \rceil - 1$

maximum element : p - 1

ORDER : 5

minimum element : $\lceil 5/2 \rceil - 1 = 3 - 1 = 2$

maximum element : 5 - 1 = 4

element : keys

ex. 1, 4, 3, 6, 7, 9, 11

1	4		

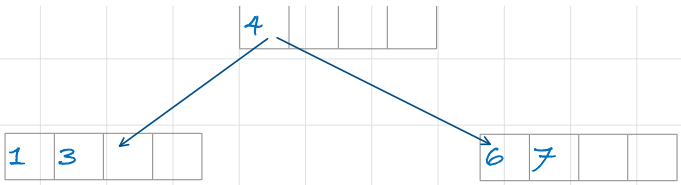
jaise hi 3 bharege toh vo sorting kardega

1	3	4	6

jaise hi 7 bharege toh vo overflow ho jayega

1	3	4	6	7

islie ab middle ko parent bana denge



database pov

(i) B tree (balanced tree)

ORDER : p

minimum element : $\lceil p/2 \rceil - 1$

maximum element : $p - 1$

Block pointer (B_p) : p

Read pointer (R_p) : $p - 1$

ORDER : 5

minimum element : $\lceil 5/2 \rceil - 1 = 3 - 1 = 2$

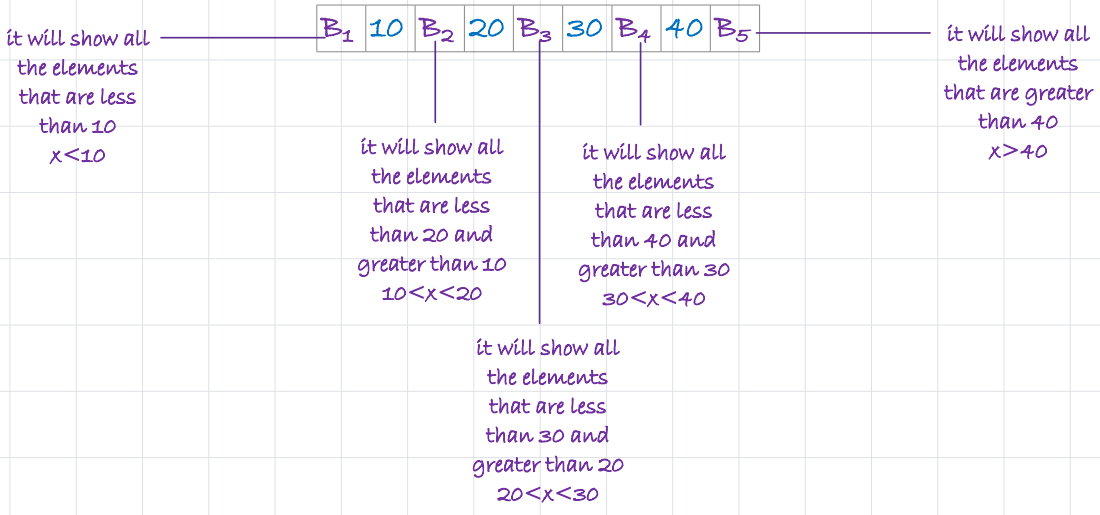
maximum element : $p - 1 = 5 - 1 = 4$

Block pointer (B_p) : $p = 5$

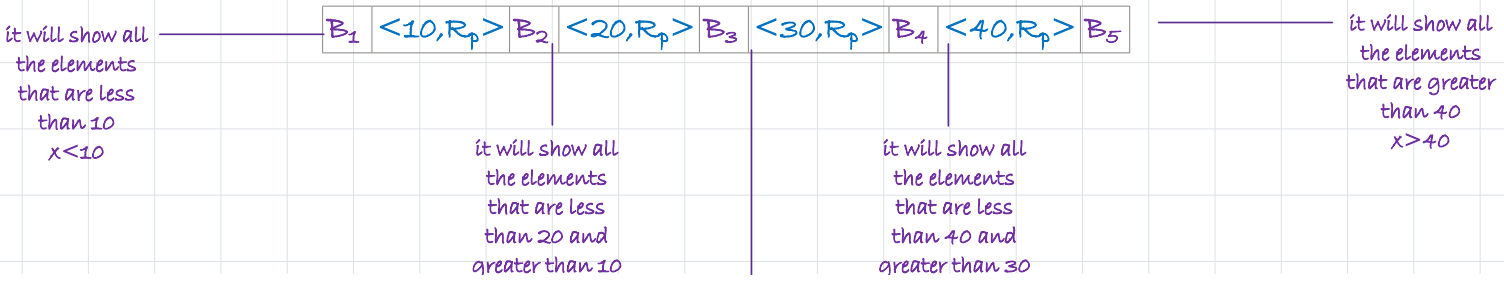
Read pointer (R_p) : $p - 1 = 5 - 1 = 4$

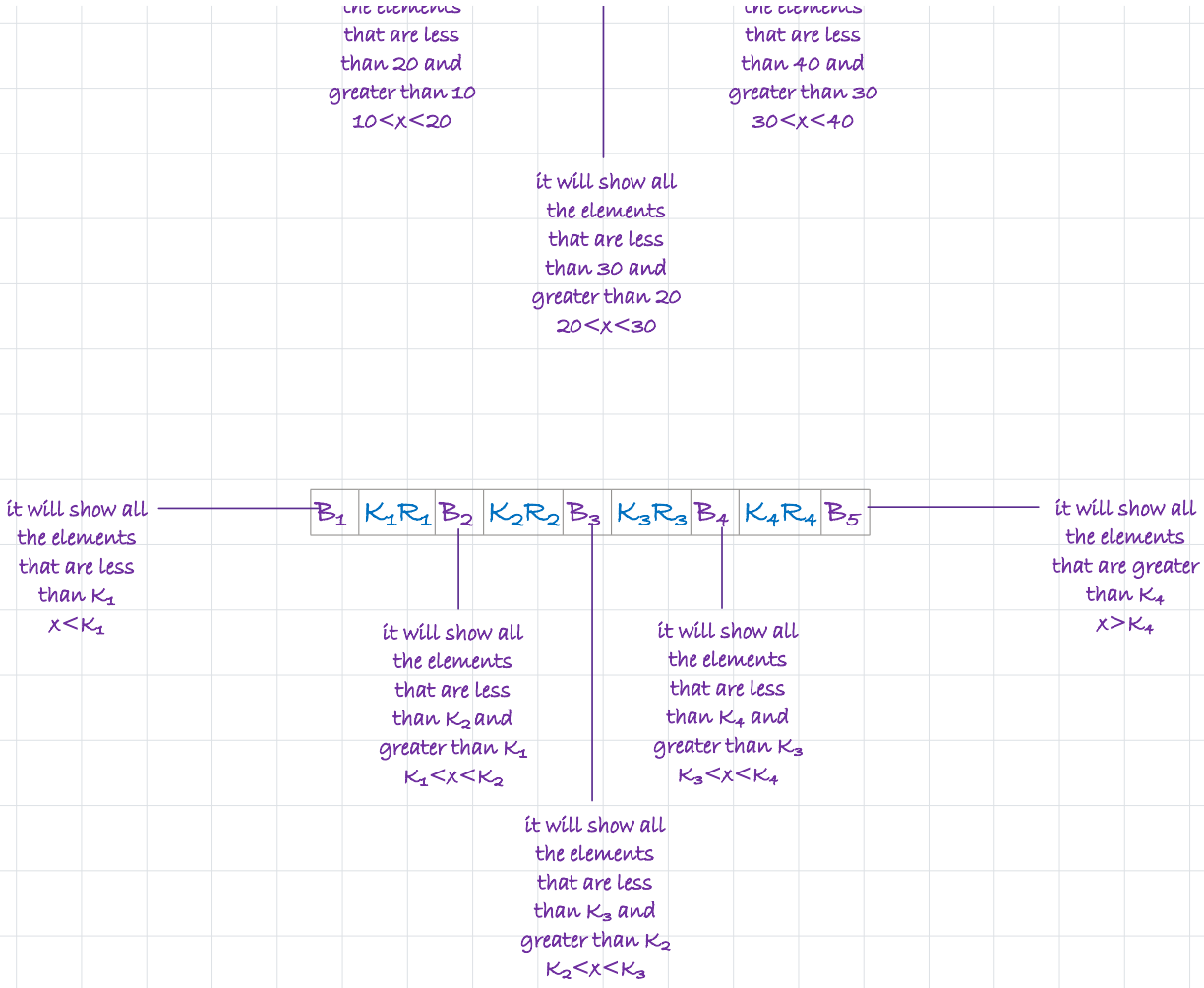
element : keys

B_1	K_1R_1	B_2	K_2R_2	B_3	K_3R_3	B_4	K_4R_4	B_5



representation :





- ex. 1, 3, 5, 7, 9, 11, 13, 16

ORDER : 5

Block pointer (B_p) : p = 5

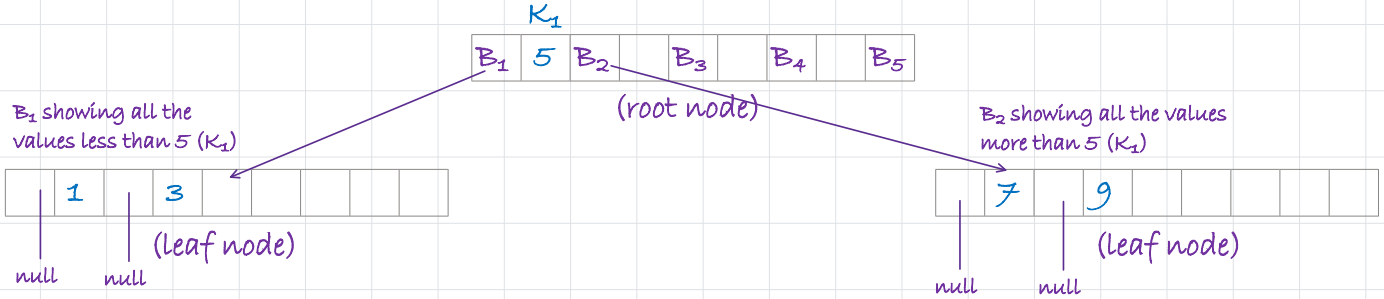
Read pointer (R_p) : p - 1 = 5 - 1 = 4

key : p - 1 = 5 - 1 = 4

B_1	1	B_2	3	B_3	5	B_4	7	B_5

B_1	1	B_2	3	B_3	5	B_4	7	B_5	← 9

as you want to fill 9, it will violate because there can be maximum 4 keys so now we will make middle element root



block pointer 5 : 5 node ko represent kar sakta hai
key 4 : har node mai maximum 4 keys ho sakti hai

- ex. 1, 3, 5, 7, 9, 11, 13, 16
  (middle in even numbers)

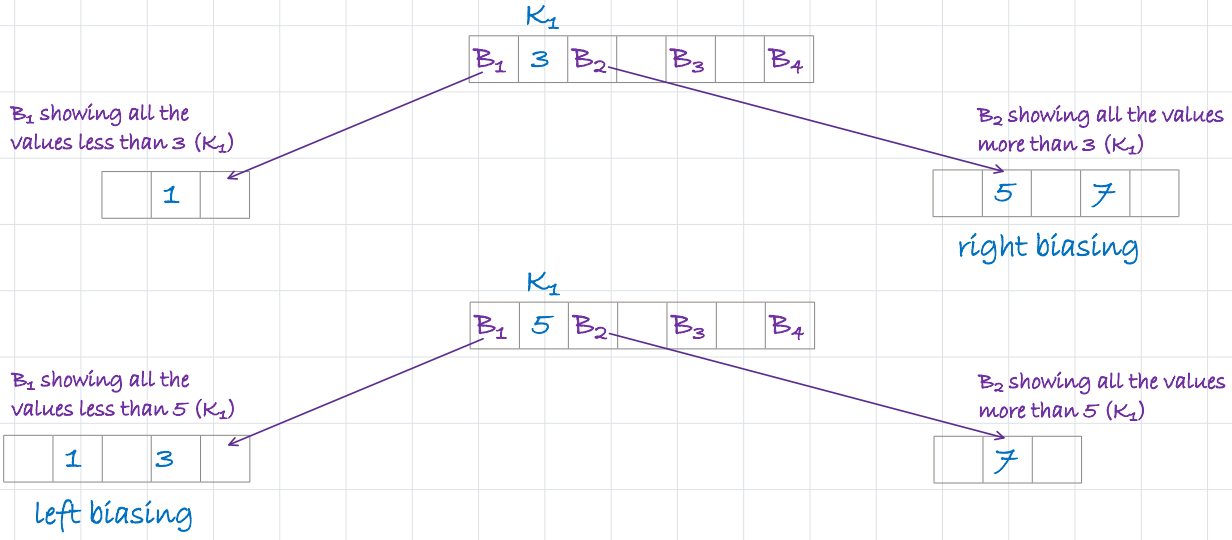
ORDER : 4

Block pointer (B_p) : p = 4

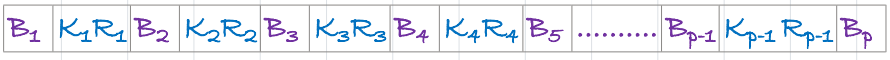
Read pointer (R_p) : p - 1 = 4 - 1 = 3

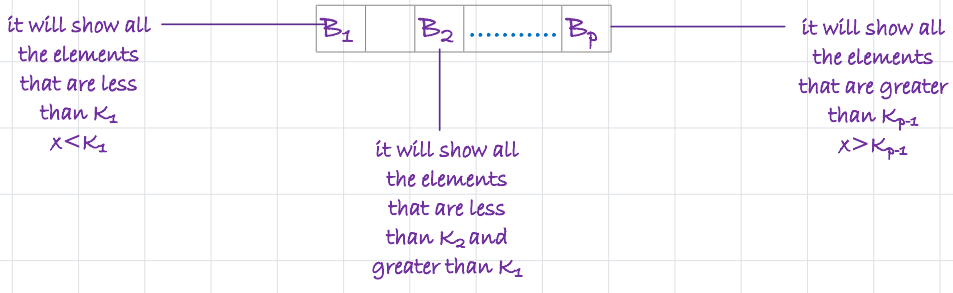
key : p - 1 = 4 - 1 = 3

as you want to fill 9, it will violate because there can be maximum 4 keys so now we will make middle element root



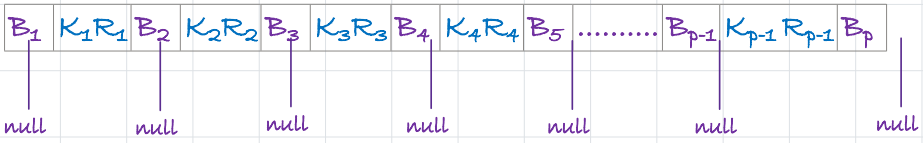
(i) structure of internal node





$K_1 < X < K_2$

(ii) structure of leaf node



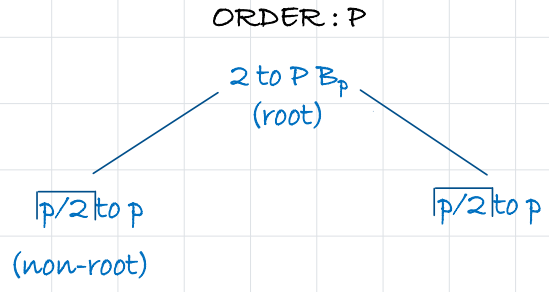
(iii) every internal node except the root node contains

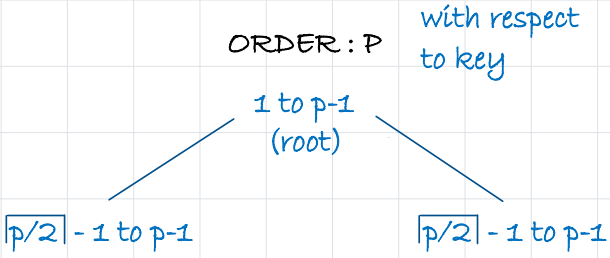
minimum :	maximum :
block pointer (Bp) : $\lceil p/2 \rceil + 1$	Block pointer (B_p) : p
keys : $\lceil p/2 \rceil - 1$	keys : p - 1
	Read pointer (R_p) : p - 1

(iv) root contains atleast 2 block pointer with (1 keys) and atmost P block pointer (p-1 keys)

(v) keys within the node should be in ascending order.

(vi) every leaf node should be at same level.





q.(i) can root have only one block pointer?
sol. never (no)

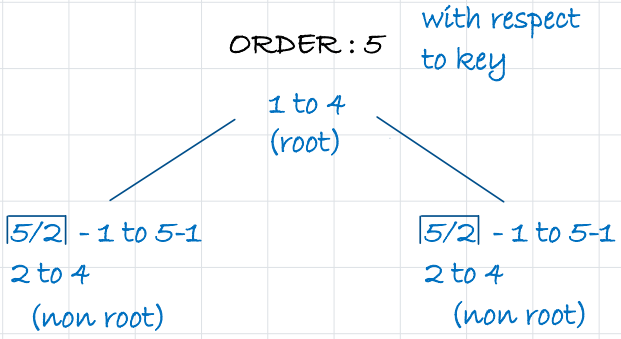
minimum 2 block pointer required.

- ORDER : P

	minimum	maximum
root (B_p)	2	P
non root (B_p)	$\lceil p/2 \rceil$	P
root (keys)	1	P-1
non root (keys)	$\lceil p/2 \rceil - 1$	P-1

- ORDER : 5

	minimum	maximum
root (B_p)	2	5
non root (B_p)	$\lceil 5/2 \rceil = 3$	5
root (keys)	1	4
non root (keys)	$\lceil 5/2 \rceil - 1 = 3-1 = 2$	4



root : 1 to 4 keys
non root : 2 to 4 keys

(i) maximum

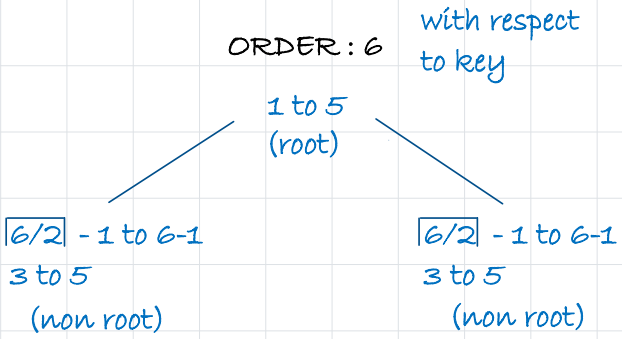
(ii) root

(iii) non-root

50% occupy

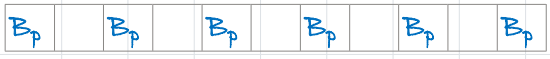
- ORDER : 6

	minimum	maximum
root (B_p)	2	6
non root (B_p)	$\lceil 6/2 \rceil = 3$	6
root (keys)	1	5
non root (keys)	$\lceil 6/2 \rceil - 1 = 3-1 = 2$	5

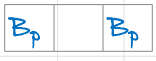


root : 1 to 5 keys
non root : 3 to 5 keys

(i) maximum



(ii) root



(iii) non-root

50% occupy

Q. can we have order 8 possible in this diagram?

yes, it i possible because minimum 3 keys

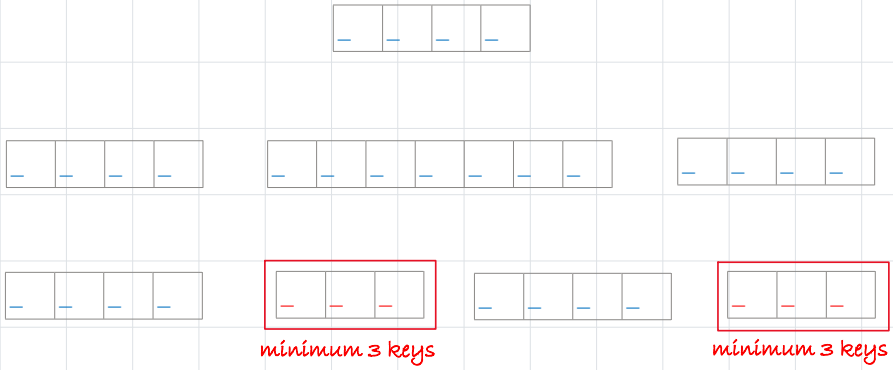
- ORDER : 8

	minimum	maximum
non root (B_p)	$\lceil 8/2 \rceil = 4$	7
non root (keys)	$\lceil 8/2 \rceil - 1 = 4-1 = 3$	7

- minimum 3 keys to maximum 7 keys
- minimum 4 block pointer to maximum 8 block pointer

Q. can we have order 9 possible in this diagram?

no, not possible because minimum 3 keys



- ORDER : 9

	minimum	maximum
non root (B_p)	$\lceil 9/2 \rceil = 5$	8

non root (B_p)	$\lceil 9/2 \rceil = 5$	8
non root (keys)	$\lceil 9/2 \rceil - 1 = 5-1 = 4$	8

- minimum 4 keys to maximum 8 keys
- minimum 5 block pointer to maximum 8 block pointer

formula for B tree question :

ORDER : P

B_1	K_1R_1	B_2	K_2R_2	B_3	K_3R_3	B_4	K_4R_4	B_5	.....	B_{p-1}	$K_{p-1}R_{p-1}$	B_p

$P \times B_p + (p-1)\ [keys + R_p] \leq$ block size

(or)

$P \times B_p + (p-1)$ keys $+ (p-1)\ R_p \leq$ block size

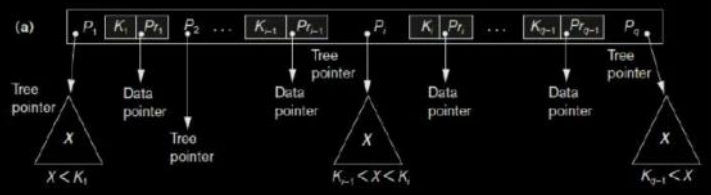
B_p : block/tree/child pointer

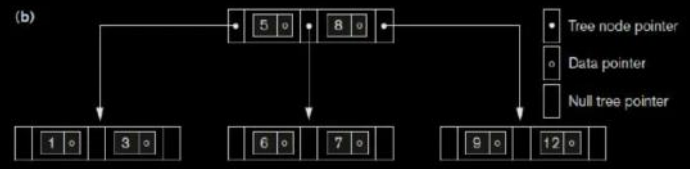
R_p : read/data/record pointer

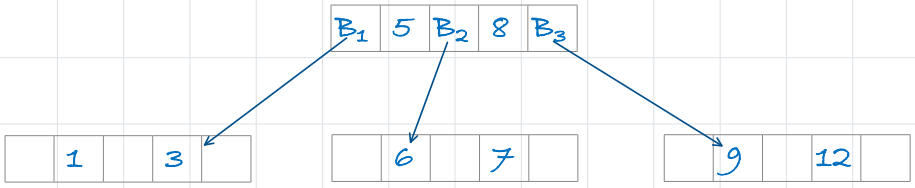
key : search key

B_p : 1
key : p-1
R_p : p-1

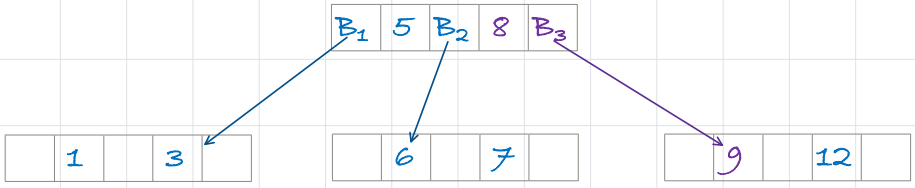
B-tree structures (a) A node in a B-tree with q−1 search values (b) A B-tree of order p=3. The values were inserted in the order 8, 5, 1, 7, 3, 12, 9, 6





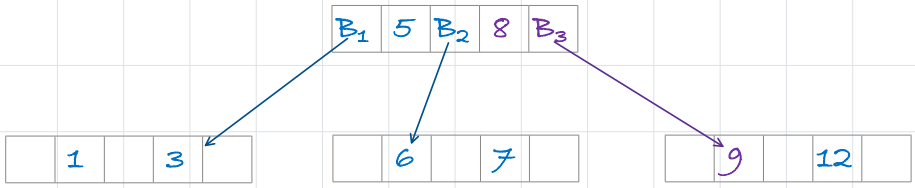


q.(i) for '9' search how many block access are required?



2 block access are required. (successful search)

q.(ii) for '13' search how many block access are required?



2 block access are required. (unsuccessful search)

note :
- if unsuccessful search that means we traverse from root to till leaf node.
- if successful search that means we traverse from root to till that key find (that block in which key is present)

by default max P block pointer

ORDER : P

$P \times Bp + (p-1)\ [keys + Rp] \leq$ block size

(or)

$P \times B_p + (p-1)\ keys + (p-1)\ R_p \leq$ block size

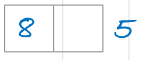
ORDER of key : P

$(P+1) \times B_p + P\ [keys + R_p] \leq$ block size

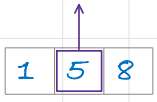
some variations :

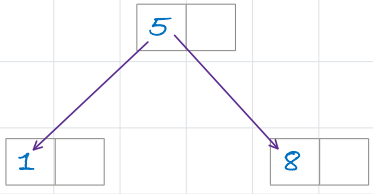
q. 8, 5, 1, 7, 3, 12, 9, 6, 11

(i)

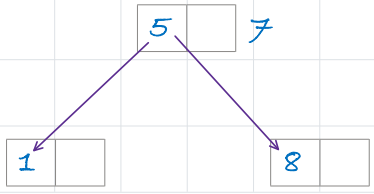


(ii)

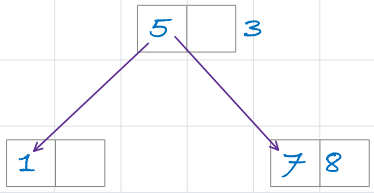




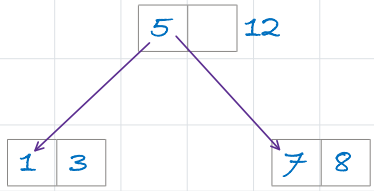
(iii)



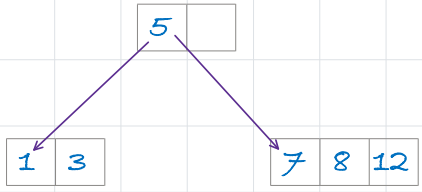
(iv)

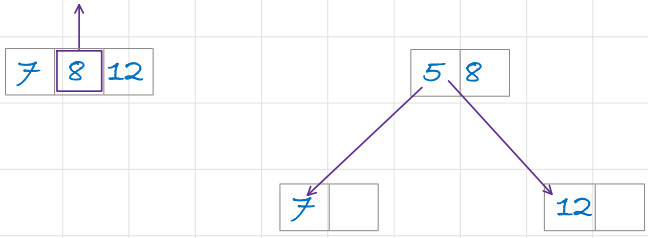


(v)

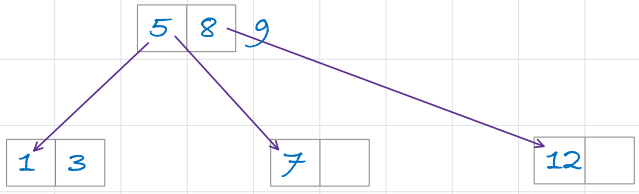


(vi)

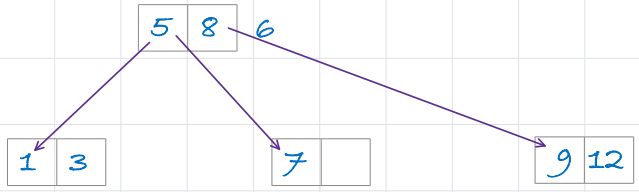




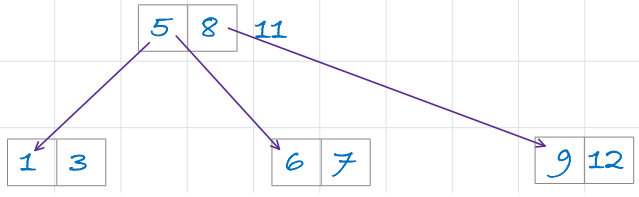
(vii)



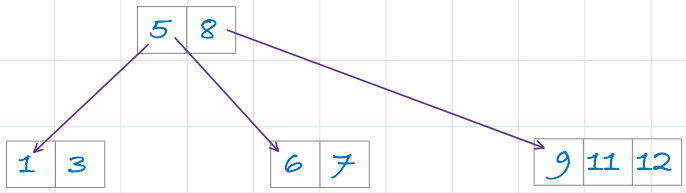
(viii)

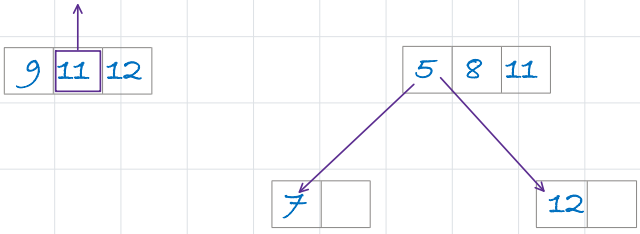


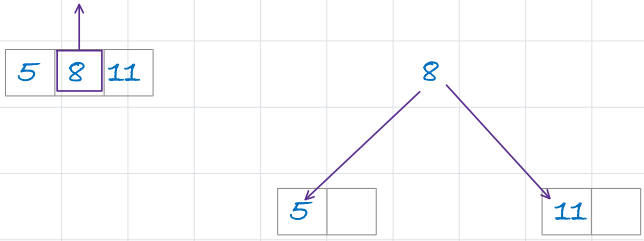
(ix)



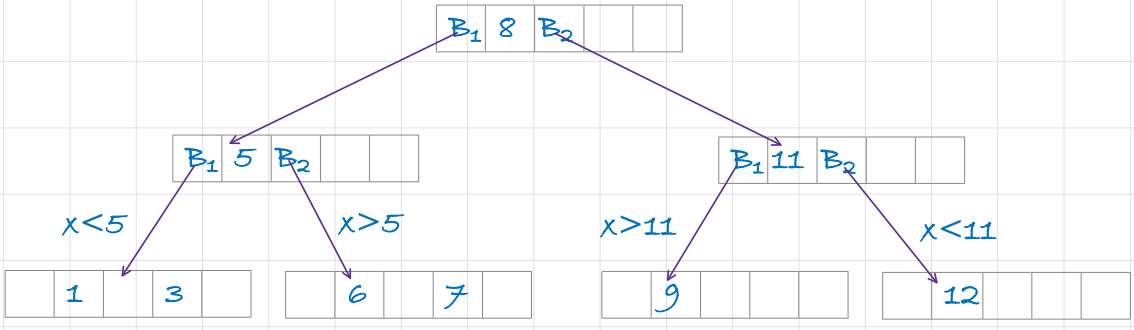
(x)







final :



- ORDER : P (minimum)

height/level	minimum no. of nodes	minimum no. of B_p	minimum no. of keys
0/1	1	2	1
1/2	2	$2(\lceil p/2 \rceil)$	$2(\lceil p/2 \rceil - 1)$
2/3	$2(\lceil p/2 \rceil)$	$2(\lceil p/2 \rceil)^2$	$2(\lceil p/2 \rceil)[(\lceil p/2 \rceil - 1)]$
3/4	$2(\lceil p/2 \rceil)^2$	$2(\lceil p/2 \rceil)^3$	$2(\lceil p/2 \rceil)^2[(\lceil p/2 \rceil - 1)]$

3/4	$2(\lceil p/2 \rceil)^2$	$2(\lceil p/2 \rceil)^3$	$2(\lceil p/2 \rceil)^2[(\lceil p/2 \rceil - 1)]$
h/h+1	$2(\lceil p/2 \rceil)^{h-1}$	$2(\lceil p/2 \rceil)^h$	$2(\lceil p/2 \rceil)^{h-1}[(\lceil p/2 \rceil - 1)]$

total minimum number of keys (for height = h and level h+1) :

$2(\lceil p/2 \rceil - 1) + 2(\lceil p/2 \rceil)[(\lceil p/2 \rceil - 1)] + 2(\lceil p/2 \rceil)^2[(\lceil p/2 \rceil - 1)]$

q. ORDER : 5 and level : 4
total minimum number of keys.

sol. $2(\lceil p/2 \rceil - 1) + 2(\lceil p/2 \rceil)[(\lceil p/2 \rceil - 1)] + 2(\lceil p/2 \rceil)^2[(\lceil p/2 \rceil - 1)]$

$2(\lceil 5/2 \rceil - 1) + 2(\lceil 5/2 \rceil)[(\lceil 5/2 \rceil - 1)] + 2(\lceil 5/2 \rceil)^2[(\lceil 5/2 \rceil - 1)]$

$2(3-1) + 2(3)[(3-1)] + 2(3)^2[3-1]$

$2(2) + 2(3)[(2)] + 2(9)[2]$

$4 + 12 + 36 = 52$

- ORDER : P (maximum)

height/level	maximum no. of nodes	maximum no. of B_p	maximum no. of keys
0/1	1	P	(P-1)
1/2	P	P^2	P(P-1)
2/3	P^2	P^3	$P^2(P-1)$
3/4	P^3	P^4	$P^3(P-1)$
h/h+1	P^n	P^{n+1}	$P^n(P-1)$

total maximum number of keys (for height = h and level h+1) :

$(P-1) + P(P-1) + P^2(P-1) + P^3(P-1) + P^n(P-1)$

$(P-1)[1 + P + P^2 + P^3 \ldots\ldots + P^n]$

q. ORDER : 5 and level : 5
total maximum number of keys.

sol. $(P-1) + P(P-1) + P^2(P-1) + P^3(P-1) + P^4(P-1)$

total maximum number of keys.

sol. $(P-1) + P(P-1) + P^2(P-1) + P^3(P-1) + P^4(P-1)$

$(5-1) + 5(5-1) + 5^2(5-1) + 5^3(5-1) + 5^4(5-1)$

$4 + 5(4) + 25(4) + 125(4) + 625(4)$

$4 + 20 + 100 + 500 + 2500 = 3124$

Q.3 Consider a table T in a relational database with a key field K. A B-tree of order p is used as an access structure on K, where p denotes the maximum number of tree pointers in B-tree index node. Assume that K is 10 bytes long; disk block size is 512 bytes; each data pointer P_D is 8 bytes long and each block pointer P_B is 5 bytes long. In order for each B-tree node to fit in a single disk block, the maximum value of p is [GATE-2004 : 2 Marks]

A 20 B 22

C 23 D 32

$P \times B_p + (p-1)[keys + R_p] \leq$ block size

$P \times 5 + (p-1)[10 + 8] \leq 512$

$P \times 5 + (18p-18) \leq 512$

$5P + 18p - 18 \leq 512$

$23P \leq 512 + 18$

$23P \leq 530$

$P \leq \frac{530}{23}$

$P \leq 23.04$

why we took 23 and not 24?

if we take ORDER : 24
and when we put the values then we will get the block size more than 512 byte.

note : B tree is more suitable for random access for some record but not for range queries

difference between B tree and B+ tree :

in B and B+ tree there are majorly three differences :

(i) all keys are available at leaf node.

(ii) no read pointer in internal node.
(iii) leaf node contains keys and read pointer + only one block.

B tree

(i) structure of internal node

B_1	K_1R_1	B_2	K_2R_2	B_3	K_3R_3	B_4	K_4R_4	B_5	.....	B_{p-1}	K_{p-1} R_{p-1}	B_p

ORDER : P (maximum number of block pointer)

$B_p = P$

keys = P-1

$R_p = P-1$

$P \times B_p + (p-1)$ [keys + R_p] ≤ block size

B+ tree

(i) structure of internal node

B_1	K_1	B_2	K_2	B_3	K_3	B_4	K_4	B_5	.....	B_{p-1}	K_{p-1}	B_p

(no read pointer)

ORDER : P (maximum number of pointer)

$P \times B_p + (p-1)$ [keys] ≤ block size

B tree

(ii) structure of leaf node

B_1	K_1R_1	B_2	K_2R_2	B_3	K_3R_3	B_4	K_4R_4	B_5	.....	B_{p-1}	K_{p-1} R_{p-1}	B_p

null null null null null null null

B+ tree

(ii) structure of leaf node

K_1R_1	K_2R_2	K_3R_3	K_4R_4	.....	K_{p-1} R_{p-1}	B_p

(only one block pointer)

ORDER of leaf node

$(P-1)[keys + R_p] + 1B_p \leq$ block size

note : in a B tree internal node and leaf node have same structure (order of internal node is same as order of leaf node in B tree) but B+ tree interal node and leaf node have different structures. (order of interal node is not same as order of leaf node in B+ tree)

q. block size : 1024 byte
key : 6 byte
B_p : 8 byte
R_p : 10 byte

order of internal node and leaf node in a B tree?

$P \times B_p + (p-1)[keys + R_p] \leq$ block size

$P \times 8 + (p-1)[6 + 10] \leq 1024$

$8P + 16p - 16 \leq 1024$

$24P \leq 1040$

$P \leq \frac{1040}{24}$

$P \leq 43$

order of interal node : 43
order of leaf node : 43

q. block size : 1024 byte
key : 6 byte
B_p : 8 byte
R_p : 10 byte

order of internal node and leaf node in a B tree?

order of interal node :

$P \times B_p + (p-1)[keys] \leq$ block size

$P \times 8 + (p-1)[6] \leq 1024$

$8P + 6p - 6 \leq 1024$

$14P \leq 1030$

$P \leq \lfloor 1030 \rfloor$

order of leaf node :

$(P-1)[keys + R_p] + 1B_p \leq$ block size

$(P-1)[6+10] + 1 \times 8 \leq 1024$

$16P - 16 + 8 \leq 1024$

$16P \leq 1032$

$P \leq \lfloor 1032 \rfloor$

$14P \leq 1030$

$P \leq \frac{1030}{14}$

$P \leq 73$

order of interal node : 73

$16P \leq 1032$

$P \leq \frac{1032}{16}$

$P \leq 64$

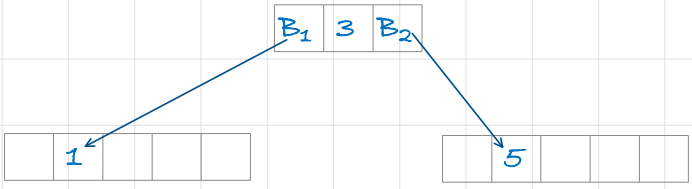
order of leaf node : 64

difference between B tree and B+ tree :

- B tree

keys : 1, 3, 5

ORDER : 3

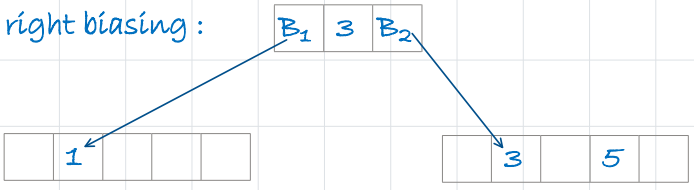


- B+ tree

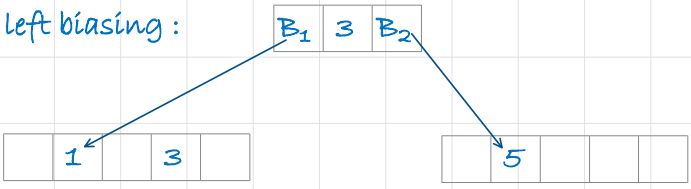
keys : 1, 3, 5

ORDER : 3

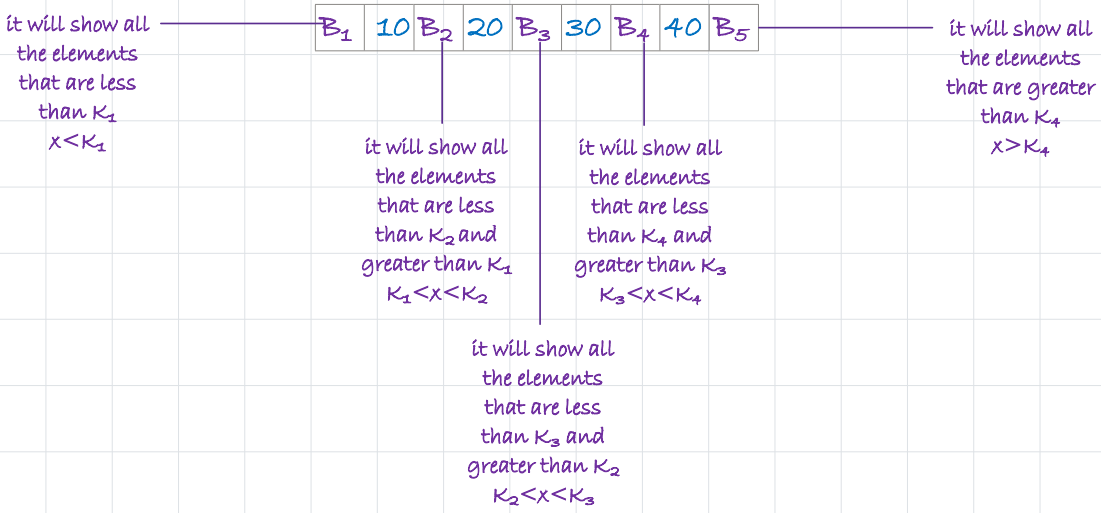
right biasing :



left biasing :

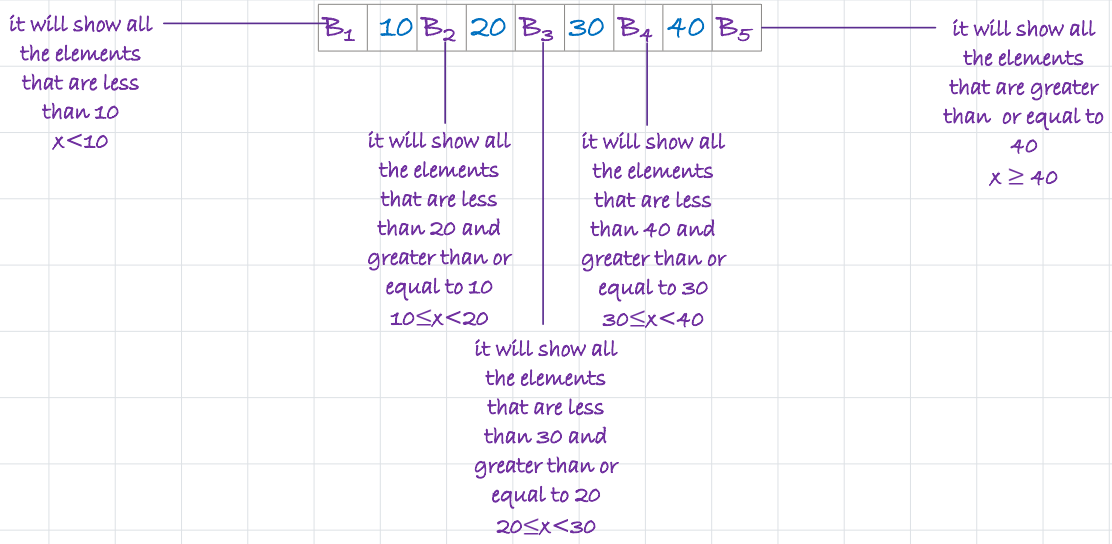


- B tree

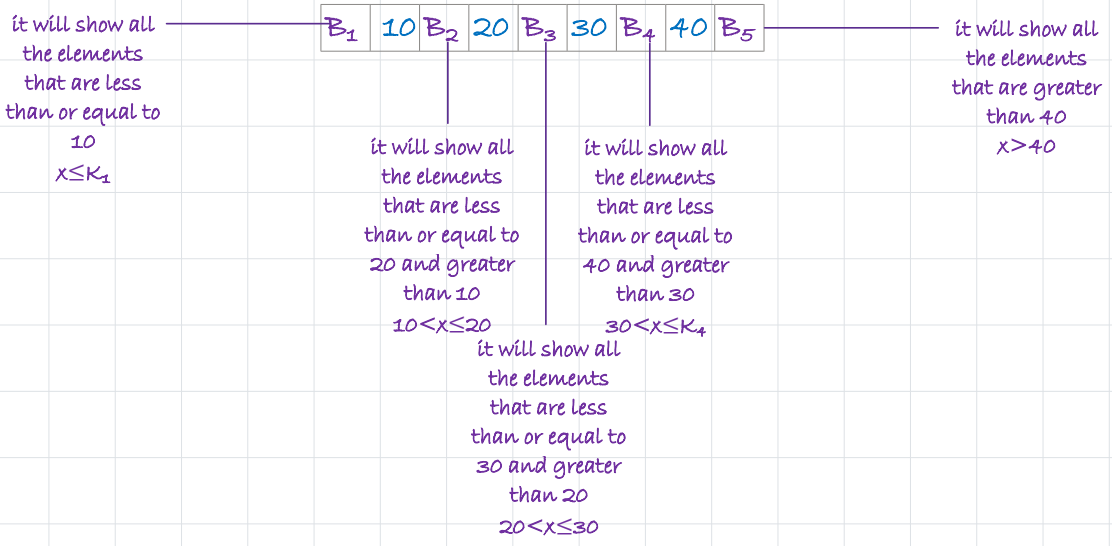


- B tree

(i) right biasing



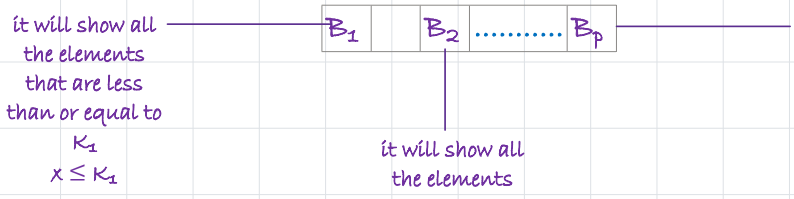
(ii) left biasing



B+ tree

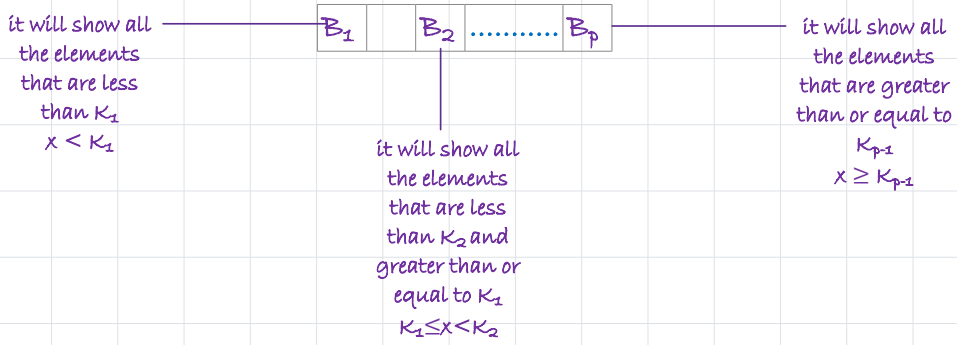
(i) structure of internal node

(i) left biasing



that are less than or equal to K_2 and greater than K_1
$K_1 < x \leq K_2$

(ii) right biasing



$P \times B_p + (p-1)$ [keys] $\leq$ block size

(ii) structure of leaf node

K_1R_1	K_2R_2	K_3R_3	K_4R_4	.....	$K_{p-1}R_{p-1}$	B_p

(only one block pointer)

ORDER of leaf node

$(P-1)$ [keys + R_p] + $1B_p \leq$ block size

(iii) every internal node except the root node contains

minimum :	maximum :
block pointer (Bp) : $\lceil p/2 \rceil + 1$	Block pointer (B_p) : p
keys : $\lceil p/2 \rceil - 1$	keys : p - 1
	Read pointer (R_p) : p - 1

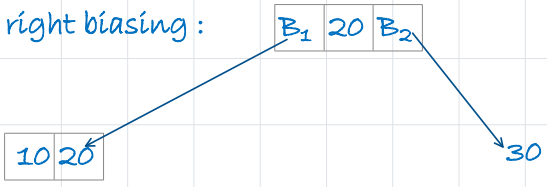
(iv) root contains atleast 2 block pointer with (1 keys) and atmost P block pointer (p-1 keys)

(v) keys within the node should be in ascending order.

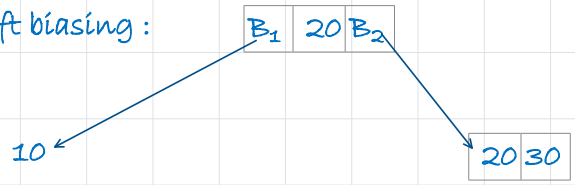
(vi) every leaf node should be at same level.

keys : 10, 20, 30
ORDER : 3

right biasing :



left biasing :



q(i) why internal node does not have R_p?

sol. all keys are available at leaf node only so read pointer is not required in the internal node so the space of read pointer is utilised to store more keys.

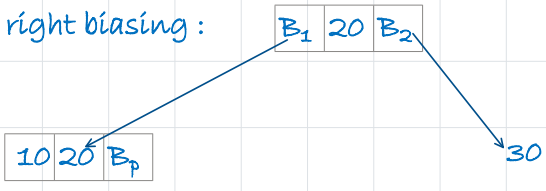
q(ii) why leaf node has only key and R_p only?

sol. all keys are available at leaf node only according to B+ tree definition so to read that keys (data/record) pointer is required.

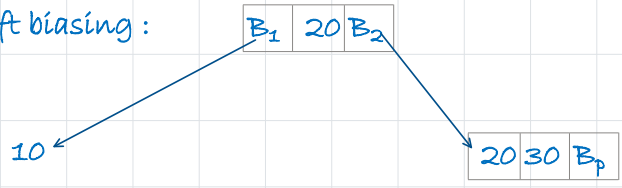
q(iii) why leaf node has only one block pointer and its use?

sol. B+ tree works is suitable for range query and in B+ tree all the keys are available at leaf node and 1 block pointer connect to next node at least (like linked list) so i/o cost is reduced.

right biasing :



left biasing :



q(iv) when all keys are available at the leaf node then why some are available in the non-leaf node?

sol. yes, some keys are available in the non-leaf node but they are useful in searching purpose only but these keys are used only for searching purpose and they don't have any read (data/record) pointer.

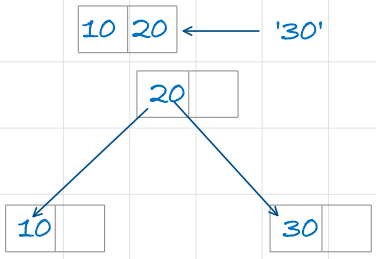
creation/insertion in B+ tree

keys : 10, 20, 30, 40, 50

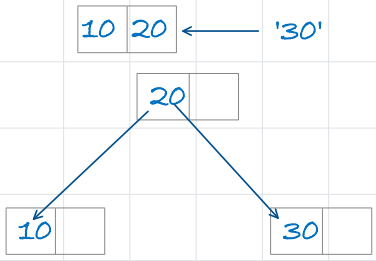
(i) in a B+ tree insertion start from leaf node.

(ii) split of internal node in B tree is same as split of internal node in B+ tree.

B tree internal node split :

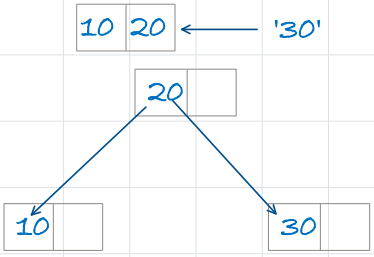


B+ tree internal node split :

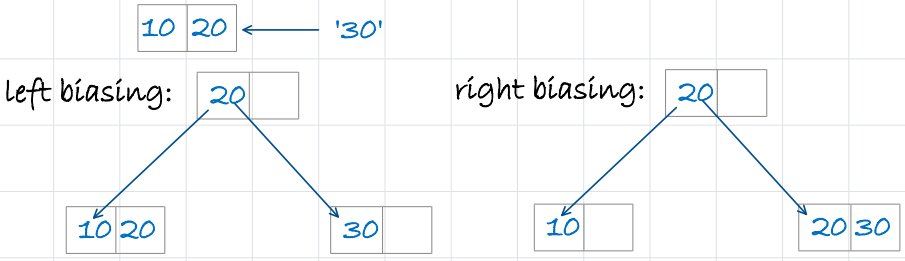


(ii) split of internal node in B tree is different as split of leaf node in B+ tree.

B tree internal node split :

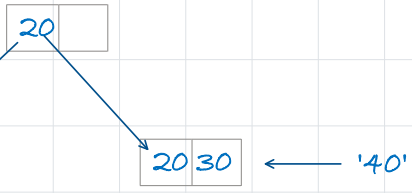


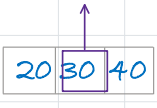
B+ tree internal node split :

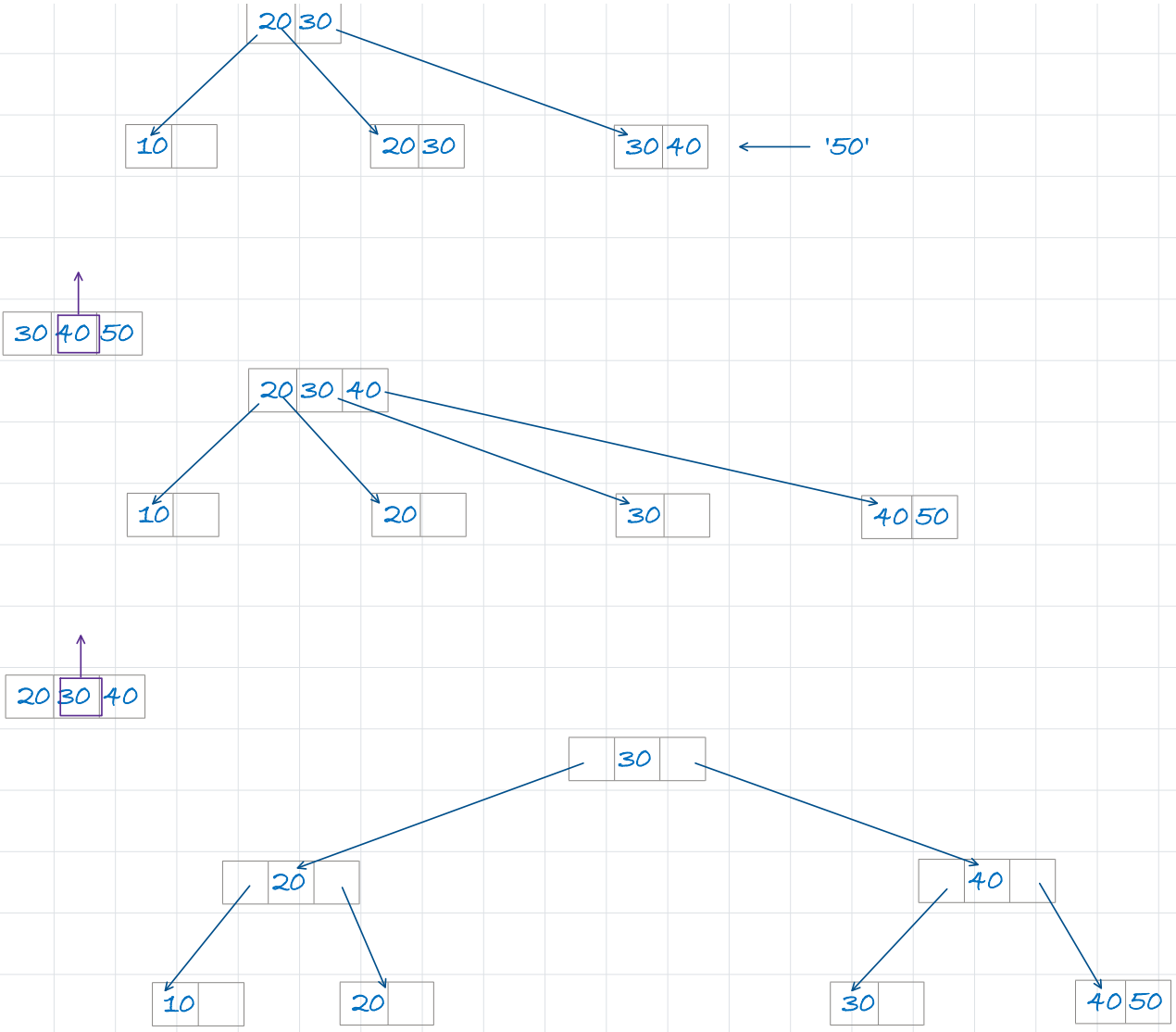


keys : 10, 20, 30, 40, 50
ORDER : 3

right biasing:







important points :

(i) every key of leaf node is not present in non leaf node.

(ii) 10, 20, 30, 40, 50 present in leaf node but not all present in non leaf node.

(iii) some keys or leaf node is present in non leaf node.

(iv) but every key of non leaf node must be present in leaf node (i.e. B+ tree all keys are available at the leaf node)